国外计算机科学教材系列

柔性字符串匹配

Flexible Pattern Matching in Strings

Practical On-Line Search Algorithms

for Texts and Biological Sequences

[美] Gonzalo Navarro
Mathieu Raffinot 著

中科院计算所网络信息安全研究组 译

電子工業出版社
Publishing House of Electronics Industry
北京 · BEIJING

内容简介

本书是一本不可多得的字符串匹配方面的专业书籍。书中对串匹配问题进行了系统化的分类，从实际效果出发，着重详细介绍了串匹配领域内效果最好的若干种算法。并且给出了具有统一接口的算法伪码，使读者能清晰理解算法原理，易于实现算法编程，从而提高专业水平。此外，书中通过严谨的理论分析和大量实验数据，说明了每种算法在实际应用中的适用范围，由此提供了良好的应用指导，解决了串匹配算法的最佳适用性问题。

本书可帮助本领域的研究人员从整体上把握字符串匹配方面的脉络，而其他相关领域的人员也可借助本书非常清晰地了解串匹配问题的概况。

Authorized translation from the English language edition published by The Press Syndicate of the University of Cambridge, England. Copyright © Cambridge University Press 2002.

All rights reserved. No part of this book may be reproduced or transmitted in any form or by any means, electronic or mechanical, including photocopying, recording or by any information storage retrieval system, without permission from the Publisher.

This edition is licensed for distribution and sale in the People's Republic of China only excluding Hong Kong, Taiwan and Macau and may not be distributed and sold elsewhere.

Simplified Chinese language edition published by Publishing House of Electronics Industry. Copyright © 2007.

本书中文简体专有翻译出版权由Cambridge University Press 授予电子工业出版社。其原文版权及中文翻译出版权受法律保护。未经许可，不得以任何形式或手段复制或抄袭本书内容。

本书中文简体字版仅限于在中华人民共和国境内（不包括香港、澳门特别行政区以及台湾地区）发行与销售，并不得在其他地区发行与销售。

版权贸易合同登记号　图字：01-2007-1057

图书在版编目（CIP）数据

柔性字符串匹配 /（美）纳瓦罗（Navarro, G.）等著；中科院计算所网络信息安全研究组译.
北京：电子工业出版社，2007.3
（国外计算机科学教材系列）
书名原文：Flexible Pattern Matching in Strings: Practical On-Line Search Algorithms for Texts and Biological Sequences
ISBN 978-7-121-03858-7
Ⅰ.柔...　Ⅱ.①纳...　②中...　Ⅲ.电子计算机－算法理论－教材　Ⅳ.TP301.6
中国版本图书馆CIP数据核字（2007）第017507号

责任编辑：史　平
印　　刷：北京市天竺颖华印刷厂
装　　订：三河市金马印装有限公司
出版发行：电子工业出版社
　　北京市海淀区万寿路173信箱　邮编：100036
开　　本：787 × 980　1/16　　印张：13.75　字数：275千字
印　　次：2007年3月第1次印刷
定　　价：38.00元

凡所购买电子工业出版社的图书有缺损问题，请向购买书店调换；若书店售缺，请与本社发行部联系。联系电话：（010）68279077。邮购电话：（010）88254888。

质量投诉请发邮件至zlts@phei.com.cn，盗版侵权举报请发邮件至dbqq@phei.com.cn。

服务热线：（010）88258888。

译 者 序

初次看到《柔性字符串匹配》这本书是在 2003 年。那时，我们小组正忙于进行一个项目，需要一种高效的字符串匹配算法。虽然当时我们已经拥有几年的研究工作基础，钻研了不少经典算法，自己也开发了几种新算法，但为获得最佳性能，我们还是忙碌了几个月才基本达到目标。这时我们意识到：虽然串匹配问题是一个非常经典的问题，但当前却面临着非常大的挑战。尤其是在信息检索、信息过滤和计算生物学等领域，需求日新月异，对实时性的要求也越来越高，经典算法已经远远不能满足需求。这促使我们对于串匹配问题进行更深入的研究。就在此时，由 Gonzalo Navarro 和 Mathieu Raffinot 合著、剑桥大学出版社出版的本书进入了我们的视野，着实给了我们一种惊喜的感觉。惊讶的是，原来在串匹配领域中，已经有人不仅在理论上而且在实践上都对问题进行了如此深入的理解和阐述；喜悦的是，这本书对于我们不只是有用处，还让我们有种茅塞顿开的感觉！

这本书的学术价值在于对串匹配问题进行了系统化的分类，使本领域的研究人员可以从整体上把握本专业的脉搏，并使相关领域人员也可以非常清晰地理解串匹配问题的概况。本书的内容分类合理，章节紧凑，问题阐述由浅入深，是串匹配方面一本不可多得的专业书籍。更重要的是，作者没有片面求全，不是一一列举串匹配领域的全部算法和技术，而是从实际效果出发，着重详细介绍效果最好的若干种算法。叙述时条理清晰，内容详尽，并给出了具有统一接口的算法伪码，使读者能清晰地理解算法原理，方便地实现算法编程，从而提高在本领域的专业水平（包括理论和实践）。这本书不仅可以在学术方面对我们有所帮助，而且在应用方面的作用也是目前其他图书所不能比拟的。作者通过严谨的理论分析和大量实验数据，说明了每一种算法在实际应用中的适用范围，为非本领域的众多其他人员提供了很好的应用指导，解决了一直困扰大家的难题：怎样选择一个最适合我这个应用（问题）的串匹配技术（或算法）？

在我们决定将本书翻译出版后，2005 年底，在 SPIRE 2005 会议上，译者和本书的作者之一 Gonzalo Navarro 先生见了面。Navarro 先生很高兴我们将他的书翻译为中文，并欣然表示要为中文版写序。Navarro 先生是字符串匹配领域的一位非常活跃的专家，

在精确匹配、近似匹配、压缩匹配、音乐检索、计算生物学等方面都有论著。他还积极组织串匹配领域的各种会议，推动了南美地区本技术的发展。我们翻译本书的目的也是希望让国内更多的人士关注字符串匹配技术，希望有朝一日在国内也能有一个串匹配技术爱好者共同讨论和交流的平台，并推动相关的研究工作更上一个新的层面。

本书是由中科院计算所信息智能与信息安全中心长期从事字符串匹配研究工作的小组人员共同翻译完成的。翻译工作得到了中心领导程学旗主任和郭莉主任的支持，白硕研究员给了我们很多翻译方面的指导，研究组组长谭建龙博士也给予了很多帮助。翻译工作还得到了中心其他同仁的帮助，他们是戴磊、李真真、王小磊、杨毅夫、赵咏，在此也表示衷心的感谢。

在本书的翻译工作中，王映负责第 1 章和第 6 章的翻译，刘燕兵负责第 2 章和第 3 章的翻译，曹京负责第 4 章的翻译，刘萍负责第 5 章和第 7 章的翻译，最后的统稿由刘萍完成。

刘萍

2007 年 1 月于中科院计算所

中译本序言

在阿根廷的布宜诺斯艾利斯举行 SPIRE 2005 会议期间，来自世界另一端的两位研究者找到了我，他们是中国科学院计算所的刘萍和谭建龙。他们提议将 Mathieu Raffinot 与我合著的一本书翻译为中文，我欣然采纳了这一建议，并答应提供尽可能多的帮助。随后，Mathieu 和本书的出版社（剑桥大学出版社）也很快同意了此事。然而遗憾的是，当时我正参与主持当年的 SPIRE 会议，忙得不可开交，无暇分身和他们做太多的交流。接下来的几个月里，我对他们的进展也知之甚少。

此书中译本的重要性是不言而喻的，它为克服语言障碍提供了便利，使得更多的学生和感兴趣的研究者有机会了解我们的专著以及字符串匹配领域。对于我来说，特地选择出版此书的中译本还有更深层的涵义。在有些语言中，信息检索中的一些传统假设往往不再成立了，而中文就是这类语言的一个绝佳范例。与西方语言不同，中文字符串不能明显地分割成一些有意义并符合实际概率分布的“单词”，因此还需要一个预处理过程来进行分词，而字符串匹配技术可能派上用武之地。此外，在中文等东方语言上直接使用字符串匹配也是一种潜在的可能途径。因此，这是我们出版中译本的另一个重要初衷。

基础性研究的美妙之处在于它可以应用到极其广泛的领域，往往超出研究者的想象。研究的问题越基础，其应用范围也越广，而且更有意思的是，研究者也更难确切地说出这个东西到底有什么用。字符串匹配成为中文信息处理中的一种不可或缺的基本技术，也说明了这一点。

仅仅一年之后，我欣喜地发现此书的中译本在他们的努力下诞生了。当我浏览书中那一长串漂亮的中文字符时（尽管我并不理解它们的意思），我觉得真正体会到了字符串匹配的精华实质。很显然，在这样陌生的文本中找出某个字符串，并不像在我们熟悉的文本上那么容易。当我们处理熟悉的文本时，由于大脑受过这种语言的训练，整个过程的复杂性已经不知不觉地被掩盖了。但是当搜索陌生文本时，你不得不借助于最原始的手工计算，这时你才能真正地感受到这个问题的挑战之处，同时也体会到钻研它的乐趣所在。

我希望中国的读者除了从这本书中获取对他们的学习、研究和开发有所裨益的知识之外，也能从中感受到算法（尤其是字符串匹配算法）的快乐和美妙之处，这种快乐和美妙也一直伴随在我的左右。

Gonzalo Navarro
智利圣地亚哥 2007 年 1 月

目　录

第 1 章　导言 ····· 1

1.1　本书的目的和侧重点 ····· 1
1.2　概况 ····· 3
1.3　基本概念 ····· 7
1.3.1　位并行和位运算 ····· 7
1.3.2　带标记的有根树和 trie ····· 8
1.3.3　自动机 ····· 9
1.3.4　复杂度表示法 ····· 11

第 2 章　字符串匹配 ····· 13

2.1　基本概念 ····· 13
2.2　基于前缀搜索的方法 ····· 15
2.2.1　Knuth-Morris-Pratt 算法的思想 ····· 15
2.2.2　Shift-And/Shift-Or 算法 ····· 16
2.3　基于后缀搜索的方法 ····· 19
2.3.1　Boyer-Moore 算法的思想 ····· 19
2.3.2　Horspool 算法 ····· 21
2.4　基于子串搜索的方法 ····· 23
2.4.1　BDM 算法的思想 ····· 24
2.4.2　BNDM 算法 ····· 26
2.4.3　BOM 算法 ····· 29
2.5　实验图 ····· 34
2.6　其他算法和参考文献 ····· 35

第 3 章　多字符串匹配 ····· 36

3.1　基本概念 ····· 36
3.2　基于前缀搜索的方法 ····· 39

3.2.1 Multiple Shift-And 算法……40
3.2.2 基本的 Aho-Corasick 算法……43
3.2.3 高级的 Aho-Corasick 算法……47
3.3 基于后缀搜索的方法……48
3.3.1 Commentz-Walter 算法的思想……48
3.3.2 Set Horspool 算法……49
3.3.3 Wu-Manber 算法……52
3.4 基于子串搜索的方法……55
3.4.1 Multiple BNDM 算法……55
3.4.2 SBDM 算法的思想……60
3.4.3 SBOM 算法……61
3.5 实验图……66
3.6 其他算法和文献……68
第 4 章 扩展字符串匹配……69
4.1 基本概念……69
4.2 字符组……70
4.2.1 模式串中的字符组……70
4.2.2 文本中的字符组……72
4.3 限长空位……73
4.3.1 Shift-And 算法扩展……74
4.3.2 BNDM 算法扩展……76
4.4 可选字符……79
4.5 通配符和重复字符……81
4.5.1 Shift-And 算法扩展……83
4.5.2 BNDM 算法扩展……85
4.6 多模式串搜索……88
4.7 其他算法和参考文献……89
第 5 章 正则表达式匹配……90
5.1 基本概念……90
5.2 构造 NFA……93

5.2.1 Thompson 自动机 ····· 93
5.2.2 Glushkov 自动机 ····· 95
5.3 搜索正则表达式的经典方法 ····· 101
5.3.1 Thompson 的 NFA 模拟 ····· 101
5.3.2 使用确定自动机 ····· 101
5.3.3 混合方法 ····· 106
5.4 位并行算法 ····· 107
5.4.1 位并行 Thompson ····· 107
5.4.2 位并行 Glushkov ····· 111
5.5 过滤方法 ····· 114
5.5.1 多字符串匹配方法 ····· 115
5.5.2 Gnu 的基于必要因子的启发式方法 ····· 119
5.5.3 基于 BNDM 的方法 ····· 120
5.6 实验结果 ····· 126
5.7 其他算法与参考资料 ····· 127
5.8 构造解析树 ····· 128
第 6 章 近似匹配 ····· 132
6.1 基本概念 ····· 132
6.2 动态规划算法 ····· 133
6.2.1 编辑距离的计算 ····· 133
6.2.2 在文本中搜索 ····· 134
6.2.3 平均情况下的改进 ····· 135
6.2.4 其他基于动态规划的算法 ····· 137
6.3 基于自动机的算法 ····· 137
6.4 位并行算法 ····· 139
6.4.1 并行化 NFA ····· 139
6.4.2 并行化动态规划矩阵 ····· 145
6.5 文本快速过滤算法 ····· 150
6.5.1 k+1 分片算法 ····· 150
6.5.2 近似 BNDM 算法 ····· 154
6.5.3 其他过滤算法 ····· 158
6.6 多模式串近似搜索 ····· 158

6.6.1 仅允许一个错误的散列算法 ····· 158
6.6.2 多模式串的 k+1 分片算法 ····· 161
6.6.3 重叠自动机算法 ····· 161
6.7 扩展字符串和正则表达式的近似搜索 ····· 164
6.7.1 基于动态规划的方法 ····· 164
6.7.2 Four-Russians 方法 ····· 166
6.7.3 位并行方法 ····· 167
6.8 实验图 ····· 168
6.9 其他算法和参考文献 ····· 170
第 7 章 总结 ····· 172
7.1 软件资源 ····· 172
7.1.1 *Gnu Grep* ····· 172
7.1.2 Wu 和 Manber 的 *Agrep* ····· 173
7.1.3 Navarro 的 *Nrgrep* ····· 174
7.1.4 Mehldau 和 Myers 的 *Anrep* ····· 175
7.1.5 计算生物学方面的资料 ····· 175
7.2 其他书籍 ····· 176
7.2.1 串匹配方面 ····· 176
7.2.2 计算生物学方面 ····· 177
7.3 其他资源 ····· 178
7.3.1 期刊 ····· 178
7.3.2 会议 ····· 178
7.3.3 在线资源 ····· 179
7.4 相关主题 ····· 179
7.4.1 索引 ····· 179
7.4.2 压缩文本中的搜索 ····· 181
7.4.3 重复和循环 ····· 183
7.4.4 二维和多维的模式匹配 ····· 184
7.4.5 树模式匹配 ····· 186
7.4.6 序列比较 ····· 186
7.4.7 特异子串检测 ····· 188
参考文献 ····· 189
索引 ····· 202

第1章 导 言

1.1 本书的目的和侧重点

字符串匹配问题可以理解为从给定的符号序列中找出一个具有某种属性的模式，最简单的例子是从给定的字符序列中找出一个给定的字符串。

字符串匹配是计算机科学中最古老、研究最广泛的问题之一，并且，字符串匹配的应用也随处可见。近年来，学术界对字符串匹配的研究兴趣与日俱增，特别是在发展迅猛的信息检索领域和计算生物学领域。

之所以有上述现象，不仅因为在这两个研究领域中需要处理的文本规模越来越大，而且由于需要在文本中进行越来越复杂的搜索。在这两个领域中，令人感兴趣的模式不仅是简单的字符串，还包括通配符、空位以及正则表达式。“匹配”的定义也允许模式和文本字符串之间存在某些细微的差异，即所谓的“近似匹配”，这在文本检索领域和计算生物学领域中得到广泛应用。

字符串匹配问题可以从多个不同的角度来阐述。通常来说，既可以从非常抽象的理论研究方面来介绍，也可依据极其实用的应用成果来介绍。虽然理论研究的成果极大程度提高了某些重要算法的性能，但很少能在实际应用中取得良好效果。在字符串匹配研究领域中，一个人所共知的事实是“算法的思想越简单，实际应用的效果越好”。有两个典型的例子可以说明这一点。一是著名的 Knuth-Morris-Pratt（KMP）算法，它在实际应用中比简单的蛮力方法还要慢一倍。二是在著名的 Boyer-Moore（BM）系列算法中，应用最成功的算法是对原始算法进行高度简化后得到的算法。

然而，很难在已有的相关书籍中找到那些思想简单的算法。在关于文本处理的现有书籍中，字符串匹配部分往往只包括了一些经典的理论算法。造成这一现象的主要原因有以下三个。

首先，非常实用的算法是近几年才出现的，而经典算法往往已经存在了几十年，因此很难在现有书籍中找到这些新的成果。新的算法通常采用了一些新的技术，例如位并行，这是随着新一代计算机的产生而出现的。

第二个原因是理论研究和实际应用脱节。那些专门从事算法研究的学者关心的只是理论上看起来很美妙的算法——具有很好的时间复杂度，涉及到复杂的算法概念。而开发人员只追求实际应用中尽可能快的算法。这两者之间从不注意对方在干些什么。将理论研究和实际应用相结合的算法（例如 BNDM 算法）只是近年才出现。现在，字符串匹配算法的发展趋势是更快、更健壮，然而这些新近出现的算法在已有的书籍中还找不到。

最后一个原因是，在现有的相关书籍中，很少涉及字符串领域中那些最新、最活跃的相关主题，包含对扩展模式串的搜索、多模式串匹配，以及近似模式匹配等。

由于上述原因，在实际应用中常常很难找到适合需求的算法——这样的算法实际上是存在的，但是只有资深专家才比较了解。考虑如下情况，一位软件开发人员，或者一位计算生物学家，或者一位研究人员，又或者一位学生，对字符串匹配领域并没有深入了解，可是现在需要处理一个文本搜索问题。于是，不得不去钻研那些汗牛充栋的典籍。这些典籍往往极具理论价值，可实现起来常常太过复杂。使得阅读者淹没在各种匹配算法的海洋中，但却没有足够的背景知识选择最适用的算法。最后，常常会导致这样的局面：选择一种最简单的算法加以实现。这往往可能导致很差的性能，从而影响整个开发系统的质量。更糟糕的是，选择了一个理论上看起来很漂亮的算法，并且花费了大量精力去实现。结果，却发现实际效果和一个简单算法差不多，甚至还不如简单算法。

本书的目的就是介绍在各种模式（包括字符串、字符串集合、扩展字符串以及正则表达式等）下现存的精确匹配算法和近似匹配算法，并深入介绍几个最实用的算法。所谓“实用”，就是指算法在实际应用中性能较好，并且一个普通程序员能在几小时内完成算法的实现代码。幸好，在字符串匹配领域中，这两个准则还是一致的。注意：本书只关注在线（on-line）搜索，即不对文本建立索引数据结构。尽管索引搜索也基于在线搜索技术，但它是个非常复杂的问题，其本身就值得单独出本书加以介绍。

本书面向各种不同的读者：计算机科学家将会通过本书了解那些最快的搜索算法并能加以实现。如果他们想更进一步研究文本搜索算法，本书中提供了相关问题的详细链接和文献索引（包括书籍、会议录和论文）。计算生物学家能够通过本书对模式匹配有更深入的了解，并找到适合他们需求的最简单、最有效的序列搜索算法。

本书的作者实现了书中所有的算法，并进行了实验，其中某些算法是作者自己提出的。此外，书中尽可能给出一些实验图，以帮助读者在最短的时间内找到最适合特定应用的算法。

当然，文本搜索领域是如此之大，一本书根本不能囊括其中所有的内容。本书更倾向于将不同的主题分别作为独立的章节加以详细阐述。在 7.2 节中给出了最近出版的相关书籍的列表。

1.2 概况

第2章 字符串匹配

一个字符串是一个定义在有限字母表 Σ 上的字符序列。例如，ATCTAGAGA 是字母表 $\Sigma = \{A, C, G, T\}$ 上的一个字符串。字符串匹配问题就是在一个大的字符串 T 中搜索某个字符串 p 的所有出现位置。其中，T 称为文本，p 称为模式串，T 和 p 都定义在同一个字母表 Σ 上。给定字符串 x, y 和 z，称 x 是 xy 的一个前缀，x 是 yx 的一个后缀，x 是 yxz 的一个因子。

根据在文本中搜索模式串方式的不同，字符串匹配算法可以归结为三种基本的方法。

第一种方法是从文本中逐个读入字符，每读入一个字符就更新相应变量，检查是否存在一个可能的匹配。Knuth-Morris-Pratt 算法就属于这种方法，另外一种是更快的 Shift-Or 算法，对其进行扩展后可以支持对更复杂模式串的搜索。

第二种方法基于滑动窗口。滑动窗口沿着文本 T 滑动，对于任意位置上的窗口，在窗口中从后向前搜索窗口中的文本和模式串 p 的公共后缀。Boyer-Moore 算法就使用了这种方法。但在一般情况下，Boyer-Moore 算法比它的一个简化版本 Horspool 要慢。实际上，Boyer-Moore 系列算法中除了 Horspool，都要比使用其他方法的一些算法慢。

第三种方法出现得较晚，实际应用中如果模式串 p 足够长，其中一些算法是最有效率的。和第二种方法一样，第三种方法也使用了滑动窗口，并在其中从后向前搜索。不同的是，它搜索的是窗口中文本的最长后缀，并且这个最长后缀同时也是模式串 p 的一个因子。最早使用这种方法的算法是 BDM 算法，当 p 足够短时，可以将其改造成更简单、更有效的算法 BNDM。对于较长的模式串，一个称为 BOM 的新算法是最快的。

本章中还给出了一幅实验图。对于给定的模式串长度和字母表大小，通过这幅实验图能很容易找到速度最快的算法。

上面的三种方法给出了一个对字符串匹配算法进行分类的框架。目前，最有效的匹配算法都属于其中的某一类。另外，也存在其他一些算法，例如基于散列的算法，但通常它们的性能不是很高。关于这些算法的参考文献，在本章的最后一节给出。

第3章 多字符串匹配

多字符串匹配算法能够在只扫描一遍文本的情况下，搜索出模式串集合 $P = \{p^1, p^2,$

$\ldots, p^r\}$中所有模式串的所有出现位置。许多单字符串搜索算法都能够扩展成多字符串搜索算法，当然这些扩展算法的性能有高有低。本章概述了那些效率最高的算法，让人困惑和恼火的是，在这些算法中，很多只是以技术报告的形式给出，如果你不是一位字符串匹配方面的专家，要找到这些算法是比较困难的。

上述三种单字符串搜索方法都能扩展到多字符串的情况。基于第一种方法的算法包括著名的 Aho-Corasick 算法。如果用$|P|$表示所有模式串的长度之和，那么，当$|P|$很小的时候，可以使用 Multiple Shift-And 算法。

基于第二种方法的算法有著名的 Commentz-Walter 算法，但其实际运行的效率不是很高。Horspool 算法的扩展是 Set Horspool 算法，当搜索一个很大字母表上的一个较小的模式串集合时，其效率很高。Wu-Manber 算法结合了后缀搜索算法和散列方法，在实际应用中效率很高。

基于第三种方法的算法中，由 BOM 算法派生出了 SBOM 算法。当模式串集合中的最小模式串长度较大时，SBOM 算法的效率很高。与 Shift-Or 算法相似，当$|P|$很小时，BNDM 派生出了 Multiple BNDM 算法。

本章也给出了一幅实验图，指明了在模式串长度之和$|P|$、模式串的最短长度以及字母表大小等因素变化的情况下，怎样选择一个合适的算法。

第 4 章　扩展字符串匹配

在许多应用中，模式串并不是简单的字符序列。本章中主要讨论的内容是：在实际应用中使用的几种对模式串的扩展，以及怎样对其进行搜索匹配。所有这些扩展都能够转换为正则表达式（参见第 5 章），但是对于某些特定情况，存在更简单、更快速的算法。

最简单的扩展是，模式串中的每个位置不是一个字符而是一个字符集合。也就是说，只要文本中的对应字符属于该位置的字符集合，即为在该位置匹配成功。同样，这种字符集合也可以出现在文本中。

第二种扩展是限长空位，即可以指定模式串中的某些位置，这些位置能够与文本中的长度介于指定的最大值和最小值之间的任意字符串匹配。这种扩展对于计算生物学中的某些应用特别重要，例如搜索 PROSITE 模式。

第三种扩展是允许可选字符和可重复字符。一个可选字符表示它在文本对应的位置上可以出现也可以不出现，而可重复字符在匹配的文本中可以出现 1 次或多次。

对上面这三种扩展模式串及其组合的模式串的搜索问题都可以通过修改后的 Shift-Or 算法和 BNDM 算法来解决。这两种算法都是使用位并行技术来模拟一个非确定自动机在文本

中进行搜索，以找到所有的成功匹配（参见 1.3 节）。构造这样一个自动机比较复杂，核心问题是怎样进行自动机的模拟。扩展后的 Shift-Or 算法不能跳过文本中的某些字符，但其效率不受模式串复杂度的影响。扩展后的 BNDM 算法通常要更快一些，但其效率受制于成功匹配的最小长度、字母表的大小、字符类的大小以及空位的大小等因素。相对而言，其他经典算法都不能容易地扩展并达到如此好的效率。

本章的最后将给出对一个由较短模式串组成的规模较小的模式串集合，可以通过与上面相似的方法进行搜索匹配。此外，本章中还给出了其他一些理论算法的参考文献，它们是针对某些特定扩展模式串的搜索。

第 5 章 正则表达式匹配

正则表达式是一个表示模式串集合的有力工具，前面讨论的各种模式串都能使用正则表达式来表示。正则表达式的定义是递归的：一个正则表达式要么是一个简单的字符串，要么是子正则表达式的连接、联合或重复。搜索正则表达式的算法相当复杂，只有在模式串不能用更简单的方法来表示的时候才会使用正则表达式来表示。

通常，在文本中搜索一个正则表达式分为如下几个步骤。首先，解析正则表达式，把它表示成容易实现的等价树形结构。在第 5 章的末尾将详细讲述这个问题，之后的步骤就围绕这个树形的表示来进行。

第二步是由模式串建立一个非确定的有限自动机（NFA, nondeterministic finite automaton）。NFA 本质上是一个状态机，当读入文本字符时，活动状态将被改变。状态机的当前状态为一个终止状态时，就表示识别出了一个模式串。从正则表达式建立 NFA 有两种办法：Thompson 算法和 Glushkov 算法。由 Thompson 算法建立的 NFA 的状态转移数目和正则表达式的长度成正比，这个性质对于某些正则表达式是非常有用的。Glushkov 算法产生的 NFA 具有最少的状态数目，并具有其他一些有用的性质。

NFA 构建完成之后就可以直接进行搜索（这种算法称为 NFA-Thompson 算法），但这样做速度会很慢，因为可能会同时存在多个活动状态。也可以考虑把 NFA 转换成确定的有限自动机（DFA, deterministic finite automaton），它在任何时刻都只有一个活动状态。DFA 很适合于文本搜索，也是正则表达式搜索中所使用的最经典算法之一，称为 DFA Classical 算法。该算法的主要问题是，转换后 DFA 的大小可能是原来的 NFA 大小的指数级，因此，只是在模式串长度较短的时候才适用。对于较长的模式串，通常使用多个较小的 DFA 构造一个大的 NFA，这种结合使用 NFA 和 DFA 的方法具有较好的效率，称为 DFA Modules 算法。

另一种办法是用位并行技术来模拟 NFA，而不是将 NFA 转换为 DFA。本章中将讨论两个比较新的算法：BPThompson 算法和 BPGlushkov 算法，二者都使用其特定的性质来模拟相应的 NFA。一般情况下，BPGlushkov 算法的效率要优于 BPThompson 算法。

第三种方法比较新颖，允许跳过文本中的某些字符。MultiStringRE 算法能够计算正则表达式的成功匹配的最小长度 *lmin*，并计算出所有可能的成功匹配的前缀（长度为 *lmin*），然后利用多模式串匹配算法（参见第 3 章）在文本中搜索这些前缀。如果找到一个这样的前缀，算法再试图将前缀扩展成完整的匹配结果。从 MultiStringRE 算法可衍生出 MultiFactRE 算法，它选择所有可能的成功匹配的因子（而不是前缀），这些因子的长度也是 *lmin*。其中，某些因子必须在所有的成功匹配中出现。本章的最后将介绍 RegularBNDM 算法，它通过模拟 Glushkov NFA 来扩展 BNDM 算法。

对于正则表达式的搜索，选择一个最好的匹配算法是非常复杂的，这是因为算法的效率通常依赖于正则表达式本身的结构。本章中试图给出一些准则，可以根据正则表达式本身的性质来判断哪个算法更合适。

第 6 章　近似匹配

所谓近似匹配，就是在文本中搜索某个模式串的所有出现位置。其中，模式串和对应的文本之间可以有稍许不同。近年来，近似匹配的应用越来越广泛。例如，在信息检索中对拼写错误的纠正，在计算生物学中的序列比对，在信号处理中对传输错误的纠正，等等。

近似匹配基于距离模型，该模型中需要一个可以度量两个字符串相似度的距离函数。在近似匹配中，通常给定模式串和一个对应的阈值 k，其中，k 指定了模式串和匹配结果之间允许的最大距离。本章中主要讨论编辑距离，即将两个字符串转变成完全相同的串所需要的最小编辑操作数目（这里的编辑操作指的是插入、删除和替换）。在实际应用中，很多问题使用了编辑距离模型或者使用其变体。

本章中将已有的近似匹配算法分为四类：

第一类算法基于动态规划方法，这是最古老的算法，同时也最容易扩展到非编辑距离的模型，但不是效率最高的算法。

第二类算法基于 NFA 搜索。该算法使用给定的模式串和阈值 k 构造 NFA，然后将其确定化。当模式串较短时，这类算法的效率不错，但不如一些更新的算法。

第三类算法基于位并行技术，这是当前应用最成功的一类算法。其中，BPR 算法和 BPD 算法用位并行技术模拟 NFA，BPM 算法用位并行技术模拟动态规划算法。BPM 和 BPD 是这类算法中最有效的，不过 BPR 算法更灵活，容易扩展到更复杂的模式串形式。

最后一类是基于过滤的算法，它用一个快速的过滤算法跳过文本中那些不可能存在成功匹配的大块区域，然后再利用一个非过滤算法来验证剩下的区域是否存在成功匹配。这类算法也是最快的算法之一，但当阈值 k 和模式串长度 m 的比值增加时，其效率迅速下降。

在众多过滤算法中，本章将介绍其中效率最高的两个。PEX 算法把模式串分成 $k+1$ 块，然后利用多模式串匹配算法在文本中搜索这些模式串的块。在任意一个成功匹配结果中，至少有一个块是和文本完全相同的。另一个算法 ABNDM 是对 BNDM 的扩展，它通过模拟 NFA 来执行近似搜索。

本章给出了这些算法的实验图。一般说来，当 k/m 值较小时，过滤算法的效率更高一些；当字母表大小较小时（例如 DNA 序列），ABNDM 算法更适用；而字母表较大时（例如蛋白序列或者自然语言），PEX 算法是最好的；对于较大的 k/m 值，最好使用位并行算法，同时位并行算法适用于在过滤算法中对不能跳过的文本区域的验证。

近来，对于更复杂的模式串的近似匹配也取得了一些进展。对于多模式串的近似匹配，目前主要使用 MultiHash 算法。虽然该算法只允许最多有 1 个错误，但是当模式串集合很大时，其效率也是很好的。MultiPEX 算法也是从每个模式串中派生出 $k+1$ 个模式块，当 k/m 值较小时具有最好的效率。MultiBP 算法利用所有模式串构造多层 NFA，然后利用构造的 NFA 进行过滤。对于一个中等的 k/m 值，本算法是最好的选择。

对于扩展字符串和正则表达式的近似匹配也存在几个算法：一个是关于正则表达式的动态规划算法，还有一个是使用多个 DFA 构造 NFA 的容错算法，再就是基于 BPR 的位并行算法。基于 BPR 的位并行算法颇为吸引人，因为它不仅简单、易于实现，同时效率也很高。

1.3 基本概念

1.3.1 位并行和位运算

位并行技术利用了计算机机器字位运算的内在并行性，即可以把多个值装入同一个机器字内，然后只需一次运算就能更新所有值。利用位并行，一个算法所执行的运算次数最多可能减少到原来的 $1/w$，这里 w 是机器字的位数。当前的系统结构中，w 为 32 或者 64，因此实际应用中的加速效果是相当显著的。

这里，先介绍一些用以描述位并行算法的符号。对于位的重复出现，可以使用指数形式来表示，例如，$0^3 1 = 0001$。长度为 ℓ 的位掩码使用位序列 $b_\ell \ldots b_1$ 来表示，位掩码存于

长度为 w 的机器字中。机器字的位运算可以使用类似于 C 语言中的句法来表示，即“|”表示位或运算，“&”表示位与运算，“^”表示异或运算，“~”表示对所有位取反，“<<”（“>>”）表示左（右）移位运算并以零填充右（左）边移入的位，例如，$b_\ell b_{\ell-1}\ldots b_2 b_1 << 3 = b_{\ell-3}\ldots b_2 b_1 000$。

在位上可以进行算术运算，例如加法或减法。在进行算术运算时，把位掩码视为数值。例如，$00010110 + 00010010 = 00101000$，$10010000 - 1 = 10001111$。

有些情况下可能需要使用多个机器字来存储一个给定的值集合，这时，上述位运算必须在多个机器字上整体进行。这对于大多数位运算来说是很直白的，但对于算术运算来说就必须小心了，因为必须考虑进位的影响。例如，假设需要用多个机器字来模拟 $Z \leftarrow X+Y$ 或者 $Z \leftarrow X-Y$，其中 $X = X_t\ldots X_1$，$Y = Y_t\ldots Y_1$，都是使用 t 个机器字来表示的。图 1.1 给出了这两项运算的算法。

```
Add(X = X_t ... X_1, Y = Y_t ... Y_1)
1.    carry ← 0
2.    For i ∈ 1...t Do
3.        Z_i ← X_i + Y_i + carry
4.        If Z_i < X_i OR (Y_i = 1^w AND carry = 1) Then carry ← 1
5.        Else carry ← 0
6.    End of for
7.    Return Z = Z_t ... Z_1

Subtract(X = X_t ... X_1, Y = Y_t ... Y_1)
8.    carry ← 0
9.    For i ∈ 1...t Do
10.       Z_i ← X_i - Y_i - carry
11.       If Z_i > X_i OR (Y_i = 1^w AND carry = 1) Then carry ← 1
12.       Else carry ← 0
13.   End of for
14.   Return Z = Z_t ... Z_1
```

图 1.1 存于多个机器字中的无符号数的加法和减法运算算法。其中第一个机器字 Z_1 是最低位。此处忽略了运算中最后可能产生的溢出，但实际上溢出信息保存在变量 carry 中

1.3.2 带标记的有根树和 trie

本书中大部分的数据结构都基于经典的字符串和有根树。一棵有根树是一个由单向边连接的节点的集合。每条单向边的源节点称为父节点，目标节点称为子节点。有一个特殊的节点没有父节点，这个节点称为根节点。树中除根节点之外的每个节点，都有且仅有一个父节点。没有子节点的节点称为叶节点。

出于一些特殊的需要，往往对树中的每条边附加一个标记，通常是字母表 Σ 中的一个字符。图 1.2 给出了一个带有标记的树的例子。

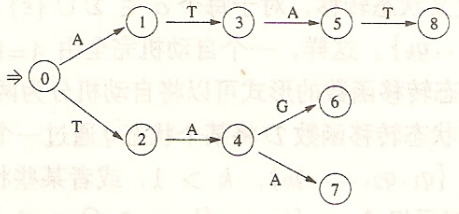


图 1.2 一棵带标记的树。状态 0 为树根，除根以外的节点有且仅有一个父节点

把带标记的有根树和一个字符串的集合关联起来时，就称这棵树为 trie。第 3 章将会比较全面地介绍 trie 结构。一个使用带标记的有根树的算法常常需要以某种特定的顺序在树中的节点上执行计算。以前缀序计算时，该算法首先在当前节点上计算，然后在其子节点上计算（如果存在子节点）。以后缀序计算时，首先在当前节点的所有子节点上计算，然后才在当前节点上计算。例如，在图 1.2 所示的树中，以前缀序计算的顺序是 0，1，3，5，8，2，4，6，7，以后缀序计算的顺序是 8，5，3，1，7，6，4，2，0。这里不关心兄弟节点之间的处理顺序。

另一种常用的访问顺序是层次遍历序。一个节点所处的层次就是它到根节点的距离，也就是从它到根节点所经历的中间节点的个数。层次遍历序就是以层次渐增的顺序处理节点。在每一层内，节点的处理顺序通常无关紧要，但有时也会在层内强加一个顺序以简化算法。图 1.3 给出了一棵树的两个层次遍历序，其中的虚线箭头所指为节点的处理顺序。

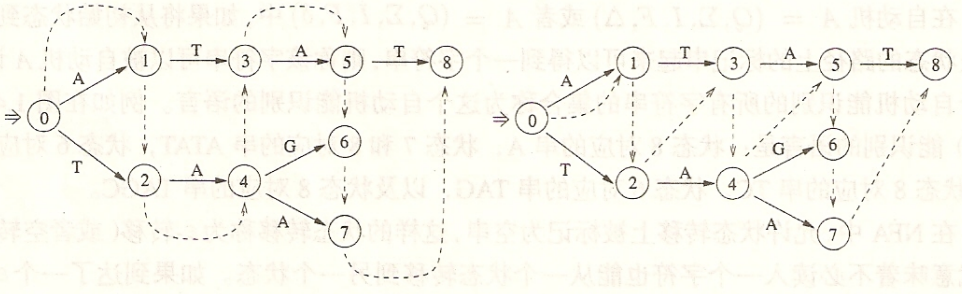


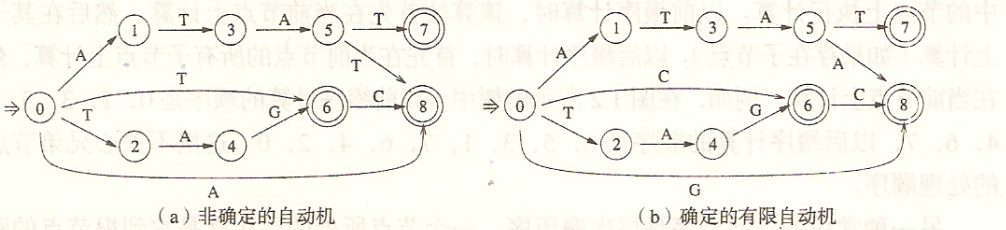
图 1.3 树的遍历。其中节点 0 为根节点，遍历的顺序如虚线箭头所指

1.3.3 自动机

在计算机科学中，自动机这个词具有多重含义。对字符串匹配领域来说，一个有限自

动机（简称为自动机）是一个有限的状态集合Q，包括：1 个初始状态（状态$I \in Q$），若干个终止状态（状态集合$F \subseteq Q$）。状态之间的转移用$\Sigma \cup \{\varepsilon\}$中的元素做标记。通常用状态转移函数$\mathcal{D}$来形式地定义状态转移，对于每个$\alpha \in \Sigma \cup \{\varepsilon\}$，$\mathcal{D}$将状态$q \in Q$关联到一个状态的集合$\{q_1, q_2, \ldots, q_k\}$。这样，一个自动机完全由$A=(Q, \Sigma, I, F, \mathcal{D})$所定义。

实际应用中，根据状态转移函数的形式可以将自动机分为两大类：一类称为非确定的有限自动机（NFA），它的状态转移函数$\mathcal{D}$将某个状态q通过一个给定字符α关联到多于 1 个的状态，即$\mathcal{D}(q, \alpha) = \{q_1, q_2, \ldots, q_k\}$，$k > 1$，或者某些状态转移能被标记为$\varepsilon$。此时，状态转移函数$\mathcal{D}$可用三元组$\Delta = \{(q, \alpha, q'),\ q \in Q, \alpha \in \Sigma \cup \{\varepsilon\}, q' \in \mathcal{D}(q, \alpha)\}$的集合来表示。另外一类称为确定的有限自动机（DFA），它的状态转移函数$\mathcal{D}$可用一个部分函数$\delta: Q \times \Sigma \to Q$来表示，如果$\mathcal{D}(q, \alpha) = \{q'\}$，那么$\delta(q, \alpha) = q'$。图 1.4 给出了这两类自动机的例子。



（a）非确定的自动机　　（b）确定的有限自动机

图 1.4　两类自动机。其中状态 0 表示初始状态，双圈状态表示终止状态。左边的自动机是非确定的，因为从状态 0 通过字符T可以到达状态 2 和状态 6。右边的是确定的，因为对于任何一个字符，每个状态都只能转移到最多一个状态

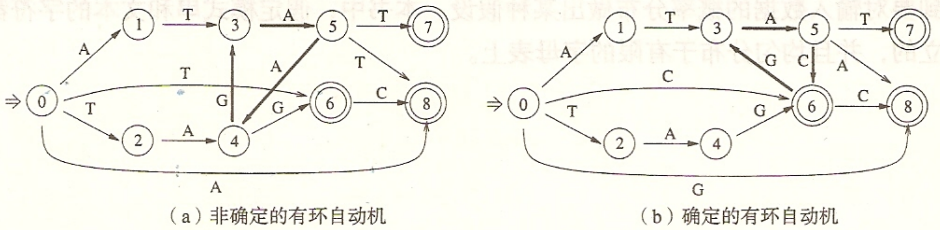
在自动机$A = (Q, \Sigma, I, F, \Delta)$或者$A = (Q, \Sigma, I, F, \delta)$中，如果将从初始状态到一个终止状态的路径上的标记串起来可以得到一个字符串，则称该字符串可以被自动机A识别。一个自动机能识别的所有字符串的集合称为这个自动机能识别的语言。例如在图 1.4 中，（a）能识别的语言是：状态 8 对应的串 A，状态 7 和 8 对应的串 ATAT，状态 6 对应的串 T，状态 8 对应的串 TC，状态 6 对应的串 TAG，以及状态 8 对应的串 TAGC。

在 NFA 中，允许状态转移上被标记为空串，这样的状态转移称为ε-转移（或者空转移）。这就意味着不必读入一个字符也能从一个状态转移到另一个状态。如果到达了一个ε转移的源状态，那么不必读入一个字符就能立即跳转到其目标状态，这也可看做是读入了一个空串。ε转移通常用来简化 NFA 的构造，但总是存在一个与之等价的不含ε转移的自动机。

在 NFA 或者 DFA 中，如果将从初始状态I到达某个状态s的路径上的标记串起来可以得到字符串x，那么就说：读入x后状态s是活动的。在任何时候，DFA 最多只有一个活动

状态，而 NFA 可能存在多个活动状态。

图 1.4 中所示的两个自动机中的状态转移不会构成环，因此是比较简单的形式，这样的自动机不论是确定的还是非确定的，都称为无环的。实际上构造一个有环的自动机很容易，它在正则表达式匹配中很有用。图 1.5 中的两个自动机就是有环的。一个有环自动机识别的语言可能是无穷的。例如，图 1.5(a)识别出 TAG，也识别出 TA·GAA·G，TA·GAA·GAA·G，TA·GAA·GAA·GAA·G，等等。



（a）非确定的有环自动机　　（b）确定的有环自动机

图 1.5　两个有环自动机。其中状态 0 为初始状态，双圈状态为终止状态。自动机中的环用粗线表示

1.3.4　复杂度表示法

字符串匹配算法的效率通常使用字符比较的次数以及其他基本操作的次数来描述，算法的效率取决于模式串的大小 m 和文本的大小 n。在算法分析中，通常不会给出一个关于 m 和 n 的精确函数，而只是给出其增长率或者复杂度阶。符号 O 用来表示这种复杂度的度量。

定义　如果存在两个常量 C 和 n_0，使得对于任意的 $n > n_0$，都有 $g(n) \leqslant C \times f(n)$，则有 $g(n) = O(f(n))$。

例如，$2n^2 + 3n + \log\log n = O(n^2)$，$n\log n + 120n + \log n = O(n\log n)$，$n^4 + 3n^3 + 15n^2 + n = O(n^4)$。关于复杂度符号及其含义更深入的介绍请参考[Sed88, CLR90]。

使用这种表示法可以比较具有不同复杂度的算法。例如，如果算法 A 耗时 $O(n)$，算法 B 耗时 $O(n\log(m)/m)$，那么就可以推出：只要 m 足够大，算法 B 的速度就会比算法 A 的速度快。这里并不知道“足够大”到底有多大，而且如果两个算法具有相同的复杂度，那么就不知道哪个算法更好。

有时，也可以使用其他的方法对算法做更细致的分析。比如：比较每个算法进行文本检查、表访问、寄存器访问等操作的精确次数。但这些精确值不仅难以获得，而且无法保

证在一台给定的计算机上能够预测哪个算法更好。这是因为在不同的体系结构上进行存储访问的代价可能不同，而且缓存和流水线的效果也会使得预测更为复杂。

符号 O 是与计算机的体系结构无关的，但其预测能力也很有限。很多情况下，必须借助于一些经验测量结果来决定在不同的实例下哪个算法更好。

在进行算法分析时，通常会用到两种复杂度：最坏复杂度是指所有可能输入的代价最大值；平均复杂度（或称为期望复杂度）是指所有可能输入的代价平均值（期望值）。这就涉及到要对输入数据的概率分布做出某种假设。本书中，假定模式串和文本的字符都是互相独立的，并且均匀分布于有限的字母表上。

第 2 章　字符串匹配

2.1　基本概念

所谓字符串匹配（简称为“串匹配”）问题，就是在一个大文本 $T = t_1 t_2 \ldots t_n$ 中找出某个特定模式串 $p = p_1 p_2 \cdots p_m$ 的所有出现位置，其中 T 和 p 都是在有限字母表 Σ 上的字符序列。设 x，y 和 z 为字符串，则称 x 为 xy 的前缀，称 x 为 yx 的后缀，称 x 为 yxz 的因子或子串。

有很多算法可以解决串匹配问题，其中最古老和最著名的是 Knuth-Morris-Pratt 算法和 Boyer-Moore 算法，它们都出现于 1977 年。前者的最坏时间复杂度为 $O(n)$（n 为文本的长度），这也是所有串匹配算法的最坏时间复杂度的下界。后者的最坏时间复杂度为 $O(mn)$，但是因为可以跳过文本的部分字符，所以其平均时间复杂度是亚线性的。在文献[Yao79]中已经证明：串匹配算法的平均时间复杂度的下界是 $O(n \log_{|\Sigma|} m/m)$。

从 1977 年至今，人们对串匹配问题进行了大量研究，以寻找更简单的算法，或者寻找平均时间复杂度最优的算法，或者寻找能够搜索扩展模式串的算法，或者寻找常量空间的算法，等等。串匹配有很多研究方向，并由此衍生出了众多不同的串匹配算法。

本章的目的不是向读者罗列尽可能多的串匹配算法，而是选择其中最有效的算法加以介绍。所谓“最有效的算法”是指：在特定字母表大小和模式串长度下能够产生最好实验结果的算法。在那些性能不相伯仲的算法中，本章将选择其中最简单的加以介绍。

根据在文本中搜索方式的不同，本章介绍的算法可以归结为三种基本的搜索方法。在所有这些算法中，都有一个搜索窗口，其长度与模式串长度相等。搜索窗口沿着文本从左向右滑动，搜索模式串的过程在窗口内进行。这些算法的主要区别在于搜索窗口的移动方式不同。图 2.1 就是基本的搜索模式，在后面的算法介绍中，都将使用这个英文示例。

一般来说，在自然语言文本上搜索字符串比在 DNA 序列上搜索字符串要简单，因为前者包含的重复字符串很少。为了展示算法可能出现的各种微妙情况，本章中也将演示在序列 AGATACGATATATAC 中搜索 ATATA 的例子。

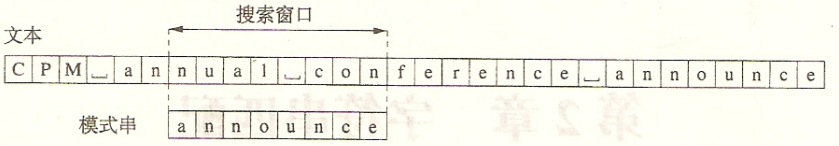


图 2.1 搜索在窗口内进行，窗口的大小与模式串的长度相等，窗口沿着文本向右滑动

下面介绍三种基本的搜索方法。

前缀搜索（图 2.2）在搜索窗口内从前向后（沿着文本的正向）逐个读入文本字符，搜索窗口中文本和模式串的最长公共前缀。Knuth-Morris-Pratt 算法使用的就是这种方法。

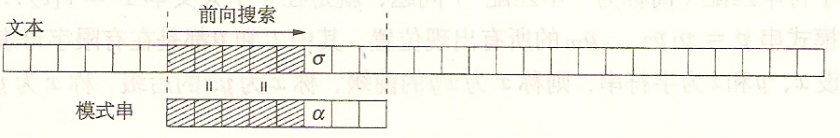


图 2.2 第一种方法：在当前窗口中搜索模式串的前缀

后缀搜索（图 2.3）在搜索窗口内从后向前（沿着文本的反向）逐个读入文本字符，搜索窗口中文本和模式串的最长公共后缀。使用这种搜索方法的算法可以跳过一些文本字符，从而具有亚线性的平均时间复杂度。其中最著名的算法是 Boyer-Moore 算法。Horspool 算法和 Sunday 算法是对 Boyer-Moore 的简化。

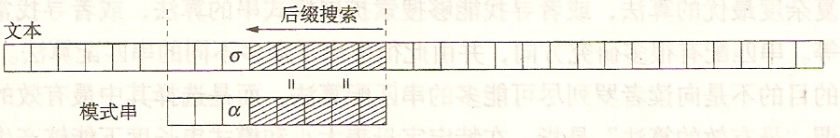


图 2.3 第二种方法：在当前窗口中搜索模式串的后缀

子串搜索（图 2.4）在搜索窗口内从后向前逐个读入文本字符，搜索满足如下条件的最长字符串 u：u 既是窗口中文本的后缀，也是模式串的子串。与后缀搜索方法一样，使用这种搜索方法的算法也具有亚线性的平均时间复杂度，甚至能达到最优的时间复杂度。这种方法的主要缺点在于需要识别模式串的所有子串，这是一个非常复杂的问题。

许多非常有效的算法都基于上述三种基本搜索方法，算法的适用范围取决于模式串长度和字母表大小，在 2.5 节中给出了相关的实验图。

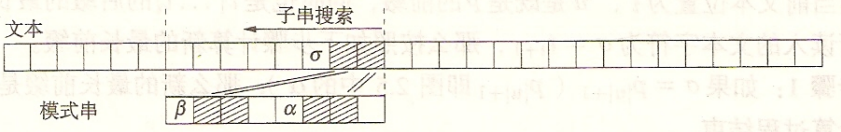


图 2.4　第三种方法：在当前窗口中搜索模式串的子串

2.2　基于前缀搜索的方法

假设已经读入直到文本位置 i 的所有字符，并且已知既是已读入文本的后缀也是模式串 p 的前缀的最长字符串长度，当该长度等于$|p|$ 时，就产生了一个成功匹配。算法要解决的主要问题是，当读入下一个文本字符时，如何快速地计算该长度。有两种经典的方法来解决这个问题：

- 第一种方法是找到一种有效的机制来计算既是已读入文本的后缀也是模式串 p 的前缀的最长字符串，并且对于每个字符而言，最好是常数的均摊时间。这就是 Knuth，Morris 和 Pratt 提出的 KMP 算法的核心思想[KMP77]。
- 第二种方法是维护一个集合，它包含了所有既是已读入文本的后缀也是模式串 p 的前缀的字符串，并且每读入一个文本字符时就更新该集合。当模式串的长度足够短时，位并行方法可以有效地维护这个集合。Shift-And 算法和 Shift-Or 算法[WM92b, BYG89b]使用了此方法。

本章中将不再给出 Knuth-Morris-Pratt 算法的伪代码，也不进行更深入的研究。这是因为在实际应用中，只有当模式串长度小于 8 时，Knuth-Morris-Pratt 才比基于后缀和基于子串的搜索方法有效。而在这个范围内，Shift-And 算法和 Shift-Or 算法能够在所有的机器上运行，速度至少是 Knuth-Morris-Pratt 的两倍，并且更易于实现。

2.2.1　Knuth-Morris-Pratt 算法的思想

每读入一个文本字符，Knuth-Morris-Pratt（KMP）算法就更新既是模式串的前缀、同时也是已读入文本的后缀的最长字符串的长度。这种机制基于如下的发现：图 2.5 中所示为对图 2.2 中的基本前缀匹配方法的扩充，其中设字符串 $v\beta$ 是下一个既是模式串 p 的前缀，同时也是文本 $t_1 \ldots t_{i+1}$ 的后缀的最长字符串。可以看出：这时 v 既是 u 的一个后缀，也是 u 的一个前缀，并且字符 β 一定与 t_{i+1}（图中的 σ）相等。这里，称 v 是 u 的一个边界。

该算法最原始的思想源于 Morris 和 Pratt 的论文[MP70]，具体如下：

- 对于模式串的每个前缀 u，预先计算 u 的最长边界 $b(u)$。

- 设当前文本位置为 i， u 是既是 p 的前缀，同时也是 $t_1 \ldots t_i$ 的后缀的最长字符串。新读入的文本字符为 $\sigma = t_{i+1}$，那么按照如下步骤计算新的最长前缀。

  步骤 1：如果 $\sigma = p_{|u|+1}$（ $p_{|u|+1}$ 即图 2.5 中的 α ），那么新的最长前缀是 $up_{|u|+1}$，计算过程结束。

  步骤 2：如果 $u = \varepsilon$，那么新的最长前缀是空字符串 ε，计算过程结束；否则，置 $u \leftarrow b(u)$，转到步骤 1 执行。

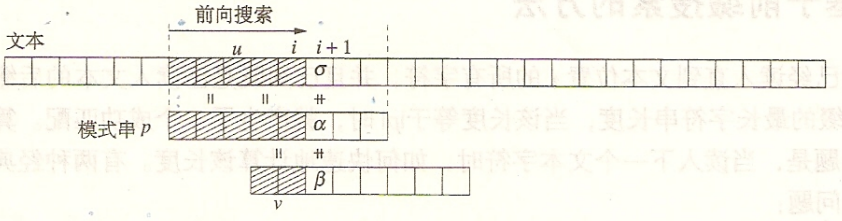


图 2.5 KMP 算法的窗口移动机制。v 既是 u 的一个后缀，也是 u 的一个前缀。由于字符 α 与 σ 不相等，因而前缀搜索无法继续识别。字符 β 与 α 也不相等

Knuth 提出了一个改进。事实上，如果 $\sigma \neq p_{|u|+1}$，则 u 的任何边界的后继字符若要与 $\sigma = t_{i+1}$ 匹配，就一定不能等于 $p_{|u|+1}$。基于这个事实，在预处理阶段，对于 p 的每个真前缀 p $(p = uw,\ w \neq \varepsilon)$，可以预先计算其最长边界 v，使得 $p_{|u|+1} \neq p_{|v|+1}$。

KMP 算法在搜索阶段的最坏时间复杂度和平均时间复杂度都是 $O(n)$。在预处理阶段，需要进行两方面的计算：第一，对于模式串的每个真前缀 u，计算 u 的最长边界 v，使得 $p_{|u|+1} \neq p_{|v|+1}$；第二，对于模式串本身，计算其最长边界。当逐个读入模式串 $p = p_1 \cdots p_m$ 的字符，并计算 $p_1 \cdots p_{i+1}$ 的最长边界时，需要计算既是 $p_1 \cdots p_{i+1}$ 的后缀，同时也是 p 的前缀的最长字符串。因此，可以用 KMP 算法来搜索 p 本身。KMP 的预处理阶段可以由 KMP 完成，时间复杂度为 $O(m)$。

这里不再对 KMP 做更多的解释，详细说明请参考[KMP77, CR94]。对 KMP 的研究以及由此产生的变体很多，在 2.6 节中给出了最主要的参考文献。

2.2.2 Shift-And/Shift-Or 算法

Shift-And 算法和 Shift-Or 算法的思想要比 KMP 简单得多，它维护一个字符串的集合，集合中的每个字符串既是模式串 p 的前缀，同时也是已读入文本的后缀。每读入一个新的文本字符，该算法即用位并行的方法更新该集合。该集合用一个位掩码 $D = d_m \ldots d_1$ 来表示。

因为 Shift-And 算法比 Shift-Or 算法更易于理解，在这里先介绍它。

D 的第 j 位被置为 1（称 D 的第 j 位是活动的）当且仅当 $p_1 \cdots p_j$ 是 $t_1 \ldots t_i$ 的后缀。

如果 p 的长度不超过机器字长 w，那么 D 可以存入一个机器字中。当 d_m 是活动的时候，就表示有一个成功匹配。

当读入下一个字符 t_{i+1} 时，需要计算新的位掩码 D'。D' 的第 $j+1$ 位是活动的当且仅当 D 的第 j 位是活动的（即 $p_1 \ldots p_j$ 是 $t_1 \ldots t_i$ 的后缀）并且 t_{i+1} 与 p_{j+1} 相等。利用位并行，D' 的计算很容易在常数时间内完成。

Shift-And 算法首先构造一个表 B，记录字母表中每个字符的位掩码 $b_m \ldots b_1$。如果 $p_j = c$，掩码 $B[c]$ 的第 j 位被置为 1，否则为 0。

首先置 $D = 0^m$，对于每个新读入的文本字符 t_{i+1}，可以用如下公式对 D 进行更新：

$$D' \leftarrow ((D << 1) \mid 0^{m-1}1) \ \& \ B[t_{i+1}] \tag{2.1}$$

直观上来讲，左移位操作 << 将 D' 的第 $i+1$ 位的值置为 D 的第 i 位。因为空字符串 ε 也是文本的后缀，所以 $D << 1$ 需要在最低位与 $0^{m-1}1$ 进行位或操作。因为要找到那些满足 $t_{i+1} = p_{j+1}$ 的位置，所以需要再将上面的结果与 $B[t_{i+1}]$ 进行位与操作。

当模式串的长度不超过几个机器字长时，公式（2.1）中的操作能够在常数时间内完成，这时 Shift-And 算法的时间复杂度为 $O(n)$。

Shift-Or 算法是 Shift-And（参见图 2.6）的一种更富有技巧性的实现。其主要思想是通过对位取反以去掉公式（2.1）中的掩码 $0^{m-1}1$，从而加速 D' 的计算。该算法使用反码来表示 B 中的位掩码和位向量 D，因为左移位操作 "<<" 会自动在 D' 右端引入 0，于是空字符串就被纳入 D' 中。

Shift-And ($p = p_1 p_2 \ldots p_m$, $T = t_1 t_2 \ldots t_n$)

```
1.  Preprocessing
2.      For c ∈ Σ Do B[c] ← 0^m
3.      For j ∈ 1...m Do B[p_j] ← B[p_j] | 0^{m-j}10^{j-1}
4.  Searching
5.      D ← 0^m
6.      For pos ∈ 1...n Do
7.          D ← ((D << 1) | 0^{m-1}1) & B[t_pos]
8.          If D & 10^{m-1} ≠ 0^m Then report an occurrence at pos − m + 1
9.      End of for
```

图 2.6　Shift-And 算法

Shift-And 算法和 Shift-Or 算法可以看做是使用一个非确定性自动机对扫描文本的过程进行模拟（参见图 2.7）。公式（2.1）对应于读入每个新字符时自动机的状态转移：当文本

字符与相应的箭头匹配时，当前状态就转移到下一个状态。

公式中的“$|\ 0^{m-1}1$”允许匹配过程从当前文本位置起始，这由自动机初始状态上的回边来进行模拟。

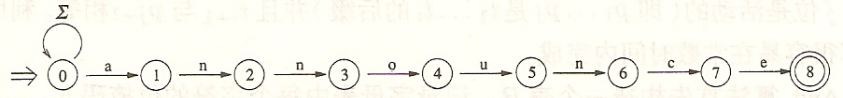


图 2.7 识别模式串“announce”所有前缀的非确定性自动机

自动机的观点对 KMP 算法同样适用，可以看做是用一种经济的方法构建确定性的扫描自动机。KMP 算法和 Shift-Or 算法之间的区别在于：前者使用确定性自动机进行模拟，而后者使用位并行方法进行模拟。然而在实际应用中，Shift-Or 算法的速度是 KMP 的两倍，并且其实现更简单，也更易于处理扩展字符串（参见第 4 章）。

英文例子 在文本“annual_announce”中搜索字符串“announce”。

$B =$

a	00000001
c	01000000
e	10000000
n	00100110
o	00001000
u	00010000
*	00000000

$D = 00000000$

1. 读入 a
   00000001
   00000001
   $D =$ 00000001

2. 读入 n
   00000011
   00100110
   $D =$ 00000010

3. 读入 n
   00000101
   00100110
   $D =$ 00000100

4. 读入 u
   00001001
   00010000
   $D =$ 00000000

5. 读入 a
   00000001
   00000001
   $D =$ 00000001

6. 读入 l
   00000011
   00000000
   $D =$ 00000000

7. 读入_
   00000001
   00000000
   $D =$ 00000000

8. 读入 a
   00000001
   00000001
   $D =$ 00000001

9. 读入 n
   00000011
   00100110
   $D =$ 00000010

10. 读入 n
    00000101
    00100110
    $D =$ 00000100

11. 读入 o
    00001001
    10001000
    $D =$ 00001000

12. 读入 u
    00010001
    00010000
    $D =$ 00010000

13. 读入 n
    00100001
    00100110
    $D =$ 00100000

14.
	01000001
读入 a	01000000
$D=$	01000000

15.
	10000001
读入 e	10000000
$D=$	10000000

D 的最高位已经被置为 1，于是报告一个匹配。

DNA 例子　在序列 AGATACGATATATAC 中搜索字符串 ATATA。

$B=$

A	10101
T	01010
*	00000

$D=00000$

1.
	00001
读入 A	10101
$D=$	00001

2.
	00011
读入 G	00000
$D=$	00000

3.
	00001
读入 A	10101
$D=$	00001

4.
	00011
读入 T	01010
$D=$	00010

5.
	00101
读入 A	10101
$D=$	00101

6.
	01011
读入 C	00000
$D=$	00000

7.
	00001
读入 G	00000
$D=$	00000

8.
	00001
读入 A	10101
$D=$	00001

9.
	00011
读入 T	01010
$D=$	00010

10.
	00101
读入 A	10101
$D=$	00101

11.
	01011
读入 T	01010
$D=$	01010

12.
	10101
读入 A	10101
$D=$	10101

D 的最高位已经被置为 1，于是报告一个匹配。

13.
	01011
读入 T	01010
$D=$	01010

14.
	10101
读入 A	10101
$D=$	10101

D 的最高位已经被置为 1，于是报告一个匹配。

15.
	01011
读入 C	00000
$D=$	00000

2.3　基于后缀搜索的方法

基于后缀的方法的主要难点在于如何安全地移动窗口，以避免遗漏可能的成功匹配。

本节首先介绍 Boyer-Moore 算法（以下简称 BM 算法）[BM77]，然后介绍对 BM 进行简化的 Horspool 算法[Hor80]。这里没有给出 BM 的伪代码，也不进行更深入的研究。因为尽管在理论上 BM 算法比其他两种基本方法（基于前缀的方法和基于子串的方法）要好，但在实际应用中，它并不是匹配速度最快的算法。

2.3.1　Boyer-Moore 算法的思想

Boyer-Moore 算法预先计算三个 shift 函数 d_1, d_2 和 d_3，它们分别对应于以下的三种情况。这三种情况都假设已经读入了一个既是搜索窗口中文本的后缀，同时也是模式串后缀

的字符串 u，并且读入的下一个文本字符 σ 与模式串的下一个字符 α 不相等（参见图 2.3）。

第一种情况下，后缀 u 在模式串 p 中的另一个位置出现。设最右出现位置（不包括在模式串末尾的出现）为 j（$u = p_{j-|u|+1} \cdots p_j$）。这种情况下安全的窗口移动方法是，将窗口移动 $m - j$ 字符，使得文本中的 u 与模式串中下一个 u 的出现位置相对齐，如图 2.8 所示。这时，对于模式串的每一个后缀，需要计算它到它的下一个出现之间的距离，这个距离用 shift 函数 d_1 来表示。如果 p 的后缀 u 不在 p 中重复出现，那么 $d_1(u)$ 被置为整个模式串的长度 m。

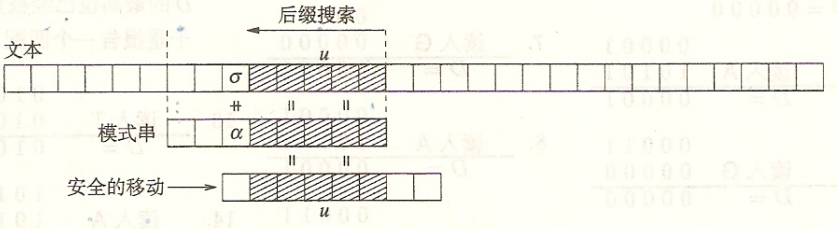


图 2.8 Boyer-Moore 的 shift 函数 d_1。模式串移动到 u 的下一个出现位置

第二种情况下，后缀 u 不出现在 p 中的任何其他位置。这时并不意味着可以安全地移动整个窗口大小的距离，因为可能会出现图 2.9 所示的情况，即 u 的后缀 v 也可能是模式串 p 的一个前缀。为了处理这种情况，需要对模式串的所有后缀计算第二个函数 d_2。对于 p 的每个后缀 u，$d_2(u)$ 表示既是 p 的前缀，同时也是 u 的后缀的最长字符串 v 的长度。

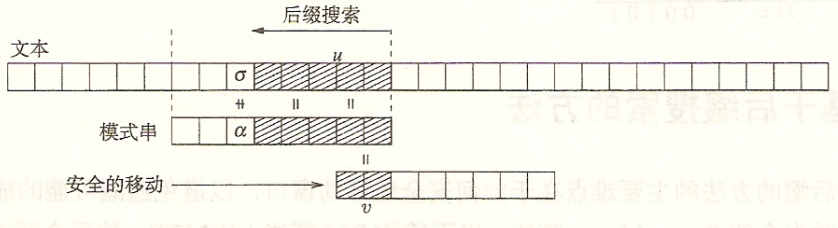


图 2.9 Boyer-Moore 的 shift 函数 d_2。u 不在 p 中重复出现。模式串移动到 p 与 u 的右端对齐的位置

第三种情况是在搜索窗口中从后向前搜索时，在文本字符 σ 处不能成功匹配。这时，如果用函数 d_1 进行窗口移动，并且对应的模式串字符不是 σ，那么将会对新的搜索窗口进行一次不必要的验证，如图 2.10 所示。shift 函数 d_3 就是用来保证下一次验证时文本字符 σ 一定与模式串中的一个字符 σ 相对应。对于字母表中的每个字符 σ，$d_3(\sigma)$ 表示 σ 在模式串中的最右出现位置到模式串末尾的距离。如果 σ 不出现在 p 中，$d_3(\sigma)$ 被设置为 m。

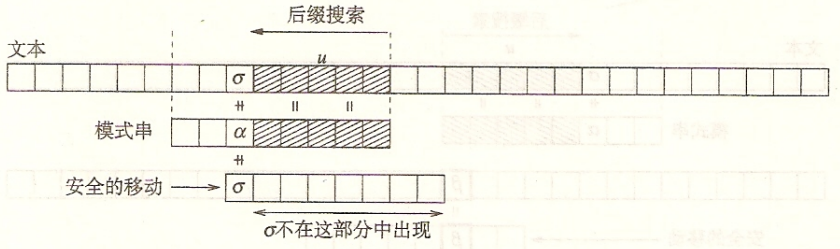


图 2.10　Boyer-Moore 的 shift 函数 d_3。模式串移动到 p 与 σ 的下一次出现位置相对齐的地方

当读入了文本字符串 u 并在字符 σ 上不匹配时，Boyer-Moore 算法对这三种 shift 函数进行两次比较，以决定窗口的移动距离。

- 第一次比较：取 $d_1(u)$ 和 $d_3(\sigma)$ 中的较大者，因为希望将 u 同它在模式串中的下一个出现对齐，并且 σ 与模式串中的下一个 σ 对齐。
- 第二次比较：取上面的比较结果和 $m-d_2(u)$ 中的较小者，因为后者是最大的安全移动距离。

经过上面两次比较得到的就是 BM 的移动距离。

然而，如果抵达了窗口的起始位置，说明已经发现了一个成功匹配，这时只使用函数 d_2 来计算窗口的移动距离。

BM 的搜索阶段的最坏时间复杂度为 $O(mn)$，而其平均时间复杂度是亚线性的。BM 算法有很多变体可以保证线性的最坏时间复杂度，最重要的参考文献将在 2.6 节中给出。

BM 的最大不便之处是要计算三个函数 d_1, d_2 和 d_3，虽然它们可以在 $O(m)$ 的时间内完成，但是很复杂[Ryt80]。下面将介绍 BM 算法的一种简化算法，在很多场合都比 BM 更有效。

2.3.2　Horspool 算法

Horspool[Hor80]是首个对 BM 算法进行简化的算法，它认为：对于较大的字母表来说，shift 函数 d_3 总会产生最大的跳跃距离。Horspool 算法对 d_3 进行了一些小的修改，使其易于计算并能产生更大的跳跃距离。该算法的工作原理如图 2.11 所示。

对于每个搜索窗口，该算法将窗口内文本的最后一个字符（图中的 β）和模式串的最后一个字符进行比较。如果相等，则需要一个验证过程。该验证过程在搜索窗口中从后向前对文本和模式串进行比较，直到完全相等或者在某个字符处不匹配（图中的 σ）。然后，无论匹配与否，都将根据 β 在模式串中的下一个出现位置将窗口向右移动。Horspool 算法的伪代码如图 2.12 所示。

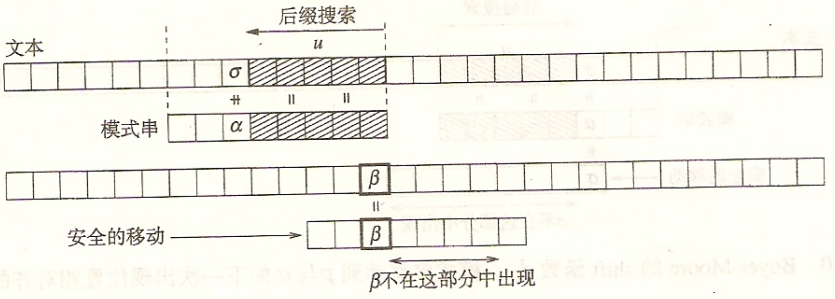


图 2.11 Horspool 算法。模式串根据搜索窗口中的最后一个字符进行移动

```
Horspool (p = p1p2...pm, T = t1t2...tn)
1.    Preprocessing
2.        For c ∈ Σ Do d[c] ← m
3.        For j ∈ 1...m − 1 Do d[pj] ← m − j
4.    Searching
5.        pos ← 0
6.        While pos ≤ n − m Do
7.            j ← m
8.            While j > 0 AND t_{pos+j} = pj Do j ← j − 1
9.            If j = 0 Then report an occurrence at pos + 1
10.           pos ← pos + d[t_{pos+m}]
11.       End of while
```

图 2.12 Horspool 算法

需要注意的是：

- 验证过程也可以从前向后进行。此外，很多具体实现都使用了机器内置的比较指令。
- 主循环过程可以被“展开”，即：首先可以不断地移动搜索窗口，直到窗口的最后一个字符和模式串的最后一个字符相等，然后再进行验证。

Sunday 的变体——Sunday 算法与 Horspool 算法不同，Sunday 算法利用窗口后的一个字符进行移动，这能带来更大的平均移动距离。该算法由 Sunday 提出[Sun90]。尽管它的移动距离要长一些，但是“展开”的 Horspool 具有更少的内存引用次数，因而一般来说比 Sunday 算法要快。

Horspool 算法在英文上的例子 在文本“CPM_annual_conference_announce”中搜索字符串“announce”。

$m=8$, $d=$

a	c	n	o	u	*
7	1	2	4	3	8

1. [CPM_annu] al_conference_announce
   u ≠ e, d[u] = 3

2. CPM [_annual_] conference_announce
   _ ≠ e, d[_] = 8

3. CPM_annual_ [conferen] ce_announce
   n ≠ e, d[n] = 2

4. CPM_annual_ [conferen] _announce
   文本窗口[nferenc[e]]中的最后一个字符与模式串的最后一个字符相等，从后向前验证，[nferen[ce]]，[nfere[nce]]，文本字符“e”和模式串字符“u”不匹配，于是根据文本窗口的最后一个字符“e”进行移动，移动距离为：d[e] = 8。

5. CPM_annual_conference [_announc] e
   c ≠ e, d[c] = 1

6. CPM_annual_conference_ [announce]
   文本窗口[announc[e]]中的最后一个字符与模式串的最后一个字符相等，从后向前验证，并发现了一个匹配。

Horspool 算法在 DNA 上的例子　在序列 AGATACGATATATAC 中搜索字符串 ATATA。

$m=5$, $d=$

A	T	*
2	1	5

1. [AGATA] CGATATATAC
   文本窗口[AGAT[A]]中的最后一个字符与模式串的最后一个字符相等，从后向前验证，[AGA[TA]]，[AG[ATA]]，文本字符“G”和模式串字符“T”不匹配，于是根据文本窗口的最后一个字符“A”进行移动，移动距离为：d[A] = 2。

2. AG [ATACG] ATATATAC
   G ≠ A, d[G] = 5

3. AGATACG [ATATA] TAC
   文本窗口[ATAT[A]]中的最后一个字符与模式串的最后一个字符相等。从后向前验证，并发现了一个匹配。然后，根据文本窗口的最后一个字符“A”进行移动，移动距离为：d[A] = 2。

4. AGATACGAT [ATATA] C
   文本窗口[ATAT[A]]中的最后一个字符与模式串的最后一个字符相等。从后向前验证，并发现了一个新的匹配。然后，根据文本窗口的最后一个字符“A”进行移动，移动距离为：d[A] = 2。窗口移动后 $pos > n - m$，搜索过程结束。

2.4　基于子串搜索的方法

在文本中的字符互相独立并且等概率出现的条件下，基于子串搜索的算法的平均时间复杂度是最优的。

在基于子串搜索的方法中，搜索窗口的移动简单而优雅。如图 2.13 所示，假设在搜索窗口中已经从后向前识别了模式串的子串 u，并且无法继续识别下一个字符σ。这意味着

σu不再是p的子串，p在文本中的出现不可能覆盖σu。所以，这时可以安全地将搜索窗口移到σ之后。

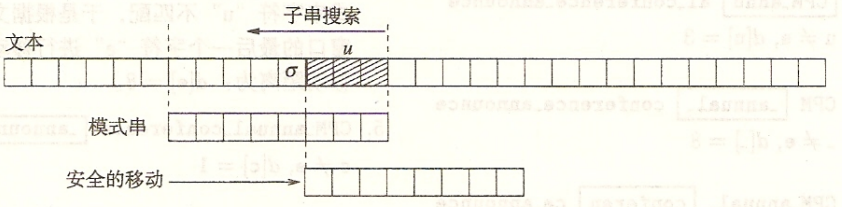


图 2.13 用子串的方法进行搜索窗口的移动。如果无法继续识别字符σ，那么σu不是p的子串，这时可以安全地将搜索窗口移到σ之后

这种方法的主要难点在于如何识别模式串的所有子串。这里首先介绍 Backward Dawg Matching（BDM）算法[CCG+94]。该算法使用了后缀自动机这种数据结构，虽然功能强大，但结构复杂。鉴于以下两点原因，本章中不对它做非常具体的介绍。

（i）当模式串的长度不超过机器字长w时，后缀自动机可以用位并行方法进行有效的模拟。位并行算法 Backward Nondeterministic Dawg Matching（BNDM）[NR00]比 BDM 算法更快，更易于实现，并且可以用于扩展串匹配（参见第 4 章）。

（ii）当模式串较长时，Backward Oracle Matching（BOM）[ACR01]算法能够获得与 BDM 算法相同的效果。但是，BOM 对基于子串的方法进行了修改，采用了一种更简单的自动机：Factor Oracle。

2.4.1 BDM 算法的思想

Backward Dawg Matching（BDM）算法使用后缀自动机搜索子串，并且对基本的子串搜索方法进行了改进。下面首先对后缀自动机做一个简单介绍，然后再详细介绍算法。

后缀自动机 目的是判别字符串u是否是模式串p的一个子串。有很多索引结构都能够在$O(|u|)$的时间内确定u是否为p的子串，其中最经典的结构是 Compact Suffix Tree[McC76]。而在这种结构中，状态转移上的标号是模式串的子串。为了进行状态转换，需要访问不定长的字符串。后缀自动机具有同样的功能，但其状态转移上的标号是单个字符，这样就可以加速字符串匹配算法的搜索过程。感兴趣的读者可以参考文献[CH97, CR94]以获得关于后缀自动机更全面的信息。这里简单回顾一下它的三个基本性质。

性质 1：可以在$O(|u|)$的时间内确定一个字符串u是否为字符p的子串。字符串u是p的一个子串当且仅当p的后缀自动机中存在一条从初始状态开始的标号为u的路径。

性质 2：可以识别模式串的所有后缀。从初始状态到某个终止状态的路径上的字符组成的字符串是模式串 p 的一个后缀。

性质 3：模式串 $p = p_1p_2\cdots p_m$ 对应的后缀自动机［用 $SA(p = p_1p_2\cdots p_m)$ 表示］可以通过在线的方法在 $O(m)$ 时间内构建完成，即依次将字符 p_j 添加到 $SA(p_1\cdots p_{j-1})$ 上以构造 $SA(p_1\cdots p_j)$。

搜索算法　BDM [CCG+94] 使用了后缀自动机的性质。图 2.13 中的搜索过程可以利用后缀自动机来进行，并且利用性质 2 可以做一个技术性的改进。

为了在文本 $T = t_1t_2\ldots t_n$ 中搜索模式串 $p = p_1p_2\cdots p_m$，需要构建模式串 p 的反转 $p^{rv} = p_mp_{m-1}\cdots p_1$ 对应的后缀自动机。算法使用后缀自动机在搜索窗口中从后向前搜索模式串的子串。在搜索过程中，如果到达了一个终止状态，并且对应的不是整个模式串，那么它在窗口中的当前位置被保存在变量 *last* 中。根据性质 2，这对应于找到了模式串 p 的一个前缀（因为 p^{rv} 的后缀是 p 的前缀的反转），并且是当前所识别的最长前缀。它从位置 *last* 开始，到窗口末端结束。这个反向搜索过程以两种可能的方式结束：

（i）在识别一个子串时失败了，即读入了一个字符 σ，而在 p^{rv} 的后缀自动机的当前状态没有 σ 转移。这时将窗口向右移动，使得它的起始位置与 *last* 对齐。这样移动窗口不会遗漏任何可能的匹配，否则将会在窗口中发现模式串的一个更长的前缀。这种情况如图 2.14 所示。

（ii）抵达了窗口的起始位置，这意味着整个模式串 p 被成功匹配。于是报告一个成功匹配，并像（i）中那样移动窗口。

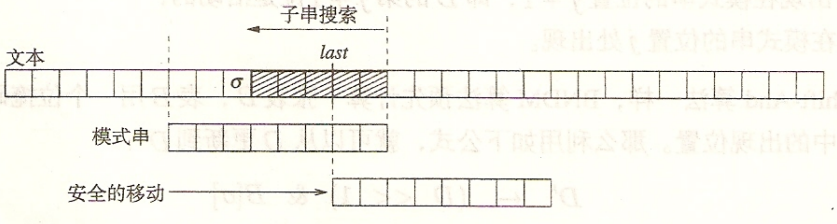


图 2.14　BDM 算法用后缀自动机进行搜索的原理。变量 *last* 保存了既是已读入文本的后缀，同时也是模式串的前缀的最长字符串的出现位置

BDM 算法的最坏时间复杂度是 $O(mn)$。然而，在文本的各个字符相互独立并且等概率出现的条件下，该算法能够达到最优的平均时间复杂度 $O(n\log_{|\Sigma|} m/m)$。

2.4.2 BNDM 算法

Backward Nondeterministic Dawg Matching（BNDM）算法的搜索方法与 BDM 算法相同，但它使用了位并行来识别子串。与原始的 BDM 相比，BNDM 更简单，内存用量更少，具有更好的引用局部性，并且易于扩展到更复杂的模式串的情形（参见第 4 章）。

在当前搜索窗口内，设已读入的字符串为 u，BNDM 算法维护一个集合，记录 u 在 p^{rv} 中的所有出现位置。同 Shift-And 算法一样，该集合可以用一个位向量 D 来表示。如果子串 $p_j \ldots p_{j+|u|-1}$ 等于 u，那么 D 的第 $m-j+1$ 位是 1，表示 p 的位置 j 是一个活动状态。图 2.15 显示了这种关系。如果模式串的长度不超过机器字长 w，那么该集合可以用一个机器字 $D = d_m \ldots d_1$ 表示。

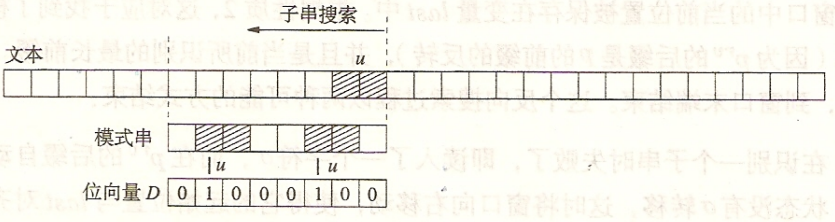


图 2.15 子串搜索的位并行方法。D 记录了 u 在 p 中的所有出现的开始位置

当读入一个新的文本字符 σ 时，要从 D 更新到 D'。D' 的一个活动状态 j 对应于 σu 在模式串中的一个起始位置，也就是说：

- u 出现在模式串的位置 $j+1$，即 D 的第 $j+1$ 位是活动的；
- σ 在模式串的位置 j 处出现。

同 Shift-And 算法一样，BNDM 算法预先计算一张表 B，表 B 用一个位掩码记录了该字符在 p 中的出现位置。那么利用如下公式，就可以从 D 更新到 D'：

$$D' \leftarrow (D << 1) \ \& \ B[\sigma] \tag{2.2}$$

然而，初始化的时候有些问题需要注意。在初始化 D 的时候，可能会将 D 初始化为 1^m，这表示模式串的每个位置都能与空串相匹配。如果这样的话，第一次位移将会得到 $(D << 1) = 1^{m-1}0$，这样会丢失第一个子串。最简单的解决方法是将 D 的大小设为 $m+1$，并初始化为 1^{m+1}。然而，这样会将搜索的最大模式串长度限制在 $w-1$ 以内。为了解决这个问题，可以将公式（2.2）拆分为两个部分。

第一部分进行操作 $D_1' \leftarrow D \ \& \ B[\sigma]$ 并且验证可能的匹配；第二部分进行移位 $D' \leftarrow D_1' << 1$。这里使用的初始化是 $D = 1^m$。如果 D_1' 的位置 d_m 是活动的，那么已读入的文本字符串也是 p 的前缀。

BNDM 与 BDM 使用的搜索方法是一样的，区别在于前者用位并行技术进行子串搜索，而后者使用后缀自动机。每当比特位 d_m 是活动的时候，窗口中的当前位置就用变量 *last* 保存。算法的伪代码如图 2.16 所示。

```
BNDM (p = p1p2...pm, T = t1t2...tn)
1.    Preprocessing
2.        For c ∈ Σ Do B[c] ← 0^m
3.        For j ∈ 1...m Do B[pj] ← B[pj] | 0^(j-1)10^(m-j)
4.    Searching
5.        pos ← 0
6.        While pos ≤ n − m Do
7.            j ← m, last ← m
8.            D ← 1^m
9.            While D ≠ 0^m Do
10.               D ← D & B[t_(pos+j)]
11.               j ← j − 1
12.               If D & 10^(m-1) ≠ 0^m Then
13.                   If j > 0 Then last ← j
14.                   Else report an occurrence at pos + 1
15.               End of if
16.               D ← D << 1
17.           End of while
18.           pos ← pos + last
19.       End of while
```

图 2.16　BNDM 的位并行伪代码

BNDM 与 BDM 具有相同的最坏时间复杂度 $O(mn)$，也都具有最优的平均时间复杂度 $O(n \log_{|\Sigma|} m/m)$。

从自动机的观点来看，基于位并行的子串搜索方法是对识别 p^{rv} 所有后缀的非确定性自动机的模拟。例如，如果要搜索模式串 “announce”，可以用图 2.17 所示的自动机进行模拟。可以证明：该非确定性自动机对应的最小确定性自动机就是 BDM 算法中使用的后缀自动机。BNDM 与 BDM 的区别，就如同 Shift-Or 和 KMP 之间的区别。前者用位并行模拟非确定性自动机，后者是相应的确定性自动机。

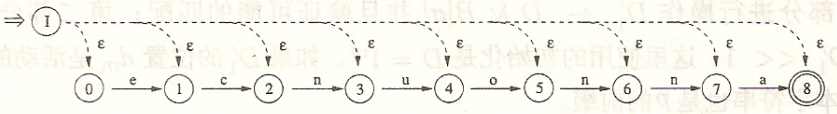


图 2.17 识别“announce”的反转“ecnuonna”的所有后缀的非确定性自动机

BNDM 算法在英文上的例子 在文本“CPM_annual_conference_announce”中搜索字符串“announce”。

$B =$

a	10000000
c	00000010
e	00000001
n	01100100
o	00010000
u	00001000
*	00000000

$D = 11111111$

1. CPM_annu **al_conference_announce**

$last \leftarrow 8$

	11111111
读入 u	00001000
$D =$	00001000
	00010000
读入 n	01100100
$D =$	00000000

2. CPM_annu **al_confe** rence_announce

$last \leftarrow 8$

	11111111
读入 e	00000001
$D =$	00000001
	00000010
读入 f	00000000
$D =$	00000000

3. CPM_annual_confe **rence_an** nounce

$last \leftarrow 8$

	11111111
读入 n	01100100
$D =$	01100100
	11001000
读入 a	10000000
$D =$	10000000

d_8是活动的，但是 $j > 0$，因此置$last \leftarrow 6$。

4. CPM_annual_conference_ **announce**

$last \leftarrow 8$

	11111111
读入 e	00000001
$D =$	00000001
	00000010
读入 c	00000010
$D =$	00000010
	00000100
读入 n	01100100
$D =$	00000100
	00001000
读入 u	00001000
$D =$	00001000
	00010000
读入 o	00010000
$D =$	00010000
	00100000
读入 n	01100100
$D =$	00100000
	01000000
读入 n	01100100
$D =$	01000000
	10000000
读入 a	10000000
$D =$	10000000

d_8是活动的，并且 $j = 0$，因此报告一个成功匹配。

BNDM 算法在 DNA 上的例子　在序列 AGATACGATATATAC 中搜索字符串 ATATA。

$$B=\left\{\begin{array}{|c|c|}\hline \texttt{A} & 10101\\ \hline \texttt{T} & 01010\\ \hline * & 00000\\ \hline\end{array}\right.$$

$D=11111$

1. [AGATA] CGATATATAC

$last \leftarrow 5$

$$\begin{array}{rr} & 11111\\ \text{读入A} & 10101\\ \hline D= & 10101\end{array}$$

d_5是活动的，但是$j>0$，因此置$last$←4。

$$\begin{array}{rr} & 01010\\ \text{读入T} & 01010\\ \hline D= & 01010\end{array}$$

$$\begin{array}{rr} & 10100\\ \text{读入A} & 10101\\ \hline D= & 10100\end{array}$$

d_5是活动的，但是$j>0$，因此置$last \leftarrow 2$。

$$\begin{array}{rr} & 01000\\ \text{读入G} & 00000\\ \hline D= & 00000\end{array}$$

2. AG [ATACG] ATATATAC

$last \leftarrow 5$

$$\begin{array}{rr} & 11111\\ \text{读入G} & 00000\\ \hline D= & 00000\end{array}$$

3. AGATACG [ATATA] TAC

$last \leftarrow 5$

$$\begin{array}{rr} & 11111\\ \text{读入A} & 10101\\ \hline D= & 10101\end{array}$$

d_5是活动的，但是$j>0$，因此置$last$←4。

$$\begin{array}{rr} \text{读入T} & 01010\\ \hline D= & 01010\end{array}$$

$$\begin{array}{rr} & 10100\\ \text{读入A} & 10101\\ \hline D= & 10100\end{array}$$

d_5是活动的，但是$j>0$，因此置$last \leftarrow 2$。

$$\begin{array}{rr} & 01000\\ \text{读入T} & 01010\\ \hline D= & 01000\end{array}$$

$$\begin{array}{rr} & 10000\\ \text{读入A} & 10101\\ \hline D= & 10000\end{array}$$

d_5是活动的，并且$j=0$，因此报告一个成功匹配。

4. AGATACGAT [ATATA] C

$last \leftarrow 5$

$$\begin{array}{rr} & 11111\\ \text{读入A} & 10101\\ \hline D= & 10101\end{array}$$

d_5是活动的，但是$j>0$，因此置$last$←4。

$$\begin{array}{rr} \text{读入T} & 01010\\ \hline D= & 01010\end{array}$$

$$\begin{array}{rr} & 10100\\ \text{读入A} & 10101\\ \hline D= & 10100\end{array}$$

d_5是活动的，但是$j>0$，因此置$last \leftarrow 2$。

$$\begin{array}{rr} & 01000\\ \text{读入T} & 01010\\ \hline D= & 01000\end{array}$$

$$\begin{array}{rr} & 10000\\ \text{读入A} & 10101\\ \hline D= & 10000\end{array}$$

d_5是活动的，并且$j=0$，因此报告一个成功匹配，然后将窗口移动到$pos+last$。而在此时，$pos>n-m$，搜索过程停止。

2.4.3　BOM 算法

当字符串长度大于机器字长w时，虽然 BDM 算法可以满足使用之需，但是后缀自动

机的构造过于复杂，使得它并不实用。最近，文献[ACR01]提出了一种解决方法，基于如下观察：在基于子串搜索的方法中，要安全地移动搜索窗口（参见图 2.13），并不需要知道 u 是否为模式串 p 的一个子串，而只需要知道 σu 不是 p 的子串就足矣。

Factor Oracle 结构可以满足上述需求。在字符串 p 上构建的 Factor Oracle，所能识别的字符串集合是 p 的子串集合的超集。所以，在窗口中从后向前识别时可能会多读入字符，这会带来一定的性能损失。但是 Factor Oracle 易于理解和实现，并且结构简洁紧凑，能够减少分页错误，因而性能上的损失可以被抵消。

为了简单起见，用 θ 表示未定义的对象。例如，在自动机中，$\delta(q,\alpha)=\theta$ 表示在状态 q 没有 α-转移（标号为 α 的转移）。

Factor Oracle

字符串 $p=p_1p_2\cdots p_m$ 的 Factor Oracle 结构是一个确定性有向无环自动机，共有 $m+1$ 个状态，以及 m 到 $2m-1$ 个状态转移。记它的状态转移函数为 δ。

Factor Oracle 的 $m+1$ 个状态对应于 p 的字符之间的 $m+1$ 个位置，其中位置 0 在整个模式串之前。状态 i ($0<i\leqslant m$)对应于 p 的前缀 $p_1p_2\ldots p_i$。

前 m 个转移将模式串的字符连成一线，即：对于每个状态 i ($0<i\leqslant m$)，构造一个从状态 $i-1$ 到状态 i 的转移，标号为 p_i。在实际操作中，这些状态和转移可以隐含在模式串之中。

之后构造外部转移，它们最多有 $m-1$ 条。对于每个状态 i，将它与另一个状态 j ($j<i$)关联，称 j 为 i 的供给状态，并用 $j=S(i)$ 表示。S 称为供给函数，在构造外部转移的过程中构建。$S(0)$ 定义为 θ。

构造算法逐个检查状态 1 到状态 m，假设已经检查过状态 $i-1$ 并且开始检查状态 i，这时沿着状态 $i-1$ 的供给函数回溯。具体的做法是，变量 k 被初始化为 $S(i-1)$，并执行如下步骤：

步骤 1 如果 $k=\theta$，那么 $S(i)\leftarrow 0$。

步骤 2 如果 $k\neq\theta$，并且状态 k 不存在 p_i-转移，那么构造从状态 k 到状态 i 的转移，转移上的标号为 p_i。然后 $k\leftarrow S(k)$，转到步骤 1 执行。

步骤 3 如果 $k\neq\theta$，并且状态 k 存在 p_i-转移，目的状态为 j，那么置 $S(i)\leftarrow j$，停止执行。

整个构建过程是很简单的，而且能够用在线方式完成，即可以逐个添加字符 p_i，并同时构造新状态 i 和新的转移。图 2.18 所示为在线构建算法的伪代码，该算法是 $O(m)$ 的。

```
Oracle_add_letter(Oracle(p = p1p2...pm), σ)
1.   Create a new state m + 1
2.   δ(m, σ) ← m + 1
3.   k ← S(m)
4.   While k ≠ θ AND δ(k, σ) = θ Do
5.       δ(k, σ) ← m + 1
6.       k ← S(k)
7.   End of while
8.   If k = θ Then s ← 0
9.   Else s ← δ(k, σ)
10.  S(m + 1) ← s
11.  Return Oracle(p = p1p2...pmσ)

Oracle-on-line(p = p1p2...pm)
12.  Create Oracle(ε) with:
13.      One single initial state 0
14.      S(0) ← θ
15.  For j ∈ 1...m Do
16.      Oracle(p = p1p2...pj) ← Oracle_add_letter(Oracle(p = p1p2...pj−1), pj)
17.  End of for
```

图 2.18　Factor Oracle 的构造过程。函数 Oracle_add_letter 在 Oracle($p = p_1p_2 \ldots p_m$) 添加一个字符 σ 构造 Oracle($p\sigma$)。Oracle_on_line 逐个添加字符 p_i，最终构造出 Oracle($p = p_1p_2 \ldots p_m$)

在 p 上构建的 Factor Oracle 结构能够识别 p 的所有子串，而且它可能识别更多，但在实际中也不会多得过度。对于长度为 m 的字符串，它只能识别出一个，那就是模式串本身。

为了实现 Factor Oracle 结构，最简单的方法是使用一个 $(m+1) \times A$ 的表，其中 A 是模式串的字母表大小。这种表示的优点是能够以 $O(1)$ 的时间进行状态之间的跳转，从而加快搜索过程。然而对于很长的模式串，则需要考虑采用 $O(m)$ 的空间来实现。

用 Factor Oracle 进行搜索

Backward Oracle Matching（BOM）算法使用 Factor Oracle 进行搜索，是子串搜索方法中最简单的（参见图 2.13）。它在当前搜索窗口内从后向前逐个读入字符，用 p^{rv} 对应的 Factor Oracle 进行子串识别。如果已经读入了字符串 u，并且无法继续识别字符 σ，则说明 σu 不是 p 的子串，这时就可以安全地将窗口移动到字符 σ 之后的位置。如果识别到了窗口的起始位置，由于 Factor Oracle 能够识别的长度为 $|p|$ 的字符串只有模式串本身，所以就可以报告一个成功匹配，并将窗口向右移动一个字符。BOM 的伪代码在图 2.19 中给出。

```
BOM(p = p1p2...pm, T = t1t2...tn)
1.   Preprocessing
2.       Oracle-on-line(p^rv)
             δ is its transition function
3.   Searching
4.       pos ← 0
5.       While pos ≤ n − m Do
6.           Current ← initial state of Oracle(p^rv)
7.           j ← m
8.           While j > 0 AND Current ≠ θ Do
9.               Current ← δ(Current, t_{pos+j})
10.              j ← j − 1
11.          End of while
12.          If Current ≠ θ Then
13.              mark an occurrence at pos + 1
14.          End of if
15.          pos ← pos + j + 1
16.      End of while
```

图 2.19 BOM 算法的伪代码

BOM 的最坏时间复杂度是 $O(mn)$。从实验结果可以推断，它在平均情况下是最优的。

英文例子 在文本“CPM_annual_conference_announce”中搜索字符串“announce”。其反转字符串对应的 Factor Oracle 自动机参见图 2.20。

1. CPM_annu al_conference_announce

   Factor Oracle 读入字符 u；
   Factor Oracle 无法继续识别下一个字符 n。

2. CPM_ann ual_conf erence_announce

   Factor Oracle 无法识别字符 f。

3. CPM_annual_conf erence_a nnounce

   Factor Oracle 读入字符 a；
   Factor Oracle 无法继续识别下一个字符_。

4. CPM_annual_conference_ announce

   Factor Oracle 读入字符 e；
   Factor Oracle 读入字符 c；
   Factor Oracle 读入字符 n；
   Factor Oracle 读入字符 u；
   Factor Oracle 读入字符 o；
   Factor Oracle 读入字符 n；
   Factor Oracle 读入字符 n；
   Factor Oracle 读入字符 a；
   到达窗口的起始位置，这时报告一个成功匹配。

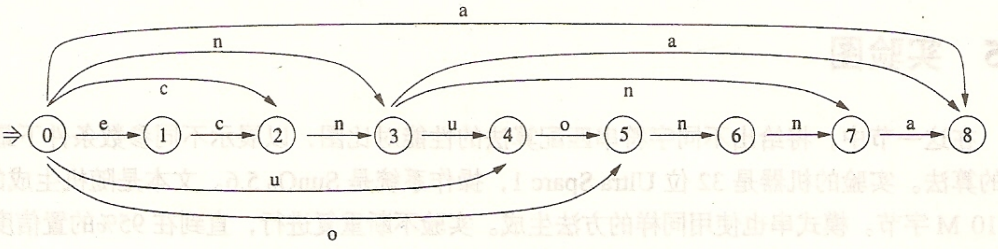


图 2.20　“announce”的反转“ecnuonna”对应的 Factor Oracle 自动机

DNA 例子　在序列 AGATACGATATATAC 中搜索字符串 ATATA。其反转字符串对应 Factor Oracle 自动机参见图 2.21。

1. AGATA CGATATATAC

   Factor Oracle 读入字符 A；
   Factor Oracle 读入字符 T；
   Factor Oracle 读入字符 A；

   Factor Oracle 无法继续识别下一个字符 G。

2. AG ATACG ATATATAC

   Factor Oracle 无法识别字符 G。

3. AGATACG ATATA TAC

   Factor Oracle 读入字符 A；
   Factor Oracle 读入字符 T；
   Factor Oracle 读入字符 A；
   Factor Oracle 读入字符 T；
   Factor Oracle 读入字符 A；

   到达窗口的起始位置，这时报告一个成功匹配。

4. AGATACGA TATAT AC

   Factor Oracle 读入字符 T；
   Factor Oracle 读入字符 A；
   Factor Oracle 读入字符 T；
   Factor Oracle 读入字符 A；

   Factor Oracle 无法识别字符 T。

5. AGATACGAT ATATA C

   Factor Oracle 读入字符 A；
   Factor Oracle 读入字符 T；
   Factor Oracle 读入字符 A；
   Factor Oracle 读入字符 T；
   Factor Oracle 读入字符 A；

   到达窗口的起始位置，这时报告一个成功匹配。

6. AGATACGATA TATAC

   Factor Oracle 无法识别字符 C。

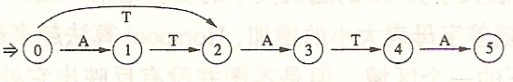


图 2.21　ATATA 的反转 ATATA 对应的 Factor Oracle 自动机

2.5 实验图

在这一节中，将给出不同字符串匹配算法的性能对比图，以展示不同参数条件下最有效的算法。实验的机器是 32 位 Ultra Sparc 1，操作系统是 SunOs 5.6。文本是随机生成的，共 10 M 字节。模式串也使用同样的方法生成。实验不断重复进行，直到在 95%的置信度下相对误差低于 2%。所有算法都采用了最优的实现，但是只有 Shift-Or，Horspool，BNDM 和 BOM 在图上有相应的分布区域，其他算法则因为太慢而没有在图上显示。

图 2.22 是实验的结果图，横轴上标示出了一个寄存器字长 w 的位置，这是用一个机器字实现的 BNDM 算法所能处理的最长字符串长度。

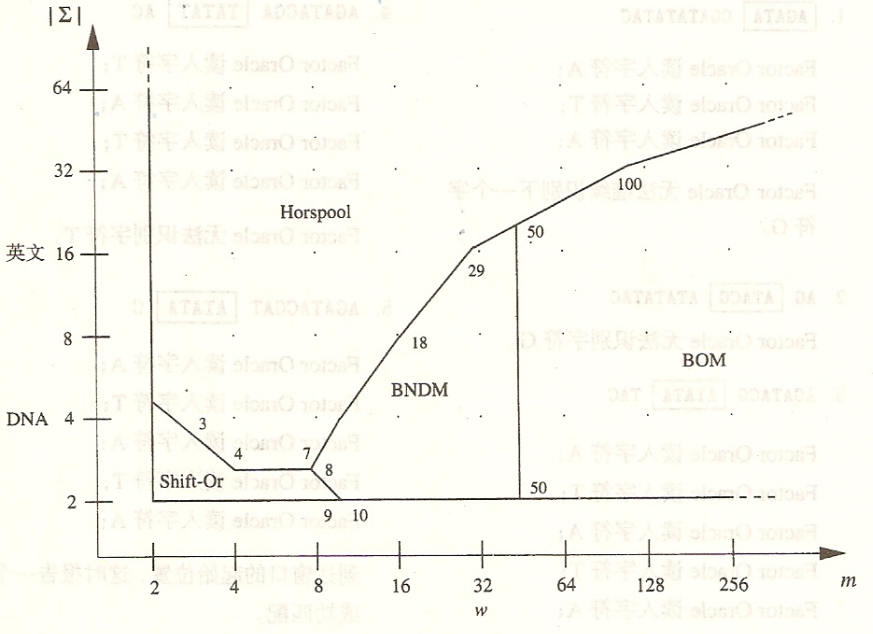


图 2.22 不同串匹配算法的性能对比图

DNA 序列的实验结果与字母表大小为 4 的随机文本的实验结果一致。出人意料的是，英文的实验结果与字母表大小为 16 的随机文本的实验结果一致。

从图中可以看出，随着字母表大小的增加，Horspool 算法越来越占据绝对优势。BNDM 算法局限于小字母表下的一个区域，但是本图并没有反映出它处理扩展字符串的能力。Shift-Or 只适用于字母表较小并且模式串长度较短的情况。

2.6　其他算法和参考文献

字符串匹配算法还有很多，在这里只给出一些重要的参考文献。

关于 KMP 算法　MP 算法[MP70]和 KMP[KMP77]算法有很多变体，其中最重要的是 Simon 算法[Sim93]。该算法表明，KMP 底层的自动机能够以一种有效的方式构建和存储。关于 KMP 更完备的分析请参考文献[Rég89]。文献[Han93]对 Simon 算法进行了分析。

关于 BM 算法　与 KMP 算法一样，BM 算法[BM77]也有很多变体，最主要的两个是 Boyer-Moore-Galil 算法[Gal79]和 Turbo-BM 算法[CGR92]。文献[BYGR90, BYR92, Col94]对 BM 算法进行了分析。文献[BYG89a, Cho90, BYCG94, BBYDS96]对底层的自动机进行了分析。文献[MRS96]对 Horspool 算法进行了分析。

关于 BDM 算法　BDM 和 KMP 一起使用可以保证线性的最坏时间复杂度，使用这种方法的算法是 TurboBDM [CCG+94] 和 TurboRF [CCG+94] 。Double Forward Dawg Matching [AR00]是最简单的线性最坏时间复杂度和最优平均时间复杂度的算法。

常量空间算法　文献[GS81]于 1981 年首次提出了只需要常量额外存储空间的线性字符串匹配算法，随后相继提出了其他很多算法[CP91, Cro92, CR95]。寻找平均时间复杂度最优的常量空间算法，仍然是一个悬而未决的问题。

散列算法　最著名的使用散列的串匹配算法是 Karp-Rabin[KR87]，文献[GBY90]对其进行了分析。

第 3 章　多字符串匹配

3.1　基本概念

单字符串匹配问题可以很自然地扩展到同时搜索一组字符串 $P=\{p^1,p^2,\ldots,p^r\}$ 的问题。其中，p^i 是定义在有限字母表 Σ 上的字符串 $p^i=p^i_1p^i_2\ldots p^i_{m_i}$。记 $|P|$ 为 P 中所有字符串长度之和，即：$|P|=\sum_{i=1}^{r}|p^i|=\sum_{i=1}^{r}m_i$。记 ℓmin 和 ℓmax 分别为 P 中最短模式串的长度和最长模式串的长度。搜索在文本 $T=t_1t_2\ldots t_n$ 上进行。

P 中的一些字符串可能是集合中其他字符串的子串、前缀、后缀，或者完全相等。例如，如果在 DNA 序列上搜索集合{ATATA, TATA}，每次发现一个 ATATA 匹配的同时 TATA 也会被成功匹配。因此，模式串总的出现次数可能是 $r\times n$。更精确地说，对于多字符串匹配，我们感兴趣的是所有满足 $t_{j-|p^i|+1}\ldots t_j=p^i$ 条件的整数对 (i,j)。

这个问题最简单的解决方法是利用第 2 章中的任何一种算法搜索 r 遍，这将导致预处理阶段的最坏时间复杂度为 $O(|P|)$，搜索阶段的最坏时间复杂度为 $O(r\times n)$。

对单模式串匹配算法进行扩展，可以将搜索阶段的最坏时间复杂度降为 $O(n+nocc)$，其中 $nocc$ 是总的匹配次数。算法的平均复杂度同样也可以改进，然而很难定义什么是“平均”，因为很多因素对算法的运行时间有影响。最重要的因素包括：字母表的大小、模式串的个数、模式串的长度分布（特别是最短模式串的长度），以及内存的用量。

与第 2 章一样，本章用 θ 表示未定义的对象。例如，While $q\neq\theta$ Do 表示：当 q 有定义时不断进行迭代。

此外，在本章中还将一直使用图 3.1 所示的例子，同时搜索三个字符串“announce”，“annual”和“annually”。

如同单个字符串时的情形，自然语言上的字符串集合包含的重复字符串通常比 DNA 序列上的要少。为了展示可能出现的各种复杂情况，本章也将在模式串集合 ATATATA，TATAT，ACGATAT 和文本 AGATACGATATATAC 上演示相关算法的执行过程。

第 2 章中的三种单模式串搜索方法都可以扩展到多模式串的情形。对于每种搜索方法，根据模式串的处理方法和移动方式的不同，可以产生很多不同的扩展。本章将介绍几个在实际应用中性能最好的算法，通常也是最简单的。

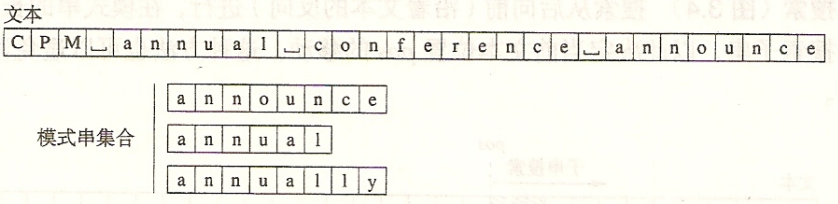


图 3.1　在文本中同时搜索三个字符串

前缀搜索（图 3.2）　搜索从前向后（沿着文本的正向）进行，逐个读入文本字符，使用在P上构建的自动机进行识别。对于每个文本位置，计算既是已读入文本的后缀，同时也是P中某个模式串的前缀的最长字符串。著名的 Aho-Corasick[AC75]算法使用的就是这种方法。

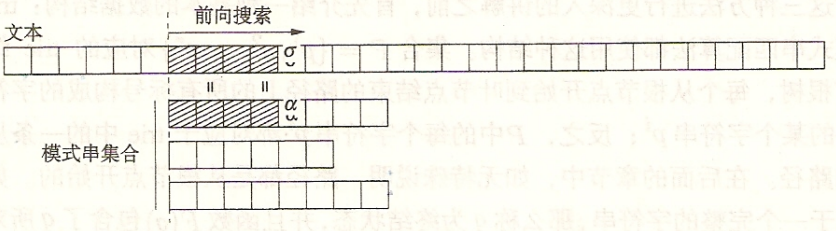


图 3.2　第一种方法：需要计算既是已读入文本的后缀也是 P 中某个模式串的前缀的最长字符串。文本中的字符至少被读入一次

后缀搜索（图 3.3）　搜索从后向前（沿着文本的反向）进行，搜索模式串的后缀。与单模式串的后缀搜索一样，根据后缀的下一次出现位置来移动当前文本位置 *pos* 。这种方法可以避免读入所有的文本字符。

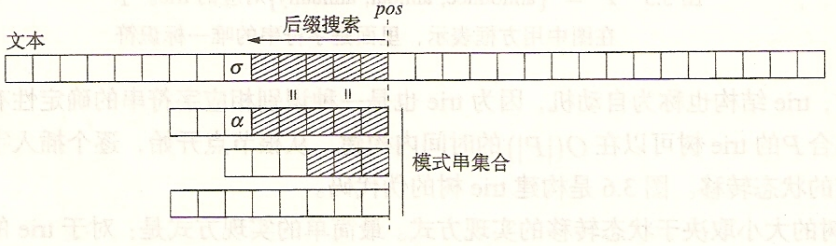


图 3.3　第二种方法：从后向前搜索某个模式串的后缀。一般来说，这种方法可以避免读入所有的文本字符

子串搜索（图 3.4） 搜索从后向前（沿着文本的反向）进行，在模式串的长度为 ℓmin 的前缀中搜索子串，以此决定当前文本位置 pos 的移动。这种方法也可以避免读入所有的文本字符。

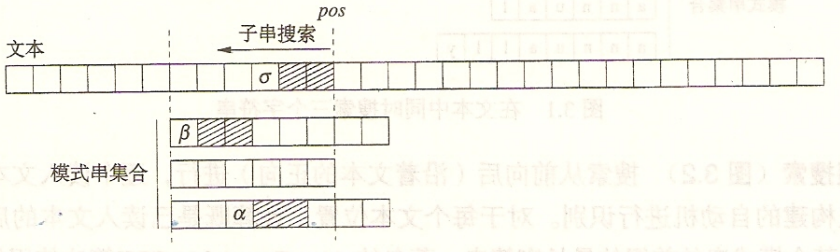


图 3.4 第三种方法：在当前窗口中搜索某个模式串的子串

在对这三种方法进行更深入的讲解之前，首先介绍一种基本的数据结构：trie。很多经典的多模式串匹配算法都使用这种结构。集合 $P = \{p^1, p^2, \ldots p^r\}$ 对应的 trie 结构是一棵有向的有根树，每个从根节点开始到叶节点结束的路径上的所有标号构成的字符串，都对应于 P 中的某个字符串 p^i；反之，P 中的每个字符串 p^i 都对应于 trie 中的一条从根节点到叶节点的路径。在后面的章节中，如无特殊说明，路径都是从根节点开始的。如果状态节点 q 对应于一个完整的字符串，那么称 q 为终结状态，并且函数 $F(q)$ 包含了 q 所对应的 P 中的字符串。图 3.5 是 $P = \{\text{announce, annual, annually}\}$ 对应的 trie 结构。

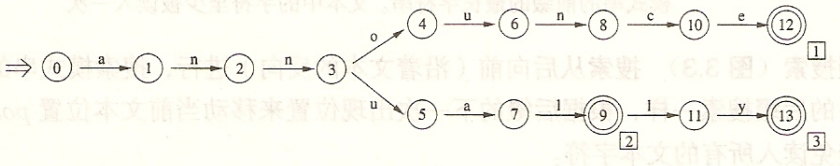


图 3.5 $P = \{\text{announce, annual, annually}\}$ 对应的 trie。F 在图中用方框表示，里面是字符串的唯一标识符

通常，trie 结构也称为自动机，因为 trie 也是一种识别相应字符串的确定性有向无环自动机。集合 P 的 trie 树可以在 $O(|P|)$ 的时间内构建，从根节点开始，逐个插入字符串 p^i，构造相应的状态转移。图 3.6 是构建 trie 树的伪代码。

trie 树的大小取决于状态转移的实现方式。最简单的实现方式是：对于 trie 的每个状态 q，用一个大小为 $|\Sigma|$ 的表存储它的转移 $\delta(q, *)$。这种 trie 的表示方式耗用的空间是最多的，大小为 $|\Sigma| \times |P|$，而它的优点是能够在 $O(1)$ 的时间内进行状态转移。这种表示方法通常用于模式串集合规模和字母表大小不是很大的情形。

```
Trie(P = {p^1, p^2, ..., p^r})
1.    Create an initial non terminal state 0
2.    For i ∈ 1...r Do
3.        Current ← initial state 0
4.        j ← 1
5.        While j ≤ m_i AND δ(Current, p_j^i) ≠ θ Do
6.            Current ← δ(Current, p_j^i)
7.            j ← j + 1
8.        End of while
9.        While j ≤ m_i Do
10.           Create a new non terminal state State
11.           δ(Current, p_j^i) ← State
12.           Current ← State
13.           j ← j + 1
14.       End of while
15.       If Current is already terminal Then F(Current) ← F(Current) ∪ {i}
16.       Else mark Current as terminal, F(Current) ← {i}
17.   End of for
```

图 3.6　在 $P = \{p^1, p^2, \ldots, p^r\}$ 上构造 trie 的伪代码。字符串是逐个添加到树中去的

因为总的状态转移最多为 $|P|$，所以可以用 $O(|P|)$ 的空间进行存储，而与字母表的大小无关。然而，进行状态转移的时间会随之增加。如果用链表存储状态的转移，那么不管是否有序，进行状态转移的最坏和平均时间复杂度都是 $O(|\Sigma|)$；通过使用平衡树[CLR90]可以将其复杂度降低为 $O(\log|\Sigma|)$，但这又使得代码实现变得复杂。

下面将对如何在一个字符串集合上使用前面的三种基本搜索方法加以具体介绍。

3.2　基于前缀搜索的方法

对基于前缀搜索方法的扩展产生了 Multiple Shift-And 算法和 Aho-Corasick 算法。与单模式串匹配一样（参见 2.2 节），假设在当前文本位置，已经找到了既是文本串 $T = t_1 \ldots t_i$ 的后缀，同时也是 P 中某个模式串 p^k 的前缀的最长字符串。问题的关键在于每读入一个新的文本字符时，如何更新这个最长字符串的长度。

在单模式串匹配里面，有两种方法来计算这个长度。一种方法是使用位并行，对于多模式串匹配，只有当模式串集合很小的时候，这种方法才是有效的，因为模式串的总长度 $|P|$ 必须不超过几个机器字长。尽管如此，这种方法在扩展匹配（参见第 4 章）和近似匹配（参见第 6 章）中得到了广泛使用，这种算法称为 Multiple Shift-And 算法。

当模式串的总长度太大以至于不能完全放入几个机器字中时，第二种方法使用一种机制，能在常数均摊时间内计算出既是已读入文本的后缀，同时也是 P 中某个模式串的前缀的最长字符串的长度，这就是 Aho-Corasick 算法，它的预处理时间为 $O(|P|)$。

3.2.1 Multiple Shift-And 算法

位并行方法仅适用于模式串集合 $P = \{p^1, \ldots, p^r\}$ 的总长度 $|P|$ 不超过几个机器字长的情形。为了简单起见，假设 $|P|$ 不超过机器字长 w。其主要思想是在一个机器字里面同时对 r 个模式串进行 Shift-And 算法的位操作[BYG89b]。

首先，将所有的模式串包装在一个机器字里面，如图 3.7 所示。然后，每读入一个新的文本字符，就对 P 中的模式串进行类似 Shift-And 那样的计算。初始化字的初始化值 DI 是所有模式串的初始化字的连接，即：

$$DI \leftarrow 0^{m_r-1}1 \ldots 0^{m_2-1}10^{m_1-1}1$$

与之类似，测试是否匹配的掩码是：

$$DF \leftarrow 10^{m_r-1} \ldots 10^{m_2-1}10^{m_1-1}$$

主循环与 Shift-And 算法一样。伪代码如图 3.8 所示。

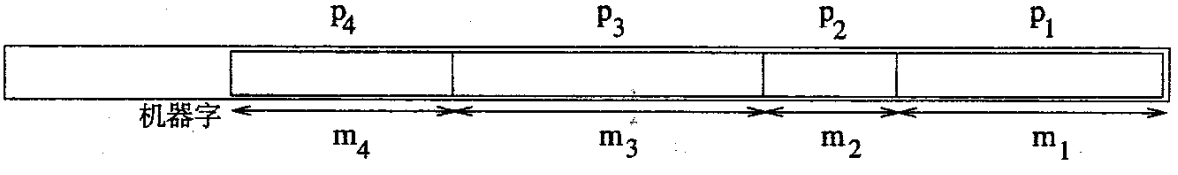


图 3.7 Multiple Shift-And 算法。模式串的总长度必须不超过 w

Multiple Shift-And($P = \{p^1, p^2, \ldots, p^r\}$, $T = t_1 t_2 \ldots t_n$)

```
1.  Preprocessing
2.      For c ∈ Σ Do B[c] ← 0^|P|
3.      ℓ ← 0
4.      For k ∈ 1...r Do
5.          For j ∈ 1...m_k Do B[p_j^k] ← B[p_j^k] | 0^(|P|-ℓ-j) 10^(ℓ+j-1)
6.          ℓ ← ℓ + m_k
7.      End of for
8.      DI ← 0^(m_r-1) 1 ... 0^(m_2-1) 10^(m_1-1) 1
9.      DF ← 10^(m_r-1) ... 10^(m_2-1) 10^(m_1-1)
10. Searching
11.     D ← 0^|P|
12.     For pos ∈ 1...n Do
13.         D ← ((D << 1) | DI) & B[t_pos]
14.         If D & DF ≠ 0^|P| Then
15.             Check which patterns match
16.             Report the corresponding occurrences ending in pos
17.         End of if
18.     End of for
```

图 3.8 Multiple Shift-And 算法。模式串的总长度 $|P|$ 必须不超过 w。记 $m_k = |p^k|$

英文例子　在文本“annual_announce”中搜索模式串集合 $P=\{$announce, annual, annually$\}$。

表 B

a	0 0 0 1 0 0 0 1 0 1 0 0 0 1 0 0 0 0 0 0 0 1
c	0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0
e	0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0
l	0 1 1 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0
n	0 0 0 0 0 1 1 0 0 0 0 1 1 0 0 0 1 0 0 1 1 0
o	0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0
u	0 0 0 0 1 0 0 0 0 0 1 0 0 0 0 0 0 1 0 0 0 0
y	1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
*	0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0

DI = 0 0 0 0 0 0 0 1 0 0 0 0 0 1 0 0 0 0 0 0 0 1
DF = 1 0 0 0 0 0 0 0 1 0 0 0 0 0 1 0 0 0 0 0 0 0
D = 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0

1. 读入 a

B[a] 0 0 0 1 0 0 0 1 0 1 0 0 0 1 0 0 0 0 0 0 0 1
$D=$ 0 0 0 0 0 0 0 1 0 0 0 0 0 1 0 0 0 0 0 0 0 1

2. 读入 n

B[n] 0 0 0 0 0 1 1 0 0 0 0 1 1 0 0 0 1 0 0 1 1 0
$D=$ 0 0 0 0 0 0 1 0 0 0 0 0 1 0 0 0 0 0 0 0 1 0

3. 读入 n

B[n] 0 0 0 0 0 1 1 0 0 0 0 1 1 0 0 0 1 0 0 1 1 0
$D=$ 0 0 0 0 0 1 0 0 0 0 0 1 0 0 0 0 0 0 0 1 0 0

4. 读入 u

B[u] 0 0 0 0 1 0 0 0 0 0 1 0 0 0 0 0 0 1 0 0 0 0
$D=$ 0 0 0 0 1 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0

5. 读入 a

B[a] 0 0 0 1 0 0 0 1 0 1 0 0 0 1 0 0 0 0 0 0 0 1
$D=$ 0 0 0 1 0 0 0 1 0 1 0 0 0 1 0 0 0 0 0 0 0 1

6. 读入 l

B[l] 0 0 1 0 0 0 1 1 1 0 0 0 1 1 0 0 0 0 0 1 1
$D=$ 0 0 1 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0

D & $DF \neq 0^{|P|}$，验证并报告可能的匹配。

7. 读入_

B[_] 0 1 0 0 0 0 0 1 0 0 0 0 0 1 0 0 0 0 0 0 0 1
$D=$ 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0

8. 读入 a

B[a] 0 0 0 0 0 0 0 1 0 0 0 0 0 1 0 0 0 0 0 0 0 1
$D=$ 0 0 0 0 0 0 0 1 0 0 0 0 0 1 0 0 0 0 0 0 0 1

9. 读入 n

B[n] 0 0 0 0 0 0 1 1 0 0 0 0 1 1 0 0 0 0 0 1 1
$D=$ 0 0 0 0 0 1 0 0 0 0 0 1 0 0 0 0 0 0 0 1 0

10. 读入 n

B[n] 0 0 0 0 0 1 0 1 0 0 0 1 0 1 0 0 0 0 0 1 0 1
$D=$ 0 0 0 0 0 1 0 0 0 0 0 1 0 0 0 0 0 0 0 1 0 0

11. 读入 o

B[o] 0 0 0 0 1 0 0 1 0 0 1 0 0 1 0 0 0 0 1 0 0 1
$D=$ 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0

12. 读入 u

B[u] 0 0 0 0 0 0 0 1 0 0 0 0 0 1 0 0 0 1 0 0 0 1
$D=$ 0 0 0 0 1 0 0 0 0 0 1 0 0 0 0 0 0 0 1 0 0 0

13. 读入 n

B[n] 0 0 0 0 0 0 0 1 0 0 0 0 0 1 0 0 1 0 0 0 0 1
$D=$ 0 0 0 0 0 1 1 0 0 0 0 1 1 0 0 0 1 0 0 1 1 0
0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0

14. 读入 c

B[c] 0 0 0 0 0 0 0 1 0 0 0 0 0 1 0 1 0 0 0 0 0 1
$D=$ 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0

15. 读入 e

B[c] 0 0 0 0 0 0 0 1 0 0 0 0 0 1 1 0 0 0 0 0 0 1
$D=$ 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0

D & $DF \neq 0^{|P|}$，验证并报告可能的匹配。

Shift-Or 算法（参见 2.2.2 节）使用的技巧在这里不再适用，因为左移操作符“<<”只能在最右端引入 0，而这里需要在每个字符串的开始位置都引入一个 0。

DNA 例子 在序列“AGATACGATATATAC”中搜索 $P = \{\text{ATATATA, TATAT, ACGATAT}\}$。

表 B

A	0 1 0 1 0 0 1 0 1 0 1 0 1 0 1 0 1 0 1
C	0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0
G	0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0
T	1 0 1 0 0 0 0 1 0 1 0 1 0 1 0 1 0 1 0
*	0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0

$DI = 0\,0\,0\,0\,0\,0\,1\,0\,0\,0\,0\,1\,0\,0\,0\,0\,0\,0\,1$
$DF = 1\,0\,0\,0\,0\,0\,0\,1\,0\,0\,0\,0\,1\,0\,0\,0\,0\,0\,0$
$D = 0\,0\,0\,0\,0\,0\,0\,0\,0\,0\,0\,0\,0\,0\,0\,0\,0\,0\,0$

1. 读入 A

$B[\text{A}] = 0\,0\,0\,0\,0\,0\,1\,0\,0\,0\,0\,1\,0\,0\,0\,0\,0\,0\,1$
$D = 0\,1\,0\,1\,0\,0\,1\,0\,1\,0\,1\,0\,1\,0\,1\,0\,1\,0\,1$
$D = 0\,0\,0\,0\,0\,0\,1\,0\,0\,0\,0\,0\,0\,0\,0\,0\,0\,0\,1$

2. 读入 G

$B[\text{G}] = 0\,0\,0\,0\,0\,1\,1\,0\,0\,0\,0\,1\,0\,0\,0\,0\,0\,1\,1$
$D = 0\,0\,0\,0\,1\,0\,0\,0\,0\,0\,0\,0\,0\,0\,0\,0\,0\,0\,0$
$D = 0\,0\,0\,0\,0\,0\,0\,0\,0\,0\,0\,0\,0\,0\,0\,0\,0\,0\,0$

3. 读入 A

$B[\text{A}] = 0\,0\,0\,0\,0\,0\,1\,0\,0\,0\,0\,1\,0\,0\,0\,0\,0\,0\,1$
$D = 0\,1\,0\,1\,0\,0\,1\,0\,1\,0\,1\,0\,1\,0\,1\,0\,1\,0\,1$
$D = 0\,0\,0\,0\,0\,0\,1\,0\,0\,0\,0\,0\,0\,0\,0\,0\,0\,0\,1$

4. 读入 T

$B[\text{T}] = 0\,0\,0\,0\,0\,1\,1\,0\,0\,0\,0\,1\,0\,0\,0\,0\,0\,1\,1$
$D = 1\,0\,1\,0\,0\,0\,0\,1\,0\,1\,0\,1\,0\,1\,0\,1\,0\,1\,0$
$D = 0\,0\,0\,0\,0\,0\,0\,0\,0\,0\,0\,1\,0\,0\,0\,0\,0\,1\,0$

5. 读入 A

$B[\text{A}] = 0\,0\,0\,0\,0\,0\,1\,0\,0\,0\,1\,1\,0\,0\,0\,0\,1\,0\,1$
$D = 0\,1\,0\,1\,0\,0\,1\,0\,1\,0\,1\,0\,1\,0\,1\,0\,1\,0\,1$
$D = 0\,0\,0\,0\,0\,0\,1\,0\,0\,0\,1\,0\,0\,0\,0\,1\,0\,1$

6. 读入 C

$B[\text{C}] = 0\,0\,0\,0\,0\,1\,1\,0\,0\,1\,0\,1\,0\,0\,0\,1\,0\,1\,1$
$D = 0\,0\,0\,0\,0\,1\,0\,0\,0\,0\,0\,0\,0\,0\,0\,0\,0\,0\,0$
$D = 0\,0\,0\,0\,0\,1\,0\,0\,0\,0\,0\,0\,0\,0\,0\,0\,0\,0\,0$

7. 读入 G

$B[\text{G}] = 0\,0\,0\,0\,1\,0\,1\,0\,0\,0\,0\,1\,0\,0\,0\,0\,0\,0\,1$
$D = 0\,0\,0\,0\,1\,0\,0\,0\,0\,0\,0\,0\,0\,0\,0\,0\,0\,0\,0$
$D = 0\,0\,0\,0\,1\,0\,0\,0\,0\,0\,0\,0\,0\,0\,0\,0\,0\,0\,0$

8. 读入 A

$B[\text{A}] = 0\,0\,0\,1\,0\,0\,1\,0\,0\,0\,0\,1\,0\,0\,0\,0\,0\,0\,1$
$D = 0\,1\,0\,1\,0\,0\,1\,0\,1\,0\,1\,0\,1\,0\,1\,0\,1\,0\,1$
$D = 0\,0\,0\,1\,0\,0\,1\,0\,0\,0\,0\,0\,0\,0\,0\,0\,0\,0\,1$

9. 读入 T

$B[\text{T}] = 0\,0\,1\,0\,0\,1\,1\,0\,0\,0\,0\,1\,0\,0\,0\,0\,0\,1\,1$
$D = 1\,0\,1\,0\,0\,0\,0\,1\,0\,1\,0\,1\,0\,1\,0\,1\,0\,1\,0$
$D = 0\,0\,1\,0\,0\,0\,0\,0\,0\,0\,0\,1\,0\,0\,0\,0\,0\,1\,0$

10. 读入 A

$B[\text{A}] = 0\,1\,0\,0\,0\,0\,1\,0\,0\,0\,1\,1\,0\,0\,0\,0\,1\,0\,1$
$D = 0\,1\,0\,1\,0\,0\,1\,0\,1\,0\,1\,0\,1\,0\,1\,0\,1\,0\,1$
$D = 0\,1\,0\,0\,0\,0\,1\,0\,0\,0\,1\,0\,0\,0\,0\,0\,1\,0\,1$

11. 读入 T

$B[\text{T}] = 1\,0\,0\,0\,0\,1\,1\,0\,0\,1\,0\,1\,0\,0\,0\,1\,0\,1\,1$
$D = 1\,0\,1\,0\,0\,0\,0\,1\,0\,1\,0\,1\,0\,1\,0\,1\,0\,1\,0$
$D = 1\,0\,0\,0\,0\,0\,0\,0\,0\,1\,0\,1\,0\,0\,0\,1\,0\,1\,0$

$D\ \&\ DF \neq 0^{|P|}$，验证相关的模式串，发现一个匹配 ACGATAT。

12. 读入 A

$B[\text{A}] = 0\,0\,0\,0\,0\,0\,1\,0\,1\,0\,1\,1\,0\,0\,1\,0\,1\,0\,1$
$D = 0\,1\,0\,1\,0\,0\,1\,0\,1\,0\,1\,0\,1\,0\,1\,0\,1\,0\,1$
$D = 0\,0\,0\,0\,0\,0\,1\,0\,1\,0\,1\,0\,0\,0\,1\,0\,1\,0\,1$

13. 读入 T

$B[\text{T}] = 0\,0\,0\,0\,0\,1\,1\,1\,0\,1\,0\,1\,0\,1\,0\,1\,0\,1\,1$
$D = 1\,0\,1\,0\,0\,0\,0\,1\,0\,1\,0\,1\,0\,1\,0\,1\,0\,1\,0$
$D = 0\,0\,0\,0\,0\,0\,0\,1\,0\,1\,0\,1\,0\,1\,0\,1\,0\,1\,0$

$D\ \&\ DF \neq 0^{|P|}$，验证相关的模式串，发现一个匹配 TATAT。

14. 读入 A

	0 0 0 0 0 0 1 0 1 0 1 1 1 0 1 0 1 0 1
B[A]	0 1 0 1 0 0 1 0 1 0 1 0 1 0 1 0 1 0 1
$D=$	0 0 0 0 0 0 1 0 1 0 1 0 1 0 1 0 1 0 1

D & $DF \neq 0^{|P|}$，验证相关的模式串，发现一个匹配 ATATATA。

15. 读入 C

	0 0 0 0 0 1 1 1 0 1 0 1 0 1 0 1 0 1 1
B[C]	0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0
$D=$	0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0

3.2.2　基本的 Aho-Corasick 算法

Aho 和 Corasick 的算法[AC75]是 KMP 算法（参见 2.2.1 节）向多模式串情形的扩展。

该算法使用一种特殊的自动机，称做 Aho-Corasick 自动机，由 P 生成。它对 trie 进行了扩展，添加了供给函数 S_{AC}。

形式化地说，用 q 表示 P 对应的 trie 的状态，用 $L(q)$ 表示从初始状态到 q 的路径上的标号组成的字符串。$S_{AC}(q)$ 定义为自动机中的另一个状态 q'，使得 $L(q')$ 是 $L(q)$ 的最长后缀，这是将“边界”的概念（参见 2.2.1 节）扩展到一组字符串。$S_{AC}(q)$ 称为 q 的供给状态，称 q 到 $S_{AC}(q)$ 的连线为供给链，称由供给链连接而成的路径为供给路径。初始状态的供给状态为 θ。

图 3.9 所示为集合 P={ATATATA, TATAT, TCGATAT}的 Aho-Corasick 自动机。例如，在这幅图上，$L(15)$ =ACGATA，既是它的后缀，同时也是某个模式串的前缀的最长字符串是 ATA，对应于状态 7，因此 $S_{AC}(15) = 7$。终结状态是那些对应于整个模式串的状态，以及如果从状态 q 到根节点的路径上存在终结状态，那么 q 也是终结状态。例如，在图 3.9 中，因为 $S_{AC}(16)$ 是终结状态，所以 16 也是终结状态。

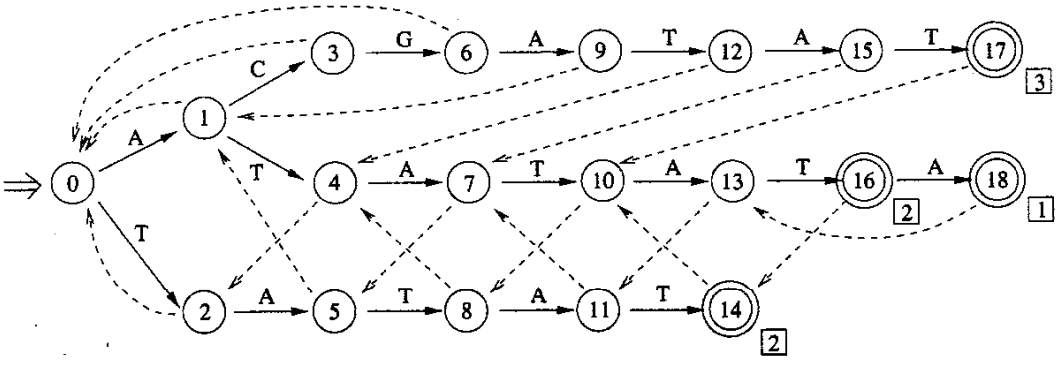


图 3.9　集合{ATATATA, TATAT, TCGATAT}的 Aho-Corasick 自动机。图中的虚线表示的是 S_{AC}，终结状态用双圈标识

假设已经读入文本的前缀 $t_1t_2\ldots t_i$，而既是 $t_1t_2\ldots t_i$ 的后缀，同时也是某个模式串的前缀的最长字符串对应于 Aho-Corasick 自动机的 *Current* 状态，记这个最长字符串为 $v = L(Current)$。当读入一个新的字符 t_{i+1} 并计算 $t_1\ldots t_it_{i+1}$ 的新的最长后缀 u 时，可能

会出现两种情况：

1. 如果状态 *Current* 存在标号为 t_{i+1} 的转移，目的状态为 f，那么 f 将成为新的 *Current* 状态。并且，$u = L(f) = ut_{i+1}$ 是 $t_1 \ldots t_{i+1}$ 的最长后缀，同时也是某个模式串的前缀。
2. 如果状态 *Current* 不存在标号为 t_{i+1} 的转移（即无法在 trie 树上继续识别字符 t_{i+1}），那么就沿着 *Current* 的供给路径回溯，直到：

   （a）找到了一个状态 q，它存在标号为 t_{i+1} 的转移。那么 q 的 t_{i+1}-转移的目的状态 f 成为新的 *Current* 状态，并且 $u = L(f)$。

   （b）如果到达了空状态 θ，那么说明要寻找的最长后缀 u 是空字符串 ε，于是从 *Current* 跳转到初始状态。

图 3.10 给出了算法的伪代码描述。搜索阶段的复杂度是很好计算的，因为沿着供给链回溯的次数不会超过已读入的文本字符数，因此，经过的供给链个数不会超过 n，经过的转移（原始的转移加上供给链）的个数不会超过 $2n$。字符比较的次数取决于转移的实现方式：如果用表实现，则时间复杂度是 $O(n + nocc)$；如果用平衡树实现，则时间复杂度是 $O(n \log |\Sigma| + nocc)$。

```
Aho-Corasick(P = {p^1, p^2, ..., p^r}, T = t_1 t_2 ... t_n)
1.   Preprocessing
2.       AC ← Build_AC(P)
3.   Searching
4.       Current ← Initial state of the automaton AC
5.       For pos ∈ 1...n Do
6.           While δ_AC(Current, t_pos) = θ AND S_AC(Current) ≠ θ Do
7.               Current ← S_AC(Current)
8.           End of while
9.           If δ_AC(Current, t_pos) ≠ θ Then
10.              Current ← δ_AC(Current, t_pos)
11.          Else Current ← initial state of AC
12.          End of if
13.          If Current is terminal Then
14.              Mark all the occurrences (F(Current), pos)
15.          End of if
16.      End of for
```

图 3.10 用 Aho-Corasick 算法搜索模式串集合 $P = \{p^1, p^2, \ldots, p^r\}$。每读入一个文本字符 t_{pos}，就计算既是 $t_1 \ldots t_{pos}$ 的后缀，同时也是某个模式串 p^k 的前缀的最长字符串

为了构造 Aho-Corasick 自动机，可以按照图 3.6 所示的算法构造 P 的 trie 结构。trie 的状态与 Aho-Corasick 自动机的状态保持一一对应，初始状态和终结状态也相应地保持不变。

然后，按照图 3.9 中的层次遍历序（transversal order）构造 S_{AC}。

假设已经按照层次遍历序计算出了 *Current* 之前所有状态的供给函数，现在考虑 *Current* 的父节点 *Parent*。假设从 *Parent* 到 *Current* 的标号为 σ，即 $Current = \delta_{AC}(Parent, \sigma)$。$S_{AC}(Parent)$ 已经计算出来了。要搜索 $v = L(Current)$ 的最长后缀 u，它同时也对应于 trie 中的一条路径。v 可以写成 $v'\sigma$ 的形式，如果 u 不是空字符串，那么 u 一定能写成 $u'\sigma$ 的形式，并且 u' 一定是 v' 的后缀。

如果 $S_{AC}(Parent)$ 有标号为 σ 的转移，并且目的状态为 h，则 $w = L(S_{AC}(Parent))$ 是 v' 的最长后缀，并且 $w\sigma$ 对应于 trie 中的一条路径。$w\sigma$ 就是最长路径 u，$S_{AC}(Current)$ 指向 h。

如果 $S_{AC}(Parent)$ 没有标号为 σ 的转移，就考虑 $S_{AC}(S_{AC}(Parent))$，依次类推。重复上述操作，直到某个状态有标号为 σ 的转移，或者抵达空状态 θ 为止。如果抵达了空状态 θ，说明 u 是空字符串 ε，这时将 $S_{AC}(Current)$ 置为初始状态。

构建 Aho-Corasick 自动机的过程与 Aho-Corasick 算法本身的搜索过程很类似，图 3.11 所示为其伪代码描述。复杂度的计算基于如下事实：回溯的供给链的个数不会超过自动机转移的数目，后者的复杂度为 $O(|P|)$。因此，总的转移数不会超过 $2 \times |P|$。与搜索阶段一样，字符比较的次数取决于自动机的转移的实现方式：如果用表实现，时间复杂度是 $O(|P|)$；如果用平衡树实现，时间复杂度是 $O(|P| \log |\Sigma|)$。

```
Build_AC(P = {p^1, p^2, ..., p^r})
1.    AC_trie ← Trie(P)
          δ_AC is its transition function
2.    Initial_state ← root of AC_trie
3.    S_AC(Initial_state) ← θ
4.    For Current in transversal order Do
5.        Parent ← parent of Current in AC_trie
6.        σ ← label of the transition from Parent to Current
7.        Down ← S_AC(Parent)
8.        While Down ≠ θ AND δ_AC(Down, σ) = θ Do
9.            Down ← S_AC(Down)
10.       End of while
11.       If Down ≠ θ Then
12.           S_AC(Current) ← δ_AC(Down, σ)
13.           If S_AC(Current) is terminal Then
14.               Mark Current as terminal
15.               F(Current) ← F(Current) ∪ F(S_AC(Current))
16.           End of if
17.       Else S_AC(Current) ← Initial_state
18.       End of if
19.   End of for
```

图 3.11　Aho-Corasick 自动机的构造过程。*Current* 以层次遍历的方式访问 *AC_trie* 的每个状态。*Down* 从 *Current* 的父节点的供给状态开始回溯，直到具有相同标号 σ 的状态为止。当 *Current* 第一次标记为终结状态时，$F(Current)$ 置为空集

英文例子 在文本“annual_announce”中搜索模式串集合 $P = \{\text{announce, annual, annually}\}$。$P$对应的 Aho-Corasick 自动机如图 3.12 所示。

$Current \leftarrow 0$.

1. 读入 a
   $Current \leftarrow 1 = \delta(0, a)$
2. 读入 n
   $Current \leftarrow 2 = \delta(1, n)$
3. 读入 n
   $Current \leftarrow 3 = \delta(2, n)$
4. 读入 u
   $Current \leftarrow 5 = \delta(3, u)$
5. 读入 a
   $Current \leftarrow 7 = \delta(5, a)$
6. 读入 l
   $Current \leftarrow 9 = \delta(7, a)$.
   状态 9 是终结状态，报告匹配 $F(9) \rightarrow$ annual。
7. 读入_
   $\delta(9, _) = \theta$，状态跳转到 $0 = S_{AC}(9)$；
   $\delta(0, _) = \theta$，状态跳转到 $\theta = S_{AC}(0)$。
   然后继续从初始状态 0 开始搜索，
   $Current \leftarrow 0$。
8. 读入 a
   $Current \leftarrow 1 = \delta(0, a)$
9. 读入 n
   $Current \leftarrow 2 = \delta(1, n)$
10. 读入 n
    $Current \leftarrow 3 = \delta(2, n)$
11. 读入 o
    $Current \leftarrow 4 = \delta(3, o)$
12. 读入 u
    $Current \leftarrow 6 = \delta(4, u)$
13. 读入 n
    $Current \leftarrow 8 = \delta(6, n)$
14. 读入 c
    $Current \leftarrow 10 = \delta(8, c)$
15. 读入 e
    $Current \leftarrow 12 = \delta(10, e)$
    状态 12 是终结状态，报告匹配 $F(12) \rightarrow$ announce。

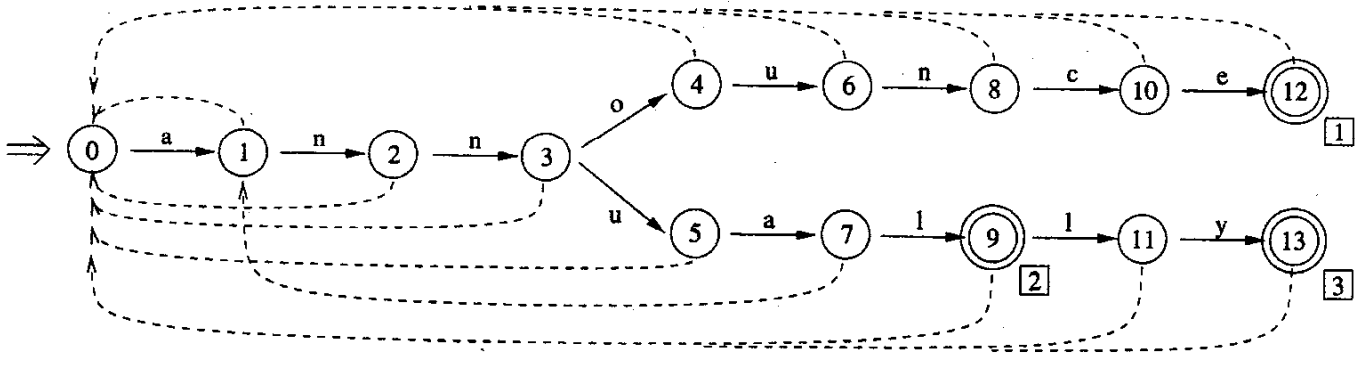


图 3.12 集合 $P = \{\text{announce, annual, annually}\}$的 Aho-Corasick 自动机。标为两个圈的状态是终结状态

DNA 例子 在序列“AGATACGATATATAC”中搜索 $P = \{\text{ATATATA, TATAT, ACGATAT}\}$。$P$对应的 Aho-Corasick 自动机已经在图 3.9 中给出。

$Current \leftarrow 0$.

1. 读入 A
   $Current \leftarrow 1 = \delta(0, A)$

2. 读入 G
   $\delta(1, G) = \theta$，状态跳转到 $0 = S_{AC}(1)$；
   $\delta(0, G) = \theta$，状态跳转到 $\theta = S_{AC}(0)$。然后继续从初始状态 0 开始搜索，$Current \leftarrow 0$。

3. 读入 A
   $Current \leftarrow 1 = \delta(0, A)$

4. 读入 T
   $Current \leftarrow 4 = \delta(1, T)$

5. 读入 A
   $Current \leftarrow 7 = \delta(4, A)$

6. 读入 C
   $\delta(7, C) = \theta$，状态跳转到 $5 = S_{AC}(7)$；
   $\delta(5, C) = \theta$，状态跳转到 $1 = S_{AC}(7)$；
   $\delta(1, C) = 3$，$Current \leftarrow 3$。

7. 读入 G
   $Current \leftarrow 6 = \delta(3, G)$

8. 读入 A
   $Current \leftarrow 9 = \delta(6, A)$

9. 读入 T
   $Current \leftarrow 12 = \delta(9, T)$

10. 读入 A
    $Current \leftarrow 15 = \delta(12, A)$

11. 读入 T
    $Current \leftarrow 17 = \delta(12, T)$，状态 17 为终结状态，报告匹配 $F(17) \rightarrow$ **ACGATAT**。

12. 读入 A
    $\delta(17, A) = \theta$，状态跳转到；$10 = S_{AC}(17)$，$\delta(10, A) = 13$，$Current \leftarrow 13$。

13. 读入 T
    $Current \leftarrow 16 = \delta(13, T)$，状态 16 为终结状态，报告匹配 $F(16) \rightarrow$ **TATAT**。

14. 读入 A
    $Current \leftarrow 18 = \delta(16, A)$，状态 18 为终结状态，报告匹配 $F(18) \rightarrow$ **ATATATA**。

15. 读入 C
    $\delta(18, C) = \theta$，状态跳转到 $13 = S_{AC}(18)$；
    $\delta(13, C) = \theta$，状态跳转到 $11 = S_{AC}(13)$；
    $\delta(11, C) = \theta$，状态跳转到 $7 = S_{AC}(11)$；
    $\delta(7, C) = \theta$，状态跳转到 $5 = S_{AC}(7)$；
    $\delta(5, C) = \theta$，状态跳转到 $1 = S_{AC}(5)$；
    $\delta(1, C) = 3$，$Current \leftarrow 3$。

3.2.3　高级的 Aho-Corasick 算法

上面的算法可以做进一步的改进。主要的做法是对供给函数模拟的状态转移预先进计算，获得一个完全的自动机（对于字母表的每个字符，所有状态都有相应的转移），称为扩展的 Aho-Corasick 自动机。

这种完全的自动机可以由供给函数计算出来。首先，对于字母表中的每个字符 σ，果 $\delta(0, \sigma) = \theta$，置 $\delta(0, \sigma) = 0$。然后，按照层次遍历的方式访问自动机，设当前状态 $Current$。如果 $\delta(Current, \sigma) = \theta$，置 $\delta(Current, \sigma) = \delta(S_{AC}(Current), \sigma)$。

这种自动机的不足之处是需要巨大的存储空间。它的空间复杂度是 $O(|P| \times |\Sigma|)$，转移的实现方式无关。在小字母表和模式串集合规模不大的情况下，这种构造是非常有

的。一个折中的处理方式是，先不构造这些新的转移，而是在匹配过程中，发现还有足够的存储空间时构造这些新的转移。著名的 UNIX 应用程序 Grep 就是这么做的。

3.3 基于后缀搜索的方法

第 2 章的实验结果表明，基于后缀的搜索方法通常比基于前缀的方法要快。因此，很自然地可以将基于后缀的搜索方法扩展到一组模式串的情形。这样的第一个算法是 Commentz-Walter 于 1979 年引入的[CW79]，是 BM 算法的直接扩展。对 Horspool 算法也可以进行扩展，但远不如单模式串那样有效。一个更好的扩展是 Wu-Manber 算法，其实际效果非常好，简单而且高效。

3.3.1 Commentz-Walter 算法的思想

Commentz-Walter 算法[CW79]是 BM 算法(参见 2.3.1 节)的自然扩展。与 Aho-Corasick 算法和后面介绍的算法相比，它的速度并不快，但是具有历史意义。因为它是第一个具有亚线性平均时间复杂度的多模式串匹配算法，并且在 UNIX 应用程序 Grep 的第二个版本中实现。现在，在应用中它不再使用，因此，这里仅介绍其基本思想。

Commentz-Walter 算法用 trie 表示 $P = \{p^1, \ldots, p^r\}$ 的反转 $P^{rv} = \{(p^1)^{rv}, \ldots, (p^r)^{rv}\}$，用它识别文本字符。为了保证不遗漏可能的成功匹配，当前位置 pos 指向 ℓmin 。对于 pos 的每个新位置，从 pos 开始，从后向前识别文本 $t_1 \ldots t_{pos}$ 的最长后缀 u ，使得 u 也是某个模式串的后缀。如果找到了一个出现，就报告一个成功匹配，然后根据在多模式串集合上扩展的 BM 的三个函数 d_1 , d_2 和 d_3 ，将当前位置向右移动。对于 trie 的每个状态，都需要计算前两个函数。当识别了最长后缀 u 并抵达状态 q 时，根据这两个函数进行移动。

- $d_1(q)$ 是使得 $u = L(q)$ 与某个模式串的子串对齐的最小移动距离 $p^j \in P$。
- $d_2(q)$ 是使得 $u = L(q)$ 的一个后缀与某个模式串的前缀相匹配的最小移动距离 $p^j \in P$。

对于字母表中的每个字符 α 和每个位置 $0 \leqslant k < \ell max$， $d_3[\alpha, k]$ 是使得位置 $pos - k$ 处的字符与模式串中的某个字符相匹配的最小移动距离 $p^j \in P$。

为了更清楚地明白这三个函数的作用，读者可以参考第 2 章中的图 2.8、图 2.9 和图 2.10，其中对 BM 的三个函数进行了具体解释。

下面考虑怎样用这三个函数来计算移动距离。假设从文本位置 pos 从后向前读入了 k

个字符，并且在自动机中抵达状态 q，那么移动距离 $s[q, pos, k]$ 由下面的公式计算：

$$s[q, pos, k] = \min \begin{cases} \max(d_1[q], d_3[t_{pos-k}, k]) \\ d_2[q] \end{cases}$$

该公式是 BM 算法窗口移动的直接扩展。因为 $d_2 \leqslant \ell min$，所以最长的跳跃距离不超过 ℓmin，这也是避免遗漏可能的成功匹配的必要条件。

Commentz-Walter 算法的最坏时间复杂度是 $O(n \times \ell max)$，但是当模式串的个数不很多时，它的平均时间复杂度是亚线性的。三个函数 d_1，d_2 和 d_3 可以在 $O(|P|)$ 的时间内计算完成。

3.3.2 Set Horspool 算法

与 BM 算法类似，Horspool 算法可以直接扩展到多模式串的情形。扩展后的新算法称为 Set Horspool，它也可以看做是对 Commentz-Walter 的简化。

Set Horspool 算法的基本原理如图 3.13 所示。为了避免遗漏可能的成功匹配，当前文本位置 pos 初始化为 ℓmin。从 pos 开始，从后向前读入文本字符。这里使用所有模式串的反转 p^{rv} 构建的 trie 来识别文本字符，如果到达了一个终结状态，就标记一个匹配。当无法继续识别读入的文本字符时，根据第一个读入的文本字符 β 来决定 pos 的移动。pos 移动到使得 β 与它在 trie 中的下一次出现位置对齐的地方。如果 β 不再重复出现，就简单地移动 ℓmin 个字符即可。

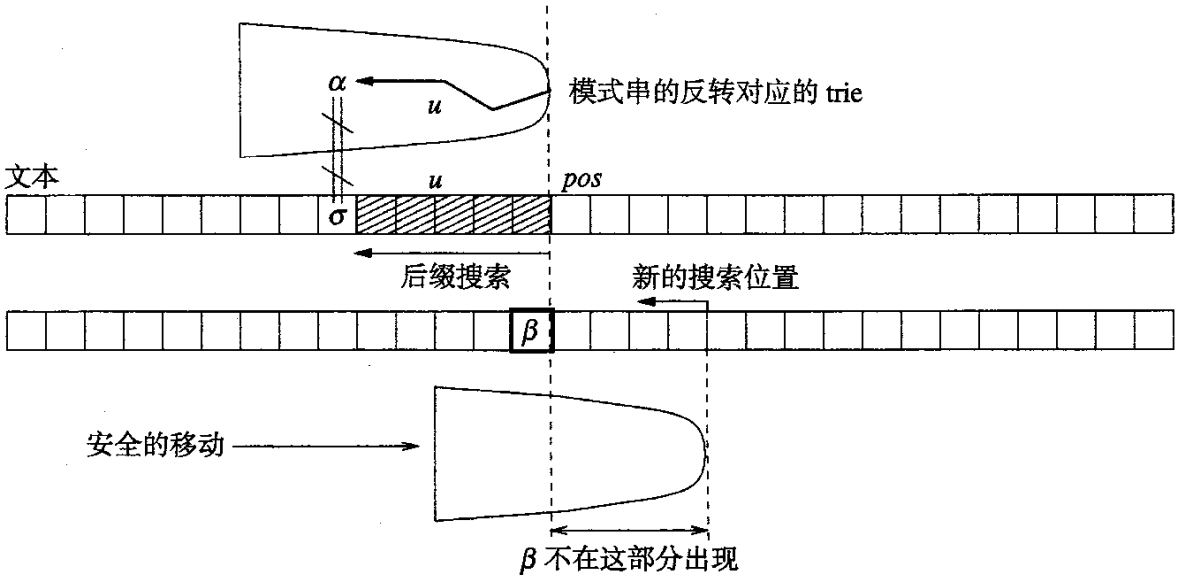


图 3.13　模式串集合上的 Horspool 算法。当前位置根据搜索窗口中的最后一个字符进行移动

Set Horspool 算法的最坏时间复杂度是 $O(n \times \ell max)$（参见图 3.14）。一般来说，它只适用于模式串集合很小并且字母表很大的场合。

Set Horspool $(P = \{p^1, p^2, \ldots, p^r\},\ T = t_1 t_2 ... t_n)$

```
1.    Preprocessing
2.        HO ← Trie(P^rv = {(p^1)^rv, ..., (p^r)^rv})
              δ_HO is its transition function
3.        For c ∈ Σ Do d[c] ← ℓmin
4.        For j ∈ 1...r Do
5.            For k ∈ 1...m_j − 1 Do d[p_k^j] ← min(d[p_k^j], m_j − k)
6.        End of for
7.    Searching
8.        pos ← ℓmin
9.        While pos ≤ n Do
10.           j ← 0, Current ← initial state of HO
11.           While pos − j > 0 AND δ_HO(t_{pos−j}, Current) ≠ θ Do
12.               If Current is terminal Then
13.                   Mark all the occurrences (F(Current), pos)
14.               End of if
15.               Current ← δ_HO(t_{pos−j}, Current)
16.               j ← j + 1
17.           End of while
18.           pos ← pos + d[t_pos]
19.       End of while
```

图 3.14　模式串集合上的 Horspool 算法。根据第一个读入的字符 t_{pos} 决定移动的距离

英文例子　在文本“CPM_annual_conference_announce”中搜索模式串集合 P={announce, annual, annually}。模式串的反转对应的 trie 如图 3.15 所示。

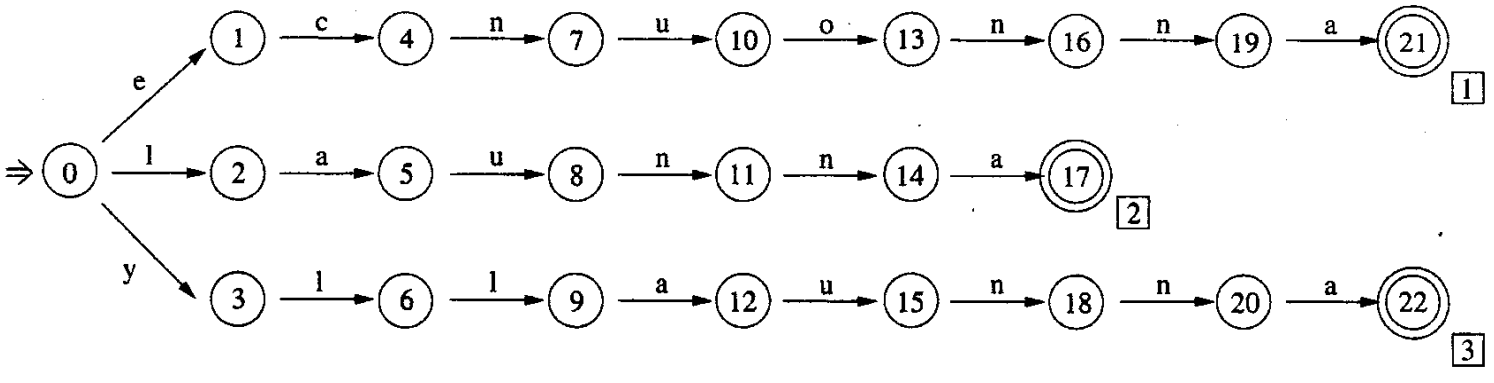


图 3.15　P = {announce, annual, annually}的反转 P^{rv} = {ecnuonna, launna, yllaunna}对应的 trie。标为两个圈的状态是终结状态

$\ell min = 6$,

$d =$

a	c	e	l	n	o	u	y	*
1	1	6	1	2	4	2	6	6

1. CPM_a [n] nual_conference_announce
   n $\notin$ {e, l, y}, d[n] = 2

2. CPM_ann [u] al_conference_announce
   u $\notin$ {e, l, y}, d[u] = 2

3. CPM_annua [l] _conference_announce
   在 trie 中依次识别文本字符 l, a, u, n, n, a, 到达终结状态 17, 报告匹配 $F(17) \rightarrow$ annual。
   我们利用第一个已经读入的字符进行移动，移动距离为 d[l]=1。

4. CPM_annual [_] conference_announce
   _ $\notin$ {e, l, y}, d[_] = 6

5. CPM_annual_confe [r] ence_announce
   r $\notin$ {e, l, y}, d[r] = 6

6. CPM_annual_conference_ [a] nnounce
   a $\notin$ {e, l, y}, d[a] = 1

7. CPM_annual_conference_a [n] nounce
   n $\notin$ {e, l, y}, d[n] = 2

8. CPM_annual_conference_ann [o] unce
   o $\notin$ {e, l, y}, d[o] = 4

9. CPM_annual_conference_announc [e]
   在 trie 中依次识别文本字符 e, c, n, u, o, n, n, a, 到达终结状态 21, 报告匹配 $F(21) \rightarrow$ announce。

DNA 例子　在序列“AGATACGATATATAC”中搜索模式串集合 P = {ATATATAT, ACGATAT}。模式串的反转对应的 trie 如图 3.16 所示。

$\ell min = 5$,

$\delta =$

A	C	G	T	*
1	5	4	1	5

1. AGAT [A] CGATATATAC
   在 trie 中依次识别文本字符 A, T, A, 无法继续识别下一个字符 G; 根据第一个读入的文本字符 A 来进行移动, d[A] = 1。

2. AGATA [C] GATATATAC
   C $\notin$ {A, T}, d[C] = 5。

3. AGATACGATA [T] ATAC
   在 trie 中依次识别文本字符 T, A, T, A, G, C, A, 到达终结状态 15, 报告匹配 $F(15) \rightarrow$ ACGATAT; 根据第一个读入的文本字符 T 来进行移动, d[T] = 1。

4. AGATACGATATA [C] ATAC
   在 trie 中依次识别文本字符 C, G, A, T, A, 无法继续识别下一个字符 G; 根据第一个读入的文本字符 A 来进行移动, d[A] = 1。

5. AGATACGATATA [T] AC
   在 trie 中依次识别文本字符 T, A, T, A, T, 到达终结状态 11, 报告匹配 $F(11) \rightarrow$ TATAT; 根据第一个读入的文本字符 T 来进行移动, d[T] = 1。

6. AGATACGATATAT [A] C
   在 trie 中依次识别文本字符 A, T, A, T, A, T, A, 到达终结状态 14, 报告匹配 $F(14) \rightarrow$ ATATATA; 根据第一个读入的文本字符 A 来进行移动, d[A] = 1。

7. AGATACGATATATA [C]

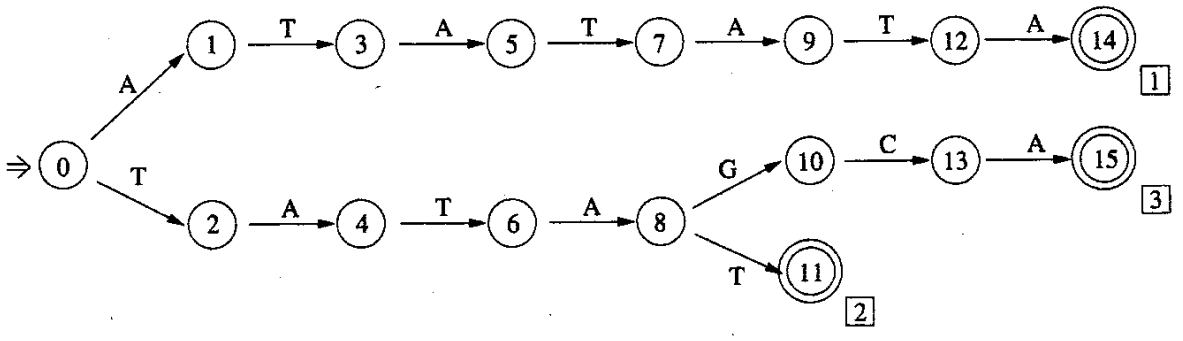


图 3.16 P = {ATATATA, TATAT, ACGATAT}的反转 P^{rv} = {ATATATA, TATAT, TATAGCA}对应的 trie。标为两个圈的状态是终结状态

3.3.3 Wu-Manber 算法

扩展的 Horspool 算法在多模式集合上性能很差，这是因为字母表中的每个字符通常都以高概率出现在某个模式串中，从而导致移动距离下降。

Wu 和 Manber 的算法[WM94]克服了这个问题。它通过读入一块字符，以降低字符块在某个模式串中的出现概率。这里设块的长度为 B。问题的难点在于，如果 B 比较大，总共可能会有 $|\Sigma|^B$ 个不同的块，这将需要非常大的存储空间。

Wu 和 Manber 使用散列函数 h_1 将所有可能的块散列到一个有限的表 $SHIFT$ 上。两个不同的块可能散列到 $SHIFT$ 中的同一个位置。对于每个新的位置，如果读入一块字符 Bl 而不是像 Horspool 那样只读入最后一个字符，则必须确保 Bl 的移动距离 $SHIFT(h_1(Bl))$ 是安全的。为了保证这一点，算法在 $SHIFT(j)$ 中存入满足 $j = h_1(Bl)$ 的所有块的移动距离的最小值。更精确地说，是按照如下方式构建 $SHIFT$ 表：

- 如果块 Bl 不出现在 P 中的任何一个模式串中，则可以向右安全地移动 $\ell min - B + 1$ 个字符。因此，将表的每一项都初始化为 $\ell min - B + 1$。
- 如果块 Bl 出现在 P 中的某个一个模式串 p^i 中，则需要找出 Bl 在 p^i 中最右边出现的末尾位置 j，然后将 $SHIFT(h_1(Bl))$ 置为 $m_i - j$。为了计算 $SHIFT$ 表的所有值，对于每个模式串 $p^i = p^i_1 \cdots p^i_{m_i}$ 的每个块 $B = p^i_{j-B+1} \cdots p^i_j$，都需要在 $SHIFT$ 中找出对应的表项 $h_1(B)$，并将 $SHIFT(h_1(B))$ 置为它的当前值和 $m_i - j$ 中的较小者。

块长 B 取决于最短关键词的长度 ℓmin、模式串集合的大小以及字母表的大小。Wu 和 Manber 的研究表明，取 $B = \log_{|\Sigma|}(2 \times \ell min \times r)$ 能够产生最好的实验结果。$SHIFT$ 的大小也可以随着可用空间的大小而变化。

一个散列表 $HASH$ 和散列函数 h_2，它的每个表项 $HASH(j)$ 是一个链表，记录了最后一块在 h_2 下映射到 j 的所有模式串。这样就可以找出那些最后一块的散列值与当前读入的本块 Bl 散列值相同的所有模式串。

搜索阶段与 Set Horspool 算法类似，将当前位置 pos 初始化为 ℓmin。对于每个当前位置 pos，从后向前读入 B 个字符的块 Bl。如果 $j = SHIFT(h_1(Bl)) > 0$，那么将窗口移动到位置 $pos + j$，并继续搜索；如果 $SHIFT(h_1(Bl)) = 0$，则用 $HASH$ 找出文本块对应的一组模式串 $HASH(h_2(Bl))$，并逐个与文本进行比较。该算法的伪代码如图 3.17 所示。

```
Wu-Manber(P = {p^1, p^2, ..., p^r},  T = t_1 t_2 ... t_n)
1.    Preprocessing
2.        Computation of B
3.        Construction of the hash tables SHIFT and HASH
4.    Searching
5.        pos ← ℓmin
6.        While pos ≤ n Do
7.            i ← h_1(t_{pos-B+1} ... t_{pos})
8.            If SHIFT[i] = 0 Then
9.                list ← HASH[ h_2(t_{pos-B+1} ... t_{pos}) ]
10.               Verify all the patterns in list one by one against the text
11.               pos ← pos+1
12.           Else pos ← pos + SHIFT[i]
13.           End of if
14.       End of while
```

图 3.17　模式串集合上的 Wu-Manber 算法

文献[WM94]中对原始算法的描述相当模糊，既没有说明如何计算 $SHIFT$ 和 $HASH$ 小的最佳值，也没有说明如何选择合适的散列函数。在实际应用中，只要参数设置得当，这个算法是相当快的。Wu-Manber 算法已经在 Agrep（参见 7.1.2 节）中实现。

下面介绍两个运行的例子。Wu-Manber 算法使用了很多散列函数和散列表，很难一一表示。在下面的例子中，没有使用真实的散列函数，但足以阐明微妙的细节。其中设 $B =$

英文例子　在文本“CPM_annual_conference_announce”中搜索模式串集 $P = \{\text{announce, annual, annually}\}$。

$SHIFT[Bl] =$

字符串	ll	no ou	an	un nc	ua al	ly	nn nu	ce	*
移动距离	1	3	4	1	0	0	2	0	5

1. CPM_ [an] nual_conference_announce
   $SHIFT$[an] = 4.

2. CPM_annu [al] _conference_announce
   $SHIFT$[al] = 0. $L = HASH$[al] = {2}.
   将 p^2 与文本进行比较，p^2 与文本匹配，于是报告一个成功匹配；然后向右移动一个字符。

3. CPM_annua [l_] conference_announce
   $SHIFT$[l_] = 5.

4. CPM_annual_con [fe] rence_announce
   $SHIFT$[fe] = 5.

5. CPM_annual_conferen [ce] _announce
   $SHIFT$[ce] = 0. $L = HASH$[ce] = {3, 1}.
   将 p^1 和 p^3 与文本进行比较，发现这两个模式串与文本皆不匹配；然后向右移动一个字符。

6. CPM_annual_conferenc [e_] announce
   $SHIFT$[e_] = 5.

7. CPM_annual_conference_ann [ou] nce
   $SHIFT$[ou] = 3.

8. CPM_annual_conference_announ [ce]
   $SHIFT$[ce] = 0. $L = HASH$[ce] = {3, 1}.
   将 p^1 和 p^3 与文本进行比较，p^1 与文本匹配，于是报告一个成功匹配。

DNA 例子　在序列“AGATACGATATATAC”中搜索模式串集合 P = {ATATATA, TATAT, ACGATAT}。

$SHIFT[Bl]$ =

字符串	GA TA	AT	CG GA	AC	*
移动距离	0	0	3	4	4

$HASH[Bl]$ =

字符串	TA	AT	*
P 中字符串的序号	1	2, 3	∅

1. AGA [TA] CGATATATAC
   $SHIFT$[TA] = 0. $L = HASH$[TA] = {1}.
   将 p^1 与文本进行比较，发现模式串与文本不匹配；然后向右移动一个字符。

2. AGAT [AC] GATATATAC
   $SHIFT$[AC] = 4.

3. AGATACGA [TA] TATAC
   $SHIFT$[TA] = 0. $L = HASH$[TA] = {1}.
   将 p^1 与文本进行比较，发现模式串与文

4. AGATACGAT [AT] ATAC
   $SHIFT$[AT] = 0. $L = HASH$[AT] = {2, 3}.
   将 p^2 和 p^3 与文本进行比较，p^3 与文本匹配，于是报告一个成功匹配；然后向右移动一个字符。

5. AGATACGATA [TA] TAC
   $SHIFT$[TA] = 0. $L = HASH$[TA] = {1}.
   将 p^1 与文本进行比较，发现模式串与文

6. AGATACGATAT [AT] AC
   $SHIFT$[AT] = 0. $L = HASH$[AT] = $\{2,3\}$.

   将 p^2 和 p^3 与文本进行比较，p^2 与文本匹配，于是报告一个成功匹配；然后向右移动一个字符。

7. AGATACGATATA [TA] C
   $SHIFT$[TA] = 0. $L = HASH$[TA] = $\{1\}$.

   将 p^1 与文本进行比较，p^1 与文本匹配，于是报告一个成功匹配；然后向右移动一个字符。

8. AGATACGATATAT [AC]
   $SHIFT$[AC] = 4.

3.4 基于子串搜索的方法

一般的基于子串的方法也可以直接扩展到一组模式串的情形。在当前文本位置，从后向前搜索文本的最长后缀 u，u 同时也是 P 中某个模式串的子串。如果无法继续识别字符 σ，那么 σu 一定不是某个字符串的子串，因此 P 中的模式串都不会覆盖 σu。

然而，有两个技术问题需要解决。第一个问题是，如何安全地移动模式串，以避免遗漏可能的成功匹配。第二个问题是，如何识别一组字符串的子串。

最早的基于子串搜索的两个算法分别是 Dawg-Match[CCG+93, CCG+99]和 MultiBDM [CR94, Raf97]，它们的目标是使得算法在平均情况和最坏情况下都较好。实际上，它们的最坏时间复杂度都是线性的，但是算法过于复杂，实际性能很差。本章的目的是介绍简单而有效的算法，所以不再具体介绍它们。

另外两种基于子串搜索的算法是 Set Backward Dawg Matching（SBDM）和 Set Backward Oracle Matching（SBOM）[AR99]，它们分别是对 BDM 和 BOM 的扩展（参见第 2 章）。这里只介绍 SBDM 的基本思想，然后在此基础上再介绍 SBOM 算法。

位并行方法只适用于模式串集合很小的情形。然而，与 Multiple Shift-And 算法（参见 3.2.1 节）一样，它可以有效地处理扩展串匹配（参见第 4 章）和近似匹配（参见第 6 章）。下面首先介绍它。

3.4.1 Multiple BNDM 算法

当 $r \times \ell min$ 不超过几个机器字长时，用位并行方法搜索一组模式串 $P = \{p^1, \ldots, p^r\}$ 是非常高效的[NR00]。为了简单起见，这里假设 $r \times \ell min \leqslant w$。

为了移动得更远，算法只保留每个模式串长度为 ℓmin 的前缀。如果前缀成功匹配了，则需要将整个模式串与文本进行比较验证。

与 BNDM 类似（参见 2.4.2 节），这里也将所有的前缀像图 3.18 所示那样连接起来，以同时搜索所有的前缀。唯一的区别是：每次移位后，都需要将一些比特位清零，防止在搜 p^i 时对 p^{i+1} 形成干扰，清零掩码 CL 如图 3.19 中所示。变量 $last$ 同样被使用，只不过现在表示的位置是其中某一个模式串的前缀开始的地方。整个算法的伪代码如图 3.19 所示。

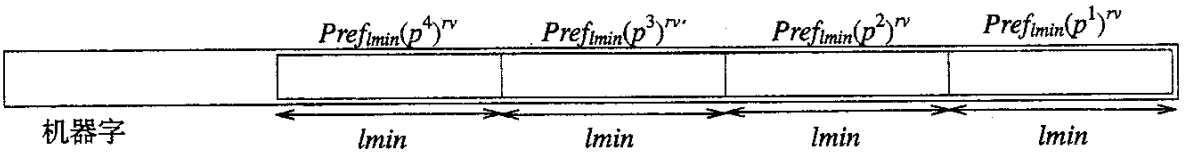


图 3.18 Multiple BNDM 算法。总长度 $r \times \ell min$ 必须不超过 w。$pref_{\ell min}(p^i)$ 表示 p^i 的长度为 ℓmin 的前缀

Multiple BNDM ($p = p_1 p_2 \ldots p_m$, $T = t_1 t_2 \ldots t_n$)

```
1.  Preprocessing
2.      For c ∈ Σ Do B[c] ← 0^|P|
3.      ℓ ← 0
4.      For k ∈ 1...r Do
5.          ℓ ← ℓ + ℓmin
6.          For j ∈ 1...ℓmin Do B[p_j^k] ← B[p_j^k] | 0^(|P|−ℓ+j−1) 10^(ℓ−j)
7.      End of for
8.      CL ← (1^(ℓmin−1) 0)^r
9.      DF ← (10^(ℓmin−1))^r
10. Searching
11.     pos ← 0
12.     While pos ≤ n − m Do
13.         j ← ℓmin, last ← ℓmin
14.         D = 1^|P|
15.         While D ≠ 0^|P| Do
16.             D ← D & B[t_(pos+j)]
17.             j ← j − 1
18.             If D & DF ≠ 0^|P| Then
19.                 If j > 0 Then last ← j
20.                 Else        /* at least one prefix matches */
21.                     Check which prefixes of length ℓmin match
                            p^i needs to be checked if
                                D & 0^(|P|−ℓmin×i) 10^(ℓmin−1) 0^(ℓmin×(i−1)) ≠ 0^|P|
22.                     Verify the corresponding string(s) against the text
23.                     Report the occurrence(s) at pos + 1
24.                 End of if
25.             End of if
26.             D ← (D << 1) & CL      /* Shifting and cleaning */
27.         End of while
28.         pos ← pos + last
29.     End of while
```

图 3.19 Multiple BNDM 使用的位并行方法

英文例子　在文本“CPM_annual_conference_announce”中搜索模式串集合$P=\{$announce, annual, annually$\}$。

$$B=\begin{cases}\texttt{a} & 1\,0\,0\,0\,1\,0\,1\,0\,0\,0\,1\,0\,1\,0\,0\,0\,0\,0\\ \texttt{l} & 0\,0\,0\,0\,0\,1\,0\,0\,0\,0\,0\,1\,0\,0\,0\,0\,0\,0\\ \texttt{n} & 0\,1\,1\,0\,0\,0\,0\,1\,1\,0\,0\,0\,0\,1\,1\,0\,0\,1\\ \texttt{o} & 0\,0\,0\,0\,0\,0\,0\,0\,0\,0\,0\,0\,0\,0\,0\,0\,1\,0\\ \texttt{u} & 0\,0\,0\,1\,0\,0\,0\,0\,0\,1\,0\,0\,0\,0\,0\,0\,1\,0\\ \texttt{*} & 0\,0\,0\,0\,0\,0\,0\,0\,0\,0\,0\,0\,0\,0\,0\,0\,0\,0\end{cases}$$

$$DF = 1\,0\,0\,0\,0\,0\,1\,0\,0\,0\,0\,0\,1\,0\,0\,0\,0\,0$$
$$CL = 1\,1\,1\,1\,1\,0\,1\,1\,1\,1\,1\,0\,1\,1\,1\,1\,1\,0$$

1. CPM_an nual_conference_announce
   $last \leftarrow 6$.
   $D \leftarrow 1\,1\,1\,1\,1\,1\,1\,1\,1\,1\,1\,1\,1\,1\,1\,1\,1\,1$

	1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
B[n]	0 1 1 0 0 0 0 1 1 0 0 0 0 1 1 0 0 1
$D=$	0 1 1 0 0 0 0 1 1 0 0 0 0 1 1 0 0 1

	1 1 0 0 0 0 1 1 0 0 0 0 1 1 0 0 1 0
B[a]	1 0 0 0 1 0 1 0 0 0 1 0 1 0 0 0 0 0
$D=$	1 0 0 0 0 0 1 0 0 0 0 0 1 0 0 0 0 0

   D & $DF \neq 0^{|P|}$ 并且 $j>0$, 于是 $last \leftarrow 4$。

	0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
B[_]	0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
$D=$	0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0

2. CPM_ annual _conference_announce
   $last \leftarrow 6$.
   $D \leftarrow 1\,1\,1\,1\,1\,1\,1\,1\,1\,1\,1\,1\,1\,1\,1\,1\,1\,1$

	1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
B[l]	0 0 0 0 0 1 0 0 0 0 0 1 0 0 0 0 0 0
$D=$	0 0 0 0 0 1 0 0 0 0 0 1 0 0 0 0 0 0

	0 0 0 0 1 0 0 0 0 0 1 0 0 0 0 0 0 0
B[a]	1 0 0 0 1 0 1 0 0 0 1 0 1 0 0 0 0 0
$D=$	0 0 0 0 1 0 0 0 0 0 1 0 0 0 0 0 0 0

	0 0 0 1 0 0 0 0 0 1 0 0 0 0 0 0 0 0
B[u]	0 0 0 1 0 0 0 0 0 1 0 0 0 0 0 0 1 0
$D=$	0 0 0 1 0 0 0 0 0 1 0 0 0 0 0 0 0 0

	0 0 1 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0
B[n]	0 1 1 0 0 0 0 1 1 0 0 0 0 1 1 0 0 1
$D=$	0 0 1 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0

	0 1 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0
B[n]	0 1 1 0 0 0 0 1 1 0 0 0 0 1 1 0 0 1
$D=$	0 1 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0

	1 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0
B[a]	1 0 0 0 1 0 1 0 0 0 1 0 1 0 0 0 0 0
$D=$	1 0 0 0 0 0 1 0 0 0 0 0 0 0 0 0 0 0

   D & $DF \neq 0^{|P|}$ 并且 $j=0$, 于是将模式串“annual”和“annually”与文本进行比较，发现“annual”匹配成功。

3. CPM_annual _confe rence_announce
   $last \leftarrow 6$.
   $D \leftarrow 1\,1\,1\,1\,1\,1\,1\,1\,1\,1\,1\,1\,1\,1\,1\,1\,1\,1$

	1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
B[e]	0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
$D=$	0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0

4. CPM_annual_confe rence_ announce
   $last \leftarrow 6$.
   $D \leftarrow 1\,1\,1\,1\,1\,1\,1\,1\,1\,1\,1\,1\,1\,1\,1\,1\,1\,1$

	1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
B[_]	0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
$D=$	0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0

5. CPM_annual_conference_ announ ce
   $last \leftarrow 6$.
   $D \leftarrow 1\,1\,1\,1\,1\,1\,1\,1\,1\,1\,1\,1\,1\,1\,1\,1\,1\,1$

	1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
B[n]	0 1 1 0 0 0 0 1 1 0 0 0 0 1 1 0 0 1
$D=$	0 1 1 0 0 0 0 1 1 0 0 0 0 1 1 0 0 1

```
      1100001100001100 10
B[u]  0001000001000000 10
D =   0000000000000000 10

      0000000000000001 00
B[o]  0000000000000001 00
D =   0000000000000001 00

      0000000000000010 00
B[n]  0110000110000110 01
D =   0000000000000010 00

      0000000000000100 00
B[n]  0110000110000110 01
D =   0000000000000100 00

      0000000000000100 00
B[a]  1000101000101000 00
D =   0000000000000100 00
```

$D \,\&\, DF \neq 0^{|P|}$ 并且 $j = 0$，于是将模式串“announce”与文本进行比较，发现“announce”匹配成功。

窗口移动后 $pos > n - \ell min$，搜索过程结束。

DNA 例子 在序列“AGATACGATATATAC”中搜索模式串集合 $P = \{$ATATATA, TATAT, ACGATAT$\}$。

$$B = \begin{cases} \texttt{A} & 1010101010100010 \\ \texttt{C} & 0000000000001000 \\ \texttt{G} & 0000000000000100 \\ \texttt{T} & 0101010101000001 \\ * & 0000000000000000 \end{cases}$$

$DF = 1000010000010000$

$CL = 1111011111011110$

1. AGATA CGATATATAC

$last \leftarrow 5$.

$D \leftarrow 1111111111111111$

```
      1111111111111111
B[A]  1010101010100010
D =   1010101010100010
```

$D \,\&\, DF \neq 0^{|P|}$ 并且 $j > 0$, 于是 $last \leftarrow 4$。

```
      0101010100000100
B[T]  0101010101000001
D =   0101010100000000
```

$D \,\&\, DF \neq 0^{|P|}$ 并且 $j > 0$, 于是 $last \leftarrow 3$。

```
      1010001000000000
B[A]  1010101010100010
D =   1010001000000000
```

$D \,\&\, DF \neq 0^{|P|}$ 并且 $j > 0$, 于是 $last \leftarrow 2$。

```
      0100010000000000
B[G]  0000000000000100
D =   0000000000000000
```

2. AG ATACG ATATATAC

$last \leftarrow 5$.

$D \leftarrow 1111111111111111$

```
      1111111111111111
B[G]  0000000000000100
D =   0000000000000100

      0000000000001000
B[C]  0000000000001000
D =   0000000000001000

      0000000000010000
B[A]  1010101010100010
D =   0000000000010000
```

$D \,\&\, DF \neq 0^{|P|}$ 并且 $j > 0$, 于是 $last \leftarrow 2$。

	0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
B[T]	0 1 0 1 0 1 0 1 0 1 0 0 0 0 1
$D=$	0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0

3. AGAT [ACGAT] ATATAC

$last \leftarrow 5$.
$D \leftarrow 1111111111111111$

	1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
B[T]	0 1 0 1 0 1 0 1 0 1 0 0 0 0 1
$D=$	0 1 0 1 0 1 0 1 0 1 0 0 0 0 1

D & $DF \neq 0^{|P|}$ 并且 $j > 0$, 于是 $last \leftarrow 4$。

	1 0 1 0 0 0 1 0 1 0 0 0 0 1 0
B[A]	1 0 1 0 1 0 1 0 1 0 0 1 0
$D=$	1 0 1 0 0 0 1 0 1 0 0 0 0 1 0

D & $DF \neq 0^{|P|}$ 并且 $j > 0$, 于是 $last \leftarrow 3$。

	0 1 0 0 0 1 0 1 0 0 0 0 1 0 0
B[G]	0 0 0 0 0 0 0 0 0 0 0 0 1 0 0
$D=$	0 0 0 0 0 0 0 0 0 0 0 0 1 0 0

	0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0
B[C]	0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0
$D=$	0 0 0 0 0 0 0 0 0 0 0 0 1 0 0 0

	0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0
B[A]	1 0 1 0 1 0 1 0 1 0 0 1 0
$D=$	0 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0

D & $DF \neq 0^{|P|}$ 并且 $j = 0$，于是将模式串 ACGATAT 与文本进行比较，发现匹配成功。

4. AGATACG [ATATA] TAC

$last \leftarrow 5$.
$D \leftarrow 1111111111111111$

	1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
B[A]	1 0 1 0 1 0 1 0 1 0 0 1 0
$D=$	1 0 1 0 1 0 1 0 1 0 0 1 0

D & $DF \neq 0^{|P|}$ 并且 $j > 0$, 于是 $last \leftarrow 4$。

	0 1 0 1 0 1 0 1 0 0 0 0 1 0 0
B[T]	0 1 0 1 0 1 0 1 0 1 0 0 0 0 1
$D=$	0 1 0 1 0 1 0 1 0 0 0 0 0 0 0

D & $DF \neq 0^{|P|}$ 并且 $j > 0$, 于是 $last \leftarrow 3$。

	1 0 1 0 0 0 1 0 0 0 0 0 0 0 0
B[A]	1 0 1 0 1 0 1 0 1 0 0 1 0
$D=$	1 0 1 0 0 0 1 0 0 0 0 0 0 0 0

D & $DF \neq 0^{|P|}$ 并且 $j > 0$, 于是 $last \leftarrow 2$。

	0 1 0 0 0 1 0 0 0 0 0 0 0 0 0
B[T]	0 1 0 1 0 1 0 1 0 1 0 0 0 0 1
$D=$	0 1 0 0 0 1 0 0 0 0 0 0 0 0 0

D & $DF \neq 0^{|P|}$ 并且 $j > 0$, 于是 $last \leftarrow 1$。

	1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
B[A]	1 0 1 0 1 0 1 0 1 0 0 1 0
$D=$	1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0

D & $DF \neq 0^{|P|}$ 并且 $j = 0$，于是将模式串 ATATATA 与文本进行比较，发现匹配成功。

5. AGATACGA [TATAT] AC

$last \leftarrow 5$.
$D \leftarrow 1111111111111111$

	1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
B[T]	0 1 0 1 0 1 0 1 0 1 0 0 0 0 1
$D=$	0 1 0 1 0 1 0 1 0 1 0 0 0 0 1

D & $DF \neq 0^{|P|}$ 并且 $j > 0$, 于是 $last \leftarrow 4$。

	1 0 1 0 0 0 1 0 1 0 0 0 0 1 0
B[A]	1 0 1 0 1 0 1 0 1 0 0 1 0
$D=$	1 0 1 0 0 0 1 0 1 0 0 0 0 1 0

D & $DF \neq 0^{|P|}$ 并且 $j > 0$, 于是 $last \leftarrow 3$。

	0 1 0 0 0 1 0 1 0 0 0 0 1 0 0
B[T]	0 1 0 1 0 1 0 1 0 1 0 0 0 0 1
$D=$	0 1 0 0 0 1 0 1 0 0 0 0 0 0 0

D & $DF \neq 0^{|P|}$ 并且 $j > 0$, 于是 $last \leftarrow 2$。

	1 0 0 0 0 0 1 0 0 0 0 0 0 0 0
B[A]	1 0 1 0 1 0 1 0 1 0 0 1 0
$D=$	1 0 0 0 0 0 1 0 0 0 0 0 0 0 0

D & $DF \neq 0^{|P|}$ 并且 $j > 0$, 于是 $last \leftarrow 1$。

```
        0 0 0 0 0 1 0 0 0 0 0 0 0 0 0
B[T]    0 1 0 1 0 1 0 1 0 1 0 0 0 0 1
D =     0 0 0 0 0 1 0 0 0 0 0 0 0 0 0
```

D & $DF \neq 0^{|P|}$ 并且 $j = 0$, 于是将模式串 TATAT 与文本进行比较，发现匹配成功。

6. AGATACGAT **ATATA** C
   $last \leftarrow 5$.
   $D \leftarrow 1\,1\,1\,1\,1\,1\,1\,1\,1\,1\,1\,1\,1\,1\,1$

```
        1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
B[A]    1 0 1 0 1 0 1 0 1 0 0 1 0
D =     1 0 1 0 1 0 1 0 1 0 0 1 0
```

D & $DF \neq 0^{|P|}$ 并且 $j > 0$, 于是 $last \leftarrow 4$。

```
        0 1 0 1 0 1 0 1 0 0 0 0 1 0 0
B[T]    0 1 0 1 0 1 0 1 0 1 0 0 0 0 1
D =     0 1 0 1 0 1 0 1 0 0 0 0 0 0 0
```

D & $DF \neq 0^{|P|}$ 并且 $j > 0$, 于是 $last \leftarrow 3$。

```
        1 0 1 0 0 0 1 0 0 0 0 0 0 0 0
B[A]    1 0 1 0 1 0 1 0 1 0 0 1 0
D =     1 0 1 0 0 0 1 0 0 0 0 0 0 0 0
```

D & $DF \neq 0^{|P|}$ 并且 $j > 0$, 于是 $last \leftarrow 2$。

```
        0 1 0 0 0 1 0 0 0 0 0 0 0 0 0
B[T]    0 1 0 1 0 1 0 1 0 1 0 0 0 0 1
D =     0 1 0 0 0 1 0 0 0 0 0 0 0 0 0
```

D & $DF \neq 0^{|P|}$ 并且 $j > 0$, 于是 $last \leftarrow 1$。

```
        1 0 0 0 0 0 0 0 0 0 0 0 0 0 0
B[A]    1 0 1 0 1 0 1 0 1 0 0 1 0
D =     1 0 0 0 0 0 0 0 0 0 0 0 0 0 0
```

D & $DF \neq 0^{|P|}$ 并且 $j = 0$, 于是将模式串 ATATATA 与文本进行比较，但是匹配不成功。

7. AGATACGATA **TATAC**
   $last \leftarrow 5$.
   $D \leftarrow 1\,1\,1\,1\,1\,1\,1\,1\,1\,1\,1\,1\,1\,1\,1$

```
        1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
B[C]    0 0 0 0 0 0 0 0 0 0 0 0 1 0 0
D =     0 0 0 0 0 0 0 0 0 0 0 0 1 0 0
        0 0 0 0 0 0 0 0 0 0 0 1 0 0 0
B[A]    1 0 1 0 1 0 1 0 1 0 0 1 0
D =     0 0 0 0 0 0 0 0 0 0 0 1 0 0 0
```

D & $DF \neq 0^{|P|}$ 并且 $j > 0$, 于是 $last \leftarrow 3$。

```
        0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
B[T]    0 1 0 1 0 1 0 1 0 1 0 0 0 0 1
D =     0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
```

2.4.2 SBDM 算法的思想

Set Backward Dawg Matching（SBDM）使用后缀自动机在长度为 ℓmin 的文本窗口中从后向前识别模式串的子串以进行窗口的移动。

模式串集合的后缀自动机

使用模式串集合P的后缀自动机可以识别P中模式串的所有后缀[BBE+87]。记 γ 对应的 trie 的状态数为 P $(\gamma \leqslant |P| + 1)$，那么后缀自动机的状态数在 γ 和 2γ 之间，它的状态转移数也是 $O(\gamma)$。

多模式串后缀自动机的构造方法是对单模式串后缀自动机的直接扩展（参见 2.4.1 节），但是构造的自动机不一定是最小的。构造过程的复杂度为 $O(|P|)$，但是构造过程复杂，构造速度缓慢。

搜索算法

算法使用的后缀自动机建立在所有模式串的长度为 ℓmin 的前缀的反转 $P^{rv}_{\ell min}$ 上，构造时间复杂度为 $O(r \times \ell min)$ [BBE+87]。搜索在大小为 ℓmin 的窗口中进行，窗口沿着文本移动。在当前窗口中，从后向前识别文本的最长后缀，它同时也是 P 中模式串的长度为 ℓmin 的前缀的子串。搜索过程中，可能出现下面两种情况：

（i）无法继续识别子串，即在自动机中无法继续识别文本字符 σ。这时，没有模式串的前缀能够完全覆盖窗口内已读入的文本字符。因此，可以安全地将当前窗口移动到字符 σ 之后。

（ii）到达了窗口的起始位置，并且抵达了自动机的状态 q。这意味着已经识别出 $F(q)$ 中某个字符串的前缀 $L(q)$（参见 3.1 节），因此需要将 $F(q)$ 中的每个模式串与当前文本进行比较验证并报告成功的匹配。然后，将窗口向右移动一个字符，开始新的搜索。

SBDM 的最坏时间复杂度是 $O(n \times |P|)$，这非常高。然而，当字母表不是太小的时候，该算法的平均复杂度是亚线性的。在实际应用中，SBDM 的后缀自动机构建代价很大。对于很大的模式串集合，构建的时间很难被搜索阶段的时间分摊抵消。而且，随着模式串规模的增大，后缀自动机耗用的存储空间也急剧增大。这里不对该方法再进行深入的介绍，也不准备给出伪代码描述，因为 SBOM 算法使用同样的搜索方法并克服了后缀自动机这个瓶颈，它使用了一种更简单的轻量级的数据结构。在所有情形下，SBOM 都比 SBDM 要快。

需要说明的是，SBDM 也可以像 BDM 那样，使用一个 $last$ 变量来做进一步的改进（参见 2.4.1 节）。

3.4.3 SBOM 算法

Set Backward Oracle Matching（SBOM）算法[AR99]使用了 Factor Oracle 结构。模式串集合 P 对应的 Factor Oracle 自动机识别的字符串集合是 P 中模式串的子串集合的超集。算法的搜索过程与 SBDM 类似，利用 Factor Oracle 自动机，在长度为 ℓmin 的文本窗口内从后向前识别字符，以进行窗口的移动。如果无法继续识别文本字符 σ，那么当前窗口可以安全地移动到 σ 之后；如果完全识别了窗口内的字符，到达了窗口的起始位置，那么需要将 P 的一个子集与文本进行比较验证。

符串集合的 Factor Oracle

Factor Oracle 的构建过程与 Aho-Corasick 自动机的构建非常相似。唯一的区别在于沿供给路径回溯以寻找标号为σ的转移。在 Aho-Corasick 自动机的构建过程中，如果σ-转不存在，则只是简单地沿着供给路径跳转到下一个状态(参见 3.2.2 节)；而在 Factor Oracle 构建过程中，需要在供给路径上的每个节点和当前状态之间增加一个σ-转移。

更确切地说，先利用图 3.6 中的算法构建P对应的 trie 结构，Factor Oracle 的每个状态与 trie 的状态相对应，包括初始状态 I 和终结状态。因此，连同初始状态一起，Factor Oracle 多有 $|P|+1$ 个状态。

外部转移最多有$|P|$条，每个状态 q 都有一个供给状态 $S_{OR}(q)$与之关联，$S_{OR}(q)$在照层次遍历序构造外部转移的过程中被同步计算出来。初始状态的 S_{OR}定义为 θ。

下面对构造过程进行解释。假设在遍历的过程中，已经计算出了 $Current$ 之前的状态 S_{OR}。设 $Current$ 在 trie 中的父节点为 $Parent$，并且从 $Parent$ 到 $Current$ 的标号为σ，$Current = \delta_{OR}(Parent, \sigma)$。由于 $S_{OR}(Parent)$ 已经计算出来了，所以将从状态 $_{OR}(Parent)$ 开始，沿着供给路径回溯。这里将变量 $Down$ 初始化为 $S_{OR}(Parent)$，并按如下步骤执行：

步骤 1 如果 $Down = \theta$，那么 $S_{OR}(Current) \leftarrow I$。

步骤 2 如果 $Down \neq \theta$ 并且 $Down$ 没有σ-转移，那么构造一条从 $Down$ 到 $Current$ σ-转移；置 $Down \leftarrow S_{OR}(Down)$，转到步骤 1 执行。

步骤 3 如果 $Down \neq \theta$ 并且 $Down$ 有σ-转移，并且其目的状态为 Im，那么就可以置 $_{OR}(Current) \leftarrow Im$。对状态 $Current$ 的处理就此完成。

构造出来的 Factor Oracle 自动机至少能够识别P中模式串的所有子串[AR99]，构造算的最坏时间复杂度为$O(|P|)$。伪代码如图 3.20 所示。

Factor Oracle 进行搜索

在P的长度为 ℓmin 的前缀的反转 $P^{rv}_{\ell min}$上构建 Factor Oracle 自动机的时间复杂度为 $(r \times \ell min)$。搜索是在长度为 ℓmin 的窗口中进行，窗口沿着文本移动。在窗口中用 Factor acle 自动机从后向前识别已读入文本的最长后缀。这时可能会出现两种情况：

(i) 无法继续识别子串，即在 Factor Oracle 自动机中无法继续识别读入的文本字符σ，因此没有模式串的前缀能够覆盖窗口内已读入的字符。这时，可以安全地将当前窗口移动到σ之后的位置。

（ii）已经识别到了窗口的起始位置，并且抵达了 Factor Oracle 自动机的状态 q。在 SBOM 中用后缀自动机识别的情况下，这时就可以确定已经识别了某个模式串的前缀。但是，在 SBOM 的 Factor Oracle 中，长度为 ℓmin 的路径抵达的状态对应的不一定是模式串的前缀。因此，首先需要将 $L(q)^{rv}$ 与 $F(q)$ 中的模式串的前缀进行比较验证。只有验证二者确实相等后，再验证 $F(q)$ 中的每个模式串是否存在匹配结果。最后，将当前窗口向右移动一个字符，并开始新的搜索。

Build_Oracle_Multiple($P = \{p^1, p^2, \ldots, p^r\}$)

1. $OR_trie \leftarrow$ **Trie**(P)
   δ_{OR} is its transition function
2. Mark the states that correspond to an entire string p^i as terminal
3. $I \leftarrow$ root of OR_trie
4. $S_{OR}(I) \leftarrow \theta$
5. **For** *Current* in transversal order **Do**
6. $Parent \leftarrow$ parent in OR_trie of *Current*
7. $\sigma \leftarrow$ label of the transition from *Parent* to *Current*
8. $Down \leftarrow S_{OR}(Parent)$
9. **While** $Down \neq \theta$ AND $\delta_{OR}(Down, \sigma) = \theta$ **Do**
10. $\delta_{OR}(Down, \sigma) \leftarrow Current$
11. $Down \leftarrow S_{OR}(Down)$
12. **End of while**
13. **If** $Down \neq \theta$ **Then**
14. $S_{OR}(Current) \leftarrow \delta_{OR}(Down, \sigma)$
15. **Else** $S_{OR}(Current) \leftarrow I$
16. **End of if**
17. **End of for**

图 3.20　构造 $P = \{p^1, p^2, \ldots, p^r\}$对应的 Factor Oracle。*Current* 以层次遍历序访问 *OR_trie* 的每个状态。*Down* 沿着 *Current* 的父节点的供给路径回溯，直到具有相同标号 σ 的状态为止。如果没有 σ-转移，那么就构造该状态到 *Current* 的 σ-转移

SBOM 的伪代码如图 3.21 所示。与 SBDM 类似，它的最坏时间复杂度是 $O(n \times |P|)$，平均时间复杂度是亚线性的。Factor Oracle 自动机的构建过程非常快，而且消耗的内存也很少，即使文本规模相对较小，也是很有效的。

SBOM($P = \{p^1, p^2, \ldots, p^r\}$, $T = t_1 t_2 \ldots t_n$)

```
1.   Preprocessing
2.       ℓmin ← minimal length of strings in p^i ∈ P
3.       Or ← Build_Oracle_Multiple({pref_ℓmin(p^1)^rv, pref_ℓmin(p^2)^rv
                                     ..., pref_ℓmin(p^r)^rv})
             δ_Or is its transition function
4.       For q state of Or Do F(q) ← ∅
5.       For i ∈ 1...r Do
6.           F(q) ← F(q) ∪ {i}, where q is the state reached by pref_ℓmin(p^i)^rv
7.       End of for
8.   Searching
9.       pos ← 0
10.      While pos ≤ n − ℓmin Do
11.          Current ← initial state of Or
12.          j ← ℓmin
13.          While j ≥ 1 AND Current ≠ θ Do
14.              Current ← δ_Or(Current, t_{pos+j})
15.              j ← j − 1
16.          End of while
17.          If Current ≠ θ Then Verify all the patterns in F(Current)
18.          one by one against the text
19.          pos ← pos +j+1
20.          End of if
21.      End of while
```

图 3.21 SBOM 算法的伪代码。$pref_{\ell min}(p^i)$ 表示 p^i 的长度为 ℓmin 的前缀

英文例子 在文本“CPM_annual_conference_announce”中搜索模式串集合 $P =$ announce, annual, annually}。集合 $P_{\ell min} =$ {announ, annual}的反转对应的 Factor Oracle 如图 3.22 所示。

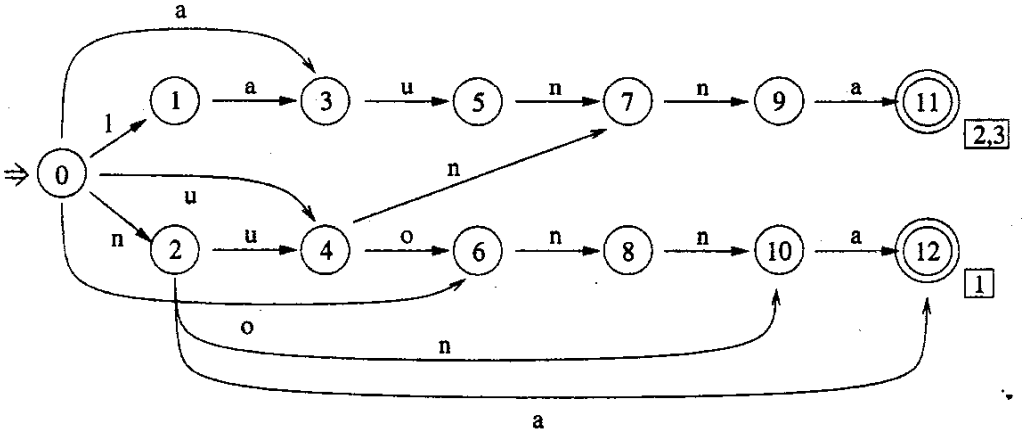


图 3.22 集合 $P_{\ell min} =$ {announ，annual}的反转对应的 Factor Oracle，标为两个圈的状态是终结状态

1. CPM_an | nual_conference_announce

   用 Factor Oracle 进行识别，依次读入字符 n，a，无法继续识别字符“_”；于是窗口移动到字符“_”之后。

2. CPM_ | annual | _conference_announce

   用 Factor Oracle 进行识别，依次读入字符 l，a，u，n，n，a，抵达窗口的起始位置，并到达状态 11，于是将 $F(11) \rightarrow$ annual 与文本进行比较，发现“annual”匹配成功；然后窗口移动一个字符。

3. CPM_a | nnual_ | conference_announce

   Factor Oracle 无法识别字符“_”，于是窗口移动到字符“_”之后。

4. CPM_annual_ | confer | ence_announce

   Factor Oracle 无法识别字符 r，于是窗口移动到字符 r 之后。

5. CPM_annual_confer | ence_a | nnounce

   用 Factor Oracle 进行识别，读入字符 a，无法继续识别字符“_”；于是窗口移动到字符“_”之后。

6. CPM_annual_conference_ | announ | ce

   用 Factor Oracle 进行识别，依次读入字符 n，u，o，n，n，a，抵达窗口的起始位置，并到达状态 12，于是将 $F(12) \rightarrow$ announce 与文本进行比较，发现匹配成功；然后窗口移动一个字符。

7. CPM_annual_conferencea_ | nnounc | e

   Factor Oracle 无法识别字符 c，于是窗口移动到字符 c 之后。此时 $pos > n - \ell min$，搜索过程结束。

DNA 例子　在序列“AGATACGATATATAC”中搜索模式串集合 P = {ATATATA, TATAT, ACGATAT}。集合 P_{lmin}={ATATA, TATAT, ACGAT}的反转对应的 Factor Oracle 如图 3.23 所示。

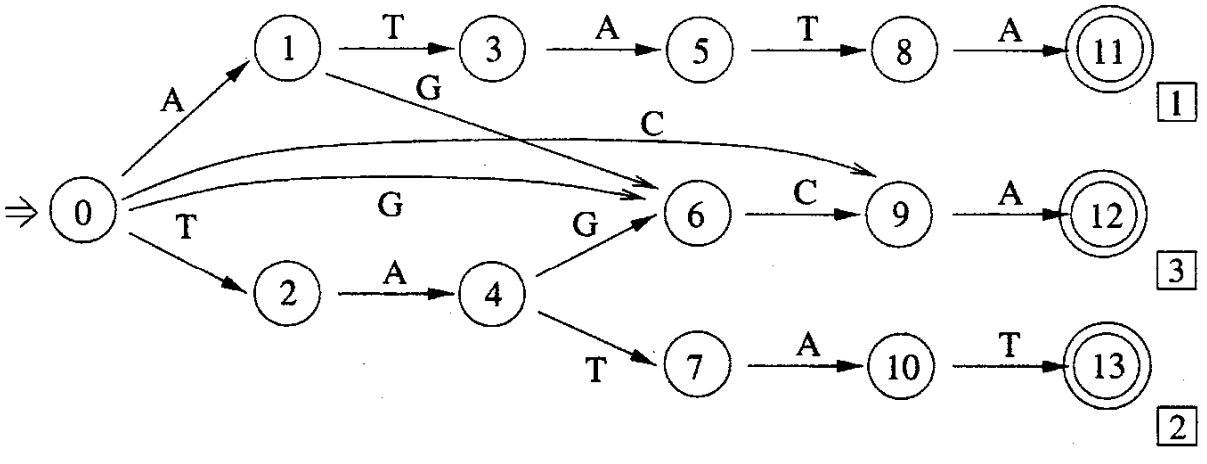


图 3.23　集合 $P_{\ell min}$ = {ATATA, TATAT, ACGAT}的反转对应的 Factor Oracle，标为两个圈的状态是终结状态

1. AGATA | CGATATATAC

用 Factor Oracle 进行识别，依次读入字符 A，T，A，无法继续识别字符 G；于是窗口移动到字符 G 之后。

2. AG | ATACG | ATATATAC

用 Factor Oracle 进行识别，依次读入字符 G，C，A，无法继续识别字符 T；于是窗口移动到字符 T 之后。

3. AGAT | ACGAT | ATATAC

用 Factor Oracle 进行识别，依次读入字符 T，A，G，C，A，抵达窗口的起始位置，并到达状态 12，于是将 $F(12) \rightarrow$ ACGATAT与文本进行比较，发现匹配成功；然后窗口移动一个字符。

4. AGATA | CGATA | TATAC

用 Factor Oracle 进行识别，依次读入字符 A，T，A，无法继续识别字符 G；于是窗口移动到字符 G 之后。

5. AGATACG | ATATA | TAC

用 Factor Oracle 进行识别，依次读入字符 A，T，A，T，A，抵达窗口的起始位置，并到达状态 11，于是将 $F(11) \rightarrow$ ATATATA 与文本进行比较，发现匹配成功；然后窗口移动一个字符。

6. AGATACGA | TATAT | AC

用 Factor Oracle 进行识别，依次读入字符 T，A，T，A，T，抵达窗口的起始位置，并到达状态 13，于是将 $F(13) \rightarrow$ TATAT与文本进行比较，发现匹配成功；然后窗口移动一个字符。

7. AGATACGAT | ATATA | C

用 Factor Oracle 进行识别，依次读入字符 A，T，A，T，A，抵达窗口的起始位置，并到达状态 11，于是将 $F(11) \rightarrow$ ATATATA 与文本进行比较，发现不匹配；然后窗口移动一个字符。

8. AGATACGATA | TATAC

用 Factor Oracle 进行识别，依次读入字符 C 和 A，无法继续识别字符 T；于是窗口移动到字符 T 之后，此时 $pos > n - \ell min$，搜索过程结束。

5 实验图

本节中将给出各种多字符串匹配算法的性能对比图，并展示不同参数条件下最有效的法。实验中的文本是随机生成的，共 10 M 字节，模式串也用同样的方式生成。实验使用机器是 32 位的 Ultra Sparc 1，操作系统是 SunOs 5.6。模式串集合分别包含 5，10，100 1000 个相同长度的字符串，字符串的长度从 5~100，以 5 为单位递增。实验对本章中介的所有算法都进行了测试，其中 Wu-Manber 算法采用了 Agrep 的实现，它的性能与散列数的选择和散列表的大小有关。

图 3.24 到图 3.27 是实验的结果图。最有效的算法是 Wu-Manber、高级 Aho-Corasick 和 OM。随着模式串集合的增大，SBOM 越来越有优势。当模式串较短时，高级 Aho-Corasick 法则更有竞争力，因为它只对文本扫描一遍。

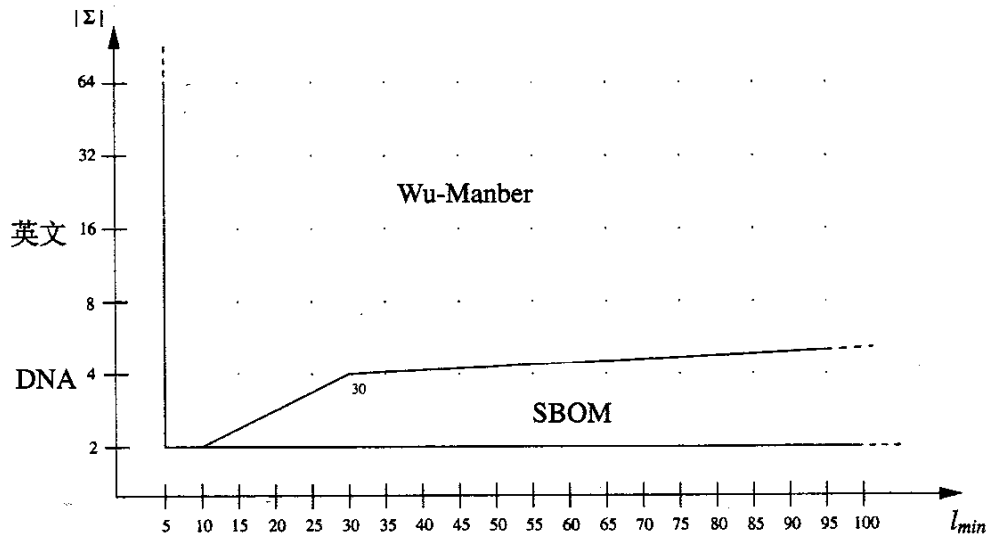


图 3.24　搜索 5 个模式串式最有效的算法

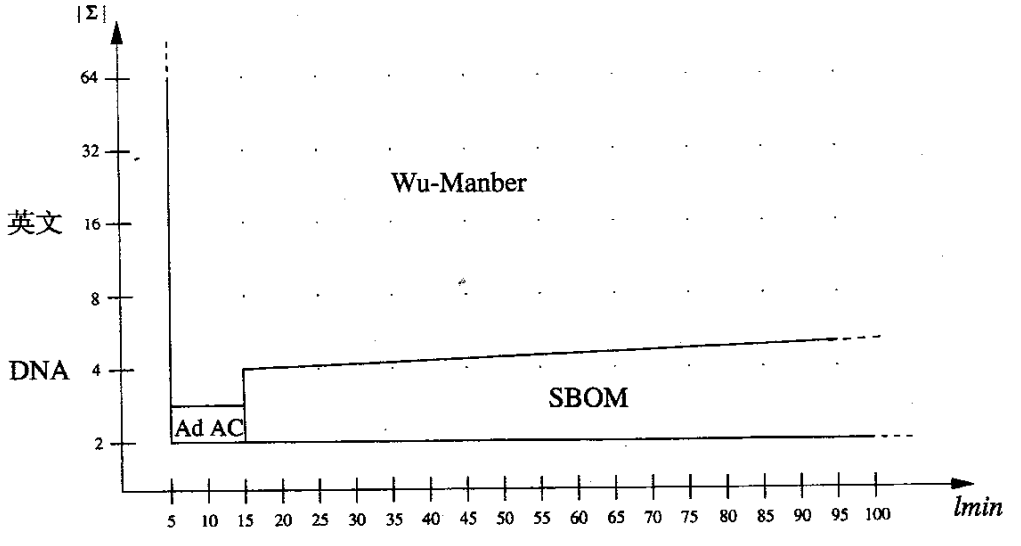


图 3.25　搜索 10 个模式串式最有效的算法

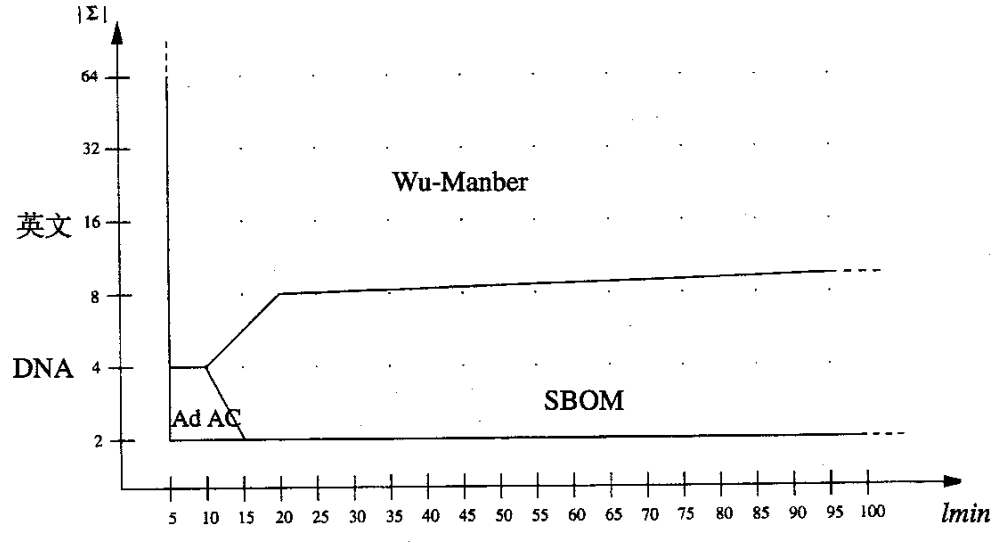


图 3.26　搜索 100 个模式串式最有效的算法

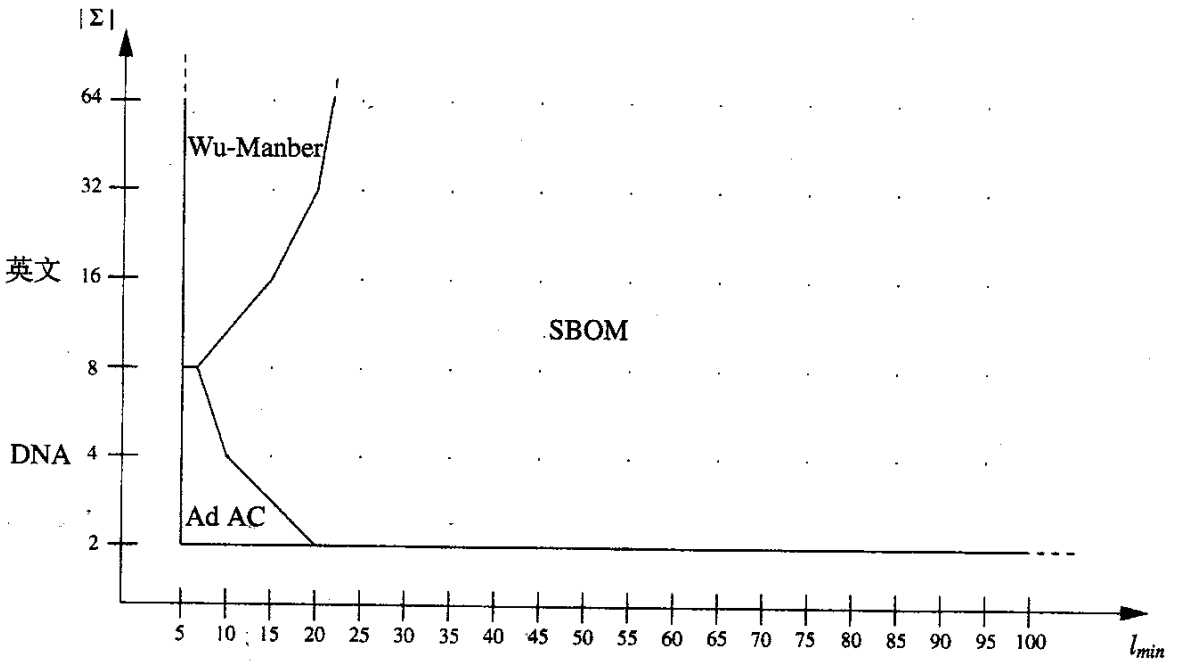


图 3.27 搜索 1000 个模式串式最有效的算法

6 其他算法和文献

动态多模式串匹配 本章中介绍的算法是对固定的模式串集合进行处理，以进行扫描。而，如果需要向模式串集合中增加一个模式串，或者从中删除一个模式串，就必须重新行预处理。如何在搜索文本的同时能够有效地更新模式串集合，这就是动态字符串匹配题。该问题近年来已经得以完全解决[SV96]，最优的时间复杂度是：（i）在$O(|P|)$的时内对模式串集合进行预处理；（ii）在$O(|p|)$的时间内增加或删除一个模式串p；（iii）在$(n + nocc)$的时间内找出文本中所有出现的模式串，其中$nocc$是模式串出现的个数。

动态串匹配可以用在变长的“don't cares”匹配中[KR95]。

关于 Commentz-Walter 算法 Commentz-Walter 算法存在几种变体，[Sri86]中通过使额外的存储空间来改善它的最坏时间复杂度，然而该算法并不实用。

无界字母表上的多字符串匹配 文献[Bre95]对无界字母表上的字符串长度均为m的字符串匹配问题进行了研究，所取得的结果是：具有$O(|P| \times m \times \log|A|)$的预处理时，需要$O((\log(|P|)/m + 1) \times n)$次比较，其中$A$是指$P$中对应的字母表。

第 4 章　扩展字符串匹配

4.1　基本概念

到目前为止，已经讨论了模式串为字符序列时的匹配算法。但在许多情况下，那些比单纯字符序列更复杂的模式串形式的匹配问题更引人关注。在本书介绍的内容中，最复杂的模式串形式是第 5 章中介绍的正则表达式。正则表达式模式串的搜索过程很耗时，并且程序实现复杂，因此只有当特别需要时，才会将具体问题归类到正则表达式匹配。而在大多数情况下，只需要一些较简单的模式串形式，从而可以使用更高效、实现更简单的算法来解决问题。

设立“扩展字符串匹配”这一章的目的是为了在简单字符串和正则表达式之间增加一个过渡。本章在简单字符串的基础上做了一些扩展，使其能比简单字符串复杂但比正则表达式模式串简单。同时，还对应地介绍了一些简单匹配算法（在简单字符串匹配算法上的扩展），其中主要关注的是那些能够同时在文本搜索和计算生物学应用中使用的算法。

本章将介绍在字符串搜索问题中对字符串的四种扩展形式：字符组、限长空位、可选字符和重复字符。第一种形式允许在模式串或文本中的任一位置匹配某一特殊字符组中的任一字符；第二种形式允许在模式串中出现限定长度的空位（这些位置在指定长度范围内可以匹配任意字符），这种扩展形式在蛋白质搜索中比较常见（例如，PROSITE 模式串[Gus97, HBFB99]）；第三种形式允许在模式串的成功匹配中，指定哪些字符可以出现或不出现；第四种形式允许在模式串的成功匹配中，指定哪些字符可以重复多次，包括通配符。本章最后将介绍这些扩展形式在小规模模式串的情况下相应搜索算法的性能。

一个扩展模式串在文本中可能有多个成功匹配，它们的长度可能会不同，而且它们的起始位置或结束位置也可能会是文本中的同一位置。在报告这些成功匹配的若干种方法中，本章中选择报告所有成功匹配的起始位置或者结束位置。至于是报告起始位置还是结束位置，将由算法自己决定，哪一种更自然（对算法来说更有意义）就报告哪种。

在本章中主要使用了位并行算法。虽然通过一些额外的工作，其他算法也能很好地解

某些扩展形式的匹配问题，但是位并行算法可以提供最佳的灵活性，通常情况下它的性能也是最好的。本章还将展示通过改变非确定自动机的模拟过程，使得 Shift-And 算法能够很好地解决各种扩展形式。BNDM 算法同样能够很好地解决，但是它面临成功匹配的模式串和原始的模式串长度可能不相同的问题。即每当窗口中的字符左移到窗口起始位置时，因为不能确认模式串是否成功匹配，所以这时要有一个验证过程。

本章中用到的所有技术都可应用于第 6 章介绍的近似字符串匹配，其中一些技术也可以与第 5 章介绍的正则表达式搜索中的技术相结合使用。

.2 字符组

.2.1 模式串中的字符组

对模式串匹配最简单的扩展形式是允许每个模式串位置能够匹配一组字符而不仅仅是一个字符。模式串 p 是 $\wp(\Sigma)$ 上的序列，即 $p = p_1 p_2 \ldots p_m$，其中 $p_j \subseteq \Sigma$。字符串 $p' \in \Sigma^*$ 是模式串 p 的一个成功匹配，前提条件是当对所有的 $j \in 1 \ldots m$，都有 $p'_j \in p_j$。简单字符串是这种模式串的一种特例。

通常使用方括号来表示一个字符组，字符组中的元素可以在方括号中枚举，也可以使用字符范围来表示（如果元素很明显是线性顺序的）。例如："[Aa]nnual" 表示可以成功匹配 "Annual" 和 "annual"；"[0–9][0–9]/[0–9][0–9]/199[0–9]" 可以用来匹配 1990~1999 年的所有日期。本章中将统一使用这种表示方式，其中符号 Σ 表示在模式串的指定位置可以匹配整个字符表（指字符表中的任一字符）。

两种简单的扩展匹配可以使用字符组来表示：1）任意字符匹配：任意字符所在的位置可以匹配字符表中的任意一个字符，只需要使用符号 Σ；2）大小写不敏感匹配：只需将原来模式串中每一位置上的单一字符替换成对应的大小写字符组，例如 "[Aa][Nn][Nn][Uu][Aa][Ll]" 可以匹配字符串 "annual" 的所有大小写不敏感的字符串。

假定已经有位并行算法，例如 Shift-And 算法或者 BNDM 算法（参见第 2 章）。模式串和文本的唯一联系是在预处理阶段构建表 B 的时候，该表提供每个字符 c 在模式串中出现位置的位掩码。现在模式串 p 变成了字符组的序列，这时位并行算法只要改变预处理过程就可以用来解决字符组扩展匹配。相应地，只要将 Shift-And 算法（参见图 2.6）中的第 3 行替换成：

For $j \in 1 \ldots m$ **Do**
　For $c \in p_j$ **Do** $B[c] \leftarrow B[c] \mid 0^{m-j}10^{j-1}$
End of for

或者，对应地将 BNDM 算法（参见图 2.16）中的第三行替换成：

For $j \in 1 \ldots m$ **Do**
　For $c \in p_j$ **Do** $B[c] \leftarrow B[c] \mid 0^{j-1}10^{m-j}$
End of for

Shift-Or 算法需要将 Shift-And 算法中表B的位掩码反转，并且将“|”改成“&”。图 4.1 给出了 Shift-And 算法中表B的一个例子。

c	$B[c]$
0	1000000000000000
1	1001000000000000
2	1000000000000000
3	1000000000000000
4	1000000000000000
5	1000000000000000
6	1000000000000000
7	1000000000000000
8	1000000000000000
9	1110000000000000
A	0000000000000001
a	0000000000001001
f	0000010000000000
l	0000000000100000
n	0000000000000110
o	0000001000000000
s	0000000010000000
_	0000100100000000

图 4.1　模式串“[Aa]nnals_of_199[0–9]”的位掩码表 B

非位并行算法也可以通过若干种扩展来解决字符组扩展匹配问题，但是都不能很好地将简单性和健壮性结合在一起。考虑一下 Horspool 算法（参见 2.3.2 节），这时需要改变图 2.12 中预处理过程的第三行代码以及匹配过程的第八行代码。然而，Horspool 算法的性能退化得很快，特别是当模式串的末尾部分有一个大字符组时，性能退化尤为突出。这是因为当末尾有大字符组时，Horspool 算法向前移动的距离会很短。因此，该算法的性能对字符组个数、每个字符组的大小、字符组在模式串中的位置都很敏感。

再来考虑一下 BDM 算法（参见 2.4.1 节）。对于确定性的后缀自动机，目前还没有很有效的扩展方式，使其能够解决字符组扩展匹配问题[NR00]。2.4.3 节中介绍的 BOM 算法也一样。

在经典算法中，扩展后性能最好的是经典 Boyer-Moore 算法（参见 2.3.1 节），然而在大部分情况下，其性能都比 BNDM[NROO]扩展算法差。

Shift-And/Shift-Or 算法的性能不受字符组扩展的影响，但它的性能通常要比 BNDM 算差。BNDM 算法扩展后其性能也受到影响，这是由于扩展后更容易在窗口中找到模式串子串。粗略的分析如下：假设S是字符组的平均大小，相当于字母表大小变成$|\Sigma|/S$，BNDM 扩展算法的平均复杂度变成了$O(n\log_{|\Sigma|/S}(m)/m)$。

在小字母表下，BNDM 扩展算法要优于 Horspool 扩展算法，而且其性能受模式串中字组大小和字符组个数的影响要比 Horspool 扩展算法小。但是当字符组个数变多或者字符大小变得很庞大时，Shift-Or 扩展算法的性能会更好，它比 Shift-And 扩展算法还稍微快些。但是对于下一节介绍的扩展形式，Shift-Or 扩展算法并不是最快的。Shift-And 扩展法将是更好的选择，因为它更加直观。

2.2 文本中的字符组

在计算生物学应用中，文本中某些位置上的字符可能会不确定。也就是说：已经知道本中某一位置上的字符肯定在某一组字符中，但是并不能确定具体是哪一个字符。这种况可以用允许在文本位置上使用字符组的扩展匹配来建立模型。对给定的一组字符，通使用新的编码来表示。

对文本中的字符组扩展匹配的形式化定义如下：文本是$\wp(\Sigma)$上的序列，即$T=t_1t_2\ldots t_n$，中$t_i\subseteq\Sigma$。模式串$p=p_1p_2...p_m$出现在文本位置$t_{i+1}\ldots t_{i+m}$，前提条件是当对于所有的$\in 1\ldots m$，均有$p_j\cap t_{i+j}\neq\emptyset$。

位并行算法给出了一个简单的方法来解决上述问题。以字符编码c来表示文本中的字组$\{c_1,\ c_2,\ \ldots,\ c_k\}$。在 Shift-And 算法或者 BNDM 算法的预处理阶段，在对文本中其正常字符构建位掩码表B之后，对于文本中的每个字符组c增加以下操作：

$B[c]\leftarrow 0^m$
For $i\in 1\ldots k$ **Do** $B[c]\leftarrow B[c]\mid B[c_i]$

样，对于所有那些能匹配若干c_i的模式串位置，字符c都能被匹配。

这种方法还可以再扩展用来支持某些特殊字符组，它们是其他特殊字符组的集合，这需要注意对上述添加操作的执行顺序要正确。更进一步，这种方法还可以与模式串中的符组扩展结合起来，只需要对正常字符构建掩码表B后再处理即可。

对于小字母表，如 DNA 序列，可以选择将字母表扩展成$\Sigma'=\{0\ldots 2^{|\Sigma|}-1\}$，然后位并行的方法来表示新的字母表。新字母表中的字符使用长度为$|\Sigma|$的位掩码形式来表

示，第 i 位为 1 的位掩码表示原字母表中的第 i 个字符。例如：原字母表为 $\{A,G,C,T\}$，这样单个字符可以表示成：$A=0001$，$G=0010$，$C=0100$，$T=1000$；字符组也可以这样表示，例如字符组 $\{A,C\}$ 可以表示为 0101。

基于上面给出的新的表示方法，算法要构建一个不同的位掩码表 B'，该表的范围在整数 $\{0\ldots 2^{|\Sigma|}-1\}$ 之间。为简单起见，假设 $\Sigma=\{0\ldots|\Sigma|-1\}$，给定位掩码表 B，新的位掩码表 B' 的构建方式如下：

$B'[0]\leftarrow 0^m$
For $c\in 0\ldots|\Sigma|-1$ **Do**
　For $j\in 0\ldots 2^c-1$ **Do** $B'[2^c+j]\leftarrow B[c]\mid B'[j]$
End of for

构建过程的时间为 $O(2^{|\Sigma|})$。对 DNA 来说，位掩码表 B' 只有 16 个元素，时间开销还是很少的。而匹配过程不需要变化，只要用 B' 来代替 B 即可。

4.3　限长空位

在蛋白质搜索中很重要的一类是对 PROSITE 模式串的搜索[Gus97，HBFB99]。在一个 PROSITE 模式串中可以包含字符组和限长空位，限长空位指在此处可以匹配限定长度范围内的任意字符串。在 PROSITE 中，使用连接号“－”来分隔模式串中的字符和字符组；使用 $x(a,b)$ 来表示一个长度范围在 a 与 b 之间的空位。通常，$x(a)=x(a,a)$，$x=x(1)$，x 等价于字母表 Σ。例如模式串 $a-b-c-x(1,3)-d-e$ 可以匹配“abcfde”，也可以匹配“abcfddde”，但是不能匹配“abcffffde”。尽管本章中只考虑 PROSITE 模式串，但是本节中介绍的算法同样可以适用于包含限长空位的其他模式串形式。

图 4.2 给出了模式串 $a-b-c-x(1,3)-d-e$ 对应的一个 NFA。在字符“c”和“d”之间增加了三个转移，由此可以接收任意字符，从而对应到空位的最大限长。两个空转移（ε-transition）使得当“abc”被识别后能够跳过随后的一个或两个状态。因此，在找到模式串最后的字符“de”之前可以跳过一至三个字符。初始自环使得该自动机能够从文本中的任意位置开始匹配。

记模式串中的符号个数为 m，每个符号表示一个字符组或者一个形如 $x(a,b)$ 的限长空位。模式串在文本中成功匹配的最小长度和最大长度为 ℓmin 和 ℓmax。由于字符组的长度为 1，限长空位 $x(a,b)$ 的最小限长为 a，最大限长为 b，因此最大长度 ℓmax 和最小长度 ℓmin 都可以在 $O(m)$ 的时间内计算得到。NFA 中的状态个数为 L，不包括初始状态。显然 $L=\ell max$。在图 4.2 的例子中：$m=6$，$\ell min=6$，$\ell max=L=8$。

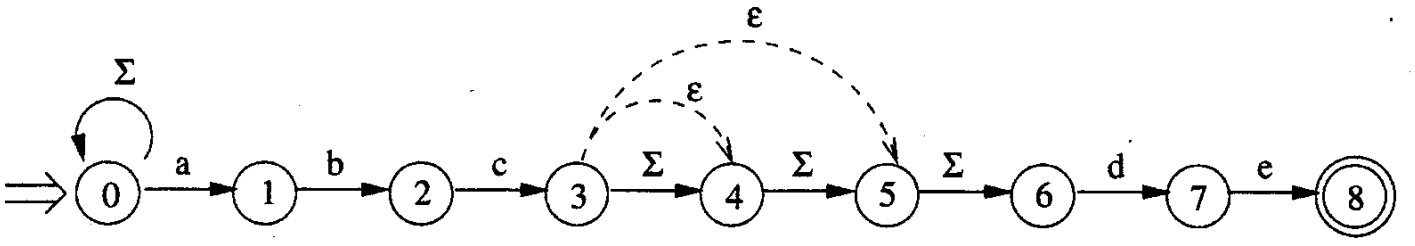


图 4.2 能识别模式串 $a-b-c-x(1,3)-d-e$ 的非确定自动机。虚箭头表示空转移，不需要任何输入字符就可以转移

下面将介绍[NR01b]中的两个位并行算法，它们分别是对 Shift-And 算法的扩展和对 NDM 算法的扩展，都能够快速地在文本中找到包含限长空位的模式串。

.3.1 Shift-And 算法扩展

本节中将讨论如何通过增加空转移对 Shift-And 算法进行扩展。“空位初始状态”指转移出发的状态 i，每个空位初始状态都对应一个空位 $x(a,b)$。“空位终止状态”指态 $(i+b-a+1)$，即从空位初始状态 i 出发的所有能够通过空转移到达的状态中最后的那个状态。在图 4.2 中，有一个空位初始状态为状态（3），有一个空位终止状态状态（6）。

创建一个位掩码 I，I 中空位初始状态对应比特位为 1（其他比特位为 0）。另外，再创一个位掩码 F，F 中空位结束状态对应比特位为 1（其他比特位为 0）。在图 4.2 中，对应位掩码 I 为 00000100，位掩码 F 为 00100000。通过执行通常的 Shift-And 算法步骤后，以使用下面的操作来模拟所有的空转移：

$$D \leftarrow D \mid ((F-(D \,\&\, I)) \,\&\, \sim F)$$

基本原理如下：$D \,\&\, I$ 使得活动的空位初始状态与其他状态相隔离，然后把它从 F 中去。这样，对于每一个空位初始状态 i 都将得到两种可能的结果。第一，如果 i 是活动，那么将使得从状态 i 到状态 $(i+b-a)$相对应的比特位均置为 1，这样就成功地将活动态从状态 i 传播到其他需要的目标状态。第二，如果 i 不是活动的，那么仅仅将状态 $+b-a+1)$对应的比特位置为 1。在减法运算得到的结果中，将通过“&~F”运算来消不需要的“1”。最后，再把通过空位传播被激活的状态与 D 中原先已经被激活的状态相。注意不同空位之间的传播不会相互影响，因为其中的减法具有局部效应。

图 4.3 详细描述了整个扩展 Shift-And 算法。为了简单起见，假设没有空位在模式串的始或结束位置，并且连续的空位已经合并为一个。预处理阶段需要消耗的时间为

$O(m|\Sigma|)$，扫描阶段需要消耗的时间为 $O(n)$。如果 $\ell max > w$，那么需要多个机器字来进行位操作，此时扫描阶段需要消耗的时间为 $O(n\lceil \ell max/w \rceil)$。

Gaps-Shift-And $(p = p_1p_2 \ldots p_m,\ T = t_1t_2 \ldots t_n)$

```
1.    Preprocessing
2.        L ← maximum length of an occurrence
3.        For c ∈ Σ Do B[c] ← 0^L
4.        I ← 0^L, F ← 0^L
5.        i ← 0
6.        For j ∈ 1...m Do
7.            If p_j is of the form x(a,b) Then
8.                I ← I | 0^{L-i}10^{i-1}
9.                F ← F | 0^{L-(i+b-a)-1}10^{i+b-a}
10.               For c ∈ Σ Do B[c] ← B[c] | 0^{L-i-b}1^b0^i
11.               i ← i+b
12.           Else /* p_j is a class of characters */
13.               For c ∈ p_j Do B[c] ← B[c] | 0^{L-i-1}10^i
14.               i ← i+1
15.           End of if
16.       End of for
17.   Searching
18.       D ← 0^L
19.       For pos ∈ 1...n Do
20.           D ← ((D << 1) | 0^{L-1}1) & B[t_pos]
21.           D ← D | ((F - (D & I)) & ~F)
22.           If D & 10^{L-1} ≠ 0^L Then report an occurrence ending at pos
23.       End of for
```

图 4.3　能够处理 PROSITE 表达式的扩展 Shift-And 算法描述

Gaps-Shift-And 算法举例　下面通过在文本“abcabcffdee”中搜索模式串 $a-b-c-x(1,3)-d-e$ 来具体说明 Gaps-Shift-And 算法的操作过程。

$B =$

a	00111001
b	00111010
c	00111100
d	01111000
e	10111000
*	00111000

$I = 00000100$
$F = 00100000$
$D = 00000000$

1. 读入字符 a 00111001
   $D =$ 00000001

   我们在D上运用传播公式。结果为$((F - (D \ \& \ I)) \ \& \ \sim F) = ((00100000 - 00000000) \ \& \ 11011111) = 00000000$，因此$D$没有变化。下面我们不再特意计算传播公式，除非对$D$发生作用。

2. 读入字符 b 00111010
   $D =$ 00000010

3. 读入字符 c 00111100
   $D =$ 00000100

   此时，空转移发生了作用：$((F - (D \ \& \ I)) \ \& \sim F)$产生$((00100000 - 00000100) \ \& \ 11011111) = 00011100$，因此状态 4 和状态 5 被激活了。这时新的$D$的值为：$D = 00011100$。

4. 读入字符 a 00111001
   $D =$ 00111001

5. 读入字符 b 00111010
   $D =$ 00110010

6. 读入字符 c 00111100
   $D =$ 00100100

   此时，传播公式再次起作用，使得：$D = 00111100$。

7. 读入字符 f 00111000
   $D =$ 00111000

8. 读入字符 f 00111000
   $D =$ 00110000

9. 读入字符 d 01111000
   $D =$ 01100000

10. 读入字符 e 10111000
    $D =$ 10000000

    D的最高位为 1，因此我们标记模式串出现。此时空位匹配文本中的“ff”。

11. 读入字符 e 10111000
    $D =$ 00000000

.3.2 BNDM 算法扩展

本节中将讨论如何扩展 BNDM 算法（参见 2.4.2 节）来解决包含限长空位扩展的匹配问题。为了识别模式串的所有反转子串，可以使用与图 4.2 中相同的方法对反转模式串构建自动机。但是没有初始自环，并且开始时所有状态都是活动的。图 4.4 展示了模式串$-b-c-x(1,3)-d-e$对应的这种自动机。只要该自动机至少存在一个活动状态，那么被该自动机读入的一个字符串就是模式串的一个子串。注意：在当前的自动机上，虽然空转移是从状态“b”出发的，但是其效果与图 4.2 中的自动机是一样的。

该自动机（可以称为“后向自动机”）的位并行模拟过程与前向自动机的模拟过程相似。需要修改的是：（a）此过程是在模式串的反转形式上进行的；（b）标记搜索过程中状态的位掩码D将被初始化为$D = 1^L$，以便能够表示初始空转移；（c）移位时不再将D与$0^{L-1}1$进行或运算（OR），因为这里没有初始自环。

后向匹配算法沿着文本移动大小为ℓmin的窗口，在每个窗口中，算法从右向左遍历文

本来识别模式串的子串。每次到达自动机的终止状态时，就表示识别出了一个模式串的前缀，这时用变量*last*来记录此时窗口中遍历到的位置。

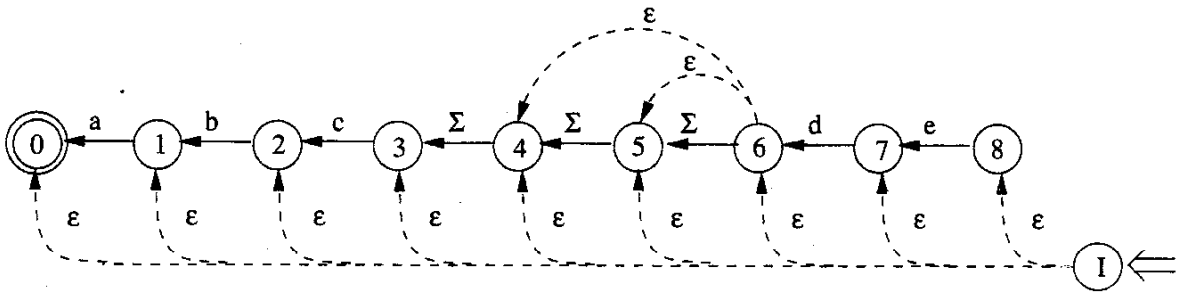


图 4.4　识别 PROSITE 模式串 $a-b-c-x(1,3)-d-e$ 的所有反转子串的非确定自动机

对于某一窗口，如果在到达窗口的起始位置之前后向搜索就失败了，那么可以将窗口移动到已经识别出的最长前缀的起始位置，这与 BNDM 算法相同。

如果已经到达了窗口的起始位置，并且自动机还有活动的状态，则表明在窗口中有长度为 ℓmin 的模式串的子串被识别。与精确字符串匹配不同，在精确字符串匹配中，所有成功匹配的长度都等于模式串的长度。但是在这里，到达窗口的起始位置并不表明已经成功匹配了整个模式串。因此，需要对这个可能的成功匹配进行验证，验证过程从窗口的起始位置开始，长度为 ℓmax。

为了实现验证过程，需要再次从该窗口的起始位置读入字符，读入过程使用类似图 4.2 中的前向自动机，但去除了初始自环。为了模拟该去除初始自环的自动机，可以使用如下操作：

$$D \leftarrow (D << 1) \ \& \ B[t_{pos}]$$
$$D \leftarrow D \mid ((F - (D \ \& \ I)) \ \& \ \sim F)$$

前向验证过程将在下面两种情况下结束：（1）自动机到达终止状态，这时表明成功匹配了一个模式串；（2）自动机不再有活动的状态，这表明在窗口的开始位置没有模式串被成功匹配。由于没有初始自环，最多读入 ℓmax 个字符后，该自动机验证过程就肯定会结束，然后通过移动匹配窗口到被识别的最长前缀的起始位置来继续搜索。

图 4.5 给出了这一算法的完整描述。该算法的最坏复杂度为 $O(n \times \ell max)$，这在理论上是很差的。更进一步来考虑在模式串中存在长度为 G 的限长空位的情况。如果 $G \geqslant \ell min$，那么文本中每个大小为 ℓmin 的窗口中的字符均为模式串的子串，于是后向扫描过程将总是遍历整个窗口，这个验证过程次数最小为 $O(n)$。所以该算法在 $G \geqslant \ell min$ 时不适用。[NR01b]中给出的实验数据表明：当 $G + 1 < \ell min/2$ 时，Gaps-BNDM 算法性能要优于 Gaps-Shift-And 算法。

```
Gaps-BNDM (p = p1p2...pm, T = t1t2...tn)
1.    Preprocessing
2.        L ← maximum length of an occurrence
3.        ℓmin ← minimum length of an occurrence
4.        For c ∈ Σ Do B[c] ← 0^L
5.        I ← 0^L, F ← 0^L
6.        i ← 0
7.        For j ∈ 1...m Do
8.            If p_j is of the form x(a,b) Then
9.                I ← I | 0^{i+b}10^{L-(i+b)-1}
10.               F ← F | 0^{i+a-1}10^{L-(i+a)}
11.               For c ∈ Σ Do B[c] ← B[c] | 0^i1^b0^{L-i-b}
12.               i ← i + b
13.           Else /* p_j is a class of characters */
14.               For c ∈ p_j Do B[c] ← B[c] | 0^i10^{L-i-1}
15.               i ← i + 1
16.           End of if
17.       End of for
18.   Searching
19.       pos ← 0
20.       While pos ≤ n − ℓmin Do
21.           j ← ℓmin, last ← ℓmin
22.           D ← 1^L
23.           While D ≠ 0^L AND j > 0 Do
24.               D ← D & B[t_{pos+j}]
25.               D ← D | ((F − (D & I)) & ∼F)
26.               j ← j − 1
27.               If D & 10^{L-1} ≠ 0^L Then /* prefix recognized */
28.                   If j > 0 Then last ← j
29.                   Else check a possible occurrence starting at pos
30.               End of if
31.               D ← D << 1
32.           End of while
33.           pos ← pos + last
34.       End of while
```

图 4.5 能够处理 PROSITE 表达式的扩展 BNDM 算法描述

aps-BNDM 算法示例 下面通过在文本 "abcabcffdee" 中搜索模式串 $a-b-c-x(1,3)--e$ 来具体说明 Gaps-BNDM 算法的操作过程。

$$B=\begin{cases}\hline \texttt{a} & 10011100\\ \texttt{b} & 01011100\\ \texttt{c} & 00111100\\ \texttt{d} & 00011110\\ \texttt{e} & 00011101\\ * & 00011100\\ \hline\end{cases}$$

$I = 00000100$
$F = 00100000$

$\ell min = 6,\ \ell max = 8$

1. abcabc ffdee

$last \leftarrow 6$

读入字符 c　00111111
$D =$　00111100

传播没有被激活，因此没有给D带来新的激活状态。

读入字符 b　01011100
$D =$　01011000

读入字符 a　10011100
$D =$　10010000

D最高位为 1，但是$j > 0$，因此置 $last \leftarrow 3$。

读入字符 c　00111100
$D =$　00100000

读入字符 b　01011100
$D =$　01000000

读入字符 a　10011100
$D =$　10000000

D最高位为 1，并且$j > 0$，因此我们再次对字符“abcabcff”进行正向验证。正向 验证自动机最后没有匹配成功。因此我们继续处理下一窗口，将窗口左移 $last \leftarrow 3$。

2. abc abcffd ee

$last \leftarrow 6$

读入字符 d　00011110
$D =$　00011110

传播机制被激活，但没有对D起作用，没有改变D值。

读入字符 f　00011100
$D =$　00011100

传播机制被激活，但仍然没有对D起作用，没有改变D值。

读入字符 f　00011100
$D =$　00011000

读入字符 c　00111100
$D =$　00110000

读入字符 b　01011100
$D =$　01000000

读入字符 c　10011100
$D =$　10000000

D最高位为 1，并用$j = 0$，因此我们再次对字符“abcffdee”进行正向验证并验证成功。因此报告在位置 4 处有成功匹配。窗口左移$last = 6$，达到文本结束位置，因此结束扫描。

4.4　可选字符

本节考虑在匹配过程中，模式串中某些位置的字符可以不在文本中出现的情况。这些位置上的字符称为“可选字符”，是通过在它们自身后面增加问号来标识的。例如：模式串“abc?d?efg?h”可以匹配“abefh”，也可以匹配“abdefgh”。识别该模式串的非确定性自动机参见图 4.6。

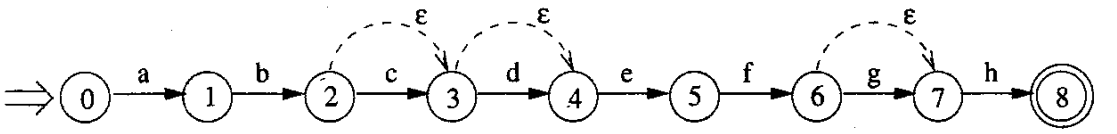


图 4.6 识别模式串“abc?d?efg?h”的非确定自动机

如图 4.6 所示，会存在多个连续的可选字符。为了避免多个连续可选字符都不出现的情况，最简单的方法是增加一个位掩码 O，该掩码置可选字符对应比特位为 1（其他比特位置为 0）（图 4.6 中，$O = 01001100$），以便能够将上一个 D 状态中的“1”传播到这些可选字符对应的比特位。因此，在正常更新 D 后（例如图 2.6 所示的 Shift-And 算法中第 7 行之后），将要增加如下操作：

$$D \leftarrow D \mid ((D << 1) \ \& \ O)$$

如果已经读入字符串“abcdef”（这时 $D = 00100000$），并且下一个读入的字符为“h”，此时由于上面对 D 的新操作，因此在将 D 与 $B[\mathrm{h}] = 10000000$ 相运算（与操作）之前，D 已经转化成 01100000，所以这种方法能够正常工作。但是如果文本为“abefgh”，即连续的可选字符均没有出现时，这种方法就无法正常工作了。

解决这个问题的通用方法是增加一个传播过程，该传播过程能将 D 中可选字符对应的前一位置上的“1”传播到该位置之后的可选字符对应位置。在上述例子中，当 D 为 00000010 时，希望传播后 D 变成 00001110。

[Nav01b]实现了这种机制，它类似于 4.3 节提到的方法。增加三个位掩码：A，I 和 F，用来标识连续可选字符块的边界。每个连续可选字符块的起始位置均为连续可选字符中第一个字符位置的前一位置，结束位置为连续可选字符中的最后一个字符位置。例如，在图 4.6 中，第一个可选字符块的起始位置为 2，结束位置为 4。位掩码 A 中第 i 位为 1 表示模式串 p 中该位置为可选字符，位掩码 I 中第 i 位为 1 表示该位置是可选字符块起始位置的前一位置，位掩码 F 中第 i 位为 1 表示该位置为可选字符块的结束位置。在图 4.6 中，$A = 01001100$，$I = 00100010$，F=01001000。在对 D 进行正常转换之后，再执行下面的操作：

$$\begin{aligned} Df &\leftarrow D \mid F \\ D &\leftarrow D \mid (A \ \& \ ((\sim (Df - I)) \ \wedge \ Df)) \end{aligned}$$

第一行操作在 D 中将可选字符块的结束位置置为 1，第二行操作将在 D 中增加一些活动状态。它们是在和 A 进行了“与”操作之后才被加入 D 的，那么就先来详细讨论一下这些操作（指和 A 相与的后面的括号内部分）。这些操作的目的是将所有可选字符块从右至

左均置为 1。首先从 Df 中减去 I，这等价于将每个可选字符块都减 1。由于 Df 通过“$|F$”（第一行）操作已经将每个可选字符块的最高位置置为 1，因此，对这些块进行减法操作并不会影响块之外的结果。减法的结果是将块中从右向左遇到第一个 1 之前的所有位置均置为 1（例如 $1000000-1=0111111$），而块中其他位置结果不变。一般而言，即 $b_xb_{x-1}\ldots b_{x-y}10^z-1=b_xb_{x-1}\ldots b_{x-y}01^z$。然后通过“~”操作将上述结果取反，得到 $\sim b_x \sim b_{x-1}\cdots \sim b_{x-y}10^z$。最后，再和 $Df=b_xb_{x-1}\ldots b_{x-y}10^z$ 进行“异或”（XOR）操作，得到最终结果 $1^{x-y+1}0^{z+1}$。

通过上述这些操作可以获得如下的效果：将块中起始位置上的 1 从右向左传播到块中其他所有位置，但该起始位置的值却被置为 0。由于这个位置上的值已经预先保留在原始 D 中，因此它的值可以通过和 D 进行 OR 操作来恢复。这种方法对活动状态在可选字符块的最左端的情况也同样有效。注意：操作的最后一定要和 A 相与，这样可以避免异或操作（XOR）将结果扩散到可选字符块之外。在 4.5 节末尾将给出一个将几种扩展形式组合到一起的例子。

注意：可选字符不能用限长空位来表示，因为可选字符会连续出现并且它不需要匹配任一字符。但是限长空位却可以用可选字符来表示，例如 $a-b-c-x(1,3)-d-e$ 等价于“abcΣΣ?Σ?de”。然而，限长空位的转换公式要比可选字符的转换公式简单，因此其算法速度要快于可选字符。

4.5　通配符和重复字符

“通配符”是一个专用术语，表示模式串中的某一位置可以匹配任意长的字符串，通配符通常用星号（“*”）来表示。例如，“ann*al”可以匹配“annal”，“annual”和“annals of biological”。本书中不使用这种表示方法，而是使用一种更加通用的形式。

通配符可以看做是更通用的概念“重复字符”的一个特例。一个重复字符表示在模式串中该位置的字符可以在文本中出现 0 次或多次。重复字符可以通过在字符或者字符组后面增加一个星号来表示。例如，AC*TCA 可以匹配 ATCA，同样也可以匹配 ACCTCA 和 ACCCCCCCTCA。另外一个例子是“[a–zA–Z_][a–zA–Z_0–9]*”，表示可以匹配大多数程序语言中的变量名。在这种定义下，通配符可以表示成“Σ*”。处理重复字符的算法比处理通配符的算法速度要快，并且更易于编程。参照正则表达式的符号表示，有 $c+=cc*$。

图 4.7 给出了一个能识别模式串“abc+def*gh”的自动机。但是，很难将这个自动机的表示方式扩展到包含连续多个重复字符的模式串，例如“abc+def*g*h”。

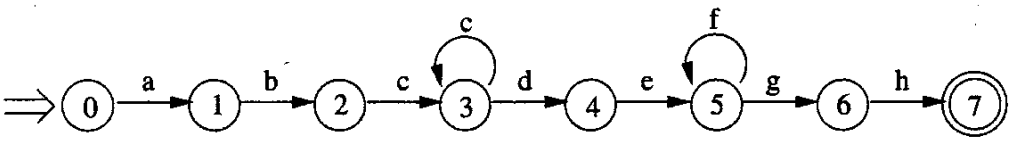


图 4.7 能识别模式串“abc+def*gh”的非确定自动机。这种表示机制不能扩展到连续多个重复字符的情况

一个通常的解决方法是将连续重复字符中的 $c*$ 表示成 $c+?$。用“+”和“?”来模拟表“*”。图 4.8 给出了使用上述方法表示的能识别“abc+def*gh”的自动机。因此，为了解通配符“*”，就必须解决重复字符和可选字符。

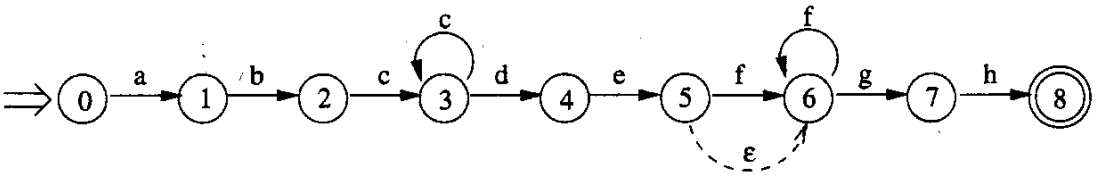


图 4.8 能识别模式串“abc+def*gh”的非确定自动机。可以解决连续多个重复字符的情况

记 m 是模式串长度，包括普通字符以及三种特别字符（指：+，?，*）。ℓmin 是一个功匹配的最小长度，即模式串中去除了由可选字符和重复字符带来的影响后的长度，它以在 $O(m)$ 时间内计算得到。对于最大长度，如果模式串中包含了重复字符，则成功匹的最大长度是没有上限的。记 L 是 NFA 的状态个数（去除第一个状态后），即模式串 p 中通字符的个数。

为了使位并行算法能够模拟操作“+”，需要建立一个表格 $S[c]$，用来记录当读入字符 c，模式串中哪些位置可以保持是活动的。参见图 4.8，$S[\mathtt{c}] = 00000100$，$S[\mathtt{f}] = 00100000$。入文本 t_{pos} 后，允许包含可选字符和重复字符的位并行模拟的完整步骤如下所示：

$$
\begin{aligned}
D &\leftarrow ((D << 1) \mid 0^{L-1}1) \ \&\ B[t_{pos}]) \ \mid \ (D \ \&\ S[t_{pos}]) \\
Df &\leftarrow D \mid F \\
D &\leftarrow D \mid (A \ \&\ ((\sim (Df - I)) \ \wedge \ Df))
\end{aligned}
$$

完整代码与 4.3 节介绍的限长空位算法类似，只是在模拟步骤中做了一些改变。下面将体介绍 Shift-And 算法和 BNDM 算法的相应扩展。

在使用了可选字符和重复字符后，不同算法的试验结果参见[Nav01b]。文中表示：在多数情况下，BNDM 扩展算法都要优于 Shift-And 扩展算法，只有当 $\ell min \leqslant 3$ 或者存在量的重复字符组时，才会考虑使用 Shift-And 扩展算法。

4.5.1 Shift-And 算法扩展

图 4.9 给出了 Shift-And 扩展算法的伪代码，其中包括了对扩展字符“+”，“*”和“?”的必要预处理过程。该伪代码假设没有可选字符或者重复字符位于模式串的起始位置或者结束位置，而且将该伪代码扩展成能支持字符组的匹配也不困难。

```
Extended-Shift-And (p = p_1 p_2 ... p_m, T = t_1 t_2 ... t_n)
1.    Preprocessing
2.        L ← number of normal characters in p
3.        A ← 0^L /* build B, S and A */
4.        For c ∈ Σ Do B[c] ← 0^L, S[c] ← 0^L
5.        i ← −1
6.        For j ∈ 1...m Do
7.            If p_j = "+" Then S[lastc] ← S[lastc] | 0^{L−i−1}10^i
8.            Else If p_j = "?" Then A ← A | 0^{L−i−1}10^i
9.            Else If p_j = "*" Then
10.               S[lastc] ← S[lastc] | 0^{L−i−1}10^i
11.               A ← A | 0^{L−i−1}10^i
12.           Else /* p_j is a character */
13.               lastc ← p_j
14.               i ← i + 1
15.               B[lastc] ← B[lastc] | 0^{L−i−1}10^i
16.           End of if
17.       End of for
18.       I ← 0^L, F ← 0^L /* build I and F */
19.       For i ∈ 0...L − 1 Do
20.           If A & 0^{L−i−1}10^i ≠ 0^L Then
21.               If F & 0^{L−i}10^{i−1} = 0^L Then
22.                   I ← I | 0^{L−i}10^{i−1}
23.                   F ← F | 0^{L−i−1}10^i
24.               Else
25.                   F ← F & 1^{L−i}01^{i−1}
26.                   F ← F | 0^{L−i−1}10^i
27.               End of if
28.           End of if
29.       End of for
30.   Searching
31.       D ← 0^L
32.       For pos ∈ 1...n Do
33.           D ← ((D << 1) | 0^{L−1}1) & B[t_pos]) | (D & S[t_pos])
34.           Df ← D | F
35.           D ← D | (A & ((∼ (Df − I)) ∧ Df))
36.           If D & 10^{L−1} ≠ 0^L Then report an occurrence ending at pos
37.       End of for
```

图 4.9　扩展 Shift-And 算法描述，可处理包含可选字符和重复字符的模式串

Extended-Shift-And 算法中的预处理阶段包括两个部分：第一部分为第 2 行至第 17 行，来构建位掩码 A、表 S 和表 B。其中表 S 用来存储重复字符信息，位掩码 A 用来存储可字符信息。操作符“*”被当做“+”后尾随“？”来处理。第二部分为第 18 行至第 29，根据位掩码 A 来构建位掩码 I 和 F，第 21 行代码用来判断当前 A 中的活动状态是否属一个新的可选字符块。如果不是，则将 F 中前一位置的 1“移动”到当前位置；如果是，将 F 和 I 中当前位置置 1。整个预处理阶段需要消耗 $O(m+|\Sigma|)$ 时间，匹配阶段需要消 $O(n\lceil L/w\rceil)$ 时间。

相对于预处理阶段，匹配阶段的代码就比较简单，主要使用前面提到的用来处理可选符和重复字符的位并行公式。

ctended-Shift-And 算法举例 下面这个例子是在文本“acccdfabdeeef”中搜索模式串 ab?c*de+f”的成功匹配的结束位置。用它来展示 Extended-Shift-And 算法的匹配过程。然 $m=9$，$L=6$。对于读入的每个字符，下面展示搜索阶段代码作用于 D 和 Df 上的果。

c	B	S
a	000001	000000
b	000010	000000
c	000100	000100
d	001000	000000
e	010000	010000
f	100000	000000
*	000000	000000

$$A = 000110$$
$$I = 000001$$
$$F = 000100$$
$$D = 000000$$

1. 读入字符 a

B	000001
S	000000
D	000001
Df	000101
D	000111

对于两个连续可选字符“b?c*”，转移公式起了作用。

2. 读入字符 c

B	000100
S	000100
D	000100
Df	000100
D	000100

这次转移公式没有起作用。

3. 读入字符 c

B	000100
S	000100
D	000100
Df	000100
D	000100

表 S 允许 D 中第三位仍然保持为活动状态。

4. 读入字符 c

B	000100
S	000100
D	000100
Df	000100
D	000100

5. 读入字符 d

B	0 0 1 0 0 0
S	0 0 0 0 0 0
D	0 0 1 0 0 0
Df	0 0 1 1 0 0
D	0 0 1 0 0 0

6. 读入字符 f

B	1 0 0 0 0 0
S	0 0 0 0 0 0
D	0 0 0 0 0 0
Df	0 0 0 1 0 0
D	0 0 0 0 0 0

7. 读入字符 a

B	0 0 0 0 0 1
S	0 0 0 0 0 0
D	0 0 0 0 0 1
Df	0 0 0 1 0 1
D	0 0 0 1 1 1

对于两个连续可选字符“b?c*”，转移公式起了作用。

8. 读入字符 b

B	0 0 0 0 1 0
S	0 0 0 0 0 0
D	0 0 0 0 1 0
Df	0 0 0 1 1 0
D	0 0 0 1 1 0

对于可选字符“c*”，转移公式起了作用。

9. 读入字符 d

B	0 0 1 0 0 0
S	0 0 0 0 0 0
D	0 0 1 0 0 0
Df	0 0 1 1 0 0
D	0 0 1 0 0 0

此次转移公式没有起作用。前一次转移允许我们忽略字符“c”。

10. 读入字符 e

B	0 1 0 0 0 0
S	0 1 0 0 0 0
D	0 1 0 0 0 0
Df	0 1 0 1 0 0
D	0 1 0 0 0 0

11. 读入字符 e

B	0 1 0 0 0 0
S	0 1 0 0 0 0
D	0 1 0 0 0 0
Df	0 1 0 1 0 0
D	0 1 0 0 0 0

表 S 允许连续读入“e”而自动机仍然有活动状态。

12. 读入字符 e

B	0 1 0 0 0 0
S	0 1 0 0 0 0
D	0 1 0 0 0 0
Df	0 1 0 1 0 0
D	0 1 0 0 0 0

13. 读入字符 f

B	1 0 0 0 0 0
S	0 0 0 0 0 0
D	1 0 0 0 0 0
Df	1 0 0 1 0 0
D	1 0 0 0 0 0

D 的最高位为 1，因此表明在文本位置 13 匹配成功。

4.5.2 BNDM 算法扩展

图 4.10 给出了 BNDM 扩展算法的伪代码。Extended-BNDM 算法的预处理阶段与 Extended-Shfit-And 算法类似，只是位掩码的次序与 Extended-Shift-And 算法相反，并且也需要计算 ℓmin。注意：计算位掩码 I 和 F 的代码没有变，这是由于块的方向没变，仍然是左边为起始位置，右边为结束位置。

```
Extended-BNDM (p = p_1 p_2 ... p_m, T = t_1 t_2 ... t_n)
1.    Preprocessing
2.        L ← number of normal characters in p
3.        ℓmin ← minimum length of an occurrence
4.        A ← 0^L /* build B, S and A */
5.        For c ∈ Σ Do B[c] ← 0^L, S[c] =← 0^L
6.        i ← −1
7.        For j ∈ 1...m Do
8.            If p_j = "+" Then S[lastc] ← S[lastc] | 0^i 10^{L−i−1}
9.            Else If p_j = "?" Then A ← A | 0^i 10^{L−i−1}
10.           Else If p_j = "*" Then
11.               S[lastc] ← S[lastc] | 0^i 10^{L−i−1}
12.               A ← A | 0^i 10^{L−i−1}
13.           Else /* p_j is a character */
14.               lastc ← p_j
15.               i ← i + 1
16.               B[lastc] ← B[lastc] | 0^i 10^{L−i−1}
17.           End of if
18.       End of for
19.       I ← 0^L, F ← 0^L /* build I and F */
20.       For i ∈ 0...L − 1 Do
21.           If A & 0^{L−i−1} 10^i ≠ 0^L Then
22.               If F & 0^{L−i} 10^{i−1} = 0^L Then
23.                   I ← I | 0^{L−i} 10^{i−1}
24.                   F ← F | 0^{L−i−1} 10^i
25.               Else
26.                   F ← F & 1^{L−i} 01^{i−1}
27.                   F ← F | 0^{L−i−1} 10^i
28.               End of if
29.           End of if
30.       End of for
31.   Searching
32.       pos ← 0
33.       While pos ≤ n − ℓmin Do
34.           j ← ℓmin − 1, last ← ℓmin
35.           D ← B[t_{pos+ℓmin}]
36.           If D & 10^{L−1} ≠ 0^L Then last ← j
37.           While D ≠ 0^L AND j > 0 Do
38.               Df ← D | F
39.               D ← D | (A & ((∼ (Df − I)) ∧ Df))
40.               D ← ((D << 1) & B[t_{pos+j}]) | (D & S[t_{pos+j}])
41.               j ← j − 1
42.               If D & 10^{L−1} ≠ 0^L Then /* prefix recognized */
43.                   If j > 0 Then last ← j
44.                   Else check a possible occurrence starting at pos
45.               End of if
46.           End of while
47.           pos ← pos + last
48.       End of while
```

图 4.10 扩展 BNDM 算法描述，可以处理包含可选字符和重复字符的模式串。假设 $\ell min > 1$

搜索阶段稍微复杂一些。首先，用窗口最后一个字符对应的表 B 中的内容来初始化位掩码 D。然后，在当前窗口中判断是否匹配成功，接着再处理下一窗口中的字符，这样循环往复直至窗口超过文本大小。对于包含限长空位的模式串，需要前向验证过程来判断窗口中的字符串是否匹配成功。

由于能够成功匹配扩展模式串的字符串长度没有上限，因此当需要验证窗口中的字符串时，并不能确定需要重新读入多少字符。这时只能一直验证下去直至自动机中没有活动状态或到达终止状态，此时才能判断是否为一个成功匹配。

Extended-BNDM 算法举例　下面通过在文本 “acccdfabdeeef” 中搜索模式串 “ab?c*de+f” 来展示 Extended-BNDM 算法的匹配过程。

c	B	S
a	100000	000000
b	010000	000000
c	001000	001000
d	000100	000000
e	000010	000010
f	000001	000000
*	000000	000000

$$
\begin{aligned}
m &= 9 \\
L &= 6 \\
\ell min &= 4 \\
\\
A &= 011000 \\
I &= 000100 \\
F &= 010000
\end{aligned}
$$

1. `accc` `dfabdeeef`

$last \leftarrow 4$

读入字符 c	B	001000
	D	001000
读入字符 c	B	001000
	S	001000
	Df	011000
	D	011000
	D	001000
读入字符 c	B	001000
	S	001000
	Df	011000
	D	011000
	D	001000
读入字符 a	B	100000
	S	000000
	Df	011000
	D	011000
	D	100000

D 最高位为 1 并且 $j = 0$，因此我们需要前向验证文本窗口 “acccdfa...”，当读入第六个字符时自动机没有活动状态，没有匹配模式串。因此我们将窗口左移 $last = 4$。

2. accc `dfab` `deeef`

$last \leftarrow 4$

读入字符 b	B	010000
	D	010000
读入字符 a	B	100000
	S	000000
	Df	010000
	D	010000
	D	100000

D 最高位为 1 但是 $j > 0$，因此我们重置 $last \leftarrow 2$。

读入字符 f	B	000001
	S	000000
	Df	110000
	D	100000
	D	000000

D 中没有活动状态，因此我们将窗口右移 $last = 2$。

3. acccdf [abde] eef

$last \leftarrow 4$

读入字符 e	B	000010
	D	000010
读入字符 d	B	000100
	S	000000
	Df	010010
	D	000010
	D	000100
读入字符 b	B	010000
	S	000000
	Df	010100
	D	011100
	D	010000
读入字符 a	B	100000
	S	000000
	Df	010000
	D	010000
	D	100000

D最高位为1并且$j = 0$，因此我们需要前向验证文本窗口“abdeeef”。显然匹配成功，因此报告在位置7处有成功匹配。然后，我们将窗口右移$last = 4$。

由于窗口已经移出文本数据长度，因此匹配过程结束。

.6 多模式串搜索

本节中讨论对多个扩展模式串的搜索问题。由于前面描述的用来解决扩展模式串的主要技术都是基于位并行的，因此这里也希望给出一个基于位并行的多模式串搜索算法。遗憾的是，在第3章中提到的多模式串算法都不是基于位并行的。

唯一有用的方法是在3.2.1节和3.4.1节中提到的将多个自动机压入到一个机器字中，然后再运用类似Shift-And或BNDM的匹配算法。如果搜索多个具有相同类型的扩展模式串，可以使用这种技术：将多个扩展类型的自动机压入到一个机器字中，并且对这个机器字使用相对应的转移公式，更新自动机的状态。特别需要注意不同模式串和它们各自对应的自动机的初始自环之间的限制。

因为在一个机器字中只能表示有限的扩展模式串，因此多扩展模式串搜索算法的能力有限。

在3.4.1节中，使用BNDM算法时要假设所有的字符串有相同的长度，否则将以最短长度对其他字符串进行截断处理。因此在扩展该BNDM算法时，由于模式串的ℓmin值不相同，需要做类似的处理：将模式串截断，使得它们具有相同ℓmin值。

3.4.1节介绍的截断过程需要在窗口中从头开始验证整个模式串。这没有给扩展形式带来多大麻烦，因为扩展形式本来就需要在窗口中进行对整个模式串的前向验证。

最简单的截断方式是使用模式串的最长前缀的长度作为ℓmin，尽管这可能带来匹配效率的下降（因为使用了模式的子串）。[Nav01b]中使用了这种方法。注意：此时的验证过程将变得更加复杂，这是因为，对窗口的前面和后面两部分都需要进行验证。

4.7　其他算法和参考文献

允许任意字符的字符串匹配问题是比较简单的，可以将它归到字符组扩展中。允许任意字符的匹配是允许模式串和文本中的某一位置可以匹配整个字母表 Σ。[FP74]中给出了一个时间复杂度为 $O(n\log^2 n)$ 的算法，该算法基于卷积。

对于允许通配符的模式串，[FP74]给出了时间复杂度为 $O(n\log^2 m\log\log m\log|\Sigma|)$ 的算法，而[Abr87]给出了复杂度为 $O(n+m\sqrt{n}\log n\sqrt{\log\log n})$ 的算法。对于包含字符组的模式串，[Pin85]给出了具有与[FP74]同样复杂度的算法。[Abr87]考虑了更通用的字符组形式，并且给出了亚二次方的算法。

所有这些算法都是理论性很强的，很难应用于实际中。[MP94]是一篇很好的综述文章，其中给出了在非标准字符学中的理论问题，并给出了现有的结果。

在计算生物学领域中，人们对限长空位的扩展匹配有很大兴趣。例如，允许空位长度为负数，即允许模式串中的空位前后部分能够重叠出现。对这类模式串的搜索可以参考[MM89，KM95，Mye96]，它们采用了近似匹配方法。近似匹配将在第 6 章中详细介绍。

第 5 章　正则表达式匹配

5.1　基本概念

本章介绍在文本和生物学序列中搜索正则表达式的算法。正则表达式常用于文本检索和计算生物学中，它可以表示比单字符串、字符串集合、扩展字符串更复杂的搜索模式。首先，给出正则表达式和它所表达语言的形式化定义。

定义　一个正则表达式 RE 是符号集合 $\Sigma \cup \{ \varepsilon, |, \cdot, *, (,) \}$ 上的一个字符串，它可以递归定义如下：

空字符 ε 是正则表达式。

任意字符 $\alpha \in \Sigma$ 是正则表达式。

如果 RE_1 和 RE_2 都是正则表达式，则 (RE_1)，$(RE_1 \cdot RE_2)$，$(RE_1 \mid RE_2)$ 和 $RE_1*)$ 亦是正则表达式。

通常将 $(RE_1 \cdot RE_2)$ 简写为 $RE_1 RE_2$。举例来说，正则表达式(((A·T)|(G·A)) (((A·G)|((A·A)A))*))可以化简为(AT|GA)((AG|AAA)*)，从而使得可读性更好。符号“*”，“·”，“|”称为操作符，可以通过为每个操作符赋予优先级来消除更多的括号。但为了可读性考虑，本章不做如此处理。此外，为方便起见，本章中使用额外的后缀操作符“+”，它的含义是 $RE+ = RE \cdot RE*$。下面，定义正则表达式所表达的语言。

定义　正则表达式 RE 所表达的语言是 Σ 上的一个字符串集合。根据 RE 的结构，可以将它递归定义如下：

- 如果 RE 是 ε，则 $L(RE) = \{\varepsilon\}$，即空串。
- 如果 RE 是 $\alpha \in \Sigma$，则 $L(RE) = \{\alpha\}$，即包含一个字符的单串。
- 如果 RE 是 (RE_1) 这种形式，则 $L(RE) = L(RE_1)$。
- 如果 RE 是 $(RE_1 \cdot RE_2)$ 这种形式，则 $L(RE) = L(RE_1) \cdot L(RE_2)$，其中 $W_1 \cdot W_2$ 可以看做是字符串集 w 的集合。其中，$w = w_1 w_2$ 并且 $w_1 \in W_1$，$w_2 \in W_2$。操作符“·”表示字符串的连接。

- 如果 RE 是 $(RE_1 \mid RE_2)$ 这种形式，则 $L(RE) = L(RE_1) \cup L(RE_2)$，是这两种语言的并集，“|” 称为并操作符。
- 如果 RE 是 (RE_1*) 这种形式，则 $L(RE) = L(RE)^* = \bigcup_{i \geqslant 0} L(RE_1)^i$，其中 $L^0 = \{\varepsilon\}$ 并且 $L^i = L \cdot L^{i-1}$，它表示字符串集合是由 0 个或多个 RE_1 表达的字符串连接而成。“*” 称为星操作符。

举例来说，L((AT|GA)((AG|AAA)*)) = {AT, GA, ATAG, GAAG, ATAAA, GAAAA, ATAGAG, ATAGAAA, ATAAAAG, ATAAAAAA, GAAGAG, GAAGAAA, ...}。注意，根据星操作符的定义，Σ^* 表示字母表 Σ 上的所有字符串。

正则表达式 RE 的规模是指它所包含的属于字母表 Σ 的字符个数，(AT|GA)((AG|AAA)*) 的规模就是 9。在下述算法复杂度分析中，它是一个重要的度量。

在文本 T 中搜索正则表达式 RE 的问题就是找出文本中所有属于语言 $L(RE)$ 的子串。本章将描述搜索过程的主要步骤。

图 5.1 中总结了在文本中搜索正则表达式的经典方法。首先，正则表达式被解析成一棵表达式树，然后将表达式树转换成非确定有限自动机（NFA）。有多种方法可以完成这种转换，本章将介绍实践中两种最常用的 NFA 构造方法，一种是 Thompson 构造法[Tho68]，另一种是 Glushkov 构造法[Glu61]。

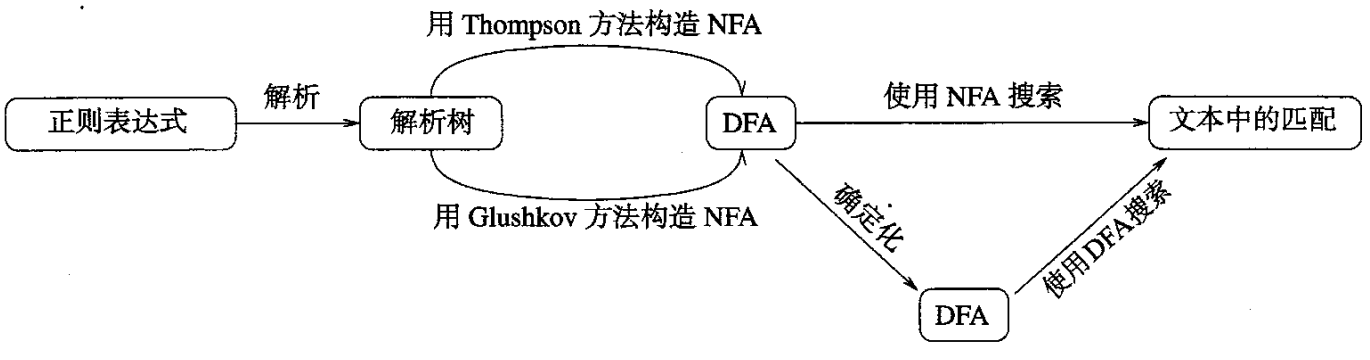


图 5.1　文本中搜索正则表达式的经典方法

直接利用 NFA 进行搜索是可行的，目前也已经提出了多种方法。由于需要保存活动状态列表，并且每读入一个新文本字符都要更新这个列表，因此这些算法处理速度通常比较慢。一般地，搜索过程的最坏情况时间复杂度是 $O(mn)$，但是需要的存储空间并不多。

另一种策略是将 NFA 转变成确定的有限自动机（DFA），这样在每个文本字符到来时可以直接进行转换，从而达到 $O(n)$ 的搜索时间。但另一方面，构造这样的一个自动机所需的最坏情况时间和空间复杂度都是 $O(2^m)$。

还有第三种策略，就是先使用多模式匹配或其他相关方法对文本进行过滤，当发现一个锚时，就预示附近区域中可能存在成功匹配。这时，可以选择前两种策略中的一种对这个区域进行仔细验证。图 5.2 描述了这一过程。

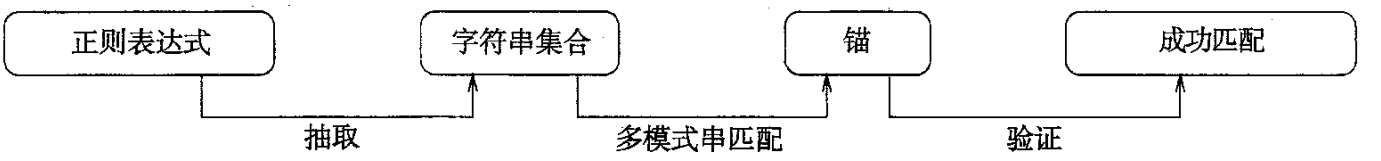


图 5.2 文本中正则表达式的过滤搜索

以上几种策略可以组合起来使用，并且可以使用位并行方法来加快搜索过程。

值得指出的是，为了便于计算，自动机构造方法大多都使用树来表示正则表达式 RE。其中的叶节点使用 RE 中属于字母表 Σ 的字符以及符号 ε 来标识，而内部节点则使用操作符来标识。标识为“|”或“·”的节点拥有两个子节点，分别表示子表达式 RE_1 和 RE_2，标识为“*”的节点只有一个对应于表达式 RE_1 的子节点。由于一些操作符之间是可交换或者可结合的，从而导致树表示并不唯一。图 5.3 是正则表达式(AT|GA)((AG|AAA)*)对应的树表示。

在 5.8 节中将解释如何解析正则表达式以获得其对应的树表示。在下面的内容中，权且认为解析树已经构造好，并且可以正确表示正则表达式。

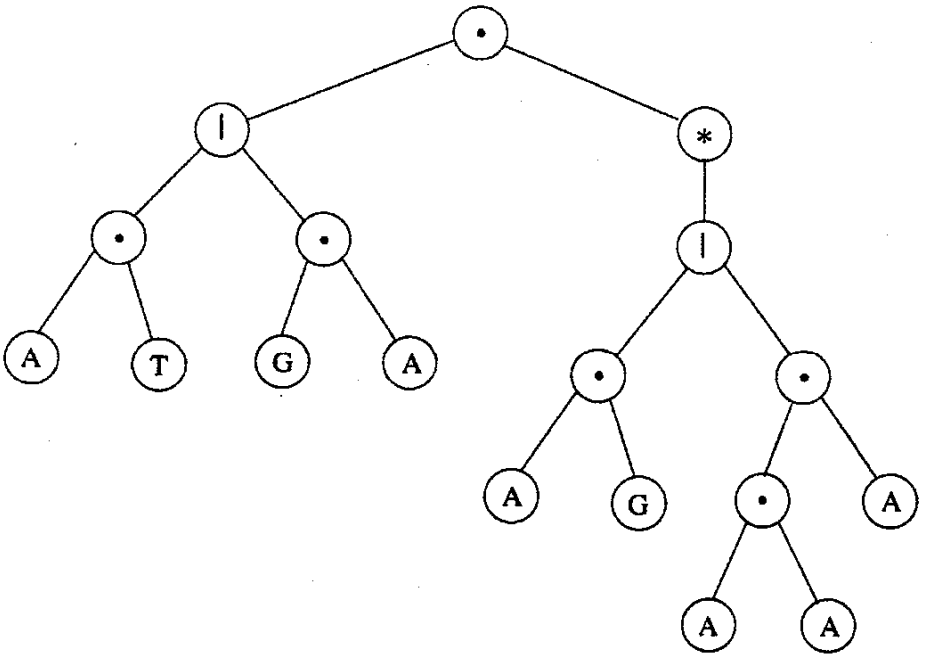


图 5.3 正则表达式(AT|GA)((AG|AAA)*)的树表示

在下述算法描述中，使用符号 $\boxed{\cdot}\ (v_l, v_r)$ 表示一棵连接树，其根节点是“·”，子节点是

v_l 和 v_r。同样地，符号 [|] (v_l, v_r) 也表示一棵连接树，其根节点是 "|"，子节点是 v_l 和 v_r。符号 [*] (v_*) 表示一棵有唯一子节点 v_* 的树。

5.2 构造 NFA

有多种方法用来从正则表达式构造 NFA [Glu61, Tho68, CP92, BS86, BK93, HSW97]，其中有两种最重要，在实践中经常使用。

Thompson 构造法[Tho68]简单，并且构造的 NFA 中状态数量（最多 $2m$ 个）和转移数量（最多 $4m$ 个）都是线性的。然而，这种自动机存在 ε-转移，即空转移，这样在不读入字符或读入空字符串 ε 时都会发生状态转移。

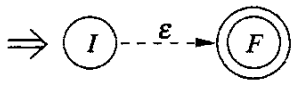
而通过 Glushkov 构造法[Glu61, BS86]构造的 NFA 中，状态数量是固定的，为 $m+1$ 个，而转移数量在最坏情况下变成为 $O(m^2)$。但是无论怎样，这种方法没有空转移。原始构造方法的时间复杂度为 $O(m^3)$，但在[BK93]中，它被降低到 $O(m^2)$。

5.2.1 Thompson 自动机

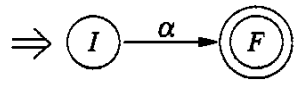
Thompson 构造法[Tho68]是一种从正则表达式构造自动机的方法。它使用自动机直接表示一个正则表达式对应的树表示，并且使用 ε-转移来简化这个表示过程。

Thompson 构造法的核心思想是形成正则表达式 RE 对应的树表示 T_{RE}，然后自底向上地对树 T_{RE} 的每个节点 v，构造一个自动机 $Th(v)$ 来识别以 v 为根的子树所表达的语言。根据不同类型的中间节点和叶节点，有不同的自动机构造方法，具体情况如下。

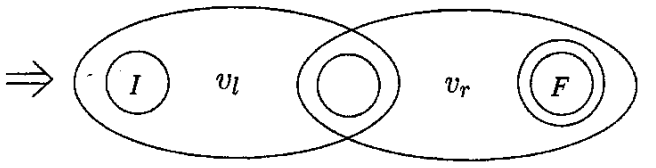
（i）空字的构造方法。自动机由 ε-转移连接两个节点而组成。



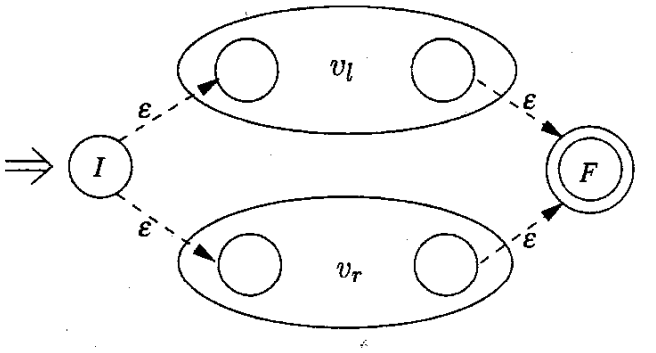
（ii）单字符 α 的构造方法，它与空词的构造方法类似，只不过转移是使用字符来标识，而不是使用空字符串。



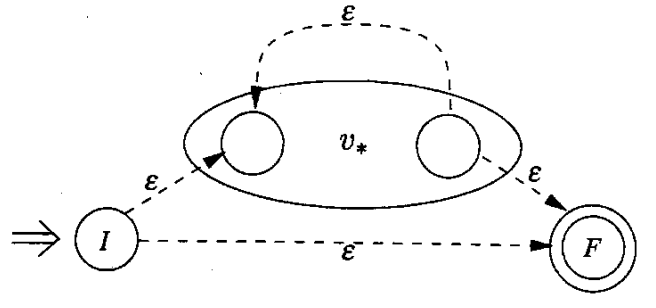
（iii）相连节点的构造法。将两个子节点 v_l 和 v_r 对应的 Thompson 自动机合并，即第一个自动机的终止状态成为第二个自动机的初始状态。



（iv）联合节点的构造法，其思想源于：必须通过两个子节点对应的自动机$Th(v_l)$和$Th(v_r)$中的一个。这种构造需要ε-转移。构造过程中，必须添加两个新的状态：一个是初始状态I，从它有两个ε-转移分别到自动机$Th(v_l)$和$Th(v_r)$的初始状态；另一个是终止状态F，从自动机$Th(v_l)$和$Th(v_r)$的终止状态分别由ε-转移到达终止状态F。因为从I到F的路径必定通过两个自动机中的一个，所以它表达的语言是$RE_{v_l} \mid RE_{v_r}$。



（v）星节点的构造方法，它使用了同联合节点构造法相同的思想。首先，因为对于语言RE_{v_*}，节点v的唯一子节点v_*可以被重复任意多次，所以需要创建一个从自动机$Th(v_*)$的终止状态指向其初始状态的ε-转移。但是，星符号也意味着自动机$Th(v_*)$可能被忽略。因此，需要创建初始节点I和终结节点F，并用一个ε-转移把它们连接起来。另外，再创建两条ε-转移分别用来从节点I指向$Th(v_*)$的初始状态以及从$Th(v_*)$的终止状态指向F。最终，自动机识别的语言是$(RE_{v_*})^*$。



整个 Thompson 算法包含自底向上的树遍历，同时保证根节点开始构造的自动机即为能够表示整个正则表达式的 Thompson 自动机。图 5.4 给出了算法的递归伪代码。

Thompson_recur(v)
1. **If** $v = [\,|\,]\,(v_l, v_r)$ OR $v = \boxed{\cdot}\,(v_l, v_r)$ **Then**
2. $\quad Th(v_l) \leftarrow$ **Thompson_recur**(v_l)
3. $\quad Th(v_r) \leftarrow$ **Thompson_recur**(v_r)
4. **Else If** $v = \boxed{*}\,(v_*)$ **Then** $Th(v_*) \leftarrow$ **Thompson_recur**(v_*)
5. **End of if**
   /* end of the recursive part, we build the automaton for the current node */
6. **If** $v = (\varepsilon)$ **Then Return** construction *(i)*
7. **If** $v = (\alpha)$, $\alpha \in \Sigma$ **Then Return** construction *(ii)*
8. **If** $v = \boxed{\cdot}\,(v_l, v_r)$ **Then Return** construction *(iii)* on $Th(v_l)$ and $Th(v_r)$
9. **If** $v = [\,|\,](v_l, v_r)$ **Then Return** construction *(iv)* on $Th(v_l)$ and $Th(v_r)$
10. **If** $v = \boxed{*}\,(v_*)$ **Then Return** construction *(v)* on $Th(v_*)$

Thompson(RE)
11. $v_{RE} \leftarrow$ **Parse**(RE\$,1) /* parse the regular expression (Section 5.8) */
12. $Th(v_{RE}) \leftarrow$ **Thompson_recur**(v_{RE}) /* build the automaton on the tree */

图 5.4　Thompson 算法。正则表达式对应的树表示的自动机递归构造

Thompson 自动机的性质　在构造树表示中的每一个节点时，自动机中相应地最多增加 2 个状态和 4 个转移。因此，在构造完成后，状态与转移的数量最多为 $2m$ 和 $4m$ 个。可以计算出更精确的上限。但值得指出的是：状态和转移的数量对 m 来说是线性的，并且，NFA 中每个节点最多只有 2 个入边和 2 个出边，整个 NFA 只有 1 个初始状态和 1 个终止状态。

另一个有趣的性质，是所有非空转移的箭头都是从状态 i 指向状态 $i+1$。在本章稍后提到的解析器中可以证明：从左向右处理正则表达式中的字符时，这种性质总是正确的。

算法复杂度　整个算法的时间复杂度是线性的，这是因为树表示中的每个节点对应的自动机都可以在常数时间内构造完成。

构造 Thompson 自动机示例　从(AT|GA)((AG|AAA)*)的树表示（参见图 5.3）来构造对应的自动机。除了连接字符的基本步骤外，构造过程如图 5.5 所示。

5.2.2　Glushkov 自动机

Berry 和 Sethi 在[BS86]中对 Glushkov 自动机[Glu61]构造法进行了改进，使它变得更通俗易懂。

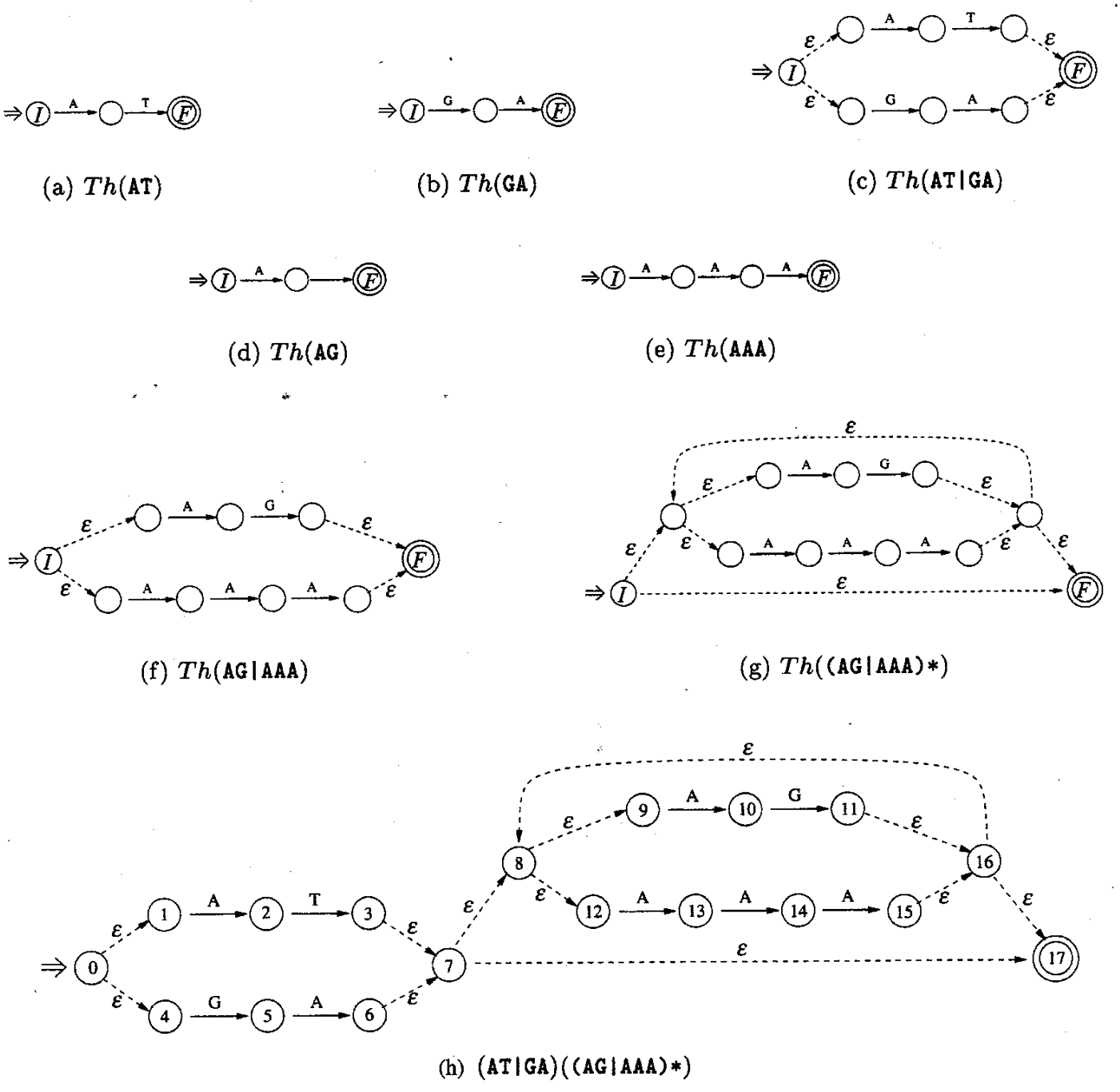


图 5.5 正则表达式(AA|AT)((AG|AAA)*)的 Thompson 自动机构造

通过只对字符计数，可以标记出字母表 Σ 中每个字符在正则表达式 RE 中的位置。如，(AT|GA)((AG|AAA)*)标记为 $(A_1T_2|G_3A_4)((A_5G_6|A_7A_8A_9)*)$。使用 $\overline{RE}$ 表示对正表达式 RE 进行标记的标记表达式，使用 $L(\overline{RE})$表示它对应的语言，其中每个字符都含它的位置索引。于是对应上例有 $L((A_1T_2|G_3A_4)((A_5G_6|A_7A_8A_9)^*)) = \{A_1T_2,\ G_3A_4,\ A_1T_2A_5G_6,\ G_3A_4A_5G_6,\ A_1T_2A_7A_8A_9,\ G_3A_4A_7A_8A_9,\ A_1T_2A_5G_6A_5G_6,\ldots\}$。用 $os(\overline{RE}) = \{1\ldots m\}$ 表示 $\overline{RE}$ 中位置的集合，用 $\overline{\Sigma}$ 表示标记后的字母表。

首先，在标记表达式 $\overline{RE}$ 上构造自动机，它可以识别语言 $L(\overline{RE})$。然后，通过消除所

有字符的位置索引，可以从中抽取出 Glushkov 自动机，从而识别语言 $L(RE)$（详见下文）。

字符位置的集合通过再加上一个初始状态 0 而被当做自动机状态集合的索引。创建 $m+1$ 个状态，标记为 0 到 m，状态 j 表示已经从文本中读取了一个字符串，该字符串在 NFA 中的位置 j 结束。当再读入一个新的字符α时，需要知道从位置 j 通过α可以到达的位置，而这可以通过 Glushkov 自动机计算出来。

为了进一步解释算法，需要以下四个新的定义。其中，使用 α_y 表示 $\overline{RE}$ 中位置 y 处的被索引字符。

定义 1　$First(\overline{RE}) = \{x \in Pos(\overline{RE}),\ \exists u \in \overline{\Sigma}^*,\ \alpha_x u \in L(\overline{RE})\}$

集合 $First(\overline{RE})$ 表示 $L(RE)$ 的初始状态集合，也就是说，在这些位置可以开始读入字符。在上述例子中，$First((A_1T_2|G_3A_4)((A_5G_6|A_7A_8A_9)*)) = \{1,3\}$。

定义 2　$Last(\overline{RE}) = \{x \in Pos(\overline{RE}),\ \exists u \in \overline{\Sigma}^*,\ u\alpha_x \in L(\overline{RE})\}$

集合 $Last(\overline{RE})$ 表示 $L(\overline{RE})\}$的终止状态集合，也就是说，在这些位置可以识别出 $L(RE)$ 中的字符串。在上述例子中，$Last((A_1T_2|G_3A_4)((A_5G_6|A_7A_8A_9)*)) = \{2,4,6,9\}$。

定义 3　$Follow(\overline{RE},x) = \{y \in Pos(\overline{RE}),\ \exists u,v \in \overline{\Sigma}^*,\ u\alpha_x\alpha_y v \in L(\overline{RE})\}$

集合 $Follow(\overline{RE},x)$ 表示在 $Pos(\overline{RE})$ 中从 x 可以到达的所有位置。在上述例子中，从位置 6 可以到达的位置集合为 $Follow((A_1T_2|G_3A_4)((A_5G_6|A_7A_8A_9)*),6) = \{7,5\}$。

此外，还需要一个函数 $Empty_{RE}$ 表示空字 ε 是否出现在 $L(RE)$ 中。

定义 4　函数 $Empty_{RE}$ 可以递归定义如下：

$$
\begin{array}{lcl}
Empty_{\varepsilon} & = & \{\varepsilon\} \\
Empty_{\alpha\in\Sigma} & = & \emptyset \\
Empty_{RE_1|RE_2} & = & Empty_{RE_1} \cup Empty_{RE_2} \\
Empty_{RE_1\cdot RE_2} & = & Empty_{RE_1} \cap Empty_{RE_2} \\
Empty_{RE^*} & = & \{\varepsilon\}
\end{array}
$$

当 ε 属于 $L(RE)$ 时，它的值为 $\{\varepsilon\}$；否则它的值为 $\emptyset$。

确定的 Glushkov 自动机 $\overline{GL}$ 可以识别语言 $L(\overline{RE})$，它通过下述方法进行构造：

$$\overline{GL} = (S, \Sigma, I, F, \overline{\delta})$$

其中：

（i）$S=\{0,1,\ldots,m\}$是状态集合，它是位置$Pos(\overline{RE})$的集合，初始状态为$I=0$。

（ii）$F=Last(\overline{RE})\cup(Empty_{RE}\cdot\{0\})$是终止状态集合。通常，如果状态（位置）$i$属于$Last(\overline{RE})$，则$i$是一个终止状态。当空字$\varepsilon$属于$L(\overline{RE})$时，初始状态0也是终止状态。这时，$Empty_{RE}=\{\varepsilon\}$，则$Empty_{RE}\cdot\{0\}=\{0\}$；否则，$Empty_{RE}\cdot\{0\}=\emptyset$。

（iii）$\overline{\delta}$是自动机的转移函数，定义为：

$$\forall x\in Pos(\overline{RE}),\ \forall y\in Follow(\overline{RE},x),\ \ \overline{\delta}(x,\alpha_y)=y \qquad (5.1)$$

通常情况下，如果状态y在状态x之后，那么从x到y有一个转移α_y。从初始状态开始的转移定义为：

$$\forall y\in First(\overline{RE}),\ \ \overline{\delta}(0,\alpha_y)=y \qquad (5.2)$$

图 5.6 中给出了标记正则表达式$(A_1T_2|G_3A_4)((A_5G_6|A_7A_8A_9)*)$对应的 Glushkov 自机。

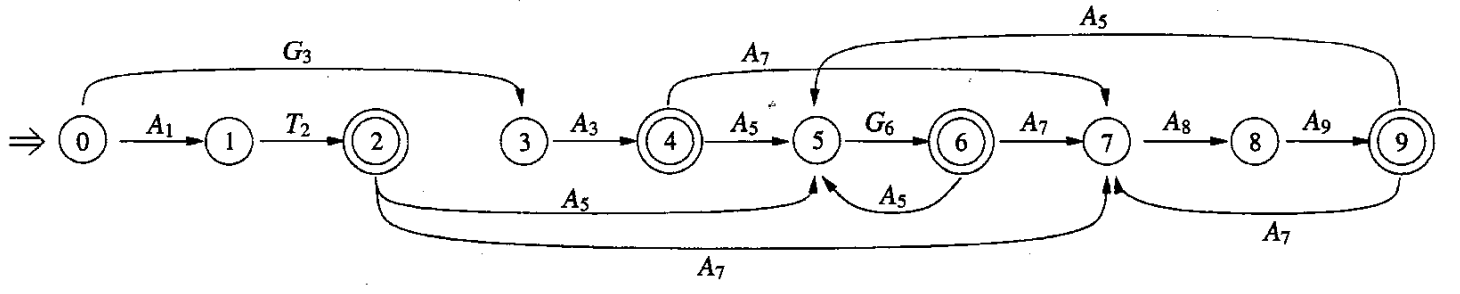


图 5.6 标记正则表达式$(A_1T_2|G_3A_4)((A_5G_6|A_7A_8A_9)*)$对应的 Glushkov 自动机。初始状态为 0，带有双圈的是终止状态

为了获得原始RE的 Glushkov 自动机，只要简单地去除标记自动机的位置索引即可。这个过程中，自动机通常会变成非确定的。新的自动机对应语言$L(RE)$。例子(AT|GA)((AG|AAA)*)对应的 Glushkov 自动机如图 5.7 所示。

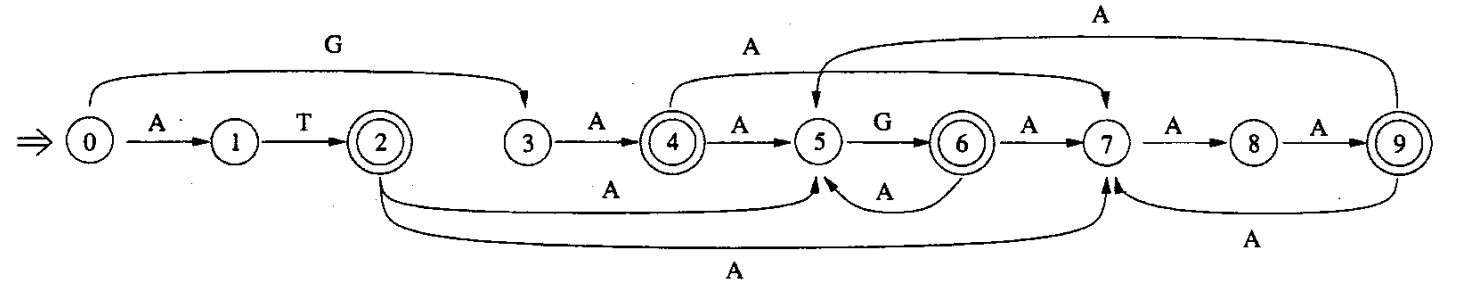


图 5.7 正则表达式(AT|GA)((AG|AAA)*)的 Glushkov 自动机，初始状态为 0，带有双圈的是终止状态。这个自动机来源于标记自动机，只是简单地删除了位置索引

Glushkov 算法基于正则表达式 RE 对应的树表示 T_{RE}（参见图 5.3），树中的每个节点 v 都代表 RE 的子表达式 RE_v。算法中使用了下列与 v 相关的变量：

- $First(v)$：表示集合 $First(\overline{RE_v})$ 中所有位置的列表。
- $Last(v)$：表示集合 $Last(\overline{RE_v})$ 中所有位置的列表。
- $Empty_v$：如果 $L(RE_v)$ 中包含空串 ε，它表示 $\{\varepsilon\}$；否则，它表示 $\emptyset$。

对于每个节点，这些变量都是后序计算的，也就是说，先计算出 v 的所有子节点的变量值，然后再计算 v 的变量值。如果 v 是 "|" 或 "·"，则把它的两个子节点分别记做 v_l 和 v_r；如果 v 表示 "*"，则把它唯一的子节点记做 v_*。

集合 $Follow(x)$ 是一个全局变量，对于每一个节点 v，$Follow(x)$ 的值要根据它在子表达式 $\overline{RE_v}$ 中位置的变化而不断更新。

图 5.8 给出了 Glushkov_variables(v_{RE}, $lpos$) 的递归算法。对正则表达式 RE 的树表示中的每个节点 v，该算法提供计算变量 $First(v)$，$Last(v)$，$Follow(x)$ 和 $Empty_v$ 的方法。为了计算每个节点 v_{RE} 的值，该算法使用后序法遍历树中所有的节点。从算法的第 9 行可以看出，每个字符的位置索引都可以快速计算出来。

整个 Glushkov 算法包括：将 RE 转化成树 v_{RE}，并且使用 Glushkov_variables (v_{RE},0) 计算它对应的变量，然后从树 v_{RE} 的根对应的变量开始构造 Glushkov 自动机。该算法的伪代码在图 5.9 中给出。

Glushkov 自动机的性质　该自动机有两个性质很重要：一是 NFA 是 ε-free 的（没有空转移的），二是所有指向状态 y 的箭头都标记着同样的字符，称为 α_y，这在公式（5.1）和公式（5.2）中显而易见。

算法复杂度算法　在最坏情况下的复杂度由函数 Glushkov_variables 决定。在该函数中，除了 "*"，所有集合的合并都可以被分解，并且在 $O(1)$ 时间内完成。第 19 行的 For 循环最坏情况是 $O(m)$。最坏情况下的复杂性主要在第 22 行，即对 "*" 的计算。因为 $Follow(x)$ 和 $First(v_*)$ 可能相交，所以集合合并的最坏时间复杂度为 $O(m)$。又因为对 "*" 的计算在 For 循环中有 $O(m)$ 次迭代，因此，整个循环的最坏时间复杂度为 $O(m^2)$。最终，可能存在的 $O(m)$ 个 "*" 运算从而导致整个算法在最坏情况下的复杂度为 $O(m^3)$。

对该算法的两个改进[BK93, CP92]可以将复杂度降至 $O(m^2)$，它们都是通过减少 For 循环中的 "*" 来实现的，但分别利用了不同的性质。[BK93]利用了这样的事实：

$$Follow(\overline{RE*}, x) = \left[\, Follow(\overline{RE*}, x) \setminus First(\overline{RE*}) \,\right] \cup First(\overline{RE*})$$

```
Glushkov_variables(v_RE, lpos)
        /* postfix computation, we compute recursively the children first */
1.  If v = [ | ] (v_l, v_r) OR v = [·] (v_l, v_r) Then
2.      lpos ← Glushkov_variables(v_l, lpos)
3.      lpos ← Glushkov_variables(v_r, lpos)
4.  Else If v = [*] (v_*) Then lpos ← Glushkov_variables(v_*, lpos)
5.  End of if
        /* end of the recursive part, we compute the values for the current node */
6.  If v = (ε) Then
7.      First(v) ← ∅, Last(v) ← ∅, Empty_v ← {ε}
8.  Else If v = (α) , α ∈ Σ Then
9.      lpos ← lpos + 1
10.     First(v) ← {lpos}, Last(v) ← {lpos}, Empty_v ← ∅, Follow(lpos) ← ∅
11. Else If v = [ | ] (v_l, v_r) Then
12.     First(v) ← First(v_l) ∪ First(v_r)
13.     Last(v) ← Last(v_l) ∪ Last(v_r)
14.     Empty_v ← Empty_{v_l} ∪ Empty_{v_r}
15. Else If v = [·] (v_l, v_r) Then
16.     First(v) ← First(v_l) ∪ (Empty_{v_l} · First(v_r)),
17.     Last(v) ← (Empty_{v_r} · Last(v_l)) ∪ Last(v_r),
18.     Empty_v ← Empty_{v_l} ∩ Empty_{v_r}
19.     For x ∈ Last(v_l) Do Follow(x) ← Follow(x) ∪ First(v_r)
20. Else If v = [*] (v_*) Then
21.     First(v) ← First(v_*), Last(v) ← Last(v_*), Empty_v ← {ε}
22.     For x ∈ Last(v_*) Do Follow(x) ← Follow(x) ∪ First(v_*)
23. End of if
24. Return lpos
```

图 5.8 Glushkov 算法的递归部分。这个函数用来计算正则表达式 $\overline{RE}$对应的树表示每一个节点 v 的 $First(v)$，$Last(v)$，$Follow(x)$和 $Empty_v$ 的值

```
Glushkov(RE)
        /* parse the regular expression (Section 5.8) */
1.  v_RE ← Parse(RE$,1)
        /* build the variables on the tree */
2.  m ← Glushkov_variables(v_RE,0)
        /* building the automaton */
3.  Δ = ∅
4.  For i ∈ 0...m Do create state i
5.  For x ∈ First(v_RE) Do Δ ← Δ ∪ {(0, α_x, x)}
6.  For i ∈ 0...m Do
7.      For x ∈ Follow(i) Do Δ ← Δ ∪ {(i, α_x, x)}
8.  End of for
9.  For x ∈ Last(v_RE) ∪ (Empty_{v_RE} · {0}) Do mark x as terminal
```

图 5.9 完整的 Glushkov 算法。通常情况下自动机是不确定性的，它的状态转移函数用符号Δ来表示。初始状态为 0

[CP92]利用的事实是：

$$Follow(\overline{RE*}, x) = Follow(\overline{RE*}, x) \cup \left[First(\overline{RE*}) \setminus Follow(\overline{RE*}, x) \right]$$

通常，正则表达式的规模同文本相比是非常小的，所以$O(m^3)$时间复杂度的算法是足够的。此外，使用位并行方法对状态集合进行操作，可以获得$O(m^2\lceil m/w \rceil)$的时间复杂度。这对于小规模的正则表达式，相当于$O(m^2)$。

5.3　搜索正则表达式的经典方法

本节将介绍在文本中搜索正则表达式的经典方法。首先会介绍两个极端的方法：纯 NFA 模拟和纯 DFA 模拟。第三种方法较为折中，利用空间换取时间。

5.3.1　Thompson 的 NFA 模拟

[Tho68]中，Thompson 在给出了 NFA 定义的同时，还在对该 NFA 直接模拟的基础上提出了一个$O(mn)$算法，这个算法称为 NFAThompson。虽然 NFAThompson 算法现在已失去了竞争力，但它是本章后面介绍的若干颇具竞争力算法的基础。

Thompson 将当前活动状态的集合存储起来，对于每次读入的新字符，依次查看当前的每一个活动状态，以获得被激活的新状态。然后，将这些新状态加入一个新的活动状态集合。对这些新的活动状态，再跟随其所有的ε-转移获得其他所有可到达状态，并将其加入新活动状态集合中。

在构造 Thompson 自动机时，每个状态都有$O(1)$的出转移，并且活动状态数为$O(m)$。因此，在对状态集合进行恰当表示的情况下（例如，使用位向量），需要$O(m)$时间产生新的活动状态集合。如果考虑ε-转移不在活动状态之间进行传播，那么也同样需要$O(m)$时间。此外，算法还需要$O(m)$的额外空间。

注意：可以使用位并行方法来存储位向量。在5.4 节中将介绍一种巧妙使用位并行方法的算法。

5.3.2　使用确定自动机

字符串匹配领域的一个早期成就，是在文本中搜索一个正则表达式，存在$O(n)$时间复杂度的算法。如上所述，该技术包括将正则表达式转化为 DFA，然后用 DFA 在文本中进行搜索。最简单的方法是首先建立 NFA（可以使用在上节中提到的几种技术：构造 Thompson 或者 Glushkov 自动机），然后将 NFA 转化为 DFA。

这种算法称为 DFAClassical 算法，可以在有关编译技术的很多经典书籍中找到相关介绍（例如[ASU86]）。该算法的核心思想是，当使用非确定的自动机遍历文本时，会经过很多转移，因此会激活一个状态集合。然而，确定自动机在一个时刻只有一个确定的活动状态，因此可以在非确定自动机的状态集合上定义相对应的确定自动机。该思想的关键在于，确定自动机的当前唯一状态就是非确定自动机的当前活动状态集合。

为了形式化地描述这些概念，首先进行如下定义：

定义 *在 NFA 中，$E(s)$表示状态 s 对应的ε闭包，它是在 NFA 中状态 s 能够通过 s-转移到达的所有状态的集合。*

注意：在类 Glushkov 的ε-free 自动机中，对所有状态 s，$E(s)=\{s\}$总是成立，但对使用 Thompson 构造法形成的自动机，情况并非如此。

现在给出从 NFA 到 DFA 转化的形式化定义。与 1.3.3 节中相一致，令 NFA 为 (Q,Σ,I,F,Δ)。DFA 定义为：

$$(\wp(Q),\ \Sigma,\ E(I),\ F',\ \delta)$$

其中，$F'=\{f \in \wp(Q),\ f\cap F=\emptyset\}$，并且$\delta(S,\sigma)=\bigcup_{s',\ \exists s\in S,\ (s,\sigma,s')\in\Delta} E(s')$。

上述定义中的$\delta(S,\sigma)$表示：对于S中所有可能的活动状态 s，可以通过标记为字符σ的转移找到所有可能的状态 s'，然后再从 s'跟随所有可能的ε-转移寻找其他状态。

由于 DFA 是基于 NFA 的状态集合构造的，因此，它的状态数在最坏情况下是指数级的$O(2^m)$，这使得该方法仅适用于小规模的正则表达式。然而在实践中，大部分状态无法从初始状态到达，因此不需要构造它们。

下面给出一个从 NFA 构造 DFA 的算法，它仅依赖于那些可到达的状态。该算法使用 NFA 的状态集合标识 DFA 中的状态，简单的办法是使用布尔数组来表示这些集合。注意：位并行的表示方法也是可行的，它不仅可以压缩存储，还能够更快速地处理集合的合并以及其他必要的集合操作。5.4 节中将给出专门的位并行算法，目前只是抽象地表示出这些状态集合。

图 5.10 给出了对 NFA 中的每个状态 s 计算ε闭包 $E(s)$的伪代码，其结果是状态 s 对应的一个状态集合。该算法由 $E(s)=\{s\}$开始，重复遍历整个自动机寻找ε-转移。对于每一个ε-转移，将目标状态的ε闭包加入源状态的ε闭包。一直重复这个过程，直到没有新的状态（或闭包）出现。

因为每次完全遍历的开销是$O(|\Delta|m)$，而且考虑到遍历每前进一个状态，ε-转移链的长度都会增加 1，所以该算法的开销为$O(|\Delta|m^2)$。又因为 NFA 中最大距离是$O(m)$，它对

于$O(m)$的遍历是足够的。而在 Thompson 构造法中，有$|\Delta| \leqslant 4m$，所以该算法的时间复杂度是$O(m^3)$的。在 Glushkov 构造法中，因为对于每个$s \in Q$，都有$E(s) = \{s\}$，所以不需要运行这个算法。

```
EpsClosure(N = (Q, Σ, I, F, Δ))
1.    For s ∈ Q Do E(s) ← {s}
2.    changed ← TRUE
3.    While changed = TRUE Do
4.        changed ← FALSE
5.        For (s, ε, s') ∈ Δ Do
6.            If E(s') ⊈ E(s) Then
7.                E(s) ← E(s) ∪ E(s')
8.                changed ← TRUE
9.            End of if
10.       End of for
11.   End of while
```

图 5.10　计算ε闭包$E(s)$

图 5.11 给出了构造 DFA 的算法伪代码。该算法首先构造初始状态I_d，然后递归调用 BuildState 过程，此过程用来从给定的一个源状态发现所有的目标状态，并对目标状态中未出现的状态继续调用自己。终止状态集合F_d的构造是和所有状态集合Q_d的构造一起进行的。

```
BuildState(S)
1.    If S ∩ F ≠ ∅ Then F_d ← F_d ∪ {S}
2.    For σ ∈ Σ Do
3.        T ← ∅
4.        For s ∈ S Do
5.            For (s, σ, s') ∈ Δ Do T ← T ∪ E(s')
6.        End of for
7.        δ(S, σ) ← T
8.        If T ∉ Q_d Then
9.            Q_d ← Q_d ∪ {T}
10.           BuildState(T)
11.       End of if
12.   End of for

BuildDFA(N = (Q, Σ, I, F, Δ))
13.   EpsClosure(N)
14.   I_d ← E(I) /* initial DFA state */
15.   F_d ← ∅ /* final DFA states */
16.   Q_d ← {I_d} /* all the DFA states */
17.   BuildState(I_d)
18.   Return (Q_d, Σ, I_d, F_d, δ)
```

图 5.11　从 NFA 到 DFA 的经典计算

很明显，这个算法只能生成从初始状态可到达的状态（即读入文本时可到达的那些状态）的集合。算法的最坏时间复杂度是 $O(|Q_d||\Sigma||\Delta|\max_s|E(s)|)$，而在 Thompson NFA 和 Glushkov NFA 中，算法都是 $O(|Q_d|m^2)$。这是因为在 Thompson NFA 中，$|\Delta| = O(m)$；而在 Glushkov NFA 中，$|E(s)| = 1$。

构造 DFA 示例 以(AT|GA)((AG|AAA)*)为例。图 5.5 中给出了它的 Thompson NFA，表 5.1 中给出了由 EpsClosure 构造的相应的 $E(s)$ 函数。

表 5.1 图 5.5 中的最终 NFA 的 ε闭包 $E(s)$

$E(0)$	$\{0,1,4\}$	$E(9)$	$\{9\}$
$E(1)$	$\{1\}$	$E(10)$	$\{10\}$
$E(2)$	$\{2\}$	$E(11)$	$\{8,9,11,12,16,17\}$
$E(3)$	$\{3,7,8,9,12,17\}$	$E(12)$	$\{12\}$
$E(4)$	$\{4\}$	$E(13)$	$\{13\}$
$E(5)$	$\{5\}$	$E(14)$	$\{14\}$
$E(6)$	$\{6,7,8,9,12,17\}$	$E(15)$	$\{8,9,12,15,16,17\}$
$E(7)$	$\{7,8,9,12,17\}$	$E(16)$	$\{8,9,12,16,17\}$
$E(8)$	$\{8,9,12\}$	$E(17)$	$\{17\}$

对于图 5.7 中所示的 Glushkov NFA，有 $E(s) = \{s\}$。图 5.12 中分别给出了从 Thompson NFA 和 Glushkov NFA 构造的 DFA。注意：除了节点的标号不同，两个 DFA 是完全一样的。

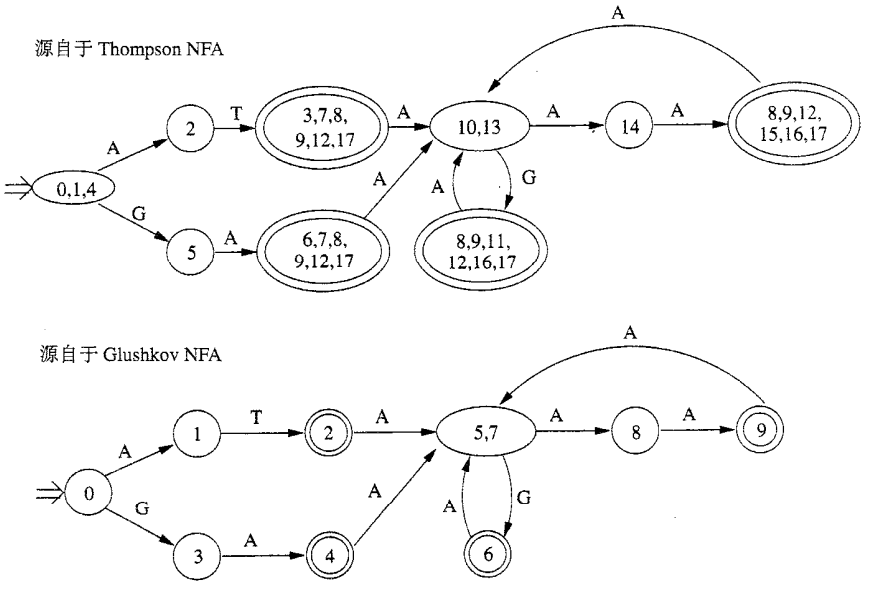


图 5.12 从 Thompson NFA 和 Glushkov NFA 中构造的 DFA

但在通常情况下，并不能确定构造的 DFA 是最小的（也就是说，没有状态更少的 DFA 可以识别这种语言）。不同的 DFA 可能识别同一种语言，并且上述构造方法也无法保证最终的结果是最小的。为了做到这一点，在 DFA 构造完成之后要对它进行最小化。DFA 最小化是一种标准技术，可以在很多经典书籍中找到（例如[ASU86]）。通常可以选择最简单的构造方法，因为在一般情况下它都能够产生大小合理的 DFA。

使用 DFA 进行搜索　构造 DFA 的目的是保证具有$O(n)$的线性搜索时间。之所以能做到这一点，是因为每读入一个字符，就可以进行一次确定的转移。然而，为了能够在文本中搜索，需要对自动机进行改进，即在 NFA 的初始状态上加一个自环，使任何字符都可以通过它。这就是说，在将 NFA 转化为 DFA 前，要使$\Delta \leftarrow \Delta \cup \bigcup_{\sigma\in\Sigma}(I,\sigma,I)$。如果原始自动机能够识别语言$L(RE)$，则改进后的自动机可以识别语言$\Sigma^*L(RE)$。图 5.13 给出了对图 5.7 中的 Glushkov NFA 加入自循环后转化得到的 DFA。

完整的搜索算法如图 5.14 所示。算法在最坏情况下的复杂度是$O(m^2 2^m+n)$，表示 DFA 所需要的额外空间是$O(m2^m)$位。

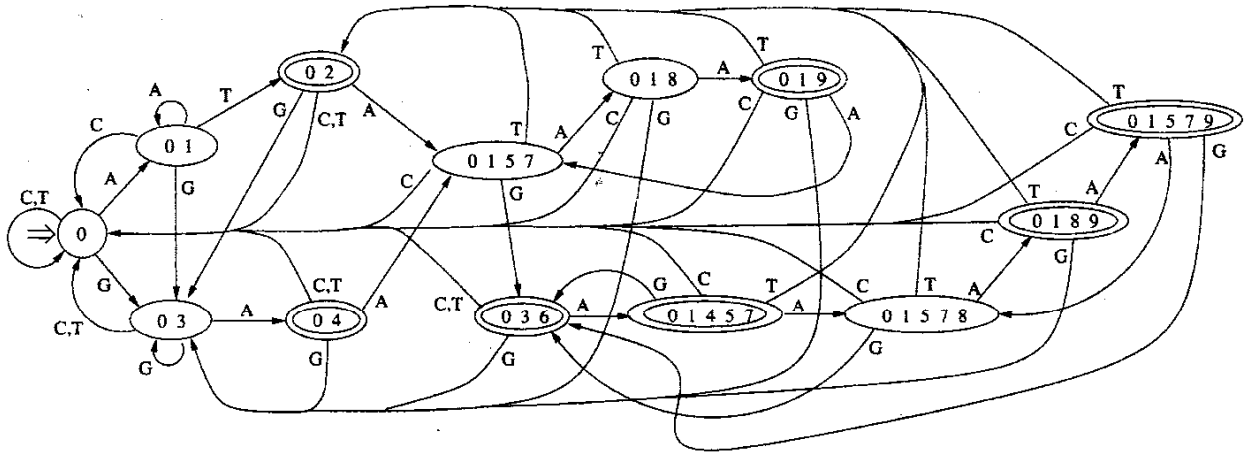


图 5.13　图 5.7 所示的 Glushkov 自动机加入初始自环后获得的自动机。它相当于正则表达式(A|C|G|T)*(AT|GA)((AG|AAA)*)

DFAClassical$(N=(Q,\Sigma,I,F,\Delta), T=t_1t_2\ldots t_n)$

```
1.    Preprocessing
2.        For σ ∈ Σ Do Δ ← Δ ∪ (I, σ, I)
3.        (Q_d, Σ, I_d, F_d, δ) ← BuildDFA(N)
4.    Searching
5.        s ← I_d
6.        For pos ∈ 1...n Do
7.            If s ∈ F_d Then report an occurrence ending at pos − 1
8.            s ← δ(s, t_pos)
9.        End of for
```

图 5.14　经典 DFA 搜索算法

3.3 混合方法

[Mye92]提出了一种折中的办法，它介于 NFA 模拟和 DFA 模拟之间。该方法的核心思是基于 Thompson 构造(参见 5.2.1 节), 它将 NFA 划分为若干个包含$O(k)$个节点的模块，后把它们确定化，并保持一个包含$O(m/k)$个模块的 NFA。该算法称为 DFAModules。

更准确地说，正则表达式的解析树按照如下方法被分成多个模块：首先选择有 k 条边子解析树构成多个模块(这些模块包含 $k/2$到 k 个叶节点), 将它们看做解析树的叶节点。旦子树模块被替换成叶节点后，就选择新的子树构成新的模块，依次类推，一直到达整表达式的根为止。

每个模块的状态都由长度为 $k+1$的位掩码来表示，它是 NFA 活动状态和不活动状态一个位图。转移表要预先计算出来，它表示在给定了当前的活动状态位掩码和一个字符σ，可以方便地获取在处理完字符σ后新的活动状态的位掩码。这个过程实际就是根据位量表示的活动状态集合，在模块的基础上构造 DFA。

对于最底层的模块，显然可以构造出这种 DFA。而对于高层模块来说，问题在于它们叶节点有可能是其他子模块。当进入子模块的边所对应的比特位被激活时，就必须激活模块的初始状态；当子模块的终止状态被激活时，就必须激活高层模块中离开子模块的条边。

由于全部子表达式都要进行模块的构造，并且Thompson 构造法保证只有一个初始状态一个终止状态，所以，模块之间的转移只需要常数时间。

因此，要模拟在高层模块上的一次计算步骤，必须首先使用预先计算出来的表以确定以到达的子模块，并且在到达时激活它们的初始状态。然后，对每个子模块递归地模拟个步骤。并且，对那些已到达终止状态的子模块，激活它们在高层模块中对应的比特位。这个预先计算的表的最后一次访问将产生最终的结果——DFA。

最后还有顺序的问题，子模块必须使用它来解决子模块之间的依赖关系。除了引入回的“*”操作符之外，NFA 可以按拓扑顺序（即源节点在目的节点之前）处理，只要有一通路即可。[Mye92]的主要观点是：对允许在源节点上存在回边的拓扑顺序，两条通路足解决所有的依赖关系。所以，在 NFA 上只需要常数数量的通路，即对每个模块的操作为(1)。

既然算法时间和模块数量成比例，而处理每个字符的时间是$O(m/k)$，并且每个确定模块需要$O(2^k)$空间和常量时间以执行它所有的内部转移，所以该算法需要$O(m2^k/k)$空间和$O(mn/k)$的时间。如果给定$O(s)$的空间，那么算法的搜索时间为$O(mn/\log s)$。

5.4 位并行算法

通过上节内容可以看到，存储 DFA 状态（也就是 NFA 的多个状态集合）的一种可行方法就是使用 $O(m)$ 位的位掩码。其中，如果 NFA 的第 i 个状态属于 DFA，那么位掩码的第 i 位就为 1。本节中将介绍两种位并行实现方法，它们都针对 NFA 和 DFA 模拟的混合算法。与经典方法相比，它们既有优点，也有缺点。

假设 NFA $(Q = \{s_0 \dots s_{|Q|-1}\}, \Sigma, I = s_0, F, \Delta)$ 表示如下：$Q_n = \{0 \dots |Q| - 1\}$，$I_n = 0^{|Q|-1}1$，$F_n = |_{s_j \in F}\ 0^{|Q|-1-j}10^j$（即对终止状态位置的位或运算），转移集合 Δ 由两个表 B_n 和 E_n 表示，其中：

$$B_n[i, \sigma] \quad = \quad |_{(s_i, \sigma, s_j) \in \Delta}\ 0^{|Q|-1-j}10^j$$

表示在不考虑 ε-转移的情况下，从状态 i 通过字符 σ 所到达的状态；

$$E_n[i] \quad = \quad |_{s_j \in E(s_i)}\ 0^{|Q|-1-j}10^j$$

表示状态 s_i 的 ε 闭包 $E(i)$（参见 5.3.2 节）。

当使用 Thompson 构造法或 Glushkov 构造法时，生成这种 NFA 的表示并不复杂。事实上，在位并行算法中，简单地使用长度为 $|Q|$ 的位掩码来表示状态集合是很方便的。当然，E_n 与 Glushkov 构造法无关，因为它的 NFA 是 ε-free 的。

5.4.1 位并行 Thompson

[WM92b]中提到了一种很有竞争力的算法 BPThompson，它源于 Thompson NFA 模拟（参见 5.3.1 节），并且巧妙地使用了位并行方法。该算法的一个重要性质（参见 5.2.1 节）是除 ε-转移外，其他所有箭头都从状态 i 指向状态 $i + 1$。

如果把状态集合打包对应成一个计算机字，使第 i 个状态与第 i 位相对应，则除了 ε-转移外的所有状态转移都可以用表 B 来模拟，这与 Shift-And 算法很相似（参见 2.2.2 节）。而模拟 ε-转移的机制可以使用预计算表 E_d。E_d 是这样构造的：对活动状态集合的每一个置位的位掩码，获取从起始状态通过 ε-转移可以到达的活动状态的新集合。其中，起始状态包括置位位掩码对应的活动状态和初始状态 0。这样可以获得初始状态 0 的 ε-闭包，从而不需要额外工作就可以模拟初始状态 0 的自环转移。

形式化定义如下：

$$E_d[D] \quad = \quad |_{i,\ i=0 \text{ OR } D\&0^{L-i-1}10^i \neq 0^L} \ E_n[i] \tag{5.3}$$

中，$L = |Q| \leqslant 2m$是 Thompson NFA 中的状态数。

由于这种机制在ε-转移上预先计算了一个 DFA，所以它并不是完整的 NFA 模拟。所有其他转移的模拟可以看做是对 NFA 的真正位并行模拟。

图 5.15 给出了创建表 B 和 E_d 的伪码。创建 B 的思想是忽略 B_n 中的源状态，即只将那些通过字符σ到达的状态存储在 $B[\sigma]$ 中：

$$B[\sigma] \quad = \quad |_{i \in 0 \ldots m} \ B_n[i, \sigma] \tag{5.4}$$

创建 E_d 的思想是使用迭代法不断地将新的最高位加入到掩码中，并利用已有的低位掩码结果来计算高位掩码结果。整个过程耗时 $O(2^L + m|\Sigma|)$。

BuildEps$(N = (Q_n, \Sigma, I_n, F_n, B_n, E_n))$
1. **For** $\sigma \in \Sigma$ **Do**
2. $\quad B[\sigma] \leftarrow 0^L$
3. $\quad$ **For** $i \in 0 \ldots L-1$ **Do** $B[\sigma] \leftarrow B[\sigma] \mid B_n[i, \sigma]$
4. **End of for**
   /* B is already built, now build E_d */
5. $E_d[0] \leftarrow E_n[0]$ /* the initial state and its closure */
6. **For** $i \in 0 \ldots L-1$ **Do**
7. $\quad$ **For** $j \in 0 \ldots 2^i - 1$ **Do** /* recall that $E_n[i]$ includes i */
8. $\quad\quad E_d[2^i + j] \leftarrow E_n[i] \mid E_d[j]$
9. $\quad$ **End of for**
10. **End of for**
11. **Return** (B, E_d)

图 5.15 E_d 的位并行构造和从 Thompson 的 NFA 生成 B。这里使用数字标识 E_d 的参数

图 5.16 给出了搜索算法。每个转移分两步进行模拟：首先利用表 B 对普通的转移使用一个类似 Shift-And 的机制，然后使用 E_d 模拟所有的ε-转移。

减少空间 一个依赖于机器和模式串的大小为 2^L 的表可能会占用非常大的存储空间，“水平划分”机制可以用来合理地使用可用内存。这里将 E_d 分成两个表：E_d^1 和 E_d^2，每个占用一半的位。这将导出下面的性质［也可以从公式（5.3）中直接获得］：

$$E_d[D_1 D_2] \quad = \quad E_d[D_1 0^{|D_2|}] \mid E_d[0^{|D_1|} D_2]$$

```
BPThompson(N = (Q_n, Σ, I_n, F_n, B_n, E_n), T = t_1 t_2 ... t_n)
1.    Preprocessing
2.        (B, E_d) ← BuildEps(N)
3.    Searching
4.        D ← E_d[I_n] /* the initial state */
5.        For pos ∈ 1...n Do
6.            If D & F_n ≠ 0^L Then report an occurrence ending at pos − 1
7.            D ← E_d [ (D << 1) & B[t_pos] ]
8.        End of for
```

图 5.16　Thompson 的位并行搜索算法

上面的性质说明：可以将 E_d 的参数分解成两部分。因此，E_d^1和E_d^2定义如下，其中，掩码的长度分别为 $\lfloor L/2 \rfloor$ 和 $\lceil L/2 \rceil$：

$$E_d^1[D] = E_d[0^{\lceil L/2 \rceil} D]\ ,\ E_d^2[D] = E_d[D0^{\lfloor L/2 \rfloor}]$$

于是有：

$$E_d[d_m \ldots d_0] = E_d^1[d_{\lfloor L/2 \rfloor - 1} \ldots d_0] \,|\, E_d^2[d_m \ldots d_{\lfloor L/2 \rfloor}]$$

举例来说，在图 5.5 中有 $E_n[3] = 1000010011100010 00$，$E_n[11] = 1110011011000000 00$，那么 $E_d^1[000001000] = 1000010011100010 00$，$E_d^2[000000100] = 1110011011000000 00$。所以，$E_d[0000001000000001000] = 1110011011100010 00$。

这样做的结果是：将得到两个大小为 $O(2^{L/2})$的更小的表，而不是一个大小为 $O(2^L)$的大表，代价是每执行一次转移必须访问两次内存。

上述方法可以按照如下方式推广。假设只能利用 $O(s)$ 的空间，如果将表 E_d 划分成 k 个表 $E_d^1 \ldots E_d^k$，每个占用参数掩码的 $\lfloor L/k \rfloor$ 或 $\lceil L/k \rceil$ 位，则全部空间需要 $O(k2^{L/k})$。如果存储空间是 s，则有 $k \approx L/\log_2 s$。因此，这种方法在使用 $O(s)$ 的空间时，搜索时间是 $O(mn/\log s)$。但在经典 DFA 算法中，是无法获得这种折中的。注意：对于长的模式（规模较大的模式），复杂度必须乘以 m/w。

在现有计算机体系结构下，即使现有内存已经能够适用于一张很大的表，但是缓存优化机制也会建议使用两个较小的表，从而获得更好的引用局部性。

BPThompson 示例　本示例在文本 AAAGATAAGATAGAAAA 中搜索模式串(AT|GA)((AG|AAA)*)，并给出模式串最后一次出现的位置。状态根据图 5.5 进行标号。因为显示整个表 E_d 的 $2^{18} = 262\,144$个表项是不实际的，因此这里只给出表 E_n。记住：E_d 的行是 E_n

的行与E_d参数中相应的位集合进行或运算获得的。这里只给出当$E(s) \neq \{s\}$时的E_n表项；否则，$E_n[s]$包含$E(0) \cup \{s\}$。

对于每次读入的字符，要分两步更新D，即分别在ε闭包之前和之后。

表E_n

0	0000000000000010011
3	1000010011100011011
6	1000010011110100111
7	1000010011100100111
8	0000010011000100111
11	1100011011000100111
15	1110010011000100111
16	1100010011000100111

A	0011100100010000100
C	0000000000000000000
G	0000001000001000000
T	0000000000000001000
*	0000000000000000000

表B

$F_n =$ 1000000000000000000
$D =$ 0000000000000010011

1. 读入A

B[A]	0011100100010000100
$D =$	0000000000000000100
$D =$	0000000000000010111

2. 读入A

B[A]	0011100100010000100
$D =$	0000000000000000100
$D =$	0000000000000010111

3. 读入A

B[A]	0011100100010000100
$D =$	0000000000000000100
$D =$	0000000000000010111

4. 读入G

B[G]	0000001000001000000
$D =$	0000000000001000000
$D =$	0000000000000110011

5. 读入A

B[A]	0011100100010000100
$D =$	0000000000001000100
$D =$	1000010011110100111

因为D & $F_n \neq 0^L$，所以标记一个成功匹配。

6. 读入T

B[T]	0000000000000001000
$D =$	0000000000000001000
$D =$	1000010011100011011

因为D & $F_n \neq 0^L$，所以标记一个成功匹配。

7. 读入A

B[A]	0011100100010000100
$D =$	0000100100000000100
$D =$	0000100100000010111

8. 读入A

B[A]	0011100100010000100
$D =$	0001000000000000100
$D =$	0001000000000010111

9. 读入G

B[G]	0000001000001000000
$D =$	0000000000001000000
$D =$	0000000000000110011

10. 读入A

B[A]	0011100100010000100
$D =$	0000000000001000100
$D =$	1000010011110100111

因为D & $F_n \neq 0^L$，所以标记一个成功匹配。

11. 读入T

B[T]	0000000000000001000
$D =$	0000000000000001000
$D =$	1000010011100011011

因为D & $F_n \neq 0^L$，所以标记一个成功匹配。

12. 读入A

B[A]	0011100100010000100
$D =$	0000100100000000100
$D =$	0000100100000010111

13. 读入 G

B[G]	0 0 0 0 0 0 1 0 0 0 0 0 1 0 0 0 0 0
$D=$	0 0 0 0 0 0 1 0 0 0 0 0 1 0 0 0 0 0
$D=$	1 1 0 0 0 1 1 0 1 1 0 0 1 1 0 0 1 1

因为D & $F_n \neq 0^L$，所以标记一个成功匹配。

14. 读入 A

B[A]	0 0 1 1 1 0 0 1 0 0 0 1 0 0 0 1 0 0
$D=$	0 0 0 0 1 0 0 1 0 0 0 1 0 0 0 1 0 0
$D=$	1 0 0 0 1 1 0 1 1 1 1 1 1 0 1 0 1 1

因为D & $F_n \neq 0^L$，所以标记一个成功匹配。

15. 读入 A

B[A]	0 0 1 1 1 0 0 1 0 0 0 1 0 0 0 1 0 0
$D=$	0 0 0 1 1 0 0 1 0 0 0 0 0 0 0 1 0 0
$D=$	0 0 0 1 1 0 0 1 0 0 0 0 0 1 0 1 1 1

16. 读入 A

B[A]	0 0 1 1 1 0 0 1 0 0 0 1 0 0 0 1 0 0
$D=$	0 0 1 1 0 0 0 0 0 0 0 0 0 0 0 1 0 0
$D=$	1 1 1 1 0 1 0 0 1 1 0 0 0 1 0 1 1 1

因为D & $F_n \neq 0^L$，所以标记一个成功匹配。

17. 读入 A

B[A]	0 0 1 1 1 0 0 1 0 0 0 1 0 0 0 1 0 0
$D=$	0 0 1 0 1 0 0 1 0 0 0 0 0 0 0 1 0 0
$D=$	1 1 1 0 1 1 0 1 1 1 1 0 0 0 1 0 1 1 1

因为D & $F_n \neq 0^L$，所以标记一个成功匹配。

5.4.2　位并行 Glushkov

另一种位并行算法[NR99a, Nav01b, NR01a]称为 BPGlushkov，使用了 Glushkov NFA，有 $m+1$ 个状态。

之所以选择 Glushkov NFA 而不是 Thompson NFA，是因为在算法中要创建并存储大小为 $2^{|Q|}$ 的表，而 Thompson NFA 的状态太多了（Q 值太大）。而使用表的代价是自动机中的转移无法分解成附加 ε-转移的前向转移。在 Glushkov 构造中，虽然没有 ε-转移，但是以字符标记的转移也不再表示一个简单的前向模式。

算法还成功地利用了 Glushkov 构造法的另一个特性：到达一个指定状态的所有箭头都标记着相同的字符。这样就可以使用表 $B[\sigma]$ 和表 $T_d[D]$ 来计算转移：$B[\sigma]$[参见公式(5.4)]给出了通过字符 σ 可以到达的状态，$T_d[D] = |_{(i,\sigma),\ D\&0^{m-i}10^i \neq 0^{m+1},\ \sigma\in\Sigma} B_n[i,\sigma]$ 给出了从 D 通过任意字符可以到达的状态。

利用特性 $\delta(D,\sigma) = T_d[D]$ & $B[\sigma]$，只需要构造和存储 T_d 和 B，而不是整个转移表。图 5.17 给出了必要的预处理。这个思想与 5.4.1 节中用来构造 E_d 和 B 的思想类似。通过使用中间表 $A[i] = |_{\sigma\in\Sigma} B[i,\sigma]$，时间开销为 $O(2^m + m|\Sigma|)$，其中，中间表 A 是后续集合（参见 5.2.2 节）的一种重要的位并行表达方式。图 5.18 给出了搜索算法，与 BPThompson 算法类似。

```
BuildTran (N = (Q_n, Σ, I_n, F_n, B_n))
1.   For i ∈ 0...m Do A[i] ← 0^{m+1}
2.   For σ ∈ Σ Do B[σ] ← 0^{m+1}
3.   For i ∈ 0...m, σ ∈ Σ Do
4.       A[i] ← A[i] | B_n[i, σ]
5.       B[σ] ← B[σ] | B_n[i, σ]
6.   End of for
                /* B and A are built, now build T_d */
7.   T_d[0] ← 0^{m+1}
8.   For i ∈ 0...m Do
9.       For j ∈ 0...2^i − 1 Do
10.          T_d[2^i + j] ← A[i] | T_d[j]
11.      End of for
12.  End of for
13.  Return (B, T_d)
```

图 5.17 使用 Glushkov NFA 对T_d和B进行位并行构造。这里使用数字来表示T_d参数

```
BPGlushkov(N = (Q_n, Σ, I_n, F_n, B_n), T = t_1 t_2 ... t_n)
1.   Preprocessing
2.       For σ ∈ Σ Do B_n[0, σ] ← B_n[0, σ] | 0^m 1 /* initial self-loop */
3.       (B, T_d) ← BuildTran(N)
4.   Searching
5.       D ← 0^m 1 /* the initial state */
6.       For pos ∈ 1...n Do
7.           If D & F_n ≠ 0^{m+1} Then report an occurrence ending at pos − 1
8.           D ← T_d[D] & B[t_pos]
9.       End of for
```

图 5.18 Glushkov 的位并行搜索算法

相对于 BPThompson，BPGlushkov 的优点是只需要$O(2^m)$的空间，而不是$O(2^{2m})$。就E_d而言，T_d也可以被水平划分从而达到$O(mn/\log s)$的时间和$O(s)$的空间。因此，通常认为 BPGlushkov 要优于 BPThompson。

BPGlushkov 示例 本示例在文本 AAAGATAAGATAGAAAA 中搜索模式串(AT|GA)(AG|AAA)*)，并给出模式串最后一次出现的位置。示例中使用 Glushkov 模拟，其中的状态根据图 5.7 进行标号。由于显示T_d表的全部$2^{10} = 1024$个表项是不实际的，这里只给出B_n，B和T_d中在搜索时需要的行。记住：T_d的行是通过B_n行与T_d参数中相应的位集进行或运算获得的。B_n中只给出那些可得出非 0 结果的表项。

B_n =

0	A	0 0 0 0 0 0 0 0 1 1
0	C	0 0 0 0 0 0 0 0 0 1
0	G	0 0 0 0 0 0 1 0 0 1
0	T	0 0 0 0 0 0 0 0 0 1
1	T	0 0 0 0 0 0 0 1 0 0
2	A	0 0 1 0 1 0 0 0 0 0
3	A	0 0 0 0 0 1 0 0 0 0
4	A	0 0 1 0 1 0 0 0 0 0
5	G	0 0 0 1 0 0 0 0 0 0
6	A	0 0 1 0 1 0 0 0 0 0
7	A	0 1 0 0 0 0 0 0 0 0
8	A	1 0 0 0 0 0 0 0 0 0
9	A	0 0 1 0 1 0 0 0 0 0

B =

A	1 1 1 0 1 1 0 0 1 1
C	0 0 0 0 0 0 0 0 0 1
G	0 0 0 1 0 0 1 0 0 1
T	0 0 0 0 0 0 0 1 0 1

F_n = 1 0 0 1 0 1 0 1 0 0
D = 0 0 0 0 0 0 0 0 0 1

1. 读入 A

$T_d[D]$ = 0 0 0 0 0 0 1 0 1 1
B[A] = 1 1 1 0 1 1 0 0 1 1
D = 0 0 0 0 0 0 0 0 1 1

2. 读入 A

$T_d[D]$ = 0 0 0 0 0 0 1 1 1 1
B[A] = 1 1 1 0 1 1 0 0 1 1
D = 0 0 0 0 0 0 0 0 1 1

3. 读入 A

$T_d[D]$ = 0 0 0 0 0 0 1 1 1 1
B[A] = 1 1 1 0 1 1 0 0 1 1
D = 0 0 0 0 0 0 0 0 1 1

4. 读入 G

$T_d[D]$ = 0 0 0 0 0 0 1 1 1 1
B[G] = 0 0 0 1 0 0 1 0 0 1
D = 0 0 0 0 0 0 1 0 0 1

5. 读入 A

$T_d[D]$ = 0 0 0 0 0 1 1 0 1 1
B[A] = 1 1 1 0 1 1 0 0 1 1
D = 0 0 0 0 0 1 0 0 1 1

因为 D & F_n $\neq$ 0^{m+1}，所以标记一个成功匹配。

6. 读入 T

$T_d[D]$ = 0 0 1 0 1 0 1 1 1 1
B[T] = 0 0 0 0 0 0 0 1 0 1
D = 0 0 0 0 0 0 0 1 0 1

因为 D & F_n $\neq$ 0^{m+1}，所以标记一个成功匹配。

7. 读入 A

$T_d[D]$ = 0 0 1 0 1 0 1 0 1 1
B[A] = 1 1 1 0 1 1 0 0 1 1
D = 0 0 1 0 1 0 0 0 1 1

8. 读入 A

$T_d[D]$ = 0 1 0 1 0 0 1 1 1 1
B[A] = 1 1 1 0 1 1 0 0 1 1
D = 0 1 0 0 0 0 0 0 1 1

9. 读入 G

$T_d[D]$ = 1 0 0 0 0 0 1 1 1 1
B[G] = 0 0 0 1 0 0 1 0 0 1
D = 0 0 0 0 0 0 1 0 0 1

10. 读入 A

$T_d[D]$ = 0 0 0 0 0 1 1 0 1 1
B[A] = 1 1 1 0 1 1 0 0 1 1
D = 0 0 0 0 0 1 0 0 1 1

因为 D & F_n $\neq$ 0^{m+1}，所以标记一个成功匹配。

11. 读入 T

$T_d[D]$ = 0 0 1 0 1 0 1 1 1 1
B[T] = 0 0 0 0 0 0 0 1 0 1
D = 0 0 0 0 0 0 0 1 0 1

因为 D & F_n $\neq$ 0^{m+1}，所以标记一个成功匹配。

12. 读入 A

$T_d[D]$ = 0 0 1 0 1 0 1 0 1 1
B[A] = 1 1 1 0 1 1 0 0 1 1
D = 0 0 1 0 1 0 0 0 1 1

13. 读入 G

$T_d[D] = 0\,1\,0\,1\,0\,0\,1\,1\,1\,1$
$B[\mathtt{G}] = 0\,0\,0\,1\,0\,0\,1\,0\,0\,1$
$D = 0\,0\,0\,1\,0\,0\,1\,0\,0\,1$

因为 $D\ \&\ F_n \neq 0^{m+1}$，所以标记一个成功匹配。

14. 读入 A

$T_d[D] = 0\,0\,1\,0\,1\,1\,1\,0\,1\,1$
$B[\mathtt{A}] = 1\,1\,1\,0\,1\,1\,0\,0\,1\,1$
$D = 0\,0\,1\,0\,1\,1\,0\,0\,1\,1$

因为 $D\ \&\ F_n \neq 0^{m+1}$，所以标记一个成功匹配。

15. 读入 A

$T_d[D] = 0\,1\,1\,1\,1\,0\,1\,1\,1\,1$
$B[\mathtt{A}] = 1\,1\,1\,0\,1\,1\,0\,0\,1\,1$
$D = 0\,1\,1\,0\,1\,0\,0\,0\,1\,1$

16. 读入 A

$T_d[D] = 1\,1\,0\,1\,0\,0\,1\,1\,1\,1$
$B[\mathtt{A}] = 1\,1\,1\,0\,1\,1\,0\,0\,1\,1$
$D = 1\,1\,0\,0\,0\,0\,0\,0\,1\,1$

因为 $D\ \&\ F_n \neq 0^{m+1}$，所以标记一个成功匹配。

17. 读入 A

$T_d[D] = 1\,0\,1\,0\,1\,0\,1\,1\,1\,1$
$B[\mathtt{A}] = 1\,1\,1\,0\,1\,1\,0\,0\,1\,1$
$D = 1\,0\,1\,0\,1\,0\,0\,0\,1\,1$

因为 $D\ \&\ F_n \neq 0^{m+1}$，所以标记一个成功匹配。

5.5 过滤方法

迄今为止，在上述正则表达式搜索方法中，都需要对文本逐个字符进行检验。很自然的联想是：前面章节中提到的关于单字符串匹配、多字符串匹配、扩展串匹配的方法能否应用于正则表达式搜索？本节的目的就是避免读入文本中的所有字符。

使用过滤法的算法通常比前几节中提到的算法要新颖，并且当正则表达式适用于过滤法时，它的搜索速度往往会更快。但是稍后就会注意到，并不是所有正则表达式都适用于过滤法，这时还要求助于前面几节介绍的技术。

由于技术原因，本节中的例子将把前面使用的例子进行反转。反转后对应的 Glushkov 自动机如图 5.19 所示。

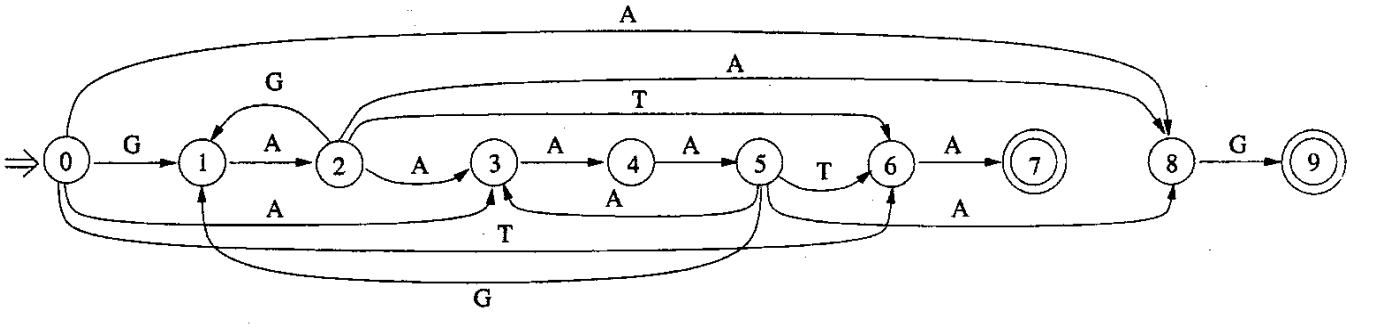


图 5.19 正则表达式((GA|AAA)*)(AT|AG)的 Glushkov 自动机

给定一个正则表达式，如果它的最短成功匹配长度为 ℓmin，则任何跳跃方法都必须对每 ℓmin 个字符中的至少 1 个进行检验，以免错过匹配位置。所以，过滤算法中通常使用长度为 ℓmin 的窗口。

图 5.20 给出了在 $O(m)$ 时间内计算 ℓmin 的递归算法，使用了正则表达式的解析树。通过求解从 NFA 的初始状态到终止状态的最短路径也可以计算 ℓmin。

```
Lmin(v)
1.  If v = [|] (v_l, v_r) Then Return min(Lmin(v_l), Lmin(v_r))
2.  If v = [·] (v_l, v_r) Then Return Lmin(v_l) + Lmin(v_r)
3.  If v = [*] (v_*) Then Return 0
4.  If v = (α) , α ∈ Σ Then Return 1
5.  If v = (ε) Then Return 0
```

图 5.20　ℓmin的计算

5.5.1　多字符串匹配方法

[Wat96]中的方法称为 MultiStringRE，算法中需要计算可以匹配正则表达式中所有长度为 ℓmin 的前缀集合 $Pref(RE)$。在正则表达式 RE = ((GA|AAA)*)(TA|AG)中，$\ell min(RE) = 2$，则与模式串相匹配的长度为 2 的前缀集合为 $Pref(RE)$ = {GA, AA, TA, AG}。下面是一个更复杂的例子，即为 RE = (AT|GA)(AG|AAA)((AG|AAA)+)，其中的 $\ell min(RE) = 6$，前缀集合为 $Pref(RE)$ = {ATAGAG, ATAGAA, ATAAAA, GAAGAG, GAAGAA, GAAAAA}。

图 5.21 给出了生成正则表达式前缀集合的伪代码。表示 $Pref$ 的一个简洁方法是使用 trie 结构，因为它容易生成，并且也可以在搜索时使用。为了简单起见，假设 NFA 是ε-free 的，最坏时间复杂度为 $O(|\Delta|^{\ell min})$。

算法中使用每一个 trie 叶节点 x 来存储 DFA 的状态 $Active(x)$，这个状态可以通过读取 trie 的每一条路径获得，之所以这样做的原因稍后会提到。这时，用 NFA 的状态集合表示 DFA。写一个 DFA 版本的 Compute_Pref 也是可行的，但这需要用另一种方法来表示 DFA 状态。

一旦计算出前缀集合后，算法就可以对 $Pref(RE)$ 使用多串匹配算法（参见第 3 章）。例如，在[Wat96]中就使用了一种类似 Commentz-Walter 的算法。因为正则表达式的每次成功匹配必然要以 $Pref(RE)$ 中的字符串开始，所以只要从 $Pref(RE)$ 在文本中的初始位置开始验证 RE 是否为成功匹配就足矣。为了从一个给定的位置开始验证是否出现成功匹配，可以使用本章中前面提到的任何方法，这时不需要加入初始自环，因为现在的匹配是从特

:位置开始的。通常情况下，匹配一个正则表达式的字符串长度是没有限制的，因此自
机将一直运行，直至遇到一个终止状态，或者是没有活动状态，或者是文本结束时才
止。

```
Pref(s, Δ, ℓmin, Trie)
1.    If ℓmin = 0 Then /* trie leaf */
2.        Active(Trie) ← Active(Trie) ∪ {s}
3.        Return Trie
4.    End of if
5.    For (s, σ, s') ∈ Δ Do
6.        If δ(Trie, σ) = θ Then
7.            Create new state Next = δ(Trie, σ)
8.            Active(Next) ← ∅
9.        End of if
10.       Pref(s', Δ, ℓmin − 1, Next)
11.   End of for

Compute_Pref(N = (Q, Σ, I, F, Δ), ℓmin)
12.   Trie ← θ
13.   Pref(I, Δ, ℓmin, Trie)
14.   Return (Trie, Active)
```

图 5.21 *Pref* 的计算。它接收一个 ε-free 的 NFA 和 *lmin*，返回 trie 形式的 *Pref* 和在叶节点的 Active

为了避免在验证阶段重读窗口的前 ℓmin 个字符，可以使用 $Active(x)$ 中的状态对自动机进行初始化，并在窗口后开始读取字符。例如，如果使用位并行方法表示 DFA，则可以将 *Active* 作为位掩码存储并直接用于自动机的初始化。

多字符串匹配过滤方法的有效性主要依赖于两个值：ℓmin 的大小（ℓmin 越大，搜索速度越快）和 $Pref(RE)$ 的大小（前缀数量越少，搜索速度越快）。注意：$Pref(RE)$ 的大小可能以 m 为指数，例如搜索(a|b)(a|b)...(a|b)。在 5.5.3 节将看到，为了避免这个问题，可以人为地减小 ℓmin 以避免过大的 trie 结构。

MultiStringRE 搜索示例 本示例在文本 AAAAGATAGAATAGAAA 中搜索模式 (GA|AAA)*)(TA|AG)，并给出成功匹配的初始位置。其中，文本是前面示例中使用文本的反转。

MultiStringRE($N = (Q, \Sigma, I, F, \Delta)$, ℓmin)

```
1.    Preprocessing
          /* Construction of Pref */
2.        (Pref, Active) ← Compute_Pref(N, ℓmin)
          /* Construction of the DFA (Figure 5.17) without initial self-loop */
3.        Produce bit-parallel version N' = (Q_n, Σ, I_n, F_n, B_n) of N
4.        (B, T_d) ← BuildTran(N')
5.    Searching
          /* Multipattern search of Pref. Check each occurrence with the DFA */
6.        For (pos, i) ∈ output of multipattern search of Pref Do
7.            D ← Active(i), j ← pos + 1
8.            While j ≤ n AND D & F_n = 0^{m+1} AND D ≠ 0^{m+1} Do
9.                D ← T_d[D] & B[t_j]
10.           End of while
11.           If D & F_n ≠ 0^{m+1} Then
12.               Report an occurrence beginning at pos + 1 − ℓmin
13.           End of if
14.       End of for
```

图 5.22　MultiStringRE 搜索算法。该算法接收一个 NFA 和一个可接受字符串的最小长度，返回匹配发生的初始位置。假设验证过程是用 5.4.2 节的 Glushkov 位并行方法进行的，此外，还假设使用位图来表示 *Active*

B_n =

0	A	0100001000
0	G	0000000010
0	T	0001000000
1	A	0000000100
2	A	0100001000
2	G	0000000010
2	T	0001000000
3	A	0000010000
4	A	0000100000
5	A	0100001000
5	G	0000000010
5	T	0001000000
6	A	0010000000
8	G	1000000000

B =

A	0110111100
C	0000000000
G	1000000010
T	0001000000

$Pref$ =

prefix	*Active*
GA	0000000100
AA	0000010000
TA	0010000000
AG	1000000000

F_n = 1010000000

ℓmin = 2

1. [AA] AAGATAGAATAGAAA

D =	0000010000
Reading A	0000100000
Reading A	0100001000
Reading G	1000000000

因为 D &F_n $\neq$ 0^{m+1}，所以报告在位置 1 处有成功匹配。

2. A [AA] AGATAGAATAGAAA

D =	0000010000
Reading A	0000100000
Reading G	0000000010
Reading A	0000000100
Reading T	0001000000
Reading A	0010000000

因为 D &F_n $\neq$ 0^{m+1}，所以报告在位置 2 处有成功匹配。

3. AA [AA] GATAGAATAGAAA

$D=$	0000010000
读入 G	0000000000

因为$D = 0^{m+1}$，所以丢弃位置3。

4. AAA [AG] ATAGAATAGAAA

$D=$ 1000000000

因为$D \ \&F_n \neq 0^{m+1}$，所以报告在位置4处有成功匹配。

5. AAAA [GA] TAGAATAGAAA

$D=$	0000000100
读入 T	0001000000
读入 A	0010000000

因为$D \ \&F_n \neq 0^{m+1}$，所以报告在位置5处有成功匹配。

6. AAAAGA [TA] GAATAGAAA
（跳过位置6）

$D=$ 0010000000

因为$D \ \&F_n \neq 0^{m+1}$，所以报告在位置7处有成功匹配。

7. AAAAGAT [AG] AATAGAAA

$D=$ 1000000000

因为$D \ \&F_n \neq 0^{m+1}$，所以报告在位置8处有成功匹配。

8. AAAAGATA [GA] ATAGAAA

$D=$	0000000100
读入 A	0100001000
读入 T	0000000000

因为$D = 0^{m+1}$，所以丢弃位置9。

9. AAAAGATAG [AA] TAGAAA

$D=$	0000010000
读入 T	0000000000

因为$D = 0^{m+1}$，所以丢弃位置10。

10. AAAAGATAGAA [TA] GAAA

$D=$ 0010000000

因为$D \ \&F_n \neq 0^{m+1}$，所以报告在位置12处有成功匹配。

11. AAAAGATAGAAT [AG] AAA

$D=$ 1000000000

因为$D \ \&F_n \neq 0^{m+1}$，所以报告在位置13处有成功匹配。

12. AAAAGATAGAATA [GA] AA

$D=$	0000000100
读入 A	0100001000
读入 A	0000010000

文本结束，没有任何成功匹配，因此丢弃文本位置14。

13. AAAAGATAGAATAG [AA] A

$D=$	0000010000
读入 A	0000100000

文本结束，没有任何成功匹配，因此丢弃文本位置15。

14. AAAAGATAGAATAGA [AA]

$D=$ 0000010000

文本结束，没有任何成功匹配，因此丢弃文本位置16。

本例中使用5.4.2节介绍的位并行Glushkov方法作为验证引擎，其中的状态根据图5.19行标号。同以前一样，只给出非零的B_n表项。

本例看上去可能有些笨拙，那是因为示例中给出的搜索模式串在文本中只产生了少量

5.5.2 Gnu 的基于必要因子的启发式方法

Gnu Grep 中使用了启发式方法，其中包含选择必要因子集合的过程，这种方法称为 MultiFactRE。在最简单的情况下，正则表达式的每一次成功匹配中都会存在一个固定的字符串，例如，搜索(AG|GA)ATA((TT)*)，字符串 ATA 是必要因子。

通常的想法是：找到一个必要因子的集合，然后使用多串匹配进行搜索。有多种方法可以用于选择合适的集合，但 *Grep* 的文档中没有详细说明所使用的技术。注意：*Pref* 只是这种方法的一个特例，它的优点在于可以知道从哪里开始匹配（因此只需要向后验证即可），而一般方法需要从发现匹配因子的地方分别向前和向后进行验证。

选择最佳必要因子集合的过程包含两部分：一是能探测到正确候选集合的算法；二是评估函数，用来评估使用候选集合进行搜索的代价，同时，也对在搜索中可能产生的匹配数目进行估算。一个好的评估方法要考虑到集合元素出现的全部可能性，但是如果能进一步考虑搜索算法所用到的知识，评估的效果可能会更好。

图 5.23 给出了一个算法，可以找出必要因子集合，并且能够选择其中最优的一个。其中，假设用来评估集合好坏的最佳选择函数 *best* 已经定义好。代码在正则表达式的解析树上递归运行，并返回 $(all, pref, suff, fact)$。其中，*all* 是所有与正则表达式匹配的字符串集合；*pref* 是最佳的前缀集合；*suff* 是最佳的后缀集合；*fact* 是最佳的因子集合。算法需要的结果是返回值中的第四个，即 *fact*。如果值为 θ，表示不存在有限的必要因子集合。

```
BestFactor(v)
1.   If v = [ | ] (v_l, v_r) OR v = [·] (v_l, v_r) Then
2.       (all_l, pref_l, suff_l, fact_l) ← BestFactor(v_l)
3.       (all_r, pref_r, suff_r, fact_r) ← BestFactor(v_r)
4.   End of if
5.   If v = [ | ] (v_l, v_r) Then
6.       Return (all_l ∪ all_r, pref_l ∪ pref_r, suff_l ∪ suff_r, fact_l ∪ fact_r)
7.   Else If v = [·] (v_l, v_r) Then
8.       Return (all_l · all_r, best(pref_l, all_l · pref_r),
                     best(suff_r, suff_l · all_r), best(fact_l, fact_r, suff_l · pref_r))
9.   Else If v = [*] (v_*) Then Return (θ, θ, θ, θ)
10.  Else If v = (α) , α ∈ Σ Then Return ({α}, {α}, {α}, {α})
11.  Else If v = (ε) Then Return ({ε}, {ε}, {ε}, {ε})
12.  End of if
```

图 5.23　最佳必要因子集合的计算。假设 θ 的行为与 Σ^* 类似，因此对于任意 A，都有 $\theta \cup A = A \cup \theta = \theta \cdot A = A \cdot \theta = \theta$。最佳选择函数 *best* 也认为 θ 有最坏的选择

最简单的情况是单字符和ε。对于“*”操作符，因为其中的字符串可以无限制地重复多次，所以对于返回值all，无法赋予它一个有限集合，因此将$\{\varepsilon\}$赋予$pref$，将θ赋予all。此时返回值是$\{\varepsilon\}$，$pref$，$suff$，$fact$和θ。对于“|”操作符，需要对两个子节点的四个返回值分别进行合并。注意：如果在子节点的位置值为θ，就必须要保留。最有趣的操作符是“·”，为了获得$all(RE_1RE_2)$，必须将$all(RE_1)$中的每个字符串分别与$all(RE_2)$中的每个字符串进行连接；为了获得最佳$pref(RE_1RE_2)$，必须在最佳$pref(RE_1)$和最佳$pref(RE_2)$中进行选择。需要注意的是，计算最佳$pref(RE_2)$时需要在前面附加$all(RE_1)$；$suff$的情况与$pref$是对称的。对于$fact(RE_1RE_2)$，可以在$fact(RE_1)$，$fact(RE_2)$，$suff(RE_1)$与$pref(RE_2)$连接中进行选择。

由于可能会找到最佳集合，因此该方法会比 MultiStringRE 产生更好的结果。在例子(GA|AAA)*)(TA|AG)中，MultiStringRE 选择了四个字符串的集合。而本方法中，可以选择更小的字符串集合：{TA, AG}。

5.5.3 基于 BNDM 的方法

对于正则表达式搜索，这里给出的最后一个可跳跃算法[NR99a, Nav01b]是扩展 BNDM（参见 2.4.2 节、4.2.2 节和 4.3.2 节），我们称之为 RegularBNDM，其优点是具有和前向搜索相等的空间。

RegularBNDM 的核心思想是基于 Glushkov 构造（参见 5.4.2 节）的位并行 DFA 模拟。通过反转箭头和初始化所有的状态可以对 DFA 进行改进，使最终的自动机能够识别所有反转表达式RE的前缀，并且只要读入的字符串是RE的一个反转因子，自动机都保持为活动状态。注意：这里的自动机并没有初始自环。图 5.24 给出了((GA|AAA)*)(TA|AG)的这种改进后的自动机。

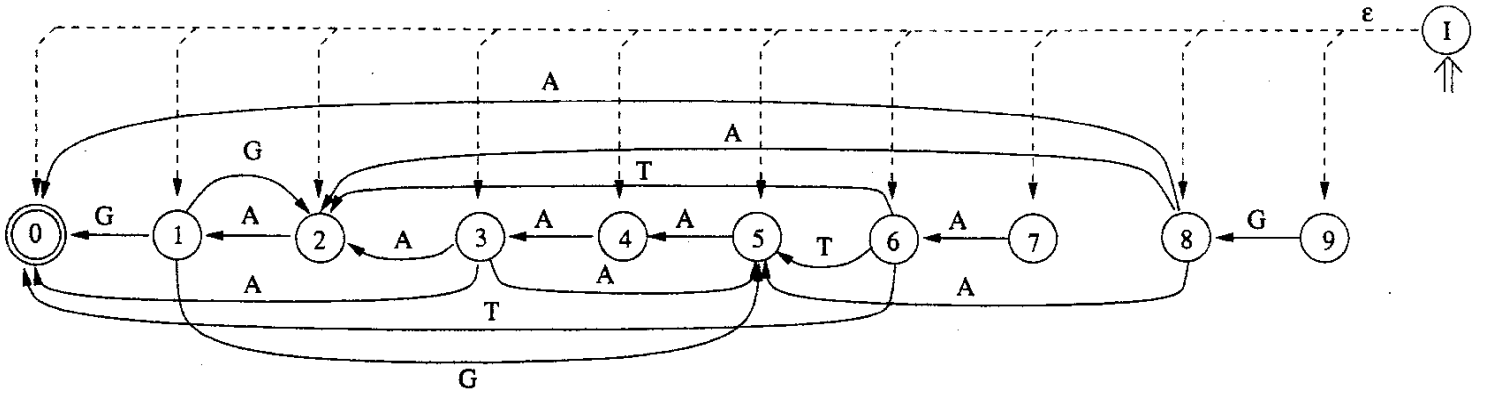


图 5.24 可以识别正则表达式((GA|AAA)*)(TA|AG)所有反转前缀的自动机

该算法在文本中滑动大小为 ℓmin 的窗口，并自后向前读取窗口中的字符。每当识别出一个前缀时，就将此时的窗口位置存入变量 $last$ 中。当窗口滑动时，必须以 $last$ 位置对齐，以便从 $last$ 位置开始继续匹配。窗口中的反向搜索在下列两种情况下可以停止：一是 DFA 中没有活动状态，这时可以滑动窗口重新开始匹配过程；二是匹配位置到达了窗口的起始位置。

对于上述第二种情况，类似于扩展串匹配（参见第 4 章），算法不能保证正则表达式一定匹配成功，只能说明匹配了一个因子。如果自动机的最终状态是位于停在窗口的起点位置，那么就可以使用正常的无自循环 DFA 自动机进行前向验证。

对上面的机制还可以进行改进。如果当前在窗口的位置为 j，$j \leqslant \ell min$，那么就不需要关心自动机中离初始状态 0 的距离大于 j 的那些状态是否为活动的。因为在窗口内部，那些状态永远不可能激活初始状态 0。所以，对于 $j \in 0 \ldots \ell min$，保存掩码 $Reach_j$，它包含了从窗口位置 j 能够影响最终结果的所有状态。通过去除那些不在 $Reach_j$ 中的活动状态，可以加快窗口的滑动速度。

从图 5.24 中可以看到，当把箭头反转后，自动机将不具有 Glushkov 的特性，即到给定的一个状态的所有箭头都标识着同样的字符。所以，这里不能直接使用 BPGlushkov 模拟。但是，如果注意到在把箭头反转后，会有另外一个特性，即所有从一个给定状态离开的箭头都标识着相同的字符，那么将会得到相似的结果。

如同前面的描述，算法仍然使用表 T_d 和 B，但在使用 T_d 前必须用 B 中的掩码进行重新计算。也就是说，要保留那些在离开它们的箭头上标识当前字符的活动状态，并且获得所有离开它们的转移。形式化表示是这样的：$\delta(D, \sigma) = T_d[\ D\ \&\ B[\sigma]\ ]$，其中 B 对应那些前向转移。

图 5.25 给出了算法的预处理过程，它产生一个前向自动机 (B, Tf_d)，一个后向自动机 Tb_d 和一张表 $Reach$。表 Tf_d 通过对输入的 NFA 在不增加初始自循环的条件下进行确定化而得到；从 (B, Tf_d) 初始状态开始，对任意字符执行 ℓmin 次转移可以得到表 $Reach$；最后，通过反转 NFA 的所有箭头可以得到 Tb_d 并进一步把它确定化。整个处理过程需要 $O(2^m + m^2|\Sigma|)$ 时间。图 5.26 给出了搜索算法。

对算法空间的改进是可行的：由于只对从初始状态 0 开始的最多经过 ℓmin 步可到达的那些状态感兴趣，所以没有必要使用整个箭头反转的自动机，而只需要保留那些属于 $Reach_{\ell min}$ 的状态。通过丢弃不必要的状态，可以达到节约空间的目的。

```
Compute_Reach (T_d, I, ℓmin)
1.    Reach_0 ← I /* the initial state */
2.    For j ∈ 1...ℓmin Do
3.        Reach_j ← Reach_{j-1} | T_d[Reach_{j-1}]
4.    Return Reach

Reverse_Arrows (N = (Q_n, Σ, I_n, F_n, Bf_n))
5.    For i ∈ 0...m, σ ∈ Σ Do
6.        Bb_n[i, σ] ← 0^{m+1}
7.        For j ∈ 0...m Do
8.            If Bf_n[j, σ] & 0^{m-i}10^i ≠ 0^{m+1} Then
9.                Bb_n[i, σ] ← Bb_n[i, σ] | 0^{m-j}10^j
10.           End of if
11.       End of for
12.   End of for
13.   Return Bb_n

BNDM_Preproc (N = (Q_n, Σ, I_n, F_n, B_n), ℓmin)
                /* (B, Tf_d) (no initial self-loop) is used for verification */
14.   (B, Tf_d) ← BuildTran(N)
                /* Reach tells reachable states */
15.   Reach ← Compute_Reach(Tf_d, I_n, ℓmin)
                /* Tb_d is a DFA for recognizing reverse prefixes */
16.   Bb_n ← Reverse_Arrows(N)
17.   (Bb, Tb_d) ← BuildTran(Nb = (Q_n, Σ, 1^{m+1}, 0^m1, Bb_n))
18.   Return (B, Tf_d, Tb_d, Reach)
```

图 5.25 基于 BNDM 算法的过程

由于在窗口位置 j 时丢弃了那些不能到达状态 0 的状态，因此，对于给定的一个活动状态，只有当它可能成为一个成功匹配中长度为 ℓmin 的前缀时，才需要保留它。这样看来，该算法只是另一个 *Pref* 搜索机制（参见 5.5.1 节），只是它没有像 MultiStringRE 算法中那样使用 trie 结构来表示搜索状态，而使用了反转箭头的 *Pref* 搜索自动机。

[Nav01b]中指出，可以通过寻找正则表达式的好的“必要因子”来改进该算法，就像 MultiFactRE 算法一样。这时，好的“必要因子”是 NFA 的一个子图，从初始状态到终止状态的所有路径都需要遍历这个子图。

RegularBNDM 搜索示例 本示例在文本 AAAAGATAGAATAGAAA 中搜索模式((GA|AA)*) (TA|AG)，并给出成功匹配的初始位置。状态根据图 5.19 标号。示例中仅给出了 Bb_n 中的非 0 表项，以及表 Tb_d 中在搜索时使用到的对应行，并且省略了前向验证的细节。

此外，执行这段代码要比简单地进行前向扫描慢，这是因为要搜索示例中这个特定的模式串，使用本方法是比较困难的。

```
RegularBNDM(N = (Q, Σ, I, F, Δ), ℓmin)
1.    Preprocessing
2.        (B, Tf_d, Tb_d, Reach) ← BNDM_Preproc(N, ℓmin)
3.    Searching
4.        pos ← 0
5.        While pos ≤ n − ℓmin Do
6.            j ← ℓmin, last ← ℓmin
7.            D ← Reach_ℓmin
8.            While D ≠ 0^{m+1} AND j > 0 Do
9.                D ← Tb_d[D & B[t_{pos+j}]] & Reach_{j−1}
10.               j ← j − 1
11.               If D & 0^m1 ≠ 0^{m+1} Then /* prefix recognized */
12.                   If j > 0 Then last ← j
13.                   Else /* check a possible occurrence starting at pos + 1 */
14.                       D ← 0^m1, j ← pos + 1
15.                       While j ≤ n AND D&F_n = 0^{m+1} AND D ≠ 0^{m+1} Do
16.                           D ← Tf_d[D] & B[t_j]
                              j←j+ 1
17.                       End of while
18.                       If D & F_n ≠ 0^{m+1} Then
19.                           Report an occurrence beginning at pos + 1
20.                       End of if
21.                   End of if
22.               End of if
23.           End of while
24.           pos ← pos + last
25.       End of while
```

图 5.26　正则表达式的扩展 BNDM 算法

Bb_n =

1	G	0 0 0 0 1 0 0 1 0 1
2	A	0 1 0 0 0 0 0 0 1 0
3	A	0 0 0 0 1 0 0 1 0 1
4	A	0 0 0 0 0 0 1 0 0 0
5	A	0 0 0 0 0 1 0 0 0 0
6	T	0 0 0 0 1 0 0 1 0 1
7	A	0 0 0 1 0 0 0 0 0 0
8	A	0 0 0 0 1 0 0 1 0 1
9	G	0 1 0 0 0 0 0 0 0 0

B =

A	0 1 1 0 1 1 1 1 0 0
C	0 0 0 0 0 0 0 0 0 0
G	1 0 0 0 0 0 0 0 1 0
T	0 0 0 1 0 0 0 0 0 0

$Reach_0$ = 0 0 0 0 0 0 0 0 0 1

$Reach_1$ = 0 1 0 1 0 0 1 0 1 1

$Reach_2$ = 1 1 1 1 0 1 1 1 1 1

ℓmin = 2

1. [AA] AAGATAGAATAGAAA

$D =$	1111011111
读入 A	
D & B[A] =	0110011100
Tb_d =	0101101111
& $Reach_1$ =	0101001011
$last$ =	1 (D & $I_n \neq 0^{m+1}$)
读入 A	
D & B[A] =	0100001000
Tb_d =	0000100101
& $Reach_0$ =	0000000001

因为 D & $I_n \neq 0^{m+1}$，所以在位置 1 开始验证。5 步之后，找到模式，报告一个成功匹配，然后根据 $last = 1$滑动窗口。

2. A [AA] AGATAGAATAGAAA

与步骤1相同，因为 D & $I_n \neq 0^{m+1}$，所以在位置2开始验证。7 步之后，找到模式，报告一个成功匹配，然后根据 $last = 1$ 滑动窗口。

3. AA [AA] GATAGAATAGAAA

与步骤 1 相同，因为 D & $I_n \neq 0^{m+1}$，所以在位置 3 开始验证。3 步之后，自动机运行到了没有活动状态，所以，丢弃位置 3，根据 $last = 1$滑动窗口。

4. AAA [AG] ATAGAATAGAAA

$D =$	1111011111
读入 G	
D & B[G] =	1000000010
Tb_d =	0100100101
& $Reach_1$ =	0100000001
$last$ =	1 (D & $I_n \neq 0^{m+1}$)
D =	0100000001
读入 A	
D & B[A] =	0100000000
Tb_d =	0000100101
& $Reach_0$ =	0000000001

因为 D & $I_n \neq 0^{m+1}$，所以在位置 4 开始验证。2 步之后，找到模式，报告一个成功匹配，然后根据 $last = 1$滑动窗口。

5. AAAA [GA] TAGAATAGAAA

$D =$	1111011111
读入 A	
D & B[A] =	0110011100
Tb_d =	0101101111
& $Reach_1$ =	0101001011
$last$ =	1 (D & $I_n \neq 0^{m+1}$)
读入 G	
D & B[G] =	0000000010
Tb_d =	0000100101
& $Reach_0$ =	0000000001

因为 D & $I_n \neq 0^{m+1}$，所以在位置 5 开始验证。4 步之后，找到模式，报告一个成功匹配，然后根据 $last = 1$滑动窗口。

6. AAAAG [AT] AGAATAGAAA

$D =$	1111011111
读入 T	
D & B[T] =	0001000000
Tb_d =	0000100101
& $Reach_1$ =	0000000001
$last$ =	1 (D & $I_n \neq 0^{m+1}$)
读入 A	
D & B[A] =	0000000000
Tb_d =	0000000000
& $Reach_0$ =	0000000000

因为 D & $I_n = 0^{m+1}$，所以根据 $last = 1$滑动窗口。

7. AAAAGA [TA] GAATAGAAA

$D =$	1111011111
读入 A	
$D \,\&\, B[\mathtt{A}] =$	0110011100
$Tb_d =$	0101101111
$\&\ Reach_1 =$	0101001011
$last =$	1 $(D \,\&\, I_n \neq 0^{m+1})$
读入 T	
$D \,\&\, B[\mathtt{T}] =$	0001000000
$Tb_d =$	0000100101
$\&\ Reach_0 =$	0000000001

因为 $D \,\&\, I_n^* \neq 0^{m+1}$，所以在位置 7 开始验证。2 步之后，找到模式，报告一个成功匹配，然后根据 $last = 1$滑动窗口。

8. AAAAGAT [AG] AATAGAAA

与步骤 4 相同，因为 $D \,\&\, I_n \neq 0^{m+1}$，所以在位置 8 开始验证。2 步之后，找到模式，报告一个成功匹配，然后根据 $last = 1$滑动窗口。

9. AAAAGATA [GA] ATAGAAA

与步骤 5 相同，因为 $D \,\&\, I_n \neq 0^{m+1}$，所以在位置 9 开始验证。4 步之后，自动机运行到了没有活动状态，所以，根据 $last = 1$滑动窗口。

10. AAAAGATAG [AA] TAGAAA

与步骤 1 相同，因为 $D \,\&\, I_n \neq 0^{m+1}$，所以在位置 10 开始验证。3 步之后，自动机运行到了没有活动状态，所以，根据 $last = 1$滑动窗口。

11. AAAAGATAGA [AT] AGAAA

与步骤 6 相同，因为 $D \,\&\, I_n = 0^{m+1}$，所以根据 $last = 1$滑动窗口。

12. AAAAGATAGAA [TA] GAAA

与步骤 7 相同，因为 $D \,\&\, I_n \neq 0^{m+1}$，所以在位置 12 开始验证。2 步之后，找到模式，报告一个成功匹配，然后根据 $last = 1$ 滑动窗口。

13. AAAAGATAGAAT [AG] AAA

与步骤 4 相同，因为 $D \,\&\, I_n \neq 0^{m+1}$，所以在位置 13 开始验证。2 步之后，找到模式，报告一个成功匹配，然后根据 $last = 1$滑动窗口。

14. AAAAGATAGAATA [GA] AA

与步骤 5 相同，因为 $D \,\&\, I_n \neq 0^{m+1}$，所以在位置 14 开始验证。4 步之后，文本结束，没有识别出模式，所以，根据 $last = 1$滑动窗口。

15. AAAAGATAGAATAG [AA] A

与步骤 1 相同，因为 $D \,\&\, I_n \neq 0^{m+1}$，所以在位置 15 开始验证。3 步之后，文本结束，没有识别出模式，所以，根据 $last = 1$滑动窗口。

16. AAAAGATAGAATAGA [AA]

与步骤 1 相同，因为 $D \,\&\, I_n \neq 0^{m+1}$，所以在位置 16 开始验证。2 步之后，文本结束，没有识别出模式，所以，根据 $last = 1$滑动窗口。

6 实验结果

对于正则表达式搜索来说，确定一个最好的搜索算法要比简单的模式串匹配困难得多。是因为正则表达式的结构对算法效率造成了复杂的影响。

相对于 DFAClassical 方法来说，位并行方法的一个明显缺点是：要构造所有 $2^{|Q|}$ 种可能组合，而 DFAClassical 只需要构造那些可到达的状态即可。因此，DFAClassical 能够产更小的自动机。

但是位并行方法也有很重要的优点：算法的代码实现简单，灵活性也更好。例如在 6 章中将看到，这种方法可以进行扩展从而允许模式串和它的成功匹配串之间有差异，这是 DFAClassical 很难做到的。此外，位并行方法还可以进行水平划分，从而尽可能减空间消耗。

在位并行方法中，BPGlushkov 要比 BPThompson 更好，因为所需要的空间更少，并且于可以在更小的表中寻址，因而有更好的引用局部性。

虽然 NFAModules 具有和 BPGlushkov 一样的空间独立复杂度 $O(mn/\log s)$，但实现起更复杂，在实践中速度也比较慢。然而，当正则表达式需要更多计算机字时，比如超过个字以上，NFAModules 就比位并行方法更具有吸引力。而且，NFAModules 还能够进行扩，从而可以处理扩展串匹配中的字符组模式和近似匹配（参见第 6 章）。

依赖于正则表达式结构的过滤法可能比上述方法都好，也可能更糟。定义一个在所有况下都能很好地预测过滤行为的参数是非常困难的，但可以给出一个好的近似值：

$$Prob\text{-}verif = \frac{|Pref|}{|\Sigma|^{\ell min}}$$

rob-verif 给出了能够匹配成功 *Pref* 中一个字符串的近似概率（*Pref* 的定义参见 5.5.1 节）。当 *Pref* 中的一个字符串被成功匹配时，都必须执行一次验证，而验证的代价很难计算，常可以近似估算为：

$$Cost\text{-}verif = \sum_{\ell \geqslant 0} \frac{|Pref_\ell|}{|\Sigma|^\ell}$$

中，$Pref_\ell$ 是正则表达式中可能出现匹配的所有长度为 ℓ 的前缀集合。获得这些集合的算同计算 *Pref* 的算法（参见 5.5.1 节）相似。

根据经验，只有在满足下面的条件时才使用过滤法：

$$Cost\text{-}filter = Prob\text{-}verif \times Cost\text{-}verif \leqslant 1$$

在基于 MultiStringRE 和 RegularBNDM 的方法中，*Pref* 值工作良好，但在 MultiFactRE 方法中，该值必须根据选择的字符串集来进行改变。

几种过滤方法的不同之处在于：当考虑文本字符的因素时，MultiStringRE 和 RegularBNDM 是相似的，尤其是当 MultiStringRE 中使用了 Multiple BNDM 或 SBDN 算法（参见 3.4 节）时。RegularBNDM 中使用了集合 *Pref* 的压缩表示，聪明地利用自动机本身取代了一个完整的 trie 结构。但是，当正则表达式规模过大时，RegularBNDM 会花费太多的时间，这时最好使用 MultiStringRE。而 MultiStringRE 的另一个优势就是不需要重读窗口。

最后，类 MultiFactRE 过滤法可以看做是对前面几种方法的一个改进。另外，*Gnu Grep*（参见 7.1.1 节）比普通的 MultiStringRE 方法工作得更好，*Nrgrep*（参见 7.1.3 节）中也包含了一个 RegularBNDM 的实现，所使用的方法同样是寻找正则表达式中的最佳必要因子。

即使是对规模较小的模式，也会发生 *Gnu Grep* 比 *Nrgrep* 更快的情况。但是，Regular BNDM 可以通过扩展来进行近似匹配，而经典的多串匹配算法通常是没有这种能力的。

在表 5.2 中，对前面给出的建议进行了总结。

表 5.2　根据模式的一些参数，建议使用的正则表达式搜索算法

	Cost-filter 值较低（1.0 以下）	*Cost-filter* 值较高（1.0 以上）
模式串较短（$m \leqslant 4w$）	RegularBNDM/ MultiFactRE	DFAClassical/ BPGlushkov
模式串校长（$m > 4w$）	MultiFactRE	NFAModuless

5.7　其他算法与参考资料

构造 NFA　构造一个 ε-free 的 NFA，理论上转移数的下限是 $O(m \log m)$ [HSW97]，但是怎样达到这个下限仍然是一个未解决的问题。[HSW97]中给出了一个在 $O(m^2)$ 时间内可以生成具有 $O(m \log^2 m)$ 个转移的 NFA 的算法，而[HM98]将它的时间复杂度改进成 $O(m \log^2 m)$。遗憾的是，该算法太复杂了。

正则表达式的集合　对正则表达式搜索问题的一个自然扩展就是对正则表达式集合（$RE_1, RE_2, \ldots, RE_r$）的搜索。原则上，这个问题可以转换为对 $RE_1 \mid RE_2 \mid \ldots \mid RE_r$ 进行搜索的单正则表达式搜索问题。然而，大多数算法都难以很好地处理大规模正则表达式集合。

NFAModules 是一种能够处理大规模正则表达式集合的算法，但它的开销会随着模式大小（在例子中是 r）的增大而线性增加。相比而言，MultiStringRE 和 MultiFactRE 要更好一些，它们利用过滤法对大规模正则表达式集合进行高效处理。

.8 构造解析树

本节将介绍如何解析正则表达式以获取其对应的解析树。通常来说，解析树并不是唯一的。在解析树中，每个叶节点都是使用$\Sigma \cup \{\varepsilon\}$中的一个字符来标识的，而每个中间节点则使用操作符集合$\{|, \cdot, *\}$中的一个进行标识。

一般的方法是：将正则表达式看做是由一个语法生成的字符串，然后利用典型的 UNIX 工具 *Lex* 和 *Yacc*，或者 Gnu 的 *Flex* 和 *Bison*，从语法产生能够识别正则表达式的自动机，最后将它转化成解析树。这些工具的原理都可以在编译书籍中找到，例如[ASU86]。

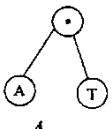
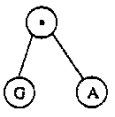
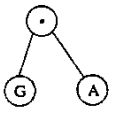
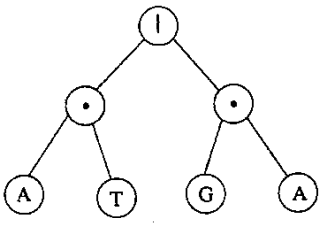
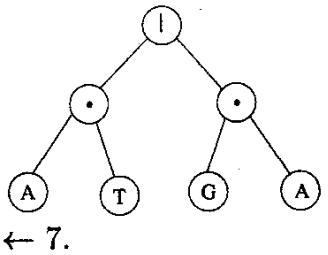
这种方法对于大型语法非常有效，例如程序语言的解析器。而对于简单的语法，*Lex* 或 *lex* 这样简单的词法分析器就足矣。正则表达式的语法对于词法分析器来说过于复杂，而对一完整的自底向上的解析器来说又过于简单。最好的方法是手工构造一个解析器，如同图 5.27 所示意的那样。这里假设：正则表达式书写正确，并且以特殊字符“$”表示结束，还假设连接符“$\cdot$”是可以隐去的。当然，在实际应用中必须修改这个简单的解析器，以使它可以处理各种类型的错误。而这个伪代码仍然不失为一个好的开端，并且可以应对简单的应用。

```
Parse(p = p_1 p_2 ... p_m, last)
1.   v ← θ
2.   While p_last ≠ $ Do
3.       If p_last ∈ Σ OR p_last = ε Then      /* normal character */
4.           v_r ← Create a node with p_last
5.           If v ≠ θ Then v ← [·](v, v_r)
6.           Else v ← v_r
7.           last ← last + 1
8.       Else If p_last = '|' Then             /* union operator */
9.           (v_r, last) ← Parse(p, last + 1)
10.          v ← [||](v, v_r)
11.      Else If p_last = '*' Then             /* star operator */
12.          v ← [*](v)
13.          last ← last + 1
14.      Else If p_last = '(' Then             /* open parenthesis */
15.          (v_r, last) ← Parse(p, last + 1)
16.          last ← last + 1
17.          If v ≠ θ Then v ← [·](v, v_r)
18.          Else v ← v_r
19.      Else If p_last = ')' Then             /* close parenthesis */
20.          Return (v, last)
21.      End of if
22.  End of while
23.  Return (v, last)
```

图 5.27 一个对于书写良好正则表达式的基本递归解析器。θ表示空树

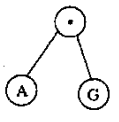
通过下面的示例，可以深入理解这个解析器是怎样工作的。示例中的正则表达式为(AT|GA)((AG|AAA)*)。

解析过程示例　本示例使用 Parse((AT|GA)((AG|AAA)*)$,1)对正则表达式(AT|GA)((AG|AAA)*)进行解析。递归调用分别标号为 Parse^1，Parse^2，依次类推，相应的变量也使用这种方法标号。

1. **Parse^1**((AT|GA)((AG|AAA)*), 1)
   $last^1 = 1$, $v^1 = \theta$,
   读入 [(] AT|GA)((AG|AAA)*)$
   第 15 行，调用：
2. **Parse^2**((AT|GA)((AG|AAA)*), 2)
   $last^2 = 2$, $v^2 = \theta$,
   读入 ([A] T|GA)((AG|AAA)*)$
   第 4 行，$v_r^2 \leftarrow$ Ⓐ
   第 6 行，$v^2 \leftarrow v_r^2$
   第 7 行，$last^2 = 3$
   转到 Parse^2 第 2 行的 While 循环处。
3. 由于 $p_{last^2} \neq \$$
   读入(A [T] |GA)((AG|AAA)*)$
   第 4 行，$v_r^2 \leftarrow$ Ⓣ
   第 5 行，$v^2 \leftarrow$ 
   第 7 行，$last^2 = 4$。
4. 输入第 8 行，
   读入(AT [|] GA)((AG|AAA)*)$
   第 9 行，调用：
5. **Parse^3**((AT|GA)((AG|AAA)*), 5)
   $last^3 = 5$, $v^3 = \theta$,
   读入(AT| [G] A)((AG|AAA)*)$
   第 4 行，$v_r^3 \leftarrow$ Ⓖ
   第 6 行，$v^3 \leftarrow v_r^3$
   第 7 行，$last^3 = 6$
   转到 Parse^3 第 2 行的 While 循环处。
6. 由于 $p_{last^3} \neq \$$,
   读入(AT|G [A])((AG|AAA)*)$
   第 4 行，$v_r^3 \leftarrow$ Ⓐ
   第 5 行，$v^3 \leftarrow$ 
   第 7 行，$last^3 = 7$。
7. 输入第 19 行，
   读入(AT|GA [)] ((AG|AAA)*)$。
   第 20 行，退出 Parse^3 函数，转到$(v^3, 7)$。
   返回到 Parse^2 的第 9 行。
   $v_r^2 \leftarrow$  ，$last^2 \leftarrow 7$。
   第 10 行，$v^2 \leftarrow$ 
8. 输入第 19 行，
   再读入(AT|GA [)] ((AG|AAA)*)$。
   第 20 行，退出 Parse^2 函数，转到$(v^2, 7)$。
   返回到 Parse^1 的第 15 行。
   $v_r^1 \leftarrow$ 
   $last^1 \leftarrow 7$.
   第 16 行，$last^1 \leftarrow 8$
   第 18 行，$v^1 \leftarrow v_r^1$
   转到 Parse^1 第 2 行的 While 循环处。
9. 由于 $p_{last^1} \neq \$$,
   读入(AT|GA) [(] (AG|AAA)*)$。
   第 15 行，调用

10. **Parse**^{2}(`(AT|GA)((AG|AAA)*)`, 9)
$last^2 = 9$, $v^2 = \theta$.
由于 $p_{last^2} \neq \$$
读入`(AT|GA)(`[`(`]`AG|AAA)*)$`
第 15 行，调用：

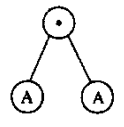
11. **Parse**^{3}(`(AT|GA)((AG|AAA)*)`, 10)
$last^3 = 10$, $v^3 = \theta$,
读入`(AT|GA)((`[`A`]`G|AAA)*)$`
第 4 行，$v_r^3 \leftarrow$ Ⓐ
第 6 行，$v^3 \leftarrow v_r^3$
第 7 行，$last^3 = 11$
转到 Parse^3 第 2 行的 While 循环处。

12. 由于 $p_{last^3} \neq \$$
读入`(AT|GA)((A`[`G`]`|AAA)*)$`
第 4 行，$v_r^3 \leftarrow$ Ⓖ
第 5 行，$v^3 \leftarrow$ 

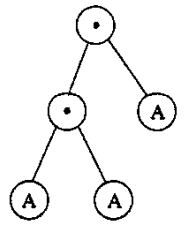
第 7 行，$last^3 = 12$。

13. 输入第 8 行，
读入`(AT|GA)((AG [ | ] AAA)*)$`
第 9 行，调用：

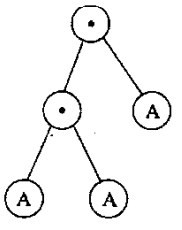
14. **Parse**^{4}(`(AT|GA)((AG|AAA)*)`, 13)
$last^4 = 13$, $v^4 = \theta$,
读入`(AT|GA)((AG|`[`A`]`AA)*)$`
第 4 行，调用：$v_r^4 \leftarrow$ Ⓐ
第 6 行，$v^4 \leftarrow v_r^4$
第 7 行，$last^4 = 14$。
转到 Parse^4 第 2 行的 While 循环处。

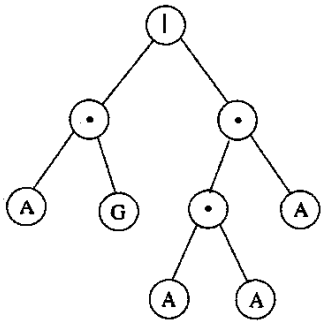
15. 由于 $p_{last^4} \neq \$$
读入`(AT|GA)((AG|A`[`A`]`A)*)$`
第 4 行，$v_r^4 \leftarrow$ Ⓐ
第 5 行，$v^4 \leftarrow$ 

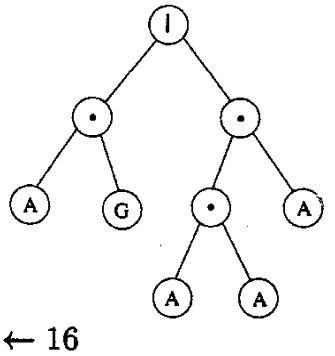
第 7 行，$last^4 = 15$。
转到 Parse^4 第 2 行的 While 循环处。

16. 由于 $p_{last^4} \neq \$$
读入`(AT|GA)((AG|AA`[`A`]`)*)$`
第 4 行，$v_r^4 \leftarrow$ Ⓐ
第 5 行，$v^4 \leftarrow$ 

第 7 行，$last^4 = 16$

17. 输入第 19 行，
读入`(AT|GA)((AG|AAA`[`)`]`*)$`。
第 20 行，退出 Parse^4 函数，转到$(v^4, 16)$。
返回到 Parse^3 的第 9 行。
$v_r^3 \leftarrow$ 

, $last^3 \leftarrow 16$。
第 10 行，$v^3 \leftarrow$ 

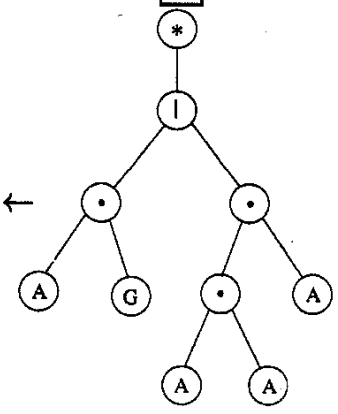
18. 输入第 19 行，
再次读入`(AT|GA)((AG|AAA`[`)`]`*)$`。
第 20 行，退出 Parse^3 函数，转到$(v^3, 16)$。
返回到 Parse^2 的第 2 行。
$v_r^2 \leftarrow$ 

$last^2 \leftarrow 16$
第 16 行，$last^2 \leftarrow 17$。
第 18 行，$v^2 \leftarrow v_r^2$。
转到 Parse^2 第 2 行的 While 循环处。

19. 由于$p_{last^2} \neq \$$

读入(AT|GA)((AG|AAA) [*])$

第 12 行，$v^2 \leftarrow$



第 13 行，$last^2 \leftarrow 18$。

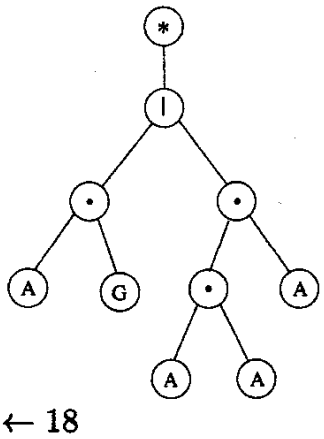
20. 输入第 19 行，

读入(AT|GA)((AG|AAA)* [ )] $

第 20 行，退出 Parse^2 函数，转到 $(v^2, 18)$。

返回到 Parse^1 的第 15 行，

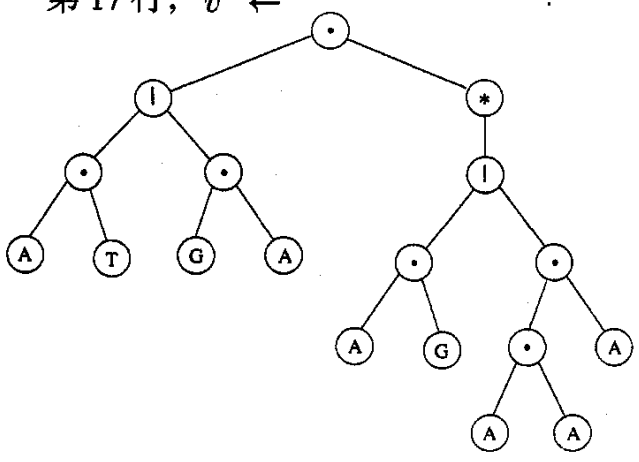
$v_r^1 \leftarrow$



$last^1 \leftarrow 18$

第 16 行，$last^1 \leftarrow 19$。

第 17 行，$v^1 \leftarrow$



“转到 Parse^1 第 2 行的 While 循环处。

21. 由于$p_{last^1} = \$$

终止该函数并返回到 $(v^1, 19)$。

第6章 近似匹配

.1 基本概念

近似串匹配，又可称为“允许误差的串匹配”，就是在文本 T 中查找模式串 p，并允许式串 p和它在文本中的出现之间存在有限的 k 个差异。

现在已经有很多度量这种“差异”的模型，我们在此重点讨论目前最流行的编辑距模型，也称为 Levenshtein 距离或编辑距离[Lev65]。另外还有一些模型，只不过非常杂，尤其是在计算生物学领域中。而编辑距离模型是应用最广泛的，并且在这个模型基础上产生了很多高效的算法，其中一些算法可以进行扩展从而应用到那些更复杂的型上。

在编辑距离模型中，一个差异等价于一个编辑操作：插入一个字符、删除一个字符，者替换一个字符。设 x，y是两个字符串，它们之间的编辑距离 $ed(x,y)$就是将 x 转变为 y 或者是将 y转变为 x ）所需的最少编辑操作次数，例如：ed(annual, annealing) = 4。因此，似串匹配也可以描述为：在文本 T中找出所有的子串 p'，使得 $ed(p,p') \leqslant k$。为了保证匹过程的输出结果个数是线性的，通常只返回 p'出现的起始位置或者终止位置。

注意，如果模式串的长度为 m，那么上面的 k 值必须满足 $0< k < m$，否则 T 中任意一长度为 m 的子串都可以通过替换 m 个字符转变为 p，从而形成一个成功匹配。当 k 取 0，近似串匹配就是精确字符串匹配。比值 $\alpha = k/m$ 称为错误水平，通过它可以对模式容错误的能力进行度量。

本章将着重关注那些在可预见的实际应用中速度最快的算法，尤其是在信息检索和计生物学领域中。通常，在很多应用中都满足 $\alpha < 1/2$。

本章将讨论四种近似串匹配的方法：第一种方法是最古老的，同时也是最灵活的，使动态规划算法来计算编辑距离；第二种方法是使用基于自动机的公式，并用各种方法来拟这个自动机；第三种方法是所有方法中最成功的，用位并行来模拟其他方法；第四种

方法首先使用一些简单的条件过滤掉文本中的大段不相关文本，然后用其他算法对剩余文本进行搜索。在错误水平较低时，过滤算法的效果是最好的。

6.2 动态规划算法

近似串匹配问题中最古老的几个算法都是基于动态规划的，从20世纪60年代以来，已经有很多动态规划的搜索算法，其中最经典的是Sellers于1980年设计的一个算法[Sel80]。尽管该算法的效率不是很好，而其时间复杂度为$O(m)$，但对它稍经修改便能适用于更复杂的距离模型。

下面，首先讨论这个算法是怎样计算两个串之间的编辑距离的。然后，介绍允许错误的模式匹配算法，并介绍怎样使该算法在平均状态下速度更快。最后，将介绍另外几个基于动态规划的算法。

6.2.1 编辑距离的计算

基于动态规划的近似串匹配算法的首要问题是计算编辑距离$ed(x,y)$。动态规划算法计算编辑距离时要使用一个矩阵$M_{0...|x|,0...|y|}$，这里$M_{i,j}$的含义是：将$x_{1...i}$变成$y_{1...j}$所需的最少编辑操作次数，亦即$M_{i,j}=ed(x_{1...i},y_{1...j})$。矩阵$\boldsymbol{M}$的计算过程如下：

$$\begin{aligned} M_{0,0} &\leftarrow 0 \\ M_{i,j} &\leftarrow \min(M_{i-1,j-1}+\delta(x_i,y_j), M_{i-1,j}+1, M_{i,j-1}+1) \end{aligned}$$

这里$\delta(a,b)=0$当且仅当$a=b$，否则$\delta(a,b)=1$；当$i>|x|$或者$j>|y|$时，$M_{i,j}$取无穷大值。那么最后$M_{|x|,|y|}=ed(x,y)$。

上式的含义如下：$M_{0,0}$是两个空串的编辑距离，值为0；对于长度分别为i和j的两个串x，y来说，假定所有长度小于i的x的子串和所有长度小于j的y的子串的编辑距离已经计算出来，则可以利用它们推导出$ed(x_{1...i},y_{1...j})$。

考虑两个串的最末字符x_i和y_j：如果x_i和y_j相等，那么只要把$x_{1...i-1}$转换成$y_{1...j-1}$就可将x转换成y，此时$M_{i,j}=M_{i-1,j-1}$；如果x_i和y_j不等，则有且只有三种办法将x通过三种编辑操作转换成y：

用y_j替换x_i，然后将$x_{1...i-1}$转换成$y_{1...j-1}$，此时$M_{i,j}=M_{i-1,j-1}+1$

从x中删除x_i，然后将$x_{1...i-1}$转换成$y_{1...j}$，此时$M_{i,j}=M_{i-1,j}+1$

从y中删除y_j，然后将$x_{1...i}$转换成$y_{1...j-1}$，此时$M_{i,j}=M_{i,j-1}+1$

这里要注意的是，在一个串中插入一个字符的效果等价于在另一个串中删除相应的字符。综合上述几种情况，便得到上面的递推公式。

从上面的推导过程可以看出，不论是最坏情况下还是平均情况下，动态规划这种算法的时间复杂度都为 $O(|x||y|)$。一个更易于编程的计算矩阵 $\boldsymbol{M}$ 的公式如下：

$$M_{i,0} \leftarrow i, \qquad M_{0,j} \leftarrow j$$

$$M_{i,j} \leftarrow \begin{cases} M_{i-1,j-1} & \text{如果 } x_i = y_j, \\ 1 + \min(M_{i-1,j-1}, M_{i-1,j}, M_{i,j-1}) & \text{否则} \end{cases} \tag{6.1}$$

这个公式和前面的递推公式是等价的。因为矩阵 $\boldsymbol{M}$ 中相邻元素的值最多相差 1，所以当 $(x_i, y_j) = 0$ 时，$M_{i-1,j-1}$不可能大于 $M_{i-1,j} + 1$或 $M_{i,j-1} + 1$。

计算出矩阵 $\boldsymbol{M}$ 后，便可以回溯得到最优路径，也就是从 $M_{0,0}$到 $M_{|x|,|y|}$最小代价的矩阵元素序列。路径可能存在多条，每条路径都对应一个比对，它是 x 中的每个字符和 y 中的每个字符之间的一幅映射图，表示 x 怎样经过匹配、替换、删除等操作而转变为 y。关于字符串比对的详细内容请参考[Gus97]。

图 6.1 给出了用动态规划算法计算 *ed*(annual, annealing)的示意图。

		a	n	n	e	a	l	i	n	g
	0	1	2	3	4	5	6	7	8	9
a	1	**0**	1	2	3	4	5	6	7	8
n	2	1	**0**	1	2	3	4	5	6	7
n	3	2	1	**0**	1	2	3	4	5	6
u	4	3	2	1	**1**	2	3	4	5	6
a	5	4	3	2	2	**1**	2	3	4	5
l	6	5	4	3	3	2	**1**	**2**	**3**	**4**

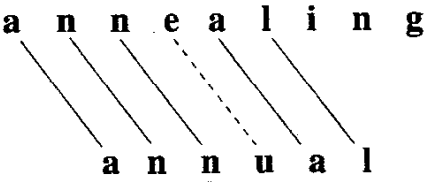


图 6.1　动态规划算法计算编辑距离矩阵示例

图 6.1 为动态规划算法示例：计算“annual”和“annealing”的编辑距离。图中的黑体字标明的矩阵元素给出了唯一的最优路径，右边给出了字符串的比对，其中虚线表示替换操作。

6.2.2 在文本中搜索

上一节中，在计算 x 和 y的编辑距离 *ed*(x , y)时，如果把 $x = p$ 看做模式，把 $y = T$看做文本，那么在 T 中搜索 p的问题就与计算 *ed*(p, T)的问题极为相似了。唯一的差别是：在 T 中搜索 p时，允许匹配结果起始于 T 中的任意位置。为了达到上述目的，可以对编辑距离矩阵 $\boldsymbol{M}$ 的初始化做如下修改：令 $M_{0,j} = 0(j \in 0 \ldots n)$。这表示空模式串可以在文本中的任何位置出现，因为它可以和任意一个长度为 0 的子串匹配。

算法的时间复杂度为$O(mn)$，由于使用编辑距离矩阵 $\boldsymbol{M}$，因此它的空间复杂度也为$O(mn)$。然而，如果稍做改进，算法的空间复杂度可下降到$O(m)$。这主要是因为在计算$M_{*,j}$时，仅仅需要知道$M_{*,j-1}$的值，因此在逐字符处理T时，只需要保存矩阵$\boldsymbol{M}$中的一列C，而不是整个矩阵。每当从T中读入一个字符，都要更新C的值，以保持等式$C_i = M_{i,j}$成立。

改进后的算法用$C_i \leftarrow i$值初始化列向量$C_{0...m}$，然后每当从T中读入一个字符t_j，用如下公式计算$C'_{0...m}$：

$$C'_i \leftarrow \begin{cases} C_{i-1} & \text{如果 } p_i = t_j \text{ ,} \\ 1 + \min(C_{i-1}, C'_{i-1}, C_i) & \text{否则} \end{cases}$$

当处理到T中的某一个字符时，如果满足$C_m \leqslant k$，那么就报告一个以当前位置为结尾的成功匹配。这里需要注意的是，向量$\boldsymbol{C} = M_{*,j-1}$是旧的列向量，而$\boldsymbol{C'} = M_{*,j}$是新的列向量，$C_{i-1}$对应于公式（6.1）中的$M_{i-1,j-1}$，$C'_{i-1}$对应于$M_{i-1,j}$，$C_i$对应于$M_{i,j-1}$。

图 6.2 中给出了用上述算法在文本“annealing”中搜索模式串“annual”的示例，阈值$k = 2$。这个例子中搜索到三个成功匹配。

		a	n	n	e	a	l	i	n	g
	0	0	0	0	0	0	0	0	0	0
a	1	0	1	1	1	0	1	1	1	1
n	2	1	0	1	2	1	1	2	1	2
n	3	2	1	0	1	2	2	2	2	2
u	4	3	2	1	1	2	3	3	3	3
a	5	4	3	2	2	1	2	3	4	4
l	6	5	4	3	3	**2**	**1**	**2**	3	4

图 6.2　利用动态规划算法搜索模式

图 6.2 为动态规划算法示例：在文本“annealing”中搜索“annual”，最多允许两个错误（$k = 2$）。图中每一列对应于每读入一个字符时的向量$\boldsymbol{C}$。图中的黑体字标出了成功匹配的终止位置。

6.2.3　平均情况下的改进

[Ukk85]对上面的动态规划算法进行了改进，在保留动态规划算法所有灵活性的前提下，把算法的平均时间复杂度降低到$O(kn)$[CL92, BYN99]，这个算法称为 DP 算法。算法的思想是：通常情况下，模式串在文本中成功匹配的数目是很少的，因此，在上面的编辑距离矩阵$\boldsymbol{M}$中，每一列的值从上往下计算很快就达到$k+1$（即不匹配了），并且如果某个

阵元素的值大于$k+1$，那么搜索结果就不会依赖于这个元素的精确值。如果一个矩阵元素的值小于或等于k，那么这个元素就称为活动的。本算法只计算到每一列的最后一个活动元素，不需要再对后面的元素进行计算。

每当从文本中读入一个字符，对于矩阵中的每一个新列，都必须重新计算它的最后活动元素。由于矩阵$\boldsymbol{M}$中相邻元素的值最多相差1，因此每当从文本中读入一个字符时，矩阵中相对应列的最后活动元素最多比前一列的最后活动元素下移一个位置。这时，只需要常数时间就可以检查新列中的下一个元素是否被激活。但是存在另外一种可能，即前一列的最后活动元素现在变成非活动的了，这时，必须从当前位置向上搜索新的最后活动元素。尽管对于某一列来说，最坏情况下计算最后活动元素需要$O(m)$的时间，但所有列的处理时间之和不会超过$O(n)$。因为在整个计算过程中，最后活动元素的值的增量最多为n；同样，减量也最多为n。因此，在分摊的最坏情况下，每列的最后活动元素的平均维护时间为$O(1)$。

图 6.3 给出了这个算法的伪码。这种避免计算非活动矩阵元素的思想广泛应用于其他算法，特别是后面将要讨论的位并行算法，就使用这种技术来减少平均搜索时间。

```
DP (p = p1p2...pm, T = t1t2...tn, k)
1.    Preprocessing
2.        For i ∈ 0...m Do Ci ← i
3.        lact ← k + 1 /* last active cell */
4.    Searching
5.        For pos ∈ 1...n Do
6.            pC ← 0, nC ← 0
7.            For i ∈ 1...lact Do
8.                If pi = tpos Then nC ← pC
9.                Else
10.                   If pC < nC Then nC ← pC
11.                   If Ci < nC Then nC ← Ci
12.                   nC ← nC + 1
13.               End of if
14.               pC ← Ci, Ci ← nC
15.           End of for
16.           While Clact > k Do lact ← lact − 1
17.           If lact = m Then report an occurrence at pos
18.           Else lact ← lact + 1
19.       End of for
```

图 6.3 一个期望时间复杂度为$O(kn)$的动态规划算法。注意，本算法只需要保存一个列向量

6.2.4 其他基于动态规划的算法

有很多算法都是基于动态规划的，从实用观点来看，其中最让人感兴趣的莫过于所谓的“列划分”算法[CL92]。这种算法的平均时间复杂度为 $O(kn/\sqrt{|\Sigma|})$ [Nav01a]，它是所有基于动态规划的算法中速度最快的。但它的缺点是很难扩展到其他更复杂的距离模型。在复杂的距离模型下，一些新的位并行算法的速度会更快。

从理论上来说，很多重要的算法都是基于动态规划的。如果要求算法的空间复杂度是关于 m 和 k 的多项式，那么在已有算法中，最好的算法可以达到 $O(m)$ 的空间复杂度，且最坏情况下的搜索时间复杂度可达到 $O(kn)$。实际应用中被认为最好的算法是在[GP90, CL94]中提出的，但相对于那些不加以最坏时间复杂度为 $O(kn)$ 限制的算法，它还是要慢一些。如果 k 远小于 m，则时间复杂度为 $O(n(1+k^4/m))$ 的算法[CH98]是最快的。在限定空间复杂度为关于 m 和 k 的多项式的前提下，动态规划算法的最坏复杂度的下界还是一个有待研究的问题。

6.3 基于自动机的算法

近似搜索问题的另一种非常有用的方法是用非确定的自动机（NFA）来模拟搜索过程。这种自动机最初以其确定化的形式在[Ukk85]中被提出，后来在[BY91, WM92b, BYN99]中给出了更明确的阐述。

考虑图 6.4 中在编辑距离模型下最多允许 $k = 2$ 个错误时的 NFA。每一行的行号表示匹配时当前的错误数，每一列表示当前已经匹配了模式的一个前缀。水平箭头表示匹配了一个字符（就是说，如果模式串和文本串中的对应字符相同，则搜索过程会在模式串和文本串中各前进一个字符）。如果模式串和文本串中的对应字符不相同，则错误数需要增加 1，此时进入下一行：垂直箭头表示需要在模式串中插入一个字符（搜索过程在文本串中前进一个字符而在模式串中保持原位置不动）；实心对角箭头表示替换一个字符（搜索过程在文本串和模式串中同时前进一个字符）；虚线对角箭头表示在模式串中删除一个字符（它们是 ε-转移，即模式串前进一个字符而文本串保持原位置不动）。初始状态的自环表示允许成功匹配出现在文本中的任何位置。当最右列中的终止状态被激活时，自动机报告一个成功匹配（报告它的终止位置）。

不难看出，一旦自动机中的某个状态被激活，则同一列中那些具有更大行号的状态也都将被激活。而且，对于一个给定的文本位置，如果收集 NFA 每一列被激活状态的最小行

，那么就能得到动态规划矩阵中对应的垂直列向量。例如，在读入文本“anneal”后，6.2 中第 7 列为 $C = [0, 1, 1, 2, 3, 2, 1]$，读者可以将其与图 6.4 中 NFA 的每一列被激活状的最小行号进行比较。

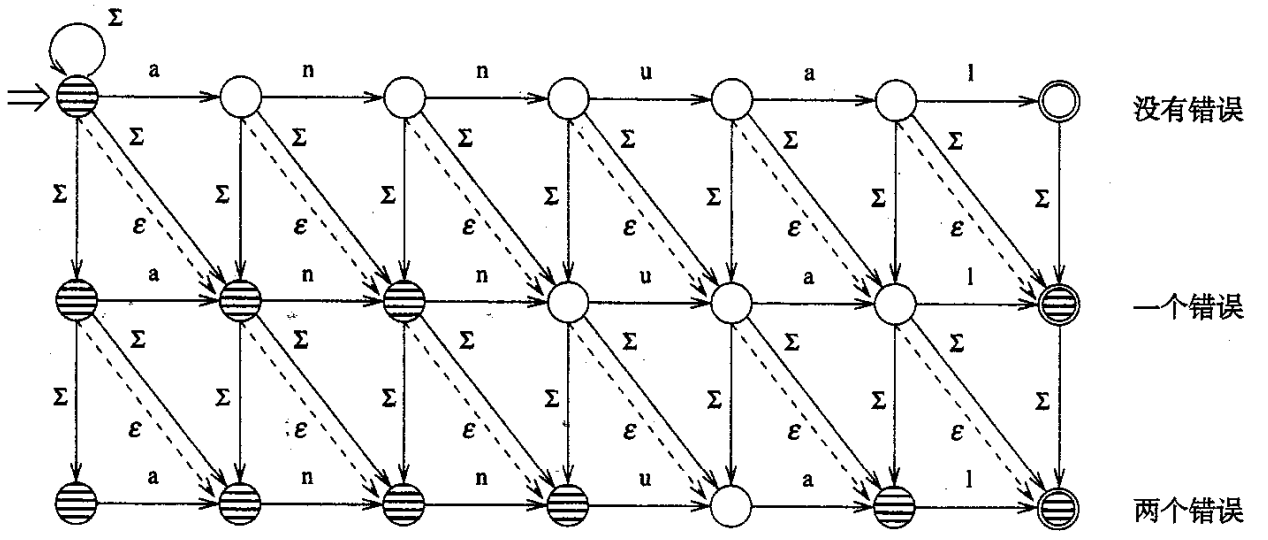


图 6.4 对模式串“annual”进行近似匹配的 NFA，最多允许两个错误。
被标为阴影的状态表示读入文本串“anneal”后被激活的状态

[Ukk85]中提出的算法是使用经典的 NFA 到 DFA 的转换算法将上述自动机确定化。过这种转换，搜索时间在最坏情况下为 $O(n)$，已经达到最优。这样，问题主要存在于FA 的构造和它所需的存储。[Ukk85]中给出了构造的 DFA 状态数目的一个上界，即为 $(\min(3^m, m(2m|\Sigma|)^k))$。实际应用中，当 $m > 20$ 时，这个自动机将无法使用。时至今，即使对于较小的 m，它也不是一个最好的选择：由于具有更好的引用局部性，位并行法具有更快的匹配速度。

另一个利用 DFA 进行搜索的方法是将 DFA 的每个状态视为动态规划矩阵的某一列。样，在预处理阶段就能计算出动态规划矩阵中各列之间对于每一个可能的输入字符所进的状态转移。

稍后的[WMM96]基于 Four-Russians 方法通过将列简化为“区域”从而解决了空间代价预处理代价的问题。图 6.5 给出了这种自动机的示意图。

给定 $O(s)$的空间，上述算法（指[WMM96]中的算法）能达到 $O(kn/\log s)$ 的平均搜索间和 $O(mn/\log s)$ 的最坏搜索时间。在实际应用中，对于较长的模式串和较高的错误水 $(\alpha > 0.7)$，这个算法是最快的。因为这个算法过于复杂，这里不再对它进行详细阐述，且 0.7 的错误水平对于大多数应用来说太高了，并不实用。

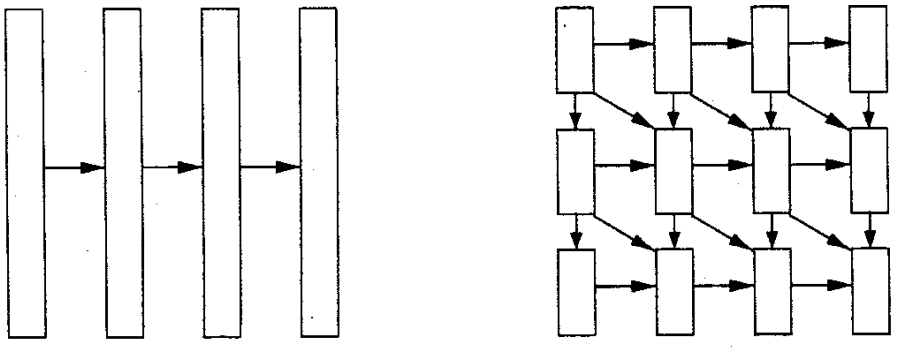


图 6.5　左边为完全 DFA，其中每一列是一个状态；右边是基于 Four-Russians 方法的自动机，其中每个区域是一个状态。图中的箭头表明了相邻区域间的依赖关系

6.4　位并行算法

近似搜索中大量使用位并行方法，很多很好的结果都是使用位并行方法得到的。位并行算法非常适用于模式串较短的情况，而现实需求中很多搜索都是对短模式串进行的。如果模式串的长度超过了计算机的字长，通常的做法是通过使用多个机器字来模拟一个长的、虚拟的机器字。然后，可以利用 6.2.3 节中 DP 算法所采用的技巧。这时，只需要更新那些包含活动数据的机器字即可。

位并行算法是用来模拟“经典”算法的。在近似搜索中可以看到，有些算法是对 NFA 的并行模拟，而其他算法是对编辑距离矩阵的并行模拟。

6.4.1　并行化 NFA

考虑图 6.4 中的第一列，它实际上是一个精确字符串匹配的 NFA，也就是 2.2.2 节中用 Shift-And 方法模拟的 NFA。现在已有多种技术可将这个思想推广到更一般的自动机上。

行式位并行

最简单的位并行 NFA 技术[WM92b]称为 BPR，它将 NFA 中的每一行 i 装入一个机器字 R_i，NFA 的每一个状态用机器字中的一比特位表示。每当从文本中读入一个字符时，自动机中的状态转移都是通过对这 $k+1$ 个位掩码的位运算来模拟的。注意：$k+1$ 个位掩码具有相同的结构，即掩码中每一比特位是和文本中同一个字符对齐的。当读到文本位置 j 时，通过下面的公式可将 R_i 的当前值更新到新值 R'_i：

$$
\begin{aligned}
R'_0 &\leftarrow ((R_0 << 1) \mid 0^{m-1}1) \ \&\ B[t_j] \\
R'_i &\leftarrow ((R_i << 1) \ \&\ B[t_j]) \mid R_{i-1} \mid (R_{i-1} << 1) \mid (R'_{i-1} << 1)
\end{aligned}
$$

中表B和 Shift-And 算法中的表B是一样的。搜索之前初始化$R_i = 0^{m-i}1^i$，这和 DP 算中初始化$C_i = i$是等价的。正如预想的那样，R_0的更新实际上就是简单的 Shift-And 过，而其余行从前面的行接收 1（即，活动状态）。在更新R'_i的公式中，各项分别反映了动机中的水平箭头、垂直箭头、对角箭头以及虚线对角箭头。图 6.6 给出了本算法的伪码。

```
BPR (p = p1p2...pm, T = t1t2...tn, k)
1.    Preprocessing
2.        For c ∈ Σ Do B[c] ← 0^m
3.        For j ∈ 1...m Do B[pj] ← B[pj] | 0^{m-j}10^{j-1}
4.    Searching
5.        For i ∈ 0...k Do Ri ← 0^{m-i}1^i
6.        For pos ∈ 1...n Do
7.            oldR ← R0
8.            newR ← ((oldR << 1) | 0^{m-1}1) & B[t_pos]
9.            R0 ← newR
10.           For i ∈ 1...k Do
11.               newR ← ((Ri << 1) & B[t_pos]) | oldR | ((oldR | newR) << 1)
12.               oldR ← Ri, Ri ← newR
13.           End of for
14.           If newR & 10^{m-1} ≠ 0^m Then report an occurrence at pos
15.       End of for
```

图 6.6 用行式位并行模拟 NFA，模式串长度必须小于w

本算法在最坏情况下和平均情况下的代价都是$O(k\lceil m/w\rceil n)$，对于典型的文本搜（即$m \leqslant w$）来说是$O(kn)$。对于短模式串，同那些具有最优的最坏时间复杂度的法相比，本算法也是很有竞争力的。稍后将会介绍一些更好的算法，不过本算法的活性最好，而且很容易扩展到更复杂的情况，例如允许错误的通配符或者正则表达搜索，等等。

BPR 示例 在文本“annealing”中搜索模式串“annual”，最多允许$k = 2$个错误。注：在任意位置上，如果在 NFA 中从右向左读入状态，则R_0，R_1和R_2的位表示都类似于 6.4 中的活动状态。但这里要除去 NFA 中的第一列，因为第一列自始至终都是活动的。别是，可以对读入文本“anneal”后的位图和图 6.4 中的活动状态进行比较。此外，对位和图 6.2 中的列值进行比较将会发现：C_i是 NFA 的第i列中活动状态的最小行号。

$$B = \begin{cases} \texttt{a} & 010001 \\ \texttt{l} & 100000 \\ \texttt{n} & 000110 \\ \texttt{u} & 001000 \\ * & 000000 \end{cases}$$

$R_0 = 000000$
$R_1 = 000001$
$R_2 = 000011$

1. 读入 a　　010001
   $R_0 =$ 000001
   $R_1 =$ 000011
   $R_2 =$ 000111

2. 读入 n　　000110
   $R_0 =$ 000010
   $R_1 =$ 000111
   $R_2 =$ 001111

3. 读入 n　　000110
   $R_0 =$ 000100
   $R_1 =$ 001111
   $R_2 =$ 011111

4. 读入 e　　000000
   $R_0 =$ 000000
   $R_1 =$ 001101
   $R_2 =$ 011111

5. 读入 a　　010001
   $R_0 =$ 000001
   $R_1 =$ 010011
   $R_2 =$ 111111

   R_2的最后一个比特位被置位，因此报告一个成功匹配。

6. 读入 l　　100000
   $R_0 =$ 000000
   $R_1 =$ 100011
   $R_2 =$ 110111

   R_2的最后一个比特位再次被置位，因此报告一个成功匹配。注意，即使在R_1中的当前位置，也找到一个成功匹配，错误数$k = 1$。

7. 读入 i　　000000
   $R_0 =$ 000000
   $R_1 =$ 000001
   $R_2 =$ 100111

   R_2的最后一个比特位被置位，因此报告一个成功匹配。注意，本次成功匹配只是前一次$k < 2$个错误数的成功匹配导致的必然结果，因为并没有更多的字符得到匹配。

8. 读入 n　　000110
   $R_0 =$ 000000
   $R_1 =$ 000011
   $R_2 =$ 000111

9. 读入 g　　000000
   $R_0 =$ 000000
   $R_1 =$ 000001
   $R_2 =$ 000111

对角式位并行

对照前述行式位并行算法，可以把经典的动态规划算法看做自动机的列式位并行，其中 NFA 的每一列对应于 C 中的一个元素，它保存了 NFA 的这一列中活动状态的最小行号。即使 NFA 的状态数目不太多，可以装入一个机器字，却也不能再提高算法的并行性，因为自动机的ε-转移会导致零时依赖。也就是说，两行或者两列的当前值是互相依赖的，不能并行计算。

[BYN99]中提出了一个对角式位并行公式，基于这个公式的算法称为 BPD 算法。这个算法把自动机的状态沿着对角线（而不是行或列）装入机器字，按照对角线箭头的方向运算。自动机中一共有 $m-k+1$ 条完整的对角线，从左向右分别以 0 到 $m-k$ 给它们编号。假设对角线 i 中活动状态的最小行号为 D_i，则对角线中其余行号更大的状态都会因为 ε-转移而被激活。在读入文本中位置 j 的字符后，计算新的 D_i' 值的公式为：

$$D_i' \leftarrow \min\left(D_i+1,\ D_{i+1}+1,\ g(i-1,t_j)\right) \qquad (6.2)$$

式中第一项表示字符替换，第二项表示字符插入，最后一项表示字符匹配。字符删除是隐藏在其中的，因为这里只关心每条对角线上具有最小行号的活动状态。接下来的主要问题是如何计算函数 g，其定义如下：

$$g(i,c) = \min\left(\{k+1\} \cup \{\, r,\ r \geqslant D_i \text{ AND } p_{i+1+r} = c \,\}\right)$$

这个公式表示：对于对角线 i 上的所有活动状态，即 $r \in \{D_i,\ D_i+1,\ \ldots,\ k\}$，首先找到那些能沿着水平箭头（即 $p_{i+1+r} = c$）转移到对角线 $i+1$ 的状态，然后从中选取行号最小的。对函数 g 的另外一个理解是：如果对角线上的某个活动状态可以通过水平边转移，那么这个活动状态也将会沿着对角线向下传播。

[BYN99]使用位并行技术模拟了上述更新过程：用一进制来表示 D_i 的值，用比特位上的算术运算来产生期望的传播效果（4.4 节中详细介绍了类似的洪泛问题的解决办法）。上述更新公式既可以从算术（在 D_i 上进行算术运算）的角度来考虑，也可以从逻辑的角度来考虑（模拟自动机中的状态转移）。算法在一个机器字 D 中装入 $m-k$ 个块，因为 D_0 对应的最左对角线中的状态一直都是活动的，所以除了 D_0，每条对角线都占一块。从左到右，$D_1 \ldots D_{m-k}$ 依次表示出来，且每块包含 $k+2$ 个比特位。每一块的最左（高）位总是为 0，从而避免了算术运算的进位而影响相邻的块；每一块的其余 $k+1$ 位用一进制来表示 D_i：i 中最右边的 D_i 个位被置 1，其余位都置 0。这里还需要使用经典的 B 表，只是需要将 B 中的所有位取反。算法还从表 B 生成另一个表 BB，使得自动机中的水平箭头和 D 中的相应位对齐。

图 6.7 给出了这个算法的伪码。算法没有包含位于最右边的完全对角线右边的状态，这将导致某些成功匹配会被漏掉。但是，这些漏掉的结果在大多数应用中是无关紧要的，因为它们肯定是那些已被找到的成功匹配的平凡扩展，这时不会有更多的模式串中的字符匹配文本中的字符（就如 BPR 算法示例中在处理完“anneali”之后找到的结果）。为了确保所有这样的成功匹配都被丢弃，一旦找到一个成功匹配，算法的第 14 行就清除了最右对角线中所有的活动状态。因此，算法只会报告那些以模式串最后一个字符结尾的成功匹配。

算法的第 11 行按照公式（6.2）通过对四个表达式的“位与”运算来更新 D，“位与”运算实际上是找出两个一进制值中的最小值。第一个表达式表示 D_i+1，第二个表达式表示 $D_{i+1}+1$，第三个表达式表示 $g(i-1,t_{pos})$，最后一个表达式表示清除块分隔位。关于第三个表达式，请注意：x 存储了前一个对角线中通过水平箭头转移到本对角线的活动状态，同时还清除了每块最后（右）的 0 的左边的 1。

BPD $(p=p_1p_2\ldots p_m,\ T=t_1t_2\ldots t_n,\ k)$

1. **Preprocessing**
2. 　　**For** $c\in\Sigma$ **Do** $B[c]\leftarrow 1^m$
3. 　　**For** $j\in 1\ldots m$ **Do** $B[p_j]\leftarrow B[p_j]\ \&\ 1^{m-j}01^{j-1}$
4. 　　**For** $c\in\Sigma$ **Do**
5. 　　$BB[c]\leftarrow 0\ s_{k+1}(B[c],0)\ \ 0\ s_{k+1}(B[c],1)\ \ \ldots\ \ 0\ s_{k+1}(B[c],m-k-1)$
6. 　　**End of for**
7. **Searching**
8. 　　$D\leftarrow(01^{k+1})^{m-k}$
9. 　　**For** $pos\in 1\ldots n$ **Do**
10. 　　$x\leftarrow(D>>(k+2))\mid BB[t_{pos}]$
11. 　　$D\leftarrow((D<<1)\mid(0^{k+1}1)^{m-k})$
　　$\&\ ((D<<(k+3))\mid(0^{k+1}1)^{m-k-1}01^{k+1})$
　　$\&\ (((x+(0^{k+1}1)^{m-k})\ \wedge\ x)>>1)\ \&\ (01^{k+1})^{m-k}$
12. 　　**If** $D\ \&\ 0^{(m-k-1)(k+2)}010^k\ =\ 0^{(m-k)(k+2)}$ **Then**
13. 　　Report an occurrence at pos
14. 　　$D\leftarrow D\mid 0^{(m-k-1)(k+2)}01^{k+1}$
15. 　　**End of if**
16. 　　**End of for**

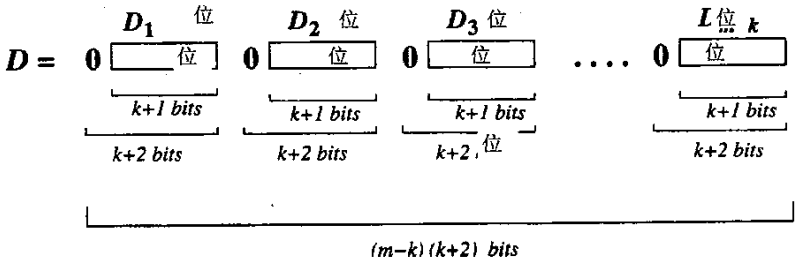


图 6.7　对角式位并行模拟 NFA，必须满足 $(m-k)(k+2)\leqslant w$。其中函数 $s_\ell(D,j)$ 表示从 D 中抽取从第 j 位到第 $(j+\ell-1)$ 位，即：$s_\ell(D,j)=(D>>j)\ \&\ 0^{(m-k)(k+2)-\ell}1^\ell$。在最下方的图中示意了怎样在位掩码 D 中排列一进制值 D_i

如果表示自动机状态的所有比特位都能装入一个机器字，那么上述算法在最坏情况下的时间复杂度为 $O(n)$，这在实际应用中是相当快的。而行式位并行算法的最坏时间复杂度

$O(kn)$。一般说来，如果不能将自动机状态装入一个机器字，那么算法的最坏时间复杂为 $O(\lceil k(m-k)/w \rceil n)$。类似于 DP 算法中只更新活动元素的技术，如果只更新包含活动态的机器字，那么算法的平均时间复杂度将会降为 $O(\lceil k^2/w \rceil n)$。本算法的思想也可以用处理其他一些情况，包括带字符组的扩展匹配，带通配符的扩展匹配，各种编辑操作具不同整数代价情况下的匹配[BYN99]，等等，但其灵活性不如行式位并行方法。

BPD 算法示例 在文本“annealing”中搜索模式串“annual”，最多允许 $k=2$ 个错误。里，比较难以从视觉上把 D 的位掩码表示和图 6.4 中的 NFA 关联起来。关联的规则是：取 NFA 中的完整对角线，除了第一条，将它们对应到 D 中的位块。从上向下读取对角线的状态，在块中从右向左读取每一位，将活动状态对应到值为 0 的位。

$B=$

a	101110
l	011111
n	111001
u	110111
*	111111

$D=0111\ 0111\ 0111\ 0111$

表 *BB*

a	0101 0011 0111 0110
l	0011 0111 0111 0111
n	0111 0110 0100 0001
u	0110 0101 0011 0111
*	0111 0111 0111 0111

1. 读入 a

$BB[\mathtt{a}]\ 0101\ 0011\ 0111\ 0110$
$D=\ 0000\ 0111\ 0111\ 0111$

2. 读入 n

$BB[\mathtt{n}]\ 0111\ 0110\ 0100\ 0001$
$D=\ 0001\ 0000\ 0111\ 0111$

3. 读入 n

$BB[\mathtt{n}]\ 0111\ 0110\ 0100\ 0001$
$D=\ 0001\ 0001\ 0000\ 0111$

4. 读入 e

$BB[\mathtt{e}]\ 0111\ 0111\ 0111\ 0111$
$D=\ 0011\ 0001\ 0001\ 0111$

5. 读入 a

$BB[\mathtt{a}]\ 0101\ 0011\ 0111\ 0110$
$D=\ 0000\ 0011\ 0011\ 0001$

$D_{m-k}=D_4$ 的最高位（D 中从右到左第三位）为零，因此报告一个成功匹配，并清理最后的对角线：

$D=0000\ 0011\ 0011\ 0111$

6. 读入 l

$BB[\mathtt{l}]\ 0011\ 0111\ 0111\ 0111$
$D=\ 0001\ 0111\ 0011\ 0011$

D_4 的最高位再次为零，因此报告一个成功匹配，并清理最后的对角线：

$D=0001\ 0111\ 0011\ 0111$

7. 读入 i

BB[i]	0111	0111	0111	0111
$D=$	0011	0111	0111	0111

不同于经典算法，这里并不报告一个成功匹配，因为并没有更多的模式串字符得到匹配。这是前一步中清理最后对角线的自然结果。

8. 读入 n

BB[n]	0111	0110	01000001	
$D=$	0001	0111	0111	0111

9. 读入 g

BB[g]	0111	0111	0111	0111
$D=$	0011	0111	0111	0111

6.4.2 并行化动态规划矩阵

对动态规划算法进行并行化时，更好的方式是保存编辑矩阵 $\boldsymbol{M}$ 中相邻两行（或者两列）之间的差值，而不是矩阵的原始值[Mye99]。定义：

$\Delta h_{i,j} = M_{i,j} - M_{i,j-1} \in \{-1,0,+1\}$为相邻元素的横向差分；

$\Delta v_{i,j} = M_{i,j} - M_{i-1,j} \in \{-1,0,+1\}$为相邻元素的纵向差分；

$\Delta d_{i,j} = M_{i,j} - M_{i-1,j-1} \in \{0,1\}$为相邻元素的对角差分。

上面这三种差分的取值范围是由编辑矩阵 $\boldsymbol{M}$ 的性质得到的。

这里介绍的算法[Hyy01]与原始算法[Mye99]稍有不同：虽然这两种方法对于文本中的任一字符的编辑操作的次数是相同的，但这里介绍的算法更容易理解。

算法中引入了如下布尔变量，其中前面的四个分别指[横向/纵向][正/负]差分，最后一个指对角零差分：

$$VP_{i,j} \equiv \Delta v_{i,j} = +1 \qquad VN_{i,j} \equiv \Delta v_{i,j} = -1$$
$$HP_{i,j} \equiv \Delta h_{i,j} = +1 \qquad HN_{i,j} \equiv \Delta h_{i,j} = -1$$
$$D0_{i,j} \equiv \Delta d_{i,j} = 0$$

注意，如下等式是成立的：

$$\Delta v_{i,j} = VP_{i,j} - VN_{i,j},$$
$$\Delta h_{i,j} = HP_{i,j} - HN_{i,j},$$
$$\Delta d_{i,j} = 1 - D0_{i,j}。$$

根据编辑矩阵 $\boldsymbol{M}$ 的定义以及上面的等式，容易证明：

$$M_{i,j} = \sum_{r=1\ldots i} \Delta v_{r,j}$$

注意到上面的值之间存在着如下依赖关系，对 DP 算法并行化的关键思想就是基于这些依赖关系。

- 如果$HN_{i,j}$为真，也就是$\Delta h_{i,j}=-1$成立，则可以推出$\Delta v_{i,j-1}=+1$，进一步得到$\Delta d_{ij}=0$，否则会违反 Δ 的取值范围。结果中的两个等式等价于关系运算式$VP_{i,j-1}$ AND $D0_{i,j}$。另一方面，也可以从结果中的两个等式都为真容易地推导出$HN_{i,j}$为真。即：$HN_{i,j}$逻辑上等价于$VP_{i,j-1}$ AND $D0_{i,j}$。
- 与上面的推导相对称，可以得到：$VN_{i,j}$逻辑等于$HP_{i-1,j}$ AND $D0_{i,j}$。
- 如果$HP_{i,j}$为真，那么$VP_{i,j-1}$不可能为真，否则$\Delta d_{ij}=\Delta h_{ij}+\Delta v_{i,j-1}=2$越界。因此$\Delta v_{i,j-1}$只能取-1 或者 0，如果取-1，就有$VN_{i,j-1}$为真；如果取 0，有$VP_{i,j-1}$为假且$D0_{i,j}$为假。而且，也可以证明：如果$VN_{i,j-1}$为真，或者$VP_{i,j-1}$和$D0_{i,j}$同时为假，都能推出$HP_{i,j}$成立。即：$HP_{i,j}$逻辑上等价于$VP_{i,j-1}$ OR (NOT $HP_{i-1,j}$ AND NOT $D0_{i,j}$)。
- 与上面相对称，有：$VP_{i,j}$逻辑等于$HN_{i-1,j}$ OR (NOT $HP_{i-1,j}$ AND NOT $D0_{i,j}$)。
- 最后，$D0_{i,j}$为真当且仅当在如下的三种情况下［对应于式（6.1）中的三种取值］：
  - 第一，如果$P_i=T_j$，那么$D0_{i,j}$为真。
  - 第二，当$M_{i,j}=1+M_{i,j-1}=M_{i-1,j-1}$时，也有$D0_{i,j}$为真，这种情况逻辑等价于$HP_{i,j}$ AND $VN_{i,j-1}$，又因为如果$VN_{i,j-1}$取真，则$M_{i,j-1}$比$M_{i-1,j-1}$小 1，由 $M_{i,j}$ 的计算方法可知 $M_{i,j}=1+M_{i-1,j}$，从而有 $HP_{i,j}$ 一定为真，即此时$VN_{i,j-1}$实际上逻辑蕴涵了 $HP_{i,j}$（注意，这种蕴涵是在 $M_{i,j}=1+M_{i,j-1}=M_{i-1,j-1}$的前提下成立的），从而$HP_{i,j}$ AND $VN_{i,j-1}$等价于$VN_{i,j-1}$。
  - 第三，当$M_{i,j}=1+M_{i-1,j}=M_{i-1,j-1}$时，也有$D0_{i,j}$为真，这种情况逻辑等价于$VP_{i,j}$ AND $HN_{i-1,j}$，同前面推导也就是等价于$HN_{i-1,j}$。

综合起来，由上述三种情况的析取，可得：$D0_{i,j}$为真当且仅当$(P_i=T_j)$ OR $VN_{i,j-1}$ OR $VN_{i,j-1}$为真。

因此，可以得到如下几个等式：

$$
\begin{aligned}
HN_{i,j} &\equiv VP_{i,j-1} \text{ AND } D0_{i,j} \\
VN_{i,j} &\equiv HP_{i-1,j} \text{ AND } D0_{i,j} \\
HP_{i,j} &\equiv VN_{i,j-1} \text{ OR NOT } (VP_{i,j-1} \text{ OR } D0_{i,j}) \\
VP_{i,j} &\equiv HN_{i-1,j} \text{ OR NOT } (HP_{i-1,j} \text{ OR } D0_{i,j}) \\
D0_{i,j} &\equiv (P_i=T_j) \text{ OR } VN_{i,j-1} \text{ OR } HN_{i-1,j}
\end{aligned}
$$

并行化后的算法也是从左向右顺序扫描文本，对于文本的当前扫描位置 j ($j=1...n$), 算法计算与每个 i ($i=1...m$)值对应的上述五个布尔值。由于每个布尔值只需要用一个比特表示，因此可以用位掩码来保存$HN, VN, HP, VP, D0$变量的值，在每读入一个目标字符后更新这些掩码。例如，掩码HN的第 i 位对应于$HN_{i,j}$的值。注意，这里当下标为 $j-1$时指的是前一个字符的掩码，而下标为 j 时才是指当前字符的掩码。

由此，算法可以首先计算$D0$，然后计算HN和HP，最后计算VN和VP。然而这里$D0_{i,j}$的计算存在着循环依赖关系：$D0_{i,j}$依赖于$HN_{i-1,j}$，$HN_{i-1,j}$依赖于$D0_{i-1,j}$，也就是说当前目标字符下的$D0_{i,j}$值依赖于当前目标字符下的其他值$D0_{i',j}$。这就出现了零时依赖问题，使得要并行地计算出当前目标字符下所有的$D0$值复杂化。对这个问题的一个解决办法是将$D0_{i,j}$的计算公式展开，得到：

$$D0_{i,j} \equiv (P_i = T_j) \text{ OR } VN_{i,j-1} \text{ OR } (VP_{i-1,j-1} \text{ AND } D0_{i-1,j}),$$

这个等式具有如下递推式形式：$D_i \equiv X_i \text{ OR } (Y_{i-1} \text{ AND } D_{i-1})$，从 $i=1$ 时开始递推：

$$\begin{aligned}
D_1 &\equiv X_1 \\
D_2 &\equiv X_2 \text{ OR } (Y_1 \text{ AND } X_1) \\
D_3 &\equiv X_3 \text{ OR } (Y_2 \text{ AND } X_2) \text{ OR } (Y_2 \text{ AND } Y_1 \text{ AND } X_1) \\
D_i &\equiv \text{ OR }_{r=1}^{i}(X_r \text{ AND } Y_r \text{ AND } Y_{r+1} \text{ AND } \ldots \text{ AND } Y_{i-1})
\end{aligned}$$

设整数 s 满足$Y_s \ldots Y_{i-1}=1$并且$Y_{s-1}=0$，那么对于满足$s \leqslant r \leqslant i$且$X_r = 1$的项将使$D_i$为真。换句话说，如果从$Y_s$到$Y_{i-1}$这段连续 $i-s$ 位都为 1，而且在这个范围内某个X_r也为 1，那么D_i的值将等于 1。如果计算$(Y+(X \ \& \ Y))$将会发现，在Y这个连续为 1 的区域内，任何一个满足$X_r = 1$的位都将会使Y_i从 0 变为 1，这一段内即使有多个$Y_r = 1$，Y_i也将变为 1 而不会重新变回 0。如果再计算$(Y+(X \ \& \ Y)) \wedge Y$（$\wedge$为异或运算），则$Y$中那些被取反的位将会等于 1。注意：如果$Y$的这一段连续的 1 中对应于多个$X_r = 1$，那么经过上述运算后，$Y$中只有第一个这样的 r 位置的值将仍然保持 1，而其余的都由于前面传递过来的进位而变成了 0。为了弥补这些X_r位以及考虑到X_i，把上面得到的结果再和X做或运算，就得到了最后的 D 值。举例如下（其中箭头方向为高位方向）：

$$\begin{aligned}
Y &= 0\,0\,0\,1\,1\,1\,1\,1\,0\,0\,0\,0\,1\,1 \\
X &= 0\,0\,0\,0\,1\,0\,1\,0\,0\,0\,0\,1\,0\,1 \\
X \ \& \ Y &= 0\,0\,0\,0\,1\,0\,1\,0\,0\,0\,0\,0\,0\,1 \\
(Y+(X \ \& \ Y)) &= 0\,0\,1\,0\,1\,0\,0\,1\,0\,0\,0\,1\,0\,0
\end{aligned}$$

$$(Y + (X \,\&\, Y)) \wedge Y = 0\,0\,1\,1\,0\,1\,1\,0\,0\,0\,0\,1\,1\,1$$
$$D0 = ((Y + (X \,\&\, Y)) \wedge Y) \mid X = 0\,0\,1\,1\,1\,1\,1\,0\,0\,0\,0\,1\,1\,1$$

$D0$的计算问题解决后，剩下的就只需要按照前述公式对其余的4个值进行计算即可。图6.8给出了计算过程的伪码。

```
BPM (p = p1p2...pm, T = t1t2...tn, k)
1.    Preprocessing
2.        For c ∈ Σ Do B[c] ← 0^m
3.        For j ∈ 1...m Do B[pj] ← B[pj] | 0^(m-j)10^(j-1)
4.        VP ← 1^m, VN ← 0^m
5.        err ← m
6.    Searching
7.        For pos ∈ 1...n Do
8.            X ← B[t_pos] | VN
9.            D0 ← ((VP + (X & VP)) ∧ VP) | X
10.           HN ← VP & D0
11.           HP ← VN | ~(VP | D0)
12.           X ← HP << 1
13.           VN ← X & D0
14.           VP ← (HN << 1) | ~(X | D0)
15.           If HP & 10^(m-1) ≠ 0^m Then err ← err + 1
16.           Else If HN & 10^(m-1) ≠ 0^m Then err ← err - 1
17.           If err ≤ k Then report an occurrence at pos
18.       End of for
```

图6.8 动态规划矩阵的位并行计算（必须满足模式长度不大于机器字长）

代码中的err保存了$C_m = M_{m,j}$的值，用$HP_{m,j}$和$HN_{m,j}$来更新err的值。注意：这种递推计算能处理差分为0的情况。

上述这种算法称为BPM算法，比前面介绍的其他位并行算法能更充分地利用机器字，它的最坏时间复杂度为$O(\lceil m/w \rceil n)$，平均时间复杂度是$O(\lceil k/w \rceil n)$。与DP算法一样，BPM算法只计算编辑距离矩阵中的活动元素。同BPD算法相比较，BPM算法的计算方法稍显复杂。因此在一般情况下，BPM算法的速度比BPD算法的稍慢一些，但因为它比BPD算法使用了更少的机器字，所以更适用于长模式串的情况。另一方面，BPM算法不如BPD算法灵活，它不适合于复杂的模式串以及不同的距离模型。

此外，只要把图6.8中伪码的第12行后面加上“$\mid 0^{m-1}1$”，算法计算出的结果便是编辑距离，这是因为第零行有一个水平增量（在位掩码中没表示出来）。

BPM算法示例 在串“annealing”中搜索串“annual”，允许最多$k = 2$个错误。要理

解以下计算过程的最好办法是把位掩码和前面的Δ值关联起来考虑，并且理解它们和图6.2中动态规划矩阵各元素值之间的关系。

有意思的是，在这里能够验证变量*err*正确地保存了DP矩阵中当前列的最后元素的值。

$B=$

a	010001
l	100000
n	000110
u	001000
*	000000

$VN = 000000$
$VP = 111111$
$err = 6$

1. 读入a 010001

$D0=$	111111
$HN=$	111111
$HP=$	000000
$VN=$	000000
$VP=$	111110
$err=$	5

2. 读入n 000110

$D0=$	111110
$HN=$	111110
$HP=$	000001
$VN=$	000010
$VP=$	111101
$err=$	4

3. 读入n 000110

$D0=$	111110
$HN=$	111100
$HP=$	000010
$VN=$	000100
$VP=$	111001
$err=$	3

4. 读入e 000000

$D0=$	000100
$HN=$	000000
$HP=$	000110
$VN=$	000100
$VP=$	110011
$err=$	3

5. 读入a 010001

$D0=$	110111
$HN=$	110011
$HP=$	001100
$VN=$	010000
$VP=$	100110
$err=$	2

这里$err \leqslant 2$，找到匹配结果。

6. 读入l 100000

$D0=$	110000
$HN=$	100000
$HP=$	011001
$VN=$	110000
$VP=$	001101
$err=$	1

这里$err \leqslant 2$，找到匹配结果。

7. 读入i 000000

$D0=$	110000
$HN=$	000000
$HP=$	110010
$VN=$	100000
$VP=$	001011
$err=$	2

这里$err \leqslant 2$，找到匹配结果。

8. 读入n 000110

$D0=$	100110
$HN=$	000010
$HP=$	110000
$VN=$	100000
$VP=$	011101
$err=$	3

9. 读入g 000000

$D0=$	100000
$HN=$	000000
$HP=$	100010
$VN=$	000000
$VP=$	011011
$err=$	4

.5 文本快速过滤算法

过滤算法的背后基于这样一种思想：判断文本中某个位置的子串和模式串不相匹配，能要比判断二者相匹配更容易。过滤算法能够过滤掉文本中那些不能产生成功匹配的区，但不能判断剩余区域中是否存在成功匹配。因此，每个过滤算法总是要和一个非过滤法结合起来使用，从而可以判断剩余区域内是否存在可能的成功匹配。

过滤算法只能提高算法在平均情况下的效率，它的优势在于可能不需要检查文本中的个字符。过滤算法的效率和它能过滤掉的文本大小有关，并且对错误水平非常敏感。大数过滤算法在错误水平较低时工作得很好，否则就很差。因此在评价一个过滤算法时，仅要考虑它的时间效率，还要考虑它的容错能力。

已有的过滤算法很多,本节中将介绍两个在一般情况下效果最好的算法:第一个是 PEX 法，当字母表不是太小（例如英文文本）时，它的效果是最好的；第二个是 ABNDM 算，对于很小的字母表上的文本（例如 DNA 序列），它的效果是最好的。

.5.1 *k*+1 分片算法

PEX 算法的核心思想是:如果把模式串分成 $k+1$ 片,那么对于任意一个近似成功匹配，少有一片在其中是精确匹配的。这显然是正确的，因为至少在编辑距离模型下， k 个错是是不可能分到 $k+1$ 片中去的。下面给出一个更一般的引理，对于证明这个结论很有用Mye94, BYN99]。

引理 1 *如果字符串 Occ 和模式串 p 的近似匹配中包含 k 个错误，其中 p 是由 j 个子模式串连接而成，即 $p = p^1 \ldots p^j$，则对于任意的非负整数 $a_1 \ldots a_j$，存在 $i \in 1 \ldots j$，使得 p^i 能与 Occ 中的某个子串近似匹配，并且最多只有 $\lfloor a_i k/A \rfloor$ 个错误，其中 $A = \sum_{i=1}^{j} a_i$。*

使用反证法很容易证明此引理：假设每个子模式串 p^i 的近似匹配中的错误个数大于或等于 $1+\lfloor a_i k/A \rfloor$，则整个模式串 p 近似匹配的错误个数将会大于或等于 $k+1$，这与题设中最多包含 k 个错误矛盾，得证。如果各个 a_i 取值均为 1，取 $A = j = k+1$，则得到此引理的一个简单情形。

[WM92b, BYN99, NBY99]的做法是：把模式串分成长度大致相等的 $k+1$ 片，使用精确的多字符串匹配算法在文本中搜索这些片，然后检查文本中那些成功匹配片之间的相邻关系。必须注意的是：文本中那些成功匹配片之间的顺序要与模式串的片的顺序一致，并

防止在使用多字符串匹配算法进行搜索时，一个成功匹配片被报告多次。

文本中相邻的成功匹配模式片必须足够大以容纳一个模式串。在编辑距离下，文本中的一个成功匹配最长不会超过 $m+k$。假设模式片 $p_{i_1 \ldots i_2}$ 在文本中的 $t_{j \ldots j+(i_2-i_1)}$ 位置被匹配，那么整个模式串在文本中的成功匹配的位置最多从 t_j 前面 i_1-1+k 个字符开始（因为在比对时，最多只能在模式串的起始位置插入 k 个字符），并最多终止于 $t_{j+(i_2-i_1)}$ 后面 $m-i2+k$ 个字符（因为在比对时，最多只能在模式串的结束位置插入 k 个字符）。这样，算法必须检查文本中从 $T_{j-(i_1-1)-k}$ 到 $T_{j+(m-i_1)+k}$ 这个区域，它的长度为 $m+2k$。注意：如果两个模式片恰好完全一样，那么必须将每个成功匹配的模式片分别和两个不同的区域进行校验。

为了构造一个具体的 PEX 算法，还需要做两个重要的选择：第一，要决定选择什么样的多字符串匹配算法（参见第 3 章）。[WM92b]使用的是 Multiple Shift-And 算法，而[BYN99, NBY99]则使用的是 Set Horspool 算法。第二，要决定选择什么样的验证算法。大部分算法设计者很少注意这个问题，一般仅仅使用最简单的动态规划算法。不过如果使用更快一些的算法，比如 BPM，那么就能将每次验证时间的复杂度从 $O(m^2)$ 降到 $O(m^2/w)$。

[BYN99]中表明：当错误水平 $\alpha < 1/(3\log_{|\Sigma|} m)$ 时，时间主要消耗在多字符串匹配阶段。当错误水平高于这个值时，时间主要消耗在验证区域是否匹配上，这时整个过滤算法的效率下降得非常快。当错误水平 α 低于 $1/(3\log_{|\Sigma|} m)$ 时，过滤算法的时间复杂度约为 $O(kn\log_{|\Sigma|}(m)/m)$。

层次验证

[NBY99]采用了所谓的“层次验证”来降低某些不必要的验证开销。层次验证的思想是：由于一次验证的开销是 $O(m^2)$，如果对每一个匹配的模式串的片都去验证整个模式串，则花费在验证上的时间开销是非常大的。而对于大部分验证，可以只使用一个较短的模式片就能判断目标不匹配，这将节省大量时间。

假设把模式串分为 $j=k+1=2^r$ 片，与原来的从左到右一次切成 $k+1$ 片不同，这里将把模式串一层一层地分解为 $k+1$ 片。首先，把模式串分成两半，按照引理 1，可以在文本中搜索这两个子模式串，使得匹配中最多不超过 $\lfloor k/2 \rfloor$ 个错误。然后，这两个子模式串又可以递归地分成两半，直到最大允许的错误数目降低到 0 为止。

图 6.9 给出了一幅示意图。其中，叶节点是实际要被搜索的模式串的片。如果在文本中成功匹配了其中的某一个叶节点，则只需要验证这个叶节点的父节点是否和成功匹配区域附近的一个较小区域匹配（例子中允许 1 个错误），而不是对整个模式串进行验证。然后使用同样的方法，在一定的模式片长度和允许的错误数目下逐级扩展文本中的成功匹配区

。如果父节点不能在给定错误数目下匹配，那么验证过程停止，继续进行下一步的多字串匹配；否则，如果从叶节点一直到根节点（即整个模式串）都能在给定错误数目下匹，那么就表明找到了一个错误数不大于 k 的成功匹配。

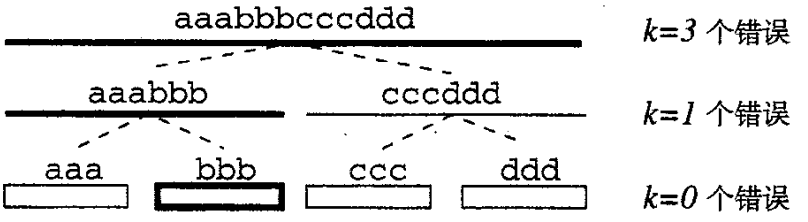


图 6.9 分层验证方法示例，把原始的模式串分成 4 个子模式串，最底层的方框表明实际被搜索的叶节点，而根节点表示完整的原始模式串。如果找到了一个完整的成功匹配，那么树的每一层至少有一个节点是在给定错误数目下存在成功匹配。图中用粗线标出了自下向上的验证过程

由引理 1 可知：对于层次树中的每一层，如果其父节点在它的错误数目下匹配成功，这一层必然存在一个子节点可以在它自己的错误数目下匹配成功。这就保证了层次验证法是正确的，它不会漏过任何可能的结果。对层次树中的任意一个节点，如果它所有的子节点都没有被成功匹配，则这个节点也不可能被成功匹配，即使在它的孙子节点中存成功匹配。

再回头看看图 6.9，如果在文本“xxxbbbxxxxxx”中搜索模式串“aaabbbcccddd”，最多许 3 个错误。当把模式串分成 4 片并且对每片进行错误数为 0 的搜索时，则模式片“bbb”会在文本中被搜索到。当使用原始的验证方法时，需要在文本的“bbb”邻域内验证是否在整个模式不超过 3 个错误的匹配结果；而在使用层次验证法时，如果可以验证“bbb”一个很小的邻域内不存在它的父节点“aaabbb”的不超过 1 个错误的匹配结果，那么就得出结论：在“bbb”邻域内不可能存在完整模式串的不超过 3 个错误的匹配结果。显然，者所花的时间要少得多。

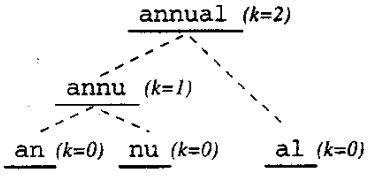
[NBY99]的分析结果表明：使用层次验证法，PEX 算法的适用范围将从 $\alpha < 1/\log_{|\Sigma|} m$)扩大到 $\alpha < 1/\log_{|\Sigma|} m$。

如果 $k+1$ 不是 2 的整数次幂，为了保证验证效率，二叉树必须要尽量保持平衡。例如，果 $k+1=3$，这时把模式串分成长度约为 $\lfloor m/3 \rfloor$ 的 3 片，即二叉搜索树将有 3 个叶节。在此二叉树中，根节点的左子节点的长度将是 $2\lfloor m/3 \rfloor$，匹配时允许的错误数目为 $2k/3\rfloor = 1$。根节点的右子节点将是个叶节点，它的长度是 $\lfloor m/3 \rfloor$，匹配时允许的错误数为 $\lfloor k/3 \rfloor = 0$。允许存在 1 个错误的左子节点然后又将分成两个叶节点。图 6.10 给出了造这样一棵二叉树的伪码，还给出了模式串为“annual”，$k=2$ 时的二叉树。而完整的EX 算法的伪码在图 6.11 中给出。

CreateTree ($p = p_i p_{i+1} \dots p_j$, k, $myParent$, idx, $plen$)

1. Create new $node$
2. $from(node) \leftarrow i$
3. $to(node) \leftarrow j$
4. $left \leftarrow \lceil (k+1)/2 \rceil$
5. $parent(node) \leftarrow myParent$
6. $err(node) \leftarrow k$
7. **If** $k = 0$ **Then** $leaf_{idx} \leftarrow node$
8. **Else**
9. 　　**CreateTree**($p_{i \dots i+left \cdot plen-1}$, $\lfloor (left \cdot k)/(k+1) \rfloor$, $node$, idx, $plen$)
10. 　　**CreateTree**($p_{i+left \cdot plen \dots j}$, $\lfloor ((k+1-left) \cdot k)/(k+1) \rfloor$, $node$, $idx + left$, $plen$)
11. **End of if**

(a) 构造层次验证二叉树的递归算法



(b) PEX 算法示例

图 6.10　构造层次验证二叉树的递归算法。图中函数 *CreateTree*( )在模式片 $p_{i \dots j}$ 上构造允许错误数为 k 的层次验证二叉树。其余的参数：*myParent* 为待构造节点的父节点；*idx* 为下一个要分配的叶节点的编号；*plen* 为模式片的长度。底部给出了字符串为“annual”，k=2 时的二叉树示例

PEX ($p = p_1 p_2 \dots p_m$, $T = t_1 t_2 \dots t_n$, k)

1. 　**Preprocessing**
2. 　　**CreateTree**(p, k, θ, 0, $\lfloor m/(k+1) \rfloor$)
3. 　　Preprocess multipattern search for
4. 　　　$\{p_{from(node)} \cdots p_{to(node)},\ node = leaf_i,\ i \in \{0 \dots k\}\}$
5. 　**Searching**
6. 　　**For** $(pos, i) \in$ output of multipattern search **Do**
7. 　　　$node \leftarrow leaf_i$
8. 　　　$in \leftarrow from(node)$
9. 　　　$node \leftarrow parent(node)$
10. 　　　$cand \leftarrow$ TRUE
11. 　　　**While** $cand$ = TRUE AND $node \neq \theta$ **Do**
12. 　　　　$p_1 \leftarrow pos - (in - from(node)) - err(node)$
13. 　　　　$p_2 \leftarrow pos + (to(node) - in + 1) + err(node)$
14. 　　　　Verify text area $T_{p_1 \dots p_2}$ for pattern piece $p_{from(node) \dots to(node)}$ allowing $err(node)$ errors
15. 　　　　**If** pattern piece was not found **Then** $cand \leftarrow$ FALSE
16. 　　　　**Else** $node \leftarrow parent(node)$
17. 　　　**End of while**
18. 　　　**If** $cand$ = TRUE **Then**
19. 　　　　Report the positions where the whole p was found
20. 　　　**End of if**
21. 　　**End of for**

图 6.11　PEX 算法，这里假定多模式匹配返回结果形式为“（文本中的位置，匹配的模式片）”

PEX 算法示例 在文本“any_annealing”中搜索模式串“annual”，最多允许 $k = 2$ 个误。模式串“annual”对应的划分如图 6.10 所示。从这里可以看出，文本中同一个匹配果可能被发现多次。

1. 发现 [an] y_annealing	
以 $k=1$ 的错误水平在文本	以 $k=2$ 的错误水平在文本
[any_a] nnealing 中搜索“annu”	an [y_annealin] g 中搜索“annual”，
没有匹配，因此停止验证过程。	发现匹配，报告位置 9，10 和 11。
2. 发现 any_ [an] nealing	3. 发现 any_anne [al] ing
以 $k=1$ 的错误水平在文本	以 $k=2$ 的错误水平在文本
any [_annea] ling 中搜索“annu”，	an [y_annealin] g 中搜索“annual”，
匹配成功，因此跳到树的上一层。	匹配成功，报告位置 9，10 和 11。

5.2 近似 BNDM 算法

通过第 2 章和第 3 章的讨论可知：在字母表较小的情况下，BDM/BNDM 算法要比 Boyer-oore 算法好。[NR00]中提出的一种 BNDM 扩展算法在 DNA 序列上要比 PEX 好，我们把个算法称为 ABNDM。

ABNDM 算法所做的修改是：构造一个 NFA，可以对模式串的反转进行允许错误的搜。这个 NFA 可以搜索模式串的任意后缀，然后在这个自动机上运行 BNDM 算法（参 2.4.2 节）。图 6.12 给出了一个这样的自动机的例子。

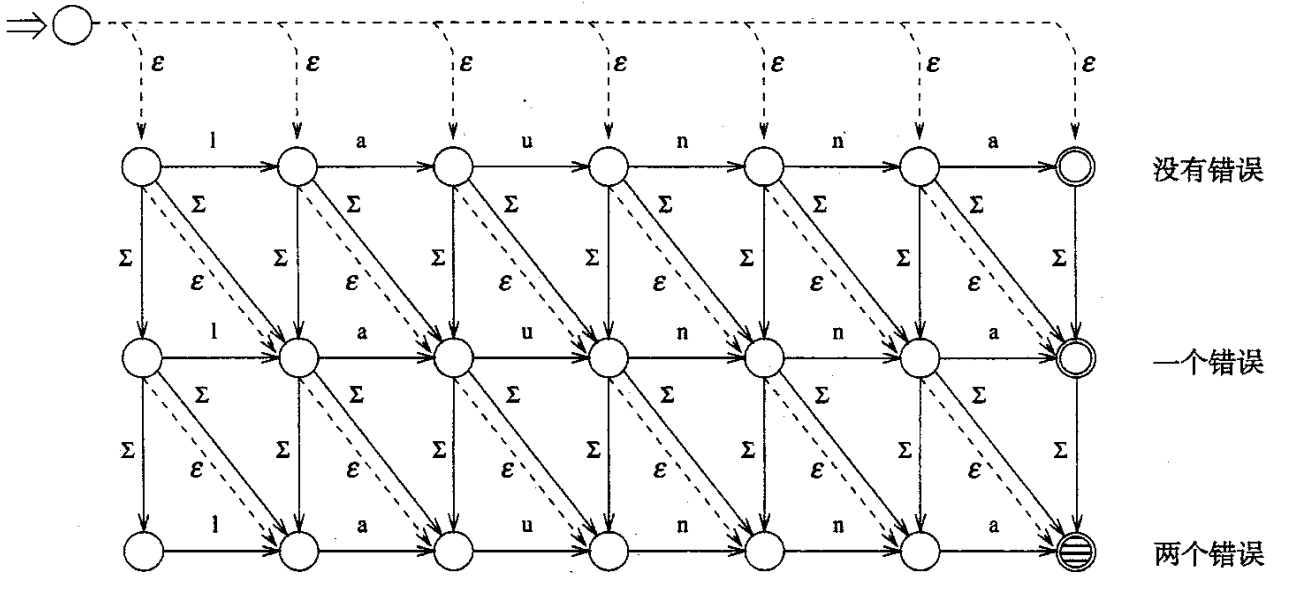


图 6.12 搜索“annual”的前缀的反转的 NFA，最多允许两个错误。图中显示了在读入文本窗口“any_”之后的活动状态

这个自动机可以识别模式串 p 的任意前缀的反转，且最多允许 k 个错误。在最多允许 k 个错误的情况下，如果读入的字符中不包含任何模式串因子，那么就可以跳过此文本窗口。这时，窗口滑动到下一个已发现的最多 k 个错误的模式串前缀处（*last* 位置）。

通过上述自动机找到的成功匹配一定是在文本窗口的起始位置，因此报告匹配结果的起始位置要比报告它的终止位置容易。当然，只要加一些额外处理，也可以不重复地报告匹配结果的终止位置。

文本窗口的长度是 $m-k$，而不是 m，这样才能保证如果在窗口内的起始位置有一个成功匹配，则在窗口的任意后缀内，必存在模式串的一个因子。

然而，到达窗口的起始位置并不保证找到一个成功匹配。因为匹配结果可能是变长的，到达窗口的起始位置，只是意味着找到了模式串的一个因子（最多允许 k 个错误）。特别是，如果没有找到模式串的某个前缀（最多允许 k 个错误），那么就不可能在窗口的起始位置找到一个成功匹配。相反，如果找到了某个模式串前缀，则必须从窗口的起始位置开始，依次计算接下来的区域的编辑距离，以确认是否找到一个匹配结果，这最多需要读入 $m+k$ 个文本字符。

上述验证过程可以使用 6.2.1 节中的算法来计算编辑距离。另一个选择是使用 BPR 算法，但需要删除图 6.4 中的初始状态的自环。除了 R_0，其余的更新公式都是一样的，R_0 的更新公式为：

$$R_0' \leftarrow (R_0 << 1)\ \&\ B[t_j]$$

改写其他的位并行算法要困难得多。

与图 6.4 中的原始 NFA 一样，有多种位并行方法可以模拟图 6.12 中的自动机。算法对于较小的 k 值效率很好，此时使用行式位并行是个好选择。特别地，针对不同的常量 k，也可以分别用特定的代码来实现算法。唯一的改变是，在初始化自动机时，必须将所有状态激活，同时还要删除自动机中的自环。

其他的一些方法，例如 BPD 和 BPM，使用起来更为复杂。使用 BPD 进行验证比使用它进行搜索需要更多的位。因为整个自动机都需要用位掩码来表示，而不仅仅是其中的完整对角线，原则上这需要 $(m+k+2)(k+2)$ 位。BPM 设计的初衷就不是用来判断自动机是否具有活动状态（但最近有这方面的工作[HN01]）。在这里，BPR 的灵活性就体现出来了。

图 6.13 给出了算法的伪代码。在读入文本窗口的第一个字符后初始化 NFA。

对于较小的字母表和比较短的模式串，ABNDM 算法工作得相当好。因为使用了位并

，所以需要满足 $m \leqslant w$。当模式串比较长时，可以使用多个机器字来模拟。但随着模式长度的增加，算法的效率急剧下降，仅仅更新包含活动状态的机器字的技巧在这里效果不佳。原因是，一方面，在初始化 NFA 时，需要将所有状态都激活，活动状态趋向均匀布于整个模式串上；另一方面，如果像 BDM 一样对自动机进行确定化，则会使得状态目以指数方式增长，就像 6.3 节中所述的 DFA 构造一样。

ABNDM $(p = p_1p_2 \ldots p_m,\ T = t_1t_2 \ldots t_n,\ k)$

1. **Preprocessing**
2. **For** $c \in \Sigma$ **Do** $B[c] \leftarrow 0^m$
3. **For** $j \in 1 \ldots m$ **Do** $B[p_j] \leftarrow B[p_j] \mid 0^{j-1}10^{m-j}$
4. **Searching**
5. $pos \leftarrow 0$
6. **While** $pos \leq n - (m - k)$ **Do**
7. $j \leftarrow m - k - 1,\ last \leftarrow m - k - 1$
8. $R_0 \leftarrow B[t_{pos+m-k}]$
9. $newR \leftarrow 1^m$
10. **For** $i \in 1 \ldots k$ **Do** $R_i \leftarrow newR$
11. **While** $newR \neq 0^m$ AND $j \neq 0$ **Do**
12. $oldR \leftarrow R_0$
13. $newR \leftarrow (oldR << 1)\ \&\ B[t_{pos+j}]$
14. $R_0 \leftarrow newR$
15. **For** $i \in 1 \ldots k$ **Do**
16. $newR \leftarrow ((R_i << 1)\ \&\ B[t_{pos+j}])$
$\mid oldR \mid ((oldR \mid newR) << 1)$
17. $oldR \leftarrow R_i,\ R_i \leftarrow newR$
18. **End of for**
19. $j \leftarrow j - 1$
20. **If** $newR\ \&\ 10^{m-1} \neq 0^m$ **Then** /* prefix recognized */
21. **If** $j > 0$ **Then** $last \leftarrow j$
22. **Else** check a possible occurrence starting at $pos + 1$
23. **End of if**
24. **End of while**
25. $pos \leftarrow pos + last$
26. **End of while**

图 6.13 对 BNDM 进行扩展以支持近似搜索，这里假定 $m - k > 1$

ABNDM 示例 在文本“any_annealing”中搜索模式串“annual”，最多允许 $k = 1$ 个误。在这里，因为 $k = 2$ 的错误水平太高以至于不好描述，所以将最多允许的错误数减到 $k = 1$。

$$B=\begin{cases}\texttt{a} & 100010\\ \texttt{l} & 000001\\ \texttt{n} & 011000\\ \texttt{u} & 000100\\ * & 000000\end{cases}$$

1. any_a nnealing

读入 a	100010
$R_0=$	100010
$R_1=$	111111

$last \leftarrow 4$.

读入_	000000
$R_0=$	000000
$R_1=$	100110

R_1的最后一位被置位，因此 $last \leftarrow 3$。

读入 y	000000
$R_0=$	000000
$R_1=$	000000

自动机中不再有活动状态，因此使用 $last=3$来移动窗口。

2. any _anne aling

读入 e	000000
$R_0=$	000000
$R_1=$	111111

$last \leftarrow 4$.

读入 n	011000
$R_0=$	000000
$R_1=$	011000

读入 n	011000
$R_0=$	000000
$R_1=$	010000

读入 a	100010
$R_0=$	000000
$R_1=$	100000

R_1的最后一位被置位，因此 $last \leftarrow 1$。

读入_	000000
$R_0=$	000000
$R_1=$	000000

自动机中不再有活动状态，因此使用 $last=1$来移动窗口

3. any_ annea ling

读入 a	100010
$R_0=$	100010
$R_1=$	111111

$last \leftarrow 4$.

读入 e	000000
$R_0=$	000000
$R_1=$	100110

R_1的最后一位被置位，因此 $last \leftarrow 3$。

读入 n	011000
$R_0=$	000000
$R_1=$	001000

读入 n	011000
$R_0=$	000000
$R_1=$	010000

读入 a	100010
$R_0=$	000000
$R_1=$	100000

R_1的最后一位被置位，并且 $j=0$，因此，计算模式串和文本前缀“anneali”之间的编辑距离。此时，发现匹配了前缀“anneal”($k \leqslant 1$)，在文本位置 5 处报告一个成功匹配。

4. any_ann ealin g

读入 n	011000
$R_0=$	011000
$R_1=$	111111

$last \leftarrow 4$.

读入 i	000000
$R_0=$	000000
$R_1=$	111000

R_1最后一位被置位，因此 $last \leftarrow 3$。

读入 l	000000
$R_0=$	000000
$R_1=$	000000

自动机中不再有活动状态，因此使用 $last=3$来移动窗口。

此时窗口移出了文本，算法终止。

5.3 其他过滤算法

除了上面两节中介绍的算法之外，还存在许多其他的过滤算法。特别是前面没有提及一些算法在某些$(m,k,|\Sigma|)$组合下会比 PEX 和 ABNDM 还要快[TU93, JTU96, BYN99, L94, ST95]。但是与这些算法相比较，PEX 算法及 ABNDM 算法的效率相差不是太大，而们编程的复杂度却要小得多。

此外，也存在一些平均时间复杂度能达到最优的算法。[CM94]中证明了近似搜索的期时间代价的下界为$O((k+\log_{|\Sigma|} m)n/m)$。同一篇文章中也提出了一个过滤算法，其平时间复杂度达到了这个下界。然而，这个复杂度只有在$\alpha < 1-e/\sqrt{|\Sigma|}$时才是有效的。个极限看起来不太可能得到改善[BYN99]，因为当错误水平达到这个极限时，文本中将太多的成功匹配。虽然[CM94]中的算法在理论上是最优的，但实际应用中效果并不那么。是否存在一个算法，既具有最优的时间复杂度，又比较实用，还是个有待研究的问题。

大多数过滤算法在$O(1/\log_{|\Sigma|} m)$时都能达到$O(k\log_{|\Sigma|}(m)n/m)$的时间复杂度。其中关键思想是，为了搜索整个模式串，首先要将模式串分解为多个子串，然后搜索所有子。搜索子串时允许比较小的错误数，而不是不允许任何错误。[CM94]中的算法正是这么的。另外一些能达到极限$\alpha < 1-e/\sqrt{|\Sigma|}$的过滤算法也采用了这种思想[BYN99]。这种术并不能跳过文本中的字符，然而在中等错误水平的应用中还是不错的选择。

6 多模式串近似搜索

在前面讨论了几种单模式串近似匹配算法以后，很自然会想到多模式串的近似匹配算。能够处理多模式串近似匹配的算法不是很多，而且这些算法几乎都是过滤算法，当错水平高到某个程度时，这些算法的效率下降很快。

6.1 仅允许一个错误的散列算法

当$k=1$时，[MM96]提出了一个很好的多模式串近似匹配算法，称为 MultiHash 算法。ultiHash 算法的基本思想是：如果p和p'在允许一个错误的前提下匹配成功，则一定$m-1$个字符完全匹配。从模式串p中任意一个位置删除一个字符，可以得到一个新，这个新串称为签名。这样，从p中可以派生出m个签名$\{p_2p_3\cdots p_m,\ p_1p_3\cdots p_m,\ p_1p_2p_4\cdots p_m,\ \cdots,\ p_1p_2\cdots p_{m-1}\}$。这里定义长度为$m$的串$x$的第$j$个签名为：

$$S_{x,j} = x_1 x_2 \ldots x_{j-1} x_{j+1} \ldots x_m$$

例如，模式串 p = “annual” 的签名有：$S_{p,1}$ = “nnual”，$S_{p,2} = S_{p,3}$ = “anual”，$S_{p,4}$ = “annal”，$S_{p,5}$ = “annul”，$S_{p,6}$ = “annua”。

如果在文本中搜索 r 个长度相同的模式串（长度为 m），每个模式可产生 m 个签名，则共有 rm 个签名。在模式长度不相同的情况下，要将所有的模式串按照最小长度截断，使它们具有相同的长度。

上述 rm 个签名存储在一个散列表里，在后面将使用这个散列表进行精确匹配。在文本中搜索时，将考虑所有长度为 m 的窗口$(t_i t_{i+1} \ldots t_{i+m-1})$。

对于每一个这样的窗口，首先提取它的签名：$t_{i+2} t_{i+3} \ldots t_{i+m-1}, t_{i+1} t_{i+3} \ldots t_{i+m-1}$，$\ldots$，$t_{i+1} t_{i+2} \ldots t_{i+m-2}$，这里把记号 $S(S_{t_i \ldots t_{i+m-1},j})$简写为 $S_{i,j}$。然后在上述散列表中搜索这些签名，如果能至少搜索到一个，那么就表明找到了一个成功匹配。

下面证明这种方法是正确的。如果模式串 p和文本中的子串 p' 只要替换一个字符就能精确匹配，即 $p_1 p_2 \ldots p_{j-1} a p_{j+1} \ldots p_m = p'_1 p'_2 \ldots p'_{j-1} b p'_{j+1} \ldots p'_m$，那么 p' 将会被上面的搜索过程发现，因为 p和窗口 p' 的第 j 个签名是一样的。如果在 p' 中插入一个字符就能使二者精确匹配，即 $p_1 p_2 \ldots p_{j-1} p_j p_{j+1} \ldots p_m = p'_1 p'_2 \ldots p'_{j-1} p'_j b p'_{j+1} \ldots p'_{m-1}$，由于窗口的长度为 m，因而存在一个窗口 $x = p'_1 p'_2 \ldots p'_{j-1} p'_j p'_{j+1} \ldots p'_{m-1} c$，使得 x 的签名 $S_{x,m}$ 和 p的签名 $S_{p,j+1}$是相同的，因此在这种情况下算法能正确找到匹配结果。最后一种情况，如果从 p' 中删除一个字符能使得二者精确匹配，即 $p_1 p_2 \ldots p_{j-1} p_j p_{j+1} \ldots p_m = p'_1 p'_2 \ldots p'_{j-1} p'_{j+1} \ldots p'_m p'_{m+1}$，则能找到窗口 $x = p'_1 p'_2 \ldots p'_{j-1} p'_j p'_{j+1} \ldots p'_m$，使得 $S_{x,j} = S_{p,m}$，因此在这种情况下，算法也能正确找到匹配结果。

在本算法中的散列函数对于整个算法的性能是非常重要的，[KR87]使用如下形式的散列函数：

$$h(x_1 \ldots x_{m-1}) = \sum_{i=1}^{m-1} x_i d^{i-1} \mod s$$

其中：整数d和s互质，s为散列表的大小，这个散列函数能够将所有的签名很均匀地分布到散列表 $H[0 \ldots s-1]$中。更妙的是，它能够利用前一个窗口的结果，在 $O(1)$ 的时间内计算出当前窗口的所有签名的散列函数值。比如：当前窗口是 $t_{i+1} t_{i+2} \ldots t_{i+m}$，它的第 j 个签名是 $S_{i+1,j} = t_{i+1} t_{i+2} \ldots t_{i+j-1} t_{i+j+1} \ldots t_{i+m}$，而前一个窗口的第 $j+1$ 个签名是 $S_{i,j+1} = t_i t_{i+1} \ldots t_{i+j-1} t_{i+j+1} \ldots t_{i+m-1}$，易得 $S_{i,j+1} t_{i+m} = t_i S_{i+1,j}$，进一步由上面的散列函数表达式得 $h(S_{i+1,j}) = ((h(S_{i,j+1}) - t_i)/d + t_{i+m} d^{m-2}) \mod s$，它的值能在常数时

]内计算出来。

由于散列函数值能在常数时间内计算，而每个窗口的 m 个签名都需要进行搜索，因此:搜索算法的时间复杂度为 $O(mn)$，与模式串的个数无关。当然，这里并没有考虑散列函:的冲突问题。一般来说，如果散列表的大小能够与待插入的签名个数成比例增长，则本:法的搜索时间可以保持不变。总的来说，本算法的时间复杂度为 $O(mn)$，空间复杂度为 $O(rm)$。

在实际应用中，即使模式串个数达到千、万数量级，本算法的效率仍然是不错的，这:其他算法所不能比的。但是另一方面，如果要把它扩展到允许多于一个错误，则代价非:昂贵：如果允许 k 个错误，为了得到新的签名，算法将不得不从每个模式串和窗口中任:删除 k 个字符，这样一个模式串或者窗口将产生 $O(m^k)$ 个签名，则算法的时间复杂度为 $O(m^k n)$，空间复杂度为 $O(m^k r)$。

图 6.14 给出了本算法的伪码，在执行过程中的任意时刻，都有 $h_j = h(S_{pos,j})$。如果考:到等式 $S_{x,j+1} = (S_{x,j} + (x_j - x_{j+1})d^j) \mod s$，则会节省部分预处理的时间和计算初:散列函数值的时间。算法总的消耗时间等于 $O(rm + mn)$，还需要加上对散列冲突的处:时间。

```
MultiHash (P = {p^1, p^2, ..., p^r}, T = t_1 t_2 ... t_n, k = 1)
1.    Preprocessing
2.        H ← empty hash table
3.        For i ∈ 1...r Do
4.            For j ∈ 1...m Do insert (S_{p^i,j}, i) in H[h(S_{p^i,j})]
5.    Searching
6.        For j ∈ 1...m − 1 Do h_j ← h(S_{1,j})
7.        For pos ∈ 1...n − m + 1 Do
8.            oldh_1 ← h_1
9.            For j ∈ 1...m Do
10.               For (x, i) ∈ H[h_j] Do
11.                   If x = S_{pos,j} Then report pattern p^i at pos
12.               End of for
13.               h_j ← ((h_{j+1} − t_{pos})/d + t_{pos+m} d^{m−2}) mod s
14.           End of for
15.           h_m ← oldh_1
16.       End of for
```

图 6.14　仅允许一个错误的散列算法

图 6.14 仅允许一个错误的基于散列的多模式串近似匹配算法。其中 H 为一个散列表，:的元素为（*key*, *value*）对的集合，对于给定的一个散列表元素，可以得到它包含的（*key*, *alue*）对集合。另外，这里假定 t_{n+1} 是可以读取的，但并不关心它的值。

6.6.2　多模式串的 k+1 分片算法

6.5.1 节介绍的 PEX 算法能够容易地扩展到多模式串的情况[BYN97]，扩展后的算法称为 MultiPEX 算法。给定 r 个模式串，把每个模式串都分成 $k+1$ 片，之后的步骤就与基本的 PEX 算法一样：对于文本中的任一位置，首先对 $r(k+1)$ 个模式片执行精确的多模式串匹配（参见第 3 章）。如果有模式片匹配成功，则在这个可能匹配的区域内验证这个模式片所属的模式串是否满足最多允许 k 个错误的匹配。如果这个匹配的模式片属于多个模式串，则需要对每个模式串都验证一遍。当然，这里也可以使用层次验证算法。

很多情况下这个算法都工作得不错，[BYN97]证明了只要是在 PEX 适用的场合，即满足 $\alpha < 1/\log_{|\Sigma|} m$的条件，则 MultiPEX 同样适用。MultiPEX 的代码只要在图 6.11 的基础上稍加修改即可。

6.6.3　重叠自动机算法

第三种算法称为 MultiBP 算法，基于图 6.4 中的 NFA。给定 r 个等长的模式串，搜索时最多允许 k 个错误，算法为每个模式串建立一个 NFA，然后对这些自动机进行叠加[BYN97]。叠加意味着自动机中的第 j 条水平箭头能通过 s 任意模式串的第 j 个字符进行转移。图 6.15 中给出了模式串为“annual”和“binary”的示例。

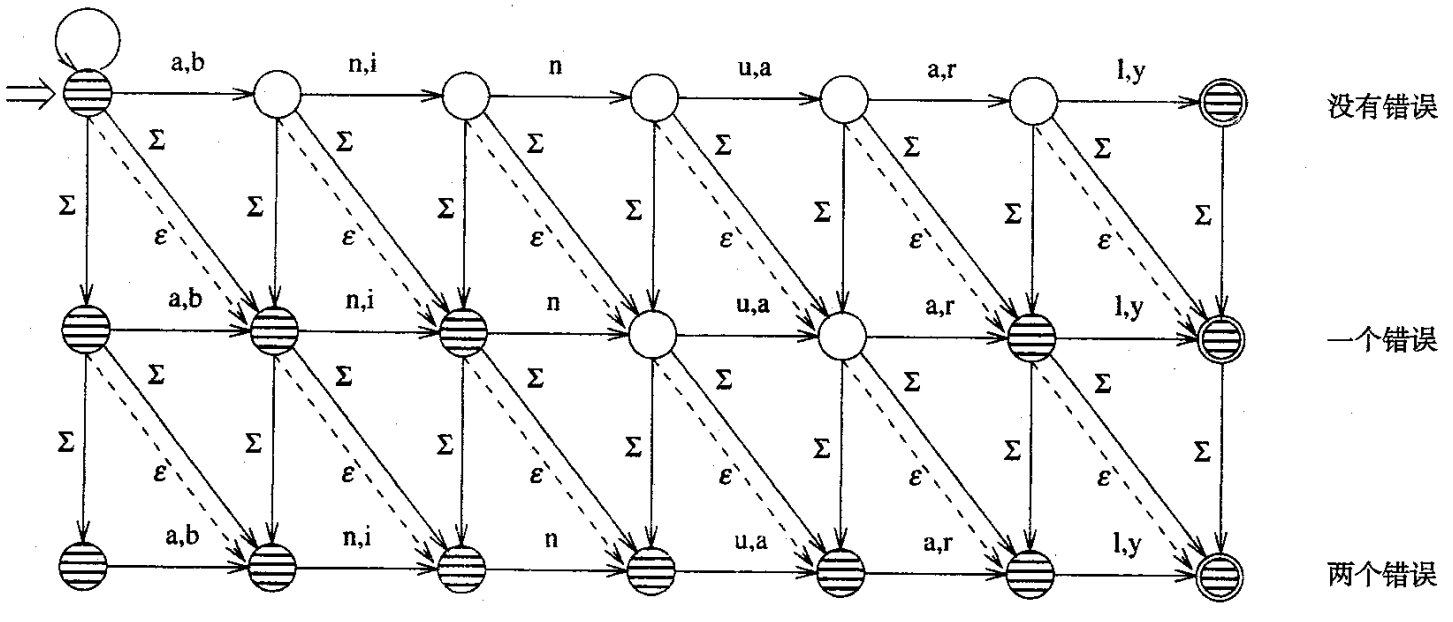


图 6.15　对模式串“annual”和“binary”进行近似搜索的 NFA，最多允许两个错误。图中的阴影状态是在读入文本“binual”后的活动状态

下面将着重介绍用位并行技术来模拟重叠自动机。设第 i 个模式串的位并行表为 $B_i[c]$，那么可以用下式生成一个新的表 B：

$$B[c] = B_1[c] \mid B_2[c] \mid \ldots \mid B_r[c]$$

然后，前面所述的单模式串上的所有算法，例如 BPR，BPD，BPM 和 ABNDM，都可以应用到这里。

由此得到的重叠自动机等价于搜索字符类形式的单模式串：将搜索 $\{p^1, p^2, \ldots, p^r\}$ 转换成搜索：

$$\{p_1^1, p_1^2, \ldots, p_1^r\}\ \{p_2^1, p_2^2, \ldots, p_2^r\}\ \ldots\ \{p_m^1, p_m^2, \ldots, p_m^r\}$$

因此，并不一定要使用 NFA，任何位并行算法都可以使用，特别是 BPM 算法。

当然，重叠自动机只能作为一个过滤器：如果要搜索“annual”和“binary”，则必会搜索到“binual”，并且只有零个错误。因此，每当重叠自动机报告一个匹配结果时，必须对所有模式串在匹配结果上进行验证。

这里，可以使用一种新的层次验证机制。如果已经把 $\{p^1, p^2, p^3, p^4\}$ 叠加起来，并利用它找到了一个可能的匹配结果，然后只需要分别验证 $\{p^1, p^2\}$ 和 $\{p^3, p^4\}$ 即可，而不是逐个验证四个模式串。在很多情况下，只需验证两个叠加的模式集，而不需要验证 r 个模式串。最后，假如叠加的模式集 $\{p^1, p^2\}$ 被验证匹配成功，那么还必须分别验证 p^1 和 p^2。

与 6.5.1 节所介绍 PEX 算法中自下向上的层次验证不同，这种层次验证机制是自上向下的。如果叠加模式集 $\{p^1, \ldots, p^r\}$ 找到了一个可能的匹配结果，则需要用两个叠加模式集 $\{p^1, \ldots, p^{\lfloor r/2 \rfloor}\}$ 和 $\{p^{1+\lfloor r/2 \rfloor}, \ldots, p^r\}$ 递归地对相关文本区域进行验证。所有的 $2r-1$ 个重叠自动机都是预先建立的。当递归验证过程结束时，要么没有找到任何成功匹配，要么找到若干个单独的模式串的成功匹配。

当叠加的模式串越来越多时，自动机就越容易通过某个水平箭头进行状态转移。实际上，在 r 个模式串叠加的自动机中，水平转移的概率约为 $r/|\Sigma|$，而单模式串自动机中水平转移的概率只有 $1/|\Sigma|$。因此，将太多模式串叠加在一起是不可取的。[BYN97]证明了模式串叠加的最佳个数是 $r' = |\Sigma|(1-\alpha)^2$。当模式串个数超过这个数目时，就必须将模式串分成若干组，每组中模式串的个数不超过 r'，搜索时分别搜索每个模式串组。

图 6.16 给出了算法的预处理伪代码，图 6.17 给出了搜索的伪代码。伪代码和怎样使用位并行模拟技术无关。当模式串能装入一个机器字时，推荐使用 BPD 算法，否则推荐使用 BPM 算法。这里假定所有的预处理信息都保存在一个对象 B 中，算法能将两个这样的对象

进行“连接”从而产生一个新的对象，新的对象反映了两个原始对象的叠加。例如；“连接”表B_1和B_2得到表B（过程 CreateSuperp 中的第 10 行），对于 BPR，ABNDM 和 BPM 来说，实际上就是：

$$\textbf{For } c \in \Sigma \textbf{ Do } B[c] \leftarrow B_1[c] \mid B_2[c]$$

对于 BPD 算法来说，“连接”反映的是表$B\,B$中的变化。

```
CreateSuperp (p^i, ..., p^j)
1.    Create new node
2.    If i = j Then
3.        B(node) ← preprocess single pattern p^i
4.        idx(node) ← i
5.        left(node) ← θ
6.        right(node) ← θ
7.    Else
8.        left(node) ← CreateSuperp(p^i ... p^⌊(i+j)/2⌋)
9.        right(node) ← CreateSuperp(p^(1+⌊(i+j)/2⌋) ... p^j)
10.       B(node) ← join B(left(node)) and B(right(node))
11.   End of if
12.   Return node
```

图 6.16　多模式串重叠搜索中层次验证的预处理

```
Verify (node, from, to)
1.    B ← B(node)
2.    For pos ∈ occurrences reported with B in T_from...to Do
3.        If left(node) = θ Then report p^idx(node) at pos
4.        Else
5.            Verify(left(node), pos − m − k + 1, pos)
6.            Verify(right(node), pos − m − k + 1, pos)
7.        End of if
8.    End of for

MultiBP (P = {p^1, p^2, ..., p^r}, T = t_1 t_2 ... t_n, k)
9.    Preprocessing
10.       tree ← CreateSuperp(p^1 ... p^r)
11.   Searching
12.       Verify(tree, 1, n)
```

图 6.17　允许错误的多模式串重叠搜索

6.7 扩展字符串和正则表达式的近似搜索

在有些应用中需要在允许错误的情况下搜索更为复杂的模式串。现在，有三类算法可以解决这个问题：第一类是将经典的动态规划算法从简单模式串扩展到正则表达式；第二类是基于 Four-Russians 的方法；第三类使用位并行技术。下面将分别介绍这三类算法，而重点放在位并行算法上，因为位并行算法的实现相对简单，并且在大部分情况下效果较好。

因为字符类的模式串的搜索比较容易用其他方法来实现，因此本节重点关注那些更复杂的模式串扩展，例如空位、可选字符、可重复字符和正则表达式等。

6.7.1 基于动态规划的方法

基于动态规划的方法是解决扩展串近似匹配和正则表达式近似匹配[MM89]的最古老方法，并且也是一种漂亮而复杂的方法。为了理解它，这里需要先回顾一下 6.2 节中的基本动态规划算法。

考虑图 6.18 中的有向带权图，每个节点对应于图 6.1 中动态规划矩阵的一个元素。图中节点之间的水平箭头表示在模式串中插入一个字符，垂直箭头表示在模式串中删除一个字符，对角箭头表示匹配/替换一个字符。水平和垂直箭头的代价都是 1，对角箭头的代价可能是 0 或者 1，取决于对应的字符相同（匹配）还是不同（替换）。

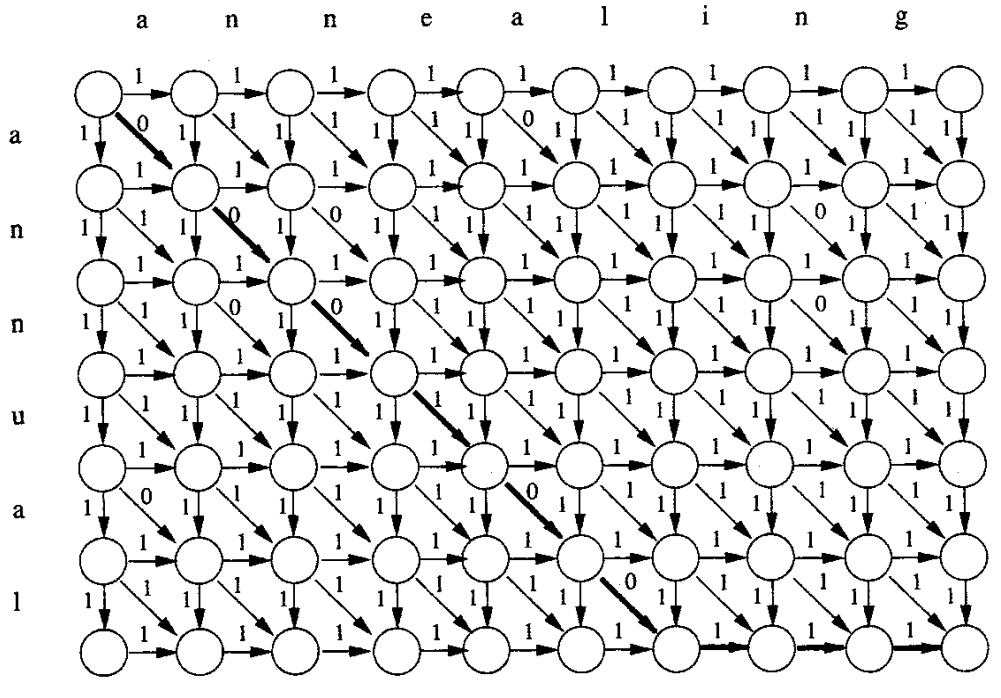


图 6.18 将计算编辑距离的问题转化为求最短路径问题。其中粗线箭头表示出了最优路径，最优路径代价为 4

这样，计算编辑距离的问题就转化为计算从左上角的节点到右下角的节点之间的最短路径问题。如果不仅仅是计算编辑距离，而是进行近似搜索，则需要将第一行的水平箭头的代价都置为 0，然后计算从第一行中的节点到最末一行的每个节点间的最小距离。

因为加权图是无环的，所以最优路径能在 $O(mn)$ 的时间内计算出来。这只是从另一个角度来阐述经典的动态规划算法，但这个角度的阐述更灵活、更容易被扩展到更为复杂的模式串。在这种最简单的情况下，模式串可用图中的一列垂直节点表示。

图 6.19 是在文本串“baa”上近似搜索的示图，其中的每个节点“竖列”都被一个正则表达式“(a|b)a*”的 NFA（使用 Thompson 方法构造，参见 5.2.1 节）所取代。可以看出，这个例子中最后求得的距离是零（即，正则表达式精确匹配了文本），最优路径是通过一个 ε-转移获得的。

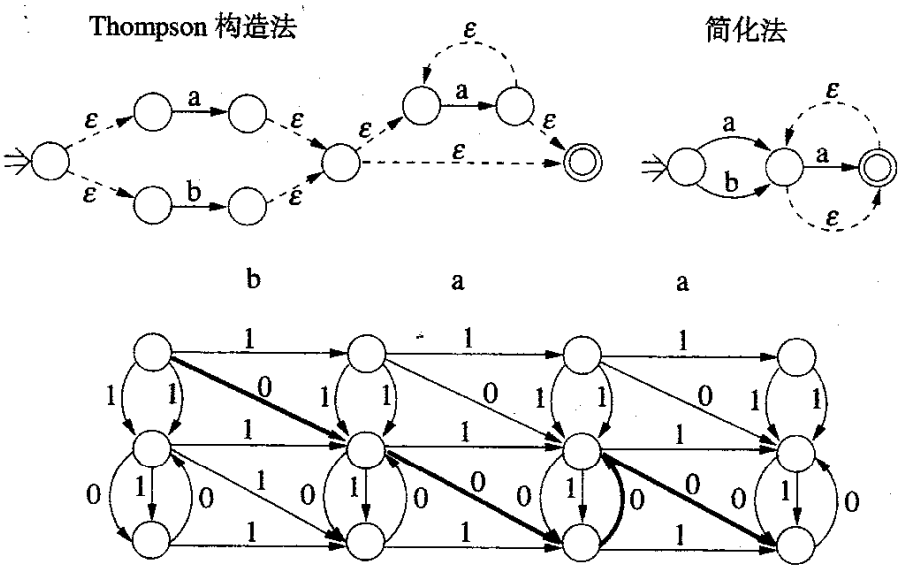


图 6.19　在文本串“baa”上搜索正则表达式“(a|b)a*”的有向带权图。粗线箭头表示最优路径，其代价为零

当有向图为无环图时，即正则表达式中不含有“*”和“+”两种运算符时，这种求最短路径的思想仍然比较容易使用。在无环的正则表达式上，能找到一个拓扑序来求解图的代价，因此总能在 $O(mn)$ 的时间内求解最短路径。但这需要有 Thompson 的保证，即在有 m 个节点的一个自动机上，最多有 $O(m)$ 条边。

然而 NFA 中的环会带来问题，因为此时找不到一种合适的序关系。当把环和删除（垂直）箭头结合在一起时，就会有这个问题，因为删除会沿着环传递并影响到状态转移的源状态。[MM89]的重要结论之一就是：这样从“*”和“+”运算符中产生的“回边”可以在第一趟扫描中忽略，然后在第二趟扫描中再考虑那些删除箭头，这同样能保证得到正确的匹配结果。要了解更详细的情况，请参考[MM89]。

.7.2 Four-Russians 方法

Four-Russians 方法已经在前面多次提到，例如 5.3.3 节中无错误的正则表达式搜索[Mye92]，以及 6.3 节中允许错误的简单串匹配[WMM96]。在 $O(s)$ 的存储空间条件下，这两个方法都能到达 $O(mn/\log s)$ 的最坏时间复杂度。如果使用 6.2.3 节中的技术，第二种方法能到达 $O(kn/\log s)$ 的平均时间复杂度。

这两种方法都是基于类似的思想：一个包含 $O(m)$ 个状态的 NFA 被划分为 r 个“区域”，每个区域包含 m/r 个状态。对于简单模式串来说，一个区域就是一个模式子串[WMM96]；对于正则表达式来说，一个区域则是 NFA 状态的一个子集[Mye92]。每个区域都使用 $O(m/r)$ 个二进位表示：对于简单模式的近似搜索，需要 $2m/r$ 位，因为每个矩阵元素和它前面的一个元素的差分值可能是–1, 0 或者+1，因此需要使用两个二进制位来表示它的值；对于正则表达式的精确匹配，每个状态只要使用一个二进位（活动的或者非活动的）表示即可。

这里，首先为每个区域预先建立一个确定的自动机，也就是一个保存所有可能的输出的表，每个区域需要 $2^{O(m/r)}$ 的存储空间。然后，用一个由区域组成的非确定自动机来模拟原始 NFA 中连接不同区域之间的箭头。这些箭头是逐个模拟的：对于简单模式串来说，将有三个箭头进入每个状态（参见图 6.5）；对于正则表达式来说，需要恰当地划分区域，Thompson 构造法保证了生成的自动机的边数为 $O(m)$，跨区域的边数为 $O(r)$。搜索时，因为必须逐个更新每个状态，所以搜索的时间复杂度为 $O(rn)$。如果有 $s = O(r)2^{O(m/r)}$ 的存储空间，那么搜索时间能降低到 $O(mn/\log s)$。

上述思想能推广到更一般的正则表达式近似匹配中[WMM95]。其基本思想和上述正则表达式精确匹配是一样的，但 NFA 的状态不仅仅是活动或非活动，而是存储了使状态激活所需的最少错误数。因为最多允许 k 个错误（这里使用值 $k+1$ 来表示所有大于 k 的值），所以在每个状态中需要保存一个取值为 $0 \ldots k+1$ 的整数，因此一个具有 m/r 个状态的确定自动机需要 $O((k+2)^{m/r})$ 的存储空间。也可以说，如果给定 $O(s)$ 的存储空间，则算法具有 $O(mn/\log_{k+2} s)$ 的搜索时间。

相对于精确搜索算法，近似搜索算法还要面临一个新的问题，即由正则表达式中的 ε-转移而产生的依赖关系。不过正如 6.7.1 节所述，一个两趟扫描的算法可以保证所有转移都能得到正确处理。

一项相关工作是[Mye96]，其中考虑了“间隔”，也就是在 PROSITE 表达式中所谓的“空位”。但不同的是：一个间隔的长度可以为负值，这意味着前一个匹配的正则表达式的一部

分可能和紧邻的下一个匹配的正则表达式相重叠。处理的思想是：首先搜索某一个正则表达式，然后用与它邻近的间隔来定义一些区域，在这些区域内搜索与其相邻的正则表达式；在搜索结果上进行扩展，直至找到整个模式串为止。[Mye96]还给出了一个最优搜索顺序，其中考虑了间隔的长度和正则表达式匹配的概率。

[Mye96]中也指出：如果将正则表达式限制为“网络表达式”，即不允许出现“*”和“+”运算符，那么可以按照到初始状态的距离从小到大来定义区域，然后使用类似于 6.2.3 节中的技术，可以获得平均时间复杂度为 $O(kn/\log_{k+2} s)$ 的算法。

长度大于 0 的空位可以转化为正则表达式来处理，但得到的 DFA 会非常大。

一般说来，Four-Russians 方法是处理大正则表达式的最好方法，但难以实现。下节中将给出一个更简单的方法，在合理大小的模式串上工作得很好。

6.7.3　位并行方法

[WM92b, Nav01b]直接对前面讨论的位并行算法进行扩展，以支持允许错误的扩展串匹配和正则表达式匹配。

如果允许通配符，则只有 BPR 算法和 BPD 算法能高效地处理，并且只有 BPR 足够灵活从而可以处理其他的所有扩展，下面主要介绍这个算法。

再回过去看看图 6.4，图中 NFA 的每一行实际都是搜索同一个模式串的非确定自动机的一个副本。这些副本通过如下规则连起来：“垂直”箭头将第 i 行中的状态和第 $i+1$ 行中的同一个状态连起来；虚线或者实线的“对角”箭头，将第 i 行中的状态 s 和第 $i+1$ 行中 s 能通过一次转移到达的状态（即 s 的“下一个”状态）连起来。

这个思想能推广到更复杂的自动机[WM92b]。特别是，如果我们将 NFA 中的每一行用第 4 章和第 5 章中的位并行自动机来代替，那么得到的 NFA 将有能力搜索相应的扩展串或正则表达式，搜索时最多允许 k 个错误。这样的自动机还可以使用 6.4.1 节中“向前”的方式搜索[WM92b]，也可以使用 6.5.2 节中“向后”的方式搜索[Nav01b]。本章中要介绍的算法的唯一变化是如何用位并行来模拟这个自动机，而基本思想是一致的。图 6.20 给出了一个搜索正则表达式的例子。

为了实现“对角”转移，需要构造一个表 T_d，对于每个状态集 D，表 T_d 给出 D 通过一步就能到达的所有状态的位掩码，这个表曾经在 5.4.2 节中构造过。而对于简单模式串来说，$T_d[D] = (D << 1)$。

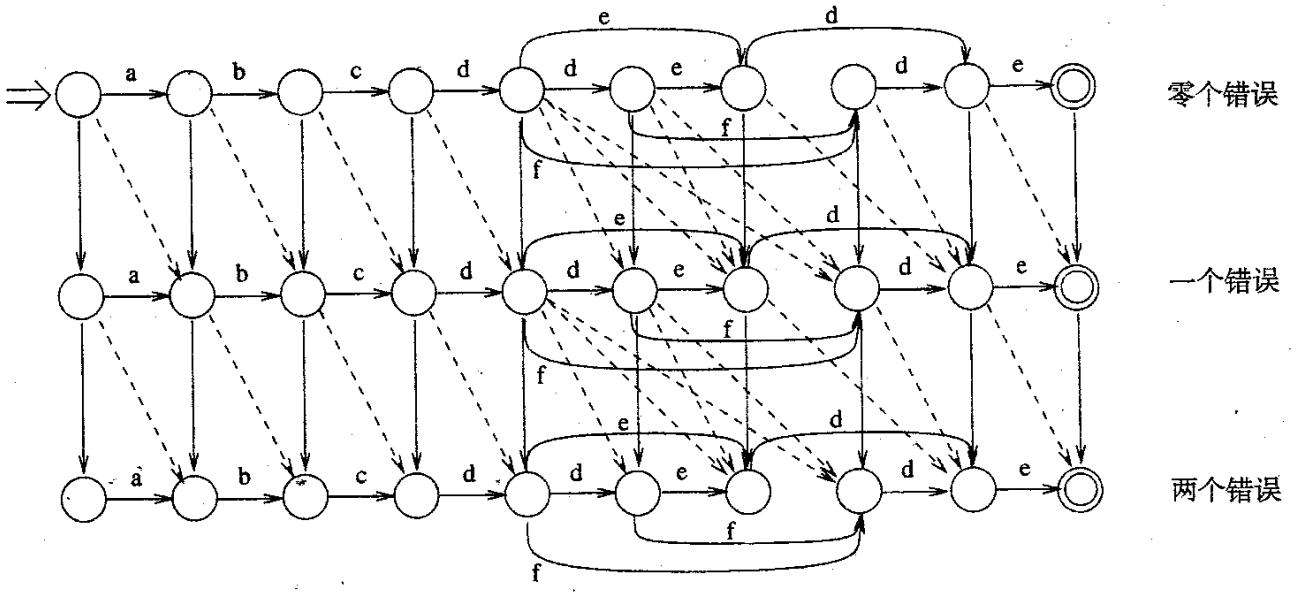


图 6.20 搜索正则表达式“abcd(d|ε)(e|f)de”的 Glushkov 的 NFA，允许最多两次插入、删除或者替换。为了简化起见，图中的虚线表示删除和替换（即，通过 $\Sigma \cup \{\varepsilon\}$ 进行状态转移），垂直线表示插入（即，通过 Σ 进行状态转移）

假定 1 表示活动，0 表示非活动。用 $f(c, D)$ 表示和模式串无关，不允许错误的无自环的更新函数，而 $f_0(c, D)$ 表示含自环的更新函数。

例如，对于简单模式串来说，f 对应于 Shift-And（参见 2.2.2 节），有：

$$\begin{aligned} f(c, D) &= (D << 1) \ \& \ B[c] \\ f_0(c, D) &= ((D << 1) \mid 0^{m-1}1) \ \& \ B[c] \end{aligned}$$

注意：上式满足 $T_d[D] = |_{c \in \Sigma} f(c, D)$。

在读入文本字符 t_{pos} 后，使用下式更新自动机的每一行：

$$\begin{aligned} &R'_0 \leftarrow f_0(t_{pos}, R_0) \\ &\textbf{For } i \in 1 \ldots k \textbf{ Do } R'_i \leftarrow f(t_{pos}, R_i) \mid R_{i-1} \mid T_d[R_{i-1} \mid R'_{i-1}] \end{aligned}$$

上面的公式可以插入到 BPR 算法和 ABNDM 算法中使用。

在少数受限情况下，如果将 BPR 算法或者 ABNDM 算法与 4.6 节中的多模式串匹配技术结合起来，进行允许错误的多扩展模式串搜索也是可能的。

6.8 实验图

本节中将给出效率最高的近似串匹配算法的性能分布图，这里只给出单模式串和多模

式串的近似匹配情况，而没有考虑扩展串的近似匹配和正则表达式的近似匹配。

对于近似串匹配，目前大约有 40 个算法，多数情况下，只有少数几个是最佳选择。为了避免在此推荐的算法过于繁多，这里忽略了那些只在少数情况下最优并且优势也不明显的算法。

图 6.21 给出了在英语文本和 DNA 文本两种情况下，各个算法在不同区域中的分布。由于每个过滤算法都需要一个非过滤算法来对可能的匹配结果进行验证，因此，在过滤算法占优势的区域，用灰色表示出了非过滤算法和重叠算法。这样，在灰色区域中，最好的选择是使用相应的过滤算法和占优势的非过滤算法作为验证引擎；而在非灰色区域中，直接使用占优势的非过滤算法更好。

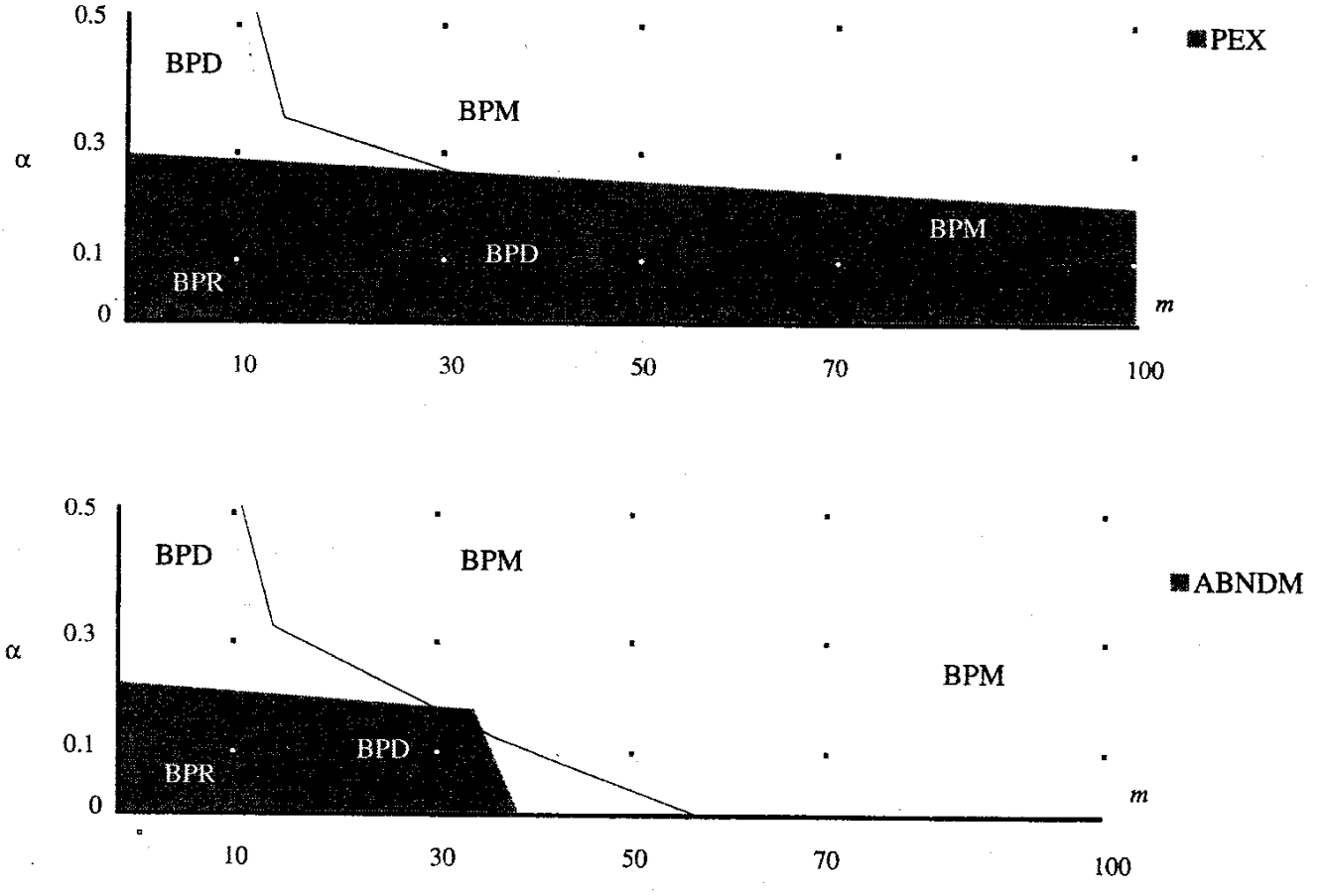


图 6.21 在不同区域中占优势的单模式串近似匹配算法。过滤算法占优势的区域被标为灰色。上图为英文文本，下图为 DNA 文本。图中对应的机器字大小为 $w = 32$ 位。对于 $w = 64$ 位来说，ABNDM 和 BPD 的区域将沿着 m 轴伸长

图 6.22 给出了多模式串搜索的情况。在英文文本上，当 $\alpha \leqslant 0.3$ 时，MultiPEX 算法的效果最好；当 $0.3 \leqslant \alpha \leqslant 0.4$ 时，MultiBP 算法的效果最好；而当错误水平更高时，没有一个算法比逐个顺序搜索 r 个模式串更好。当 $k = 1$ 并且模式串的个数很多时，MultiHash 算法更

子。对于更长的机器字来说，MultiBP 算法占优势的区域将沿着 m 轴向右伸长。

在 DNA 文本上，可选的算法只有很少几个：当 $k = 1$ 时，MultiHash 算法总是最好的选择；当 $k > 1$ 且错误水平较小时，MultiPEX 算法较好。总的来说，MultiBP 算法对 DNA 文本不太适用，因为 $|\Sigma| = 4$ 太小，使用重叠技术时，最少只需要4 个模式串就可以在每个文本位置都产生可能的匹配结果，从而导致几乎所有位置都需要验证。

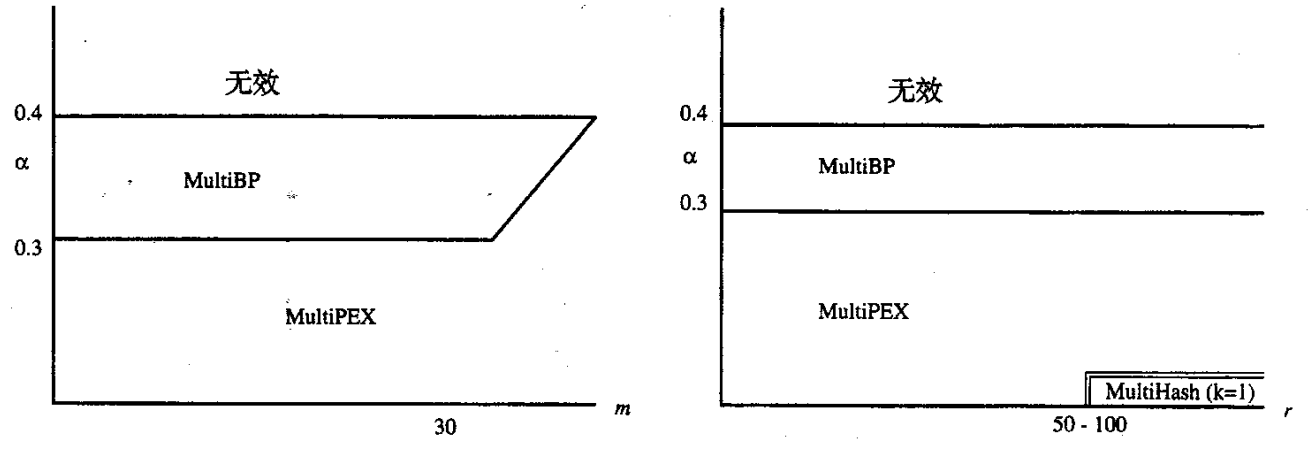


图 6.22 对于英文文本，随着不同的α，m和r值，各个多模式串近似匹配算法在不同的区域内占优势。左图中（m变化），假定r小于 50；右图中，假定模式串的长度小于w

6.9 其他算法和参考文献

如果要考虑更复杂的距离函数，那么动态规划方法将是最灵活的。例如，如果编辑操作具有不同的代价，在计算 $M_{i,j}$ 时，每次增加编辑操作相应的代价，而不是简单地加 1，即：

$$\begin{aligned} M_{0,0} &\leftarrow 0 \\ M_{i,j} &\leftarrow \min(M_{i-1,j-1} + \delta(x_i, y_j), M_{i-1,j} + \delta(x_i, \varepsilon), M_{i,j-1} + \delta(\varepsilon, y_j)) \end{aligned}$$

其中 $\delta(x, \varepsilon)$ 和 $\delta(\varepsilon, y)$ 是插入和删除字符的代价。

对于那些不允许某些编辑操作的距离模型，只要将相应项从求最小值的公式中去掉即可，或者将相应的代价函数赋值为无穷大。对于对换操作（即允许 $ab \to ba$ 在一个编辑操作内完成），可以引入第四条规则，即：如果 $x_{i-1}x_i = y_j y_{j-1}$，则 $M_{i,j}$ 可能等于 $M_{i-2,j-2} + 1$ [LW75]。

基于自动机的方法能处理各种编辑操作分别具有不同整数代价的情况。某些简化过的编辑距离模型对应到自动机上，只需要改变或者删除某些箭头。例如，如果插入一个字符

的代价是 2 而不是 1，那么可以将图 6.4 中的垂直箭头从第 i 行指向第 $i+2$ 行。对换操作更复杂些，但也可做类似处理[Mel96, Nav01b]。上述思想都可以引入到 BPR 算法和 ABNDM 算法中。BPD 算法也可以适应一些具有整数代价并且附加某些限制的情况。最近也有一些工作是关于如何扩展 BPM 算法以处理各种编辑操作具有不同代价的情况[BH01]，其中包括允许对换操作[Hyy01]，以及将 BPM 集成到 ABNDM 中[HN01]。

PEX 过滤算法应用到其他距离函数模型下也不困难。存在的主要问题是：要确定一个编辑操作可能破坏多少个片？在错误数超过阈值之前最多能进行多少次编辑操作？如此等等。例如，一个对换操作可能会破坏两个片，因此至少要把模式串划分为 $2k+1$ 片，以保证至少有一片保持不变。一个更好的解决办法是：在相邻的两片之间预留一个字符的位置，从而一个对换操作不会破坏这相邻的两块[Nav01a]。

如果读者想更深入地了解单模式串的近似搜索问题，请参看最近的一篇综述[Nav01a]。如果读者对在计算生物学应用中的距离模型和模式感兴趣，请参考[SK83, KM95]。

还有一些近似搜索模型，与前面所述的模型相差很远。举例来说，完全不一样的距离函数，比如 Hamming 距离（请参看短综述[Nav98]）、反转操作（允许反转子串）[KS95]、块距离（允许重新排列子串）[Ukk92, LT97]，以及交换操作（不相邻字符的对换）[KLPC99]，等等。虽然 Hamming 距离是编辑距离的一种简化，但在这种距离模型之上也存在一些特定的算法，其效果超过了编辑距离模型下的算法。

关于对象搜索，对象可能不仅仅是符号序列。当前存在一些扩展算法，例如多维文本上的近似搜索（请参看短综述[BYN00b]）和图像近似搜索[ALL97, Nav00]等。对上下文无关语法的近似搜索也有研究[Mye95]。这些领域都尚未发展成熟，现有的一些算法都依赖于经典的近似搜索算法。

最后需要提一下非常规算法，例如近似算法（不要把它和前述的近似搜索的精确算法相混淆）、概率算法和并行算法[TU88, AGM+90, LV89]等。[MP94]是一篇关于非常规字符串学理论问题的很好的综述，包含了一些在 Hamming 距离方面的结论。

第7章 总 结

在结束本书之前，很高兴在这里给出一些或许你能感兴趣的资料。

首先，了解一些可以免费获得的在线文本搜索软件是非常有用的。因此，在这里把一些已有的软件介绍给大家。

其次，对那些想在文本搜索领域做更深入了解的人，相应给出了一些其他相关的书籍、期刊、会议和在线资源。读者可能会对其中的一些特殊算法问题更感兴趣，而这是本书中或者一些免费软件中所没有涉及的。

最后，用一节的篇幅来概括一些与组合模式匹配相关的问题。其目的是在基本文本搜索问题的基础上，给出几个不同的扩展问题，解释它们的主要概念和已有结果，并给出相关的资源。

本书相关信息以及勘误表可以在如下地址获得：http://www.dcc.uchile.cl/~gnavarro/PMbook。

7.1 软件资源

本节将介绍一些可免费获得的在线模式匹配软件。

7.1.1 *Gnu Grep*

它是什么？ GNU（http://www.gnu.org）是一个致力于自由软件发展的组织。作为它的一个产品，*Grep* 可以支持在一组文件中进行快速单字符串、多字符串、正则表达式等的搜索，但是它不支持近似搜索。*Gnu Grep* 的搜索速度是经典 UNIX 软件 *Grep* 的两倍。

Grep 默认的是报告在文件中匹配成功的行。当然，通过设置不同的参数可以允许报告没有匹配成功的行，或者是匹配成功的行数，或者是匹配成功的文件，等等。该软件还提供了包括运算符在内的非常强大的语法来支持正则表达式匹配。

它是怎样工作的？ 在 *Grezp* 中，单串匹配使用的是 Boyer-Moore-Gosper 算法（与 Horspool

算法相似，参见 2.3.2 节）。多串匹配使用的是一个类 Commentz-Walter 算法（参见 3.3.1 节）。正则表达式匹配使用了一个懒惰的确定自动机（参见 5.3.2 节），这个 DFA 的状态和转移都是在前向搜索文本的过程中逐个创建的。为了加快复杂模式串的搜索速度，*Grep* 抽取了必要的最长子串，把它们作为过滤器并使用多串匹配方法对文本进行搜索。这种技术在 5.5.2 节中进行了描述，它使得 *Grep* 在搜索模式复杂度增加的情况下，搜索性能平滑下降，因此获得最好的平均性能。

从哪里可以获得它？ *Gnu Grep* 当前可获取的稳定版本是 2.4.2（2000 年 3 月），它发布的源码是 C 语言的，可以从 http://www.gnu.org/software/grep/grep.html 或者 ftp://ftp.gnu.org/pub/gnu/grep/处获得并免费使用。

7.1.2 Wu 和 Manber 的 *Agrep*

它是什么？ *Agrep*（指近似 *Grep*）是 1992 年由 Sun Wu 和 Udi Manber 在 Arizona 大学开发的。它是一个在线的、带索引的搜索工具集系列的开始，这个工具集还包括 *Glimpse*，*WebGlimpse*（http://glimpse.cs.arizona.edu/）和 *Harvest*（http://www.tardis.ed.ac.uk/harvest/）。

Agrep 是一个在线模式匹配软件，支持精确的和近似的单字符串、单扩展字符串、单正则表达式的匹配，也支持多字符串精确匹配。*Agrep* 有一套与 *Grep* 近似的语法，但是功能比 *Grep* 稍弱。扩展字符串包括通配符和字符组，其他的可以看做是正则表达式。*Agrep* 真正令 *Grep* 不能匹敌的显著特点是近似搜索功能，而且它还具有更灵活的结果返回方式：可以通过对边界的定义，以“记录”的方式返回匹配成功的结果，而不只是返回匹配成功的行（例如：在邮件文档中返回一个完整的邮件）。

它是怎样工作的？ *Agrep* 的算法原理已经在[WM92b]中阐述了，软件本身在[WM92a]中阐述。在 *Agrep* 中没有使用单一的算法，而是用一组启发式算法来解决不同的搜索问题。因此，*Agrep* 通常总是能选择最好的算法，但是试验结果显示，它的性能会随着搜索模式串复杂度的轻微变化而产生剧烈的变化。更进一步，*Agrep* 对模式串的长度和选项之间的组合都有很多严格限制。如果不考虑这些缺点，*Agrep* 在很多普通应用中是非常高效的。

简单字符串的搜索在长度不超过 400 时使用的是 Horspool 算法的一个变体（参见 2.3.2 节），更长字符串的搜索使用了类似的技术，不同的是在创建 shift 表时，使用了字符对而不是单个字符。多字符串的搜索使用了 3.3.3 节描述的 Wu-Manber 算法。

对于其他的模式串搜索，*Agrep* 依赖于位并行技术，更精确而言是 Shift-And 算法（参见 2.2.2 节）的扩展算法。对带字符组和带通配符的扩展字符串的搜索使用了第 4 章描述的扩展 Shift-And 算法。类似地，正则表达式的搜索使用了位并行算法 BPThompson（参见 5.4.1 节）。

近似搜索有两种实现方法：对简单串的低错误搜索，*Agrep* 中使用了 PEX（参见 6.5.1 节）；其他情况使用了位并行算法 BPR（参见 6.4.1 节中的“行式位并行”小节），它的扩展算法（参见 6.7.3 节）用来解决正则表达式的近似搜索。

从哪里可以获得它？ 商业用途是需要付费的，而学术研究和美国政府组织都可以免费使用 *Agrep*，并且可以获取源代码（C 语言）。

Agrep 的旧版本可以从 ftp://ftp.cs.arizona.edu 处下载。1994 年之后的最新版本是 3.0，可以通过从 http://webglimpse.net/download.html 处下载 *Glimpse*（1994 年之后的任何版本）获得，只要查找名字为“agrep/”的顶级子目录即可。

Agrep 的一个 Windows 版本可以从 http://www.tgries.de/agrep 处获得。

.1.3 Navarro 的 *Nrgrep*

它是什么？ *Nrgrep*（nondeterministic reverse *Grep*，不确定反转 *Grep*）是 2000 年由onzalo Navarro 在智利大学开发的一个在线模式串匹配软件。其基本功能与 *Agrep* 相似，l是 *Nrgerp* 不支持多串匹配。

它是怎样工作的？ *Nrgrep* 几乎全部基于 BNDM 算法以及对它的扩展[NR00, NR99a, R01a]，软件的具体描述参见[Nav01b]。建立在这种单一技术上的软件，性能随着搜索问复杂度的增加平滑下降，而不是像 *Agrep* 那样的剧烈下降。该软件还展示了 BNDM 的灵性，它可以快速搜索复杂模式串和正则表达式，不论是精确搜索还是允许错误的搜索。

单字符串的搜索使用了 2.4.2 节描述的基本 BNDM 算法，对扩展字符串的搜索，包括符组、可选字符、可重复字符，使用了第 4 章描述的扩展 BNDM 算法。正则表达式的搜使用了 Regular-BNDM 算法（参见 5.5.3 节）。

与 *Agrep* 相同，近似搜索使用了两种方法：一是使用 6.5.1 节中描述的 PEX 算法，其片断搜索使用 Multiple BNDM（参见 3.4.1 节）；二是使用 ABNDM 算法（参见 6.5.2 节），可以进行字符之间的跳跃，不仅可以用于单字符串，而且可以用于扩展字符串和正则表式（参见 6.7.3 节）。

Nrgrep 中将性能的平滑性看做是模式复杂度的一个函数。为了做到这一点，算法自动择最好的子串作为搜索过滤器，而且还能在不考虑语法的情况下正确探测模式的类型，样就能应用最简单有效的搜索算法。如果一旦预测到使用 BNDM 算法搜索的代价太大，将更换为前向式扫描算法（Shift-And）。

从哪里可以获得它？ *Nrgrep* 的源码，C 语言版本，可以从 http://www.dcc.uchile.cl/~gnavarro/ıbcode 免费下载，版本为 1.1（2001 年）。

7.1.4 Mehldau 和 Myers 的 *Anrep*

它是什么？ *Anrep* 是 1993 年由 Gerhard Mehldau 和 Gene Myers 在 Arizona 大学开发的，它在 DNA 和蛋白质搜索中［在从单字符串到网络表示和空间表示的范围中，找到精确的和近似的匹配（参见 4.3 节和 6.7 节）］是非常有影响的。匹配模式还包括在生物比对中很重要的一些模式。用户可以使用一种称做 A 的语言来指定搜索模式，A 语言表达清晰，格式自由，并且有很强的确定性。

它是怎样工作的？ *Anrep* 在[MM91]中进行了描述，它的算法原理参见[Mye96]。它强大的语言描述功能是生物应用中必需的，但是对于简单的算法来说是不实用的。*Anrep* 以 6.7.2 节中描述的算法为基础进行近似搜索，其中搜索过程以直接使用空间中的网络表达为代价。这种算法结合了动态规划方法和优化的回溯过程，动态规划方法用于允许错误的网络表达匹配，使用优化的回溯过程来确定那些正确的出现位置。

需要指出的是 *Anrep* 与前面提到的几个程序的一个不同之处：由于 *Anrep* 可以搜索非常复杂的模式，因此它的速度要慢得多。

从哪里可以获得它？ *Anrep* 的 C 语言版本可以从 http://www.cs.arizona.edu/people/gene/CODE/anrep.tar.z 处免费获得。

7.1.5 计算生物学方面的资料

除了 *Anrep* 之外，还有很多适用于计算生物学应用的资源，这里不一一进行详细叙述。因为它们涉及到很少的串匹配问题，更多的是那些怎样决定相关子序列的统计问题，另外还使用了大量的计算生物学知识。算法通常是近似搜索的复杂变体，使用了复杂的代价函数、空位惩罚，等等。搜索过程结合使用了动态规划方法和过滤方法（参见第 6 章），还有对空位的启发式运用。这里主要介绍其中最著名的两个系统。

BLAST 是 Basic Local Alignment Search Tool 的首字母缩写。它是由 Altschul 等人于 1990 年创立的，包括一组用于在序列数据库中寻找蛋白质或者 DNA 的近似搜索程序。它的主要目的是在尽可能小的牺牲精确度的前提下快速定位所有可能的出现位置，匹配结果以一种已定义的含有统计意义的形式给出。BLAST 使用了一种启发式算法，试图发现局部的序列比对，因此可以检测出两条远源基因序列间的部分匹配关系。BLAST 的算法原理参见[AGM+90]，不同体系结构机器上的软件版本都可以在 http://www.ncbi.nlm.nih.gov/BLAST/ 免费获得，而且还可以在线测试你的系统。BLAST 当前的版本是 2.0（1997 年）。

FASTA 是由 Pearson 和 Lipman 在 1988 年创立的，它也是一个在生物序列数据库中搜

同构子序列并寻找局部线性最优的系统，包含了几个程序以提供不同的速度/精确度。ASTA 的目标与 BLAST 相似，它们的主要差别在于对匹配所赋予意义的方法[Pea91]。ASTA 的算法原理参见[SW81, PL88]，系统源码可以从 ftp://ftp.virginia.edu/pub/fasta/免费得。FASTA 当前的版本是 3.2（1998 年）。

.2 其他书籍

2.1 串匹配方面

这里给出的所有书籍，力求尽可能全面覆盖串匹配技术涉及的领域。

Handbook of algorithms and data structures, G. Gonnet and R. Baeza-Yates, Addison-Wesley, cond edition, 1991

此书是涉及到通常意义上的算法，其中一章描述了精确串匹配。本书以一系列短篇组而成，对每一个算法给出了一个关于主要思路的简短解释，然后是代码和结果分析。

此书适用于那些为了解决某个串匹配问题而急于找到一个算法的人，因为他可以看到法的结果分析并复制源代码。然而如果要学习算法为什么这样设计以及它是怎样工作的，书是不够的。另一个问题是此书缺乏 1992 年之后的技术发展，比如近似搜索技术。

实际上，在算法类的书籍中有很多都包含了串匹配章节，例如[Knu73, AHU83, Meh84, aa88, Sed88, Man89, CLR90]，但是大都只提到 KMP 和 BM。选择上述这些书，是因为它对通常算法的涉及比较全面。一些编译和形式化语言方面的书籍，例如[ASU86, HU79]，述了使用经典 DFA 方法解决正则表达式匹配的问题。

Text algorithms，M. Crochemore and W. Rytter，Oxford University Press，1994

此书是一本很好的关于文本搜索算法的技术综述书，重点是理论定义。例如：它没有到任何位并行算法，而且它涉及到的很多算法在我们这本书中都忽略了，是因为这些算在实际运行中效果太差。此书也只涉及了精确串匹配。

此书适用于那些对串匹配算法的理论和组合部分感兴趣的人员。但是对于只是需要一实用的串匹配算法而不想深入此领域的读者，不推荐此书。

String searching algorithms，G. Stephen, World Scientific Press，1994

此书较全面地涉及到精确和近似串匹配。对于精确串匹配，它介绍了很多算法，着重 Boyer-Moore 系列上，而没有涉及 BDM 系列和位并行算法。多串匹配和扩展串匹配也有涉及。本书对近似串匹配的介绍很不错，使用了很长的一章来阐述对不同字符串相似

性的度量方法，并且另外使用章节对近似串匹配技术做了很全面的综述（至1994年）。但是，从1994年之后，这个领域有了很大的进展，因此缺少对当前运行最快的一些算法的介绍。本书还涉及了一些用于文本搜索的索引数据结构，例如后缀树。

String pattern matching strategies, J. Aoe (Editor), IEEE Computer Science Press, 1994

此书涉及了大部分基本的串匹配算法，包括单串匹配、多串匹配、近似匹配和多维的串匹配。它的不足之处也是缺乏对运行最快的最新算法的介绍。

Pattern matching algorithms, A. Apostolico and Z. Galil (Editors), Oxford University Press, 1997

此书的各个章节是由多位研究者分别完成的，每章都经过精心筛选，涵盖了从在线的精确和近似模式匹配，到并行以及关于字符串、树、matrix的索引搜索等较广的范围。本书高度理论化，推荐的理由与 *Text algorithms* 一书相同。

Modern information retrieval, R. Baeza-Yates and B. Ribeiro-Neto, Addison-Wesley, 1999

此书的重点在于信息检索，部分地方涉及算法问题，其中一章主要介绍在线串匹配。它的意图在于通过给出隐藏在信息检索后面的串匹配技术，来提高读者对信息检索的兴趣，而对于解决串匹配问题来说，本书是不够的。

7.2.2 计算生物学方面

与串匹配和计算生物学交叉相关的书籍有下面这一本：

Algorithms on strings, trees and sequences: Computer science and computational biology, D. Gusfield, Cambridge University Press, 1997

此书是一本综述性的书籍，对在计算生物学中使用到的以序列、树等为数据结构的主要算法性技术进行了概述，事实上这是一个非常大的领域。书中对这些技术做了较全面的描述，包括使用很长的篇幅介绍了索引，还特别介绍了后缀树以及建立在其上的算法。

除上面介绍的这本书之外，还有很多关于计算生物学的书籍，但是它们大多很少涉及串匹配技术。下面选择了其中三本来介绍：

Computational molecular biology: An algorithmic approach, P. A. Pevzner, MIT Press, 2000

这本近期的著作结合算法的发展介绍了计算分子生物学的几个主题，包括基因预测、限制性酶切做图、序列拼接、剪接信号位点寻找、基因序列重排以及蛋白质组学中的计算问题等。

Introduction to computational biology, M. S. Waterman, Chapman & Hall, 1995

此书在已有的研究基础上，介绍了许多计算生物学领域公认的主题，其中很多结果现在已经被认为是“经典”。

Time warps, string edits, and macromolecules: The theory and practice of sequence nparison, D. Sankoff and J. B. Kruskal, Addison-Wesley, 1983

此书是最早的关于计算生物学的著作之一，它包含了若干不同主题的文章，其中大部都清晰描述了计算生物学中的问题并给出了解决问题的算法。虽然这些算法太旧以至于在几乎不能应用，但是书中提出的问题是很有趣的，并且都描述清晰。

3 其他资源

3.1 期刊

模式匹配的文章往往分散在许多不同的期刊上，其中较有指导性的是 ACM Computing rveys；算法方面的有：Algorithmica，Journal of the ACM，Journal of Algorithms，mmunications of the ACM，Information and Computation，Information and Control，ormation Processing Letters，Information Science，Journal of Computer Systems Science，rdic Journal of Computing，Random Structures and Algorithms，SIAM Journal on mputing，Theoretical Computer Science，Journal of Discrete Algorithms；实现方面的有：ftware Practice & Experience，IEEE Trans. on Software Engineering，Information Processing d Management 和 ACM Journal of Experimental Algorithmics。上述列表中，没有给出组合式匹配方面的文章，因为这是一个有很强理论基础的领域。实际上，字符串匹配是组合式匹配中的一个简单分支。

此外，还有一些关于计算生物学的期刊，例如：Bioinformatics（它的前身是 CABIOS），cleic Acids Research，Journal of Computational Biology，Genome Research 和 Journal of olecular Biology。

3.2 会议

与本领域关系密切的会议有很多，其中最好的是 Combinatorial Pattern Matching（CPM），mputing and Combinatorics（COCOON），ACM Computational Molecular Biology RECOMB），String Processing and Information Retrieval（SPIRE）和 Intelligent Systems for olecular Biology（ISMB）。

其他也收录模式匹配领域文章的会议有 Data Compression Conference（DCC），European mposium on Algorithms（ESA），IEEE Foundations on Computer Science（FOCS），

Foundations of Software Technology and Theoretical Computer Science（FSTTCS），Automata，Languages and Programming（ICALP），IFIP World Computer Congress，Algorithms and Computation（ISAAC），Mathematical Foundations of Computer Science（STACS），ACM Theory of Computing（STOC），Scandinavian Workshop on Algorithmic Theory（SWAT），Workshop on Algorithm Engineering（WAE）和 Workshop on Algorithms and Data Structures（WADS）。

7.3.3 在线资源

字符串匹配方面最好的网站是 Pattern Matching Pointers，在这里可以找到与本领域相关的研究人员、会议、书籍、软件、期刊、新闻组和讨论区，它由 Stefano Lonardi 来维护，网址是 http://www.cs.purdue.edu/homes/stelo/pattern.html。

其他资源还包括：Pattern Matching and Data Mining Research，由 Mika Klemettinen 维护，网址是 http://www.cs.helsinki.fi/research/pmdm；The Bioinformatics Resources，网址是 http://hgmp.mrc.ac.uk/ccp11。在 http://www.purdue.cs.edu/homes/stelo/pmdb 上有相关主题的讨论区。相关的新闻组有 comp.theory，comp.theory.info-retrieval，comp.text 和 comp.infosystems。相关的邮件列表有 theorynet，dbworld 和 dmanet。

需要注意的是，在线资源的链接会经常变化。

7.4 相关主题

在本书的最后部分，将给出与本书内容相关的其他一些主题。这些主题中的任何一个都可以单独作为一章来阐述，在这里只给出与之相关的最重要的一些参考内容，尽管这是远远不够的。

7.4.1 索引

本书的重点是在文本和序列上的在线搜索，即不需要在文本上建立任何数据结构。对于单独的一个搜索请求，这是一种好的解决策略，但是当在一个文本上有多个搜索请求时，通常要事先在文本上建立一个称为“索引”的数据结构，以节省其后的搜索时间，提高搜索速度。

通常，考虑是否使用在线搜索技术的因素有：（1）文本的大小。如果文本非常小，则维护其上的索引所需代价就相对较大；（2）搜索请求的变化率与文本的变化率。考虑到建立和维护索引的代价，通常认为如果搜索请求的变化率要小于文本变化频率时，可以使用

线搜索技术；（3）可用空间。对文本建立索引需要额外的空间，在某些情况下这需要昂的代价，甚至是不可能的。即使使用了索引技术，但是在线搜索技术仍然很重要，因为多索引技术中也变相使用了一些在线搜索技术。

通索引

通过建立索引，在长度为 n 的文本上精确搜索长度为 m 的字符串，需要 $O(m)$ 或者 $(m \log n)$ 的时间，而建立索引的时间为 $O(n)$，但有时为 $O(n \log n)$，需要的额外空间为 (n)，它的常量系数通常为文本长度的 2 到 30 倍。索引的数据结构有很多，它依赖于搜类型的不同以及可使用内存空间的大小。最常使用的一类是 *suffix trie*[AG85]，它是建立文本所有后缀基础上的一种 *trie* 结构（参见第 3 章）。对于给定的一个模式串，通过遍历个结构可以找到模式串在文本中的所有出现位置。

最有效的数据结构是压缩 *suffix trie* 的结构，包括紧密后缀树[AG85, Gus97]、后缀自动或者 DAWG（一种可以识别文本所有后缀的自动机[CR94]）、紧密后缀自动机或者 DAWG[CV97a, CV97b, IHS+01]、后缀数组（一种使用字典序存储文本所有后缀的数组 IM93, GBYS92]）。

上述数据结构也可以用来搜索扩展字符串、正则表达式，还可以用来进行近似搜索 IBY91, BYG96, NBY00]。搜索时间或者是 $O(mn^{\lambda})$，$0 \leqslant \lambda \leqslant 1$ 或者是 m 的指数级（在许 k 个错误的近似匹配中是 k 的指数级）。有些时候需要附加乘以一个 $O(\log n)$ 的因子，取决于采用的数据结构。

然语言上的索引

对于自然语言文本，最常使用的索引是倒排文件或者称为倒排索引，使用它只能检索个完整的词或者短语，而不能检索任意的子串。倒排索引由文本中所有不同词（字典）成。每个词后面对应着这个词在文本中出现的所有位置。这种结构通常也用于信息检索，息检索中不仅使用模式串匹配技术，还要计算每个文档与检索查询的相似度。详细阐述排索引的书籍有[WMB99, BYRN99]。

倒排索引技术有很多变形，如果考虑查找单词的问题，那么最重要的应该是索引粒度，度最精细的索引是存储每个词的每个位置，这需要额外的 30%空间（相对于文本大小）。外，也有索引是把文本集合分成若干的文档，对每个词只需要存储它出现在哪些文档中，需要额外的 15%空间。*Glimpse*（[MW94]，http://glimpse.cs.arizona.edu/）令人惊讶地将引空间降低为文本大小的 2% ~ 4%。它将文本分成等长的块，索引中指出词出现在哪些中，而不是具体的位置。这种技术称为“块寻址”。在它上面进行检索时，只能顺序地进

行搜索。[BYN00a]中的分析指出，在保证搜索时间是亚线性的前提下，可以存在亚线性规模的索引（相对于文本大小）。*Glimpse* 还介绍了搜索扩展字符串和正则表达式的技术以及近似搜索技术，它们通过扫描文本上的亚线性规模词表而达到词内级别的扫描。

7.4.2 压缩文本中的搜索

压缩文本上的搜索问题是指在不解压文本的基础上找出一个模式串在压缩文本中出现的所有位置。对这一问题的研究从 1992 年开始活跃起来。主要动机为：一是 CPU 处理器速度的提高远远大于 I/O 设备速度的提高；二是有发现表明：在某些情况下，直接在压缩文本中进行搜索的速度比解压缩之后再进行搜索的速度要快。

压缩算法

压缩是计算机科学中一个大而活跃的领域，在这里不可能做详细阐述。文本压缩是其中的一个子领域，力图用最好的算法来对文本文件进行压缩，[BCW90]是这个领域中很好的一本书。比文本压缩更引人关注的一个领域是压缩文本数据库，它关注的是能在压缩文本上进行高效检索的压缩技术，这也是下面要阐述的内容。

在文本压缩数据库中，压缩格式必须保证能够高效解压缩并对文本进行随机访问。这种格式中最流行的有：Huffman[Huf51]（文本中的每个符号被变长编码取代，出现频率越高的符号编码长度越短）、Ziv-Lempel（对于文本中当前位置的字符串，编码器使用指向前面已经出现的该字符串的指针来代替它，算法的一个变体 LZ77[ZL77]没有对前面出现的位置加以限制，而在变体 LZ78[ZL78]中使用了由前面的重复出现再加上后面的一个字符而组成的字符串）、成对编码（Byte-Pair encoding，BPE）[Gag94]（对成对出现的字符对使用一个没有在文本中出现的字符来代替，一直到没有可用的字符为止）。压缩技术的一个重要特征就是压缩率，这里定义压缩率为：压缩后文本的大小/原文件大小。举例来说，在 DNA 上的压缩，Huffman 的压缩率约为 25%，Ziv-Lempel 大约为 25% ~ 30%，BPE 约为 30%，而在自然语言文本上的压缩，Huffman 的压缩率约为 60%，Ziv-Lempel 大约为 30% ~ 40%，BPE 约为 70%。

压缩文本上的在线模式匹配

压缩匹配问题是由 Amir 和 Benson 在[AB92a]中首次提出的，目的是直接在压缩文本中进行字符串匹配而不需要解压缩。给定文本 $T = t_1 \ldots t_u$，相应的压缩字符串为 $Z = z_1 \ldots z_n$，模式串为 $P = p_1 \ldots p_m$，压缩匹配问题就是只用 P 和 Z，找出 P 在 T 中出现的所有位置。原始的算法是首先对 Z 进行解压缩，然后使用经典的串匹配算法，它的时

复杂度为$O(m+u)$。优化后算法的时间复杂度在最坏情况下为$O(m+n)$。

在线模式串匹配算法中最实用的是基于 BPE 压缩算法以及它的若干变体[Man97, MT+00, TSM+01]。它们可以用比在原文本上更快的速度在压缩文本上搜索，这也许是因原文本使用了它自身来进行压缩。但是这种方法的压缩率很低。

还有众多的研究工作是在 Ziv-Lempel 压缩文本上进行的，这种压缩算法有较好的压缩。第一个在基于Ziv-Lempel压缩上进行精确串匹配的算法是[ABF96]，它可以在$O(m^2+n)$时间和空间内对 LZ78 压缩文本进行搜索。在 LZ77 压缩文本上的工作较少，其中一个是T98]，它可以在$O(m+n\log^2(u/n))$时间内判断一个模式串是否在文本中出现。

稍后的进展出现在[NR99b, KTSA99, NT00]中。粗略地讲，在压缩文本中搜索的时间有能只是先解压再搜索的时间的一半。

在 Ziv-Lempel 压缩格式上还开展了许多其他方面的研究。在[ABF96]基础上扩展的对模式串搜索的工作参见[KTS+98]，它有$O(m^2+n)$的时间和空间复杂度，其中m是所模式串长度之和。压缩文本上的近似搜索问题在[AB92b]中被提出，第一个理论性的工作[KNU00, MKT+00]，更实用的算法是最近才出现的[NKT+01]。上述工作都是基于 LZ78 缩格式的。在 LZ78 上进行正则表达式的搜索参见[Nav01c]。

[KTSA99]做了一项很有用的工作，称为“collage systems”，它对以模式串匹配为目的缩格式进行了抽象的描述，在该系统上设计的很多算法都可以适用于不同的压缩格式，TSA99]中就设计了基于 collage systems 的 KMP 算法。随后，一些研究组（上一段中提的）又发展了这一概念。

最后需要注意的是，在自然语言文本（压缩的）中，有很多有效的算法来搜索完整的和短语。[MNZBY00]中使用了基于词的 Huffman 编码（编码中的单位是文本中的词和分词，而不是字符）压缩文本，并支持其上的单模式串和扩展模式串的精确和近似搜索。缩率也很好，在大约 10 M 的英文文本上压缩率为 25% ~ 30%，压缩文本上的单模式串搜速度与在原文本上的搜索速度相当，而在压缩文本上的复杂模式串精确搜索和近似搜索速度要比在原文本上搜索的速度快 8 倍。

缩文本上的索引模式匹配

对于大型的文本数据库，很自然的想法就是要对压缩文本建立压缩索引。在[KU96, ir99, KS98, GV00]中都考虑了为文本搜索而设计的压缩的数据结构，但它们都是将未压缩原文本与数据结构作为整体而考虑的。

最近提出的一些数据结构看起来前景不错，它们通常是将结构与文本一起进行压缩，

[Sad99, Sad00, FM00, FM01] 中给出的是压缩后缀数组，某些情况下，它们可以用比原文本更小的空间来表示索引和文本。

使用词对自然语言文本检索时，需要建立倒排索引，通常对文本和索引都是单独进行压缩的。对索引的压缩利用了每个词出现的位置是递增的这个事实，只对位置差异进行编码，因此可以选取对小数字编码效率高的方法。在出现位置列表中，地址的差别越小，压缩率就越高。[NMN+00]中给出了一个用基于词的 Huffman 方法对文本进行压缩的系统，在对索引压缩时结合了块寻址的索引方式。[WMB99]中描述了 MG 系统（可以从 ftp://munnari.oz.au/pub/mg 免费下载），在对压缩文本上的倒排索引压缩时，它结合了文件寻址的索引方式。

7.4.3 重复和循环

对文本或者序列的循环匹配方面的研究工作有很多，大多涉及生物学领域。对重复和循环的定义有很多不同的版本，而且在计算生物学中，至今仍然不清楚该怎样定义重复和循环，以及对近似循环的理解也不甚清晰。这里只给出一个简要的说明。

精确循环

对循环的第一个解释是指在文本中连续出现两次以上的一个子串 u，即 $u^k v$，$k \geqslant 2$ 且 v 是 u 的一个前缀。很明显，文本中可能包含二次方个这样的循环，例如当文本 $T = a^n$ 时。首先得到广泛研究的循环形式是 u^2，很多算法就是为了寻找二次循环的位置（参见[CR94]或[KK99]）。如果考虑完整的循环，则最大循环（有时也称为“连续出现”或者“最大周期”）可通过紧密方式将它们表示出来。当一个循环在文本中仅能通过折断自己才能向左或者向右扩展时，它就是一个最大循环。一个文本中最多可以有 $O(n)$ 个最大循环，并且可以在 $O(n)$ 时间内找到它们[KK99]。

对循环的第二个解释经常用在不连续循环中。重复是指在文本中至少出现两次的子串。最大重复是指仅能通过折断自己才能在文本中向左或者向右扩展的重复。一个文本中最多可以有线性个最大重复，并且可以在 $O(n)$ 时间内找出它们[Gus97]。但是，上述定义都没有考虑多个重复之间的位置关系。一个 u 对是指在文本中的一个形似 uvu 的出现，最大对是指不能扩展的对，类似最大重复或者最大循环。最令人感兴趣的对是那些两个 u 之间的距离不太近也不太远的对，即 $\delta_1 \leqslant |v| \leqslant \delta_2$。[BLPS99]中提出了一个时间复杂度为 $O(n \log n + nocc)$ 的算法，可以列举文本中的对（不一定是最大的），其中 $nocc$ 是出现的个数。如果上限 δ_2 是变化的，则时间可以降低到 $O(n + nocc)$ [BLPS99]。

似循环

近似循环也是要考虑的，例如在计算生物学中，子序列不总是能精确匹配；在音乐学，经常要检索重复主旋律。然而近似循环的定义更复杂，因为它通常是在精确循环的基上扩展的，而这种扩展根据循环体之间近似关系的不同而不同。

循环的近似概念通常称为纵向重复，常用来表示形如uv的两次连续循环，其中u和v近似匹配的。[Sch98]中给出了时间复杂度为$O(n^2 \log n)$，空间复杂度为$O(n^2)$的算法。果最大错误数限制在k以内，则可以在$O(kn \log k \log(n/k))$时间内找到所有的不覆盖纵重复[KM93]。这些算法更多的是具有理论价值，因为$O(n^2)$算法对基因序列的实际应用说太高了。然而，只考虑两次的重复限制性太强，在计算生物学上通常需要寻找那些连出现两次以上的部分。对这个问题，[SM98]给出了一个算法，可以找到小的卫星，它对环的定义更灵活（允许部分不匹配），但是对长度有严格的限制（不少于40个基本单位）。个算法的思想是过滤文本，结合[FLSS92]中的高效验证算法，[Ben98]中给出了一个速度快的算法/软件，称为“纵向重复寻找器”。在http://c3.biomatch.mssm.edu/trf.html可以找不同操作系统下的可执行文件（软件的源码是受保护的）。

对的近似概念通常称为非纵向重复。如果要寻找那些不覆盖的对，[Sch98]中用于纵向复的算法也适用，只是真正的问题在于如何管理规模巨大的出现位置信息。

计算生物学中的近似循环问题是最近才提出的，它是一个发展快、变化也快的主题。个常用的软件产品是*RepeatMasker*（*Phrap*的部分包，http://www.phrap.org/），它可以隐DNA序列中的部分重复区域。

.4.4 二维和多维的模式匹配

在二维文本（例如图像文件）中进行模式匹配是字符串匹配的扩展，其中很多有效的法都是前面介绍的一维文本上匹配算法的扩展。但是，在这个领域中还存在很多特殊的题，[Aoe94, AG97]中主要介绍了这个领域的相关内容。

本节中的讨论是基于$O(n^2)$大小的文本（$n \times n$个元素），模式大小为$O(m^2)$（$m \times m$元素）。这样做是为了使描述简单，许多算法都可以处理类似的矩形甚至是非矩形的文本模式。

维模式匹配

二维精确串匹配问题首先是由Bird和Baker[Bir77, Bak78]提出的，并给出了最坏情况

下为 $O(n^2)$ 的算法。平均复杂度较好的算法参见 Zhu 和 Takaoka 的[ZT89]，Baeza-Yates 和 Régnier 的[BYR93]。在 Karkkäinen 和 Ukkonen 的[KU94]中获得了较优的 $O(n^2 \log_\sigma m/m^2)$ 的平均时间复杂度。

二维近似串匹配通常只考虑矩形模式的替换操作，比起通常意义上的插入和删除操作要简单很多。这是因为允许插入和删除操作之后，模式的行、列可以匹配文本中不等长的片断。

考虑最多允许 k 次替换的模式匹配，在最坏情况下，最好的算法之一是在[AL91]中给出的，具有 $O((k+\log\sigma)n^2)$ 的时间复杂度和 $O(n^2)$ 的空间复杂度。另一相似的算法由 Crochemore 和 Rytter 在[CR94]中提出。另一方面，Ranka 和 Heywood 在[RH91]中使用 $O((k+m)n^2)$ 的时间和 $O(kn)$ 的空间解决了这个问题。Amir 和 Landau 也提出了一个不同的算法，时间为 $O(n^2 \log n \log\log n \log m)$。平均而言，最好的算法是 Karkkäinen 和 Ukkonen 提出的[KU94, Par96]，它的期望时间复杂度为 $O(n^2 k \log_\sigma m/m^2)$，当 $k \leqslant m^2/(4\log_\sigma m)$ 时，空间复杂度为 $O(m^2)$ [平均为 $O(k)$]。这个理想复杂度是最优的。

对编辑距离概念的扩展是困难的。Krithivasan 和 Sitalakshmi 的[KS87]中定义二维中的编辑距离为相应行中的编辑距离之和，称为 KS 模型。使用这种定义，他们获得了 $O(m^2n^2)$ 的搜索时间。Krithivasan 在[Kri87]中使用了同样的定义模型给出了一种时间为 $O(m(k+\log m)n^2)$，空间为 $O(mn)$ 的算法。Amir 和 Landau 在[AL91]中给出了一个最坏时间复杂度为 $O(k^2n^2)$，空间复杂度为 $O(n^2)$ 的算法。（注意：k 可以大于 m，所以这个算法未必一定比前面的算法好。）Amir 和 Farach 在[AF91]中考虑了非矩形模式，算法的时间复杂度可以达到 $O(k(k+\sqrt{m\log m}\sqrt{k\log k})n^2)$。最后，Navarro 和 BaezaYates 在 KS 模型上获得了 $O(n^2 k \log_\sigma m/m^2)$ 的平均时间复杂度，其中 $k < m(m+1)/(5\log_\sigma m)$，空间为 $O(m^2)$。

对于许多二维匹配问题，KS 距离不能很好地反映在不同设置下的近似匹配。在[BYN00b]中给出了新的距离定义和搜索算法。在这种新模型下，错误可以出现在行或者列上。

其他多维模式匹配

与图像比较问题有关的另外一些问题是允许对旋转图像的搜索[FNU01]和对按比例缩放图像的搜索[ABL00]。

对于图像匹配，还有许多与上面介绍的方法很不一样的其他方法（属于组合模式匹配领域）。其中，需要注意的是那些用在模式匹配方面的涉及人工智能的技术，例如图像处理

神经网络；那些用在数据库中的技术，例如从图像中抽取特征（如彩色直方图）。

前面所述问题都可以泛化为多于二维的问题，并且很多二维算法都可以进行扩展以支多维的问题[AL91, KU94, GG97, FU98, BYN00b]。

4.5 树模式匹配

树模式匹配是文本中模式串匹配的另一个扩展分支。这个问题来源于计算生物学中对NA 结构的比对。在程序的编译过程中，也常使用树模式匹配技术在程序的指令树中寻找些特殊的模式来进行优化。在结构化文本数据库中也需要这种方式的匹配。

对于精确的树模式匹配，有两种有序树分别称为“文本”和“模式”，它们是对字符串配中的文本和模式的扩展。树模式匹配问题就是在文本中找到模式出现的所有位置，即本中所有与模式匹配的子树。如果用 n 来表示文本的大小，用 m 来表示模式的大小，则始的算法需要 $O(mn)$ 的最坏时间复杂度，其中包括对树中每一个节点的校验。

有不少算法改进了最坏情况下的时间复杂度，但大多只有理论意义。[Kos98]中给出了个 $O(nm^{0.75}\log m)$ 的算法，它将树模式匹配问题与字符串匹配中的“don't care”符号问结合起来考虑。接着，[DGM94]中通过对预测模式树中的路径改进了上述算法，达到了 $(nm^{0.5}\log m)$的时间。随后，同样是这些作者，持续将算法不断改进，最终在[CHI99]中到了 $O(n\log^3 n)$。对这个主题的主要描述参见[ZS97]。

近似的树模式匹配问题主要来源于计算生物学中树的对比。这个问题中隐含了对树的离的定义[ZSW94, SZ97]。虽然已经有了不少算法，但这个问题仍然处于不断研究之中，是由于对树的距离的定义需要符合树所表示对象的生物学特性。树通常被认为是第二大据结构[ZWM99]。

与树和匹配都相关的另外一个问题是最大匹配子树问题，即给定两棵有根树，其叶节取自同一个集合（比如两棵进化树），最大子树问题就是找一个最大子集，使得两棵树在子集合上的限制是同构的。对于通常的二叉树情形，存在一个 $O(n\log n)$ 的算法，参见FCH+01]。

结构化文本数据库中的树模式匹配概念与本书中介绍的“扩展串匹配”类似。这时，式是一棵树，但是每个模式的边界都可以匹配文本树中的一条绝对路径。当然，也可以求成功的匹配与模式中的节点顺序保持一致。这样，搜索复杂度从多项式时间变为P-complete[Kil92, KM92]。

4.6 序列比较

序列比较是计算两个或多个字符串之间的相似程度。它与近似搜索（参见第 6 章）有

着密切关系，并在很多领域有着广泛应用，比如计算生物学、语音识别、科学计算、编码理论、色谱学等。以上应用中，常常需要计算两个符号序列之间的相似性，通常的目标是对符号串不断执行基本的操作，直到它们相等为止。这些基本的操作都有相应的代价，而所寻找的就是要达到目标的最小代价。寻找最小代价的理由因应用的不同而不同，但是通常会认为：如果序列可以通过一系列的替换而变成另外一个或者多个序列，则替换代价最小的序列之间的相似性最大。

两个符号串之间的“距离”定义为使它们相等的最小代价。哪些是基本的操作取决于不同的应用，典型的有禁止字符、插入字符、替换字符、交换字符（相邻的或者不相邻的）、反转字符、移动子串、压缩一串相同的字符、扩展一个字符为一串，等等。每种操作在相应的模型中都有合理的意义，因此不同的应用对每个操作也给出了合理的代价。例如，最小代价表示最大相似性。最简单的方法是用处理过程中执行一个操作的概率的对数作为该操作的代价，这样可使得符号串之间相异。因此，代价之和就对应了所有操作概率的对数，如果它们之间是独立的，则表明这是一个很好的模型。

两个使用最广的相似性度量是 Levenshtein 距离和 indel 距离。前者允许字符的插入、删除和替换，代价都为 1。后者只允许字符的插入和删除，类似于两个符号串之间的最大公共子序列（LCS，longest common subsequence）问题。由于问题描述的简单性、算法的高效性以及较好的数学性质，以上两个模型得到了广泛应用。Levenshtein 距离详见第 6 章。

和上述计算序列相似性相反，在很多应用中，经常要求计算出两个序列之间的相异性，通常使用“联配”这个概念来描述（参见 6.2.1 节）。有很多方法可以描述联配关系：一个方法是将两个符号串重叠，然后使用空格将它们的字符分离，使得相似的字符落在相同的列中。两个符号串的相异程度可以参考空格的数量；另一个方法是根据字符串中相同的字符，画出一组“路径”（参见图 6.1），路径的线条用来连接两个字符串相对齐的字符；第三个方法是罗列出可以在符号串上进行的所有操作，这个方法虽然应用得不很广泛，但是有实用的数学意义。

在有些情况下，对相似度进行计算比对相异度进行计算（例如上述的“距离”）更有用处。一个例子就是 LCS，它是一个非常成熟的计算方法。其他例子包括最短公共序列（SCS）、最长公共子串（LGG，与 LCS 不同的是 LGG 必须是所有序列的连续子串）以及最短公共串（SCG），它们都适用于多串。

序列比对通常使用动态规划算法，因为它可以充分利用前面计算出来的结果进行后面的计算。当然，回溯过程也是需要的。对于长度为 n 的两个序列，使用最简单的距离，例如 Levenshtein 距离或者 indel 距离，即使使用任意的操作代价，也可以在 $O(n^2)$ 时间内计算出它们之间的距离。对于 LCS 或者 SCS 同样也可以。对于 N 个串，时间增至 $O(n^N)$，

且当 N 为任意时它是 NP 完全的。对于两个串，计算 LCS 和 SCG 距离需要 $O(n)$ 时间。有一些复杂的算法是对不同假设前提下更复杂情况进行的改进。另一方面，如果考虑块动的问题，在某些情况下距离的计算变为 NP 完全的。这也表明，问题的难易程度取决使用哪种距离。

序列比对中的另一个话题是统计。两个二进制串之间的 LCS 距离是它们长度的 80%的能性有多大？这是否表明它们很接近？如果两个串是随机的，这种情况还会发生吗？对个随机串之间的 LCS 距离的理想值有很多研究，已知的是它与 n 呈线性关系，但是精确常量还未知，只有它的上界和下界。

推荐关注这个主题的读者阅读[SK83]，也可以参阅本书的计算生物学部分的书籍（参 7.2.2 节）。

4.7 特异子串检测

如何在序列中检测某个特异子串，是一个得到广泛研究的问题。一般说来，此问题涉如下四个方面：

第一，必须要对“正常”序列定义一个产生概率模型，比如 Markov 模型。这样，以此型为基准，才可能判定哪些子串是“正常”的，哪些是特异的。

第二，必须有公式来计算“正常”子串出现的概率或其他有意义的参数，比如两次“正”子串出现之间距离的期望值等。[RS97]中给出了近似串匹配意义下每次“正常”出现概率，[NSF01]中给出了算法，可以计算在 Bernoulli 或 Markov 产生模型下，单个字符串、个字符串、正则表达式出现次数的期望值。然而，上述公式的计算通常具有很高的时间杂度[RS97]。Algolib 是一个使用 Maple 语言开发的软件包，实现了上述公式的求解，可 http://algo.inria.fr/libraries/software.html 下载。

第三，给定一个“正常”序列的产生概率模型，以及对子串各个相关参数的计算公式，需要设计判别方法，以确定文本中哪些子串是“正常”的，哪些是特异的。

第四，在给定模型、参数计算公式以及判别方法之后，由于特异子串通常有 $O(n^2)$ 个，此还需要设计高效的搜索算法，以及可视化方法。

关于上述问题的一个简短综述参见[Pev00]。目前为止，还未见到一个比较完备的综述这些问题全部归类。在软件工具方面，Verbumculus[ABLX00]软件包可以检测出符合其定定义的特异子串，参见 http://www.cs.purdue.edu/homes/stelo/Verbumculus。

参考文献

[AB92a] A. Amir and G. Benson. Efficient two-dimensional compressed matching. In *Proceedings of the 2th Data Compression Conference*, pages 279–288. IEEE Computer Society Press, 1992.

[AB92b] A. Amir and G. Benson. Two-dimensional periodicity and its applications. In *Proceedings of the 3rd ACM-SIAM Annual Symposium on Discrete Algorithms*, pages 440–452. ACM Press, 1992.

[ABF96] A. Amir, G. Benson, and M. Farach. Let sleeping files lie: Pattern matching in Z-compressed files. *Journal of Computer and Systems Sciences*, 52(2):299–307, 1996.

[ABL00] A. Amir, A. Butman, and M. Lewenstein. Real scaled matching. In *Proceedings of the 11th ACM-SIAM Annual Symposium on Discrete Algorithms*, pages 815–816. ACM Press, 2000.

[ABLX00] A. Apostolico, M. E. Bock, S. Lonardi, and X. Xu. Efficient detection of unusual words. *Journal of Computational Biology*, 7(1/2):71–94, 2000.

[Abr87] K. Abrahamson. Generalized string matching. *SIAM Journal on Computing*, 16(6):1039–1051, 1987.

[AC75] A. V. Aho and M. J. Corasick. Efficient string matching: an aid to bibliographic search. *Communications of the ACM*, 18(6):333–340, 1975.

[ACR01] C. Allauzen, M. Crochemore, and M. Raffinot. Efficient experimental string matching by weak factor recognition. In *Proceedings of the 12th Annual Symposium on Combinatorial Pattern Matching*, number 2089 in Lecture Notes in Computer Science, pages 51–72. Springer-Verlag, 2001.

[AF91] A. Amir and M. Farach. Efficient 2-dimensional approximate matching of non-rectangular figures. In *Proceedings of the 2nd ACM-SIAM Annual Symposium on Discrete Algorithms*, pages 212–223, 1991.

[AG85] A. Apostolico and Z. Galil, editors. *Combinatorial Algorithms on Words*, volume 12. Springer-Verlag, 1985.

[AG97] A. Apostolico and Z. Galil, editors. *Pattern Matching Algorithms*. Oxford University Press, 1997.

[AGM+90] S. F. Altschul, W. Gish, W. Miller, E. W. Myers, and D. J. Lipman. A basic local alignment search tool. *Journal of Molecular Biology*, 215:403–410, 1990.

[AHU83] A. V. Aho, J. E. Hopcroft, and J. D. Ullman. *Data Structures and Algorithms*. Addison-Wesley, 1983.

[AL91] A. Amir and G. M. Landau. Fast parallel and serial multidimensional approximate array matching. *Theoretical Computer Science*, 81(1):97–115, 1991.

[ALL97] A. Amir, M. Lewenstein, and N. Lewenstein. Pattern matching in hypertext. In *Proceedings of the 5th Workshop on Algorithms and Data Structures*, number 1272 in Lecture Notes in Computer Science, pages 160–173. Springer-Verlag, 1997.

[Aoe94] J.-I. Aoe, editor. *String pattern matching strategies*. IEEE Computer Society Press, 1994.

[AR99] C. Allauzen and M. Raffinot. Factor oracle of a set of words. Technical report 99-11, Institut Gaspard-Monge, Université de Marne-la-Vallée, 1999.

[AR00] C. Allauzen and M. Raffinot. Simple optimal string matching. *Journal of Algorithms*, 36:102–116, 2000.

[ASU86] A. V. Aho, R. Sethi, and J. D. Ullman. *Compilers – Principles, Techniques and Tools*. Addison-Wesley, 1986.

[Baa88] S. Baase. *Computer Algorithms – Introduction to Design and Analysis*. Addison-Wesley, 1988.

[Bak78] T. P. Baker. A technique for extending rapid exact-match string matching to arrays of more than one dimension. *SIAM Journal on Computing*, 7(4):533–541, 1978.

[BBE+87] A. Blumer, J. Blumer, A. Ehrenfeucht, D. Haussler, and R. McConnel. Complete inverted files for efficient text retrieval and analysis. *Journal of the ACM*, 34(3):578–595, 1987.

[BBYDS96] V. Bruyère, R. A. Baeza-Yates, O. Delgrange, and R. Scheihing. About the size of Boyer-Moore automata. In N. Ziviani, R. Baeza-Yates, and K. Guimarães, editors, *Proceedings of the 3rd South American Workshop on String Processing*, pages 31–46, Recife, Brazil. Carleton University Press, 1996.

[BCW90] T. Bell, J. Cleary, and I. Witten. *Text Compression*. Prentice Hall, 1990.

[Ben98] G. Benson. An algorithm for finding tandem repeats of unspecified pattern size. In *Proceedings of the 2nd Annual International Conference on Computational Molecular Biology*, pages 20–29. ACM Press, 1998.

[Ben99] G. Benson. Tandem repeats finder: a program to analyze DNA sequences. *Nucleic Acids Research*, 27:573–580, 1999.

[BH01] A. Bergeron and S. Hamel. Vector algorithms for approximate string matching. *International Journal of Foundations of Computer Science*, 2001. To appear.

[Bir77] R. S. Bird. Two-dimensional pattern matching. *Information Processing Letters*, 6(5):168–170, 1977.

[BK93] A. Brüggemann-Klein. Regular expressions into finite automata. *Theoretical Computer Science*, 120(2):197–213, 1993.

[BLPS99] G. S. Brodal, R. B. Lyngsø, C. N. S. Pedersen, and J. Stoye. Finding maximal pairs with bounded gap. In *Proceedings of the 10th Annual Symposium on Combinatorial Pattern Matching*, number 1645 in Lecture Notes in Computer Science, pages 134–149. Springer-Verlag, 1999.

[BM77] R. S. Boyer and J. S. Moore. A fast string searching algorithm. *Communi-*

cations of the ACM, 20(10):762–772, 1977.

[Bre95] D. Breslauer. Dictionary matching on unbounded alphabets: uniform length dictionaries. *Journal of Algorithms*, 18(2):278–295, 1995.

[BS86] G. Berry and R. Sethi. From regular expression to deterministic automata. *Theoretical Computer Science*, 48(1):117–126, 1986.

[BY91] R. Baeza-Yates. Some new results on approximate string matching. In *Workshop on Data Structures*, Dagstuhl, Germany, 1991. Abstract.

[BYCG94] R.A. Baeza-Yates, C. Choffrut, and G.H. Gonnet. On Boyer-Moore automata. *Algorithmica*, 12(4/5):268–292, 1994.

[BYG89a] R. A. Baeza-Yates and G. H. Gonnet. Boyer-Moore automata. Report, University of Waterloo, 1989.

[BYG89b] R. A. Baeza-Yates and G. H. Gonnet. A new approach to text searching. In N. J. Belkin and C. J. van Rijsbergen, editors, *Proceedings of the 12th International Conference on Research and Development in Information Retrieval*, pages 168–175. ACM Press, 1989.

[BYG96] R. A. Baeza-Yates and G. H. Gonnet. Fast text searching for regular expressions or automaton searching on tries. *Journal of the ACM*, 43(6):915–936, 1996.

[BYGR90] R. A. Baeza-Yates, G. H. Gonnet, and M. Régnier. Analysis of Boyer-Moore type string searching algorithms. In *Proceedings of the 1st ACM-SIAM Annual Symposium on Discrete Algorithms*, pages 328–343. ACM Press, 1990.

[BYN97] R. A. Baeza-Yates and G. Navarro. Multiple approximate string matching. In *Proceedings of the 5th Workshop on Algorithms and Data Structures*, number 1272 in Lecture Notes in Computer Science, pages 174–184. Springer-Verlag, 1997. Extended version to appear in *Random Structures and Algorithms* (Wiley).

[BYN99] R. A. Baeza-Yates and G. Navarro. Faster approximate string matching. *Algorithmica*, 23(2):127–158, 1999.

[BYN00a] R. Baeza-Yates and G. Navarro. Block-addressing indices for approximate text retrieval. *Journal of the American Society for Information Science*, 51(1):69–82, 2000.

[BYN00b] R. Baeza-Yates and G. Navarro. New models and algorithms for multidimensional approximate pattern matching. *Journal of Discrete Algorithms*, 1(1):21–49, 2000.

[BYR92] R. A. Baeza-Yates and M. Régnier. Average running time of the Boyer-Moore-Horspool algorithm. *Theoretical Computer Science*, 92(1):19–31, 1992.

[BYR93] R. A. Baeza-Yates and M. Régnier. Fast two-dimensional pattern matching. *Information Processing Letters*, 45(1):51–57, 1993.

[BYRN99] R. A. Baeza-Yates and B. Ribeiro-Neto. *Modern Information Retrieval*. Addison-Wesley, 1999.

[CCG+93] M. Crochemore, A. Czumaj, L. Gąsieniec, S. Jarominek, T. Lecroq, W. Plandowski, and W. Rytter. Fast practical multi-pattern matching. Rapport 93-3, Institut Gaspard Monge, Université de Marne-la-Vallée, 1993.

[CCG+94] M. Crochemore, A. Czumaj, L. Gąsieniec, S. Jarominek, T. Lecroq,

W. Plandowski, and W. Rytter. Speeding up two string matching algorithms. *Algorithmica*, 12(4/5):247–267, 1994.

[CCG+99] M. Crochemore, A. Czumaj, L. Gąsieniec, T. Lecroq, W. Plandowski, and W. Rytter. Fast practical multi-pattern matching. *Information Processing Letters*, 71(3/4):107–113, 1999.

[CFCH+01] R. Cole, M. Farach-Colton, R. Hariharan, T. Przytycka, and M. Thorup. An $O(n \log n)$ algorithm for the maximum agreement subtree problem for binary trees. *SIAM Journal on Computing*, 30(5):1385–1404, 2001.

[CGR92] M. Crochemore, L. Gąsieniec, and W. Rytter. Turbo-BM. Rapport LITP 92.61, Universités Paris 6 et 7, France, 1992.

[CH97] M. Crochemore and C. Hancart. Automata for matching patterns. In G. Rozenberg and A. Salomaa, editors, *Handbook of Formal Languages*, volume 2: Linear Modeling: Background and Application, chapter 9, pages 399–462. Springer-Verlag, 1997.

[CH98] R. Cole and R. Hariharan. Approximate string matching: A simpler faster algorithm. In *Proceedings of the 9th ACM-SIAM Annual Symposium on Discrete Algorithms*, pages 463–472. ACM Press, 1998.

[CHI99] R. Cole, R. Hariharan, and P. Indyk. Tree pattern matching and subset matching in deterministic $O(n \log^3 n)$-time. In *Proceedings of the 10th ACM-SIAM Annual Symposium on Discrete Algorithms*, pages 245–254. ACM Press, 1999.

[Cho90] C. Choffrut. An optimal algorithm for building the Boyer-Moore automaton. *Bulletin of the European Association of Theoretical Computer Science*, 40:217–225, 1990.

[CL92] W. I. Chang and J. Lampe. Theoretical and empirical comparisons of approximate string matching algorithms. In *Proceedings of the 3rd Annual Symposium on Combinatorial Pattern Matching*, number 664 in Lecture Notes in Computer Science, pages 175–184. Springer-Verlag, 1992.

[CL94] W. I. Chang and E. L. Lawler. Sublinear approximate string matching and biological applications. *Algorithmica*, 12(4/5):327–344, 1994.

[CLR90] T. H. Cormen, C. E. Leiserson, and R. L. Rivest. *Introduction to Algorithms*. MIT Press, 1990.

[CM94] W.I. Chang and T. Marr. Approximate string matching with local similarity. In *Proceedings of the 5th Annual Symposium on Combinatorial Pattern Matching*, number 807 in Lecture Notes in Computer Science, pages 259–273. Springer-Verlag, 1994.

[Col94] R. Cole. Tight bounds on the complexity of the Boyer-Moore string matching algorithm. *SIAM Journal on Computing*, 23(5):1075–1091, 1994.

[CP91] M. Crochemore and D. Perrin. Two-way string-matching. *Journal of the ACM*, 38(3):651–675, 1991.

[CP92] C.-H. Chang and R. Paige. From regular expression to DFA's using NFA's. In *Proceedings of the 3rd Annual Symposium on Combinatorial Pattern Matching*, number 664 in Lecture Notes in Computer Science, pages 90–110. Springer-

Verlag, 1992.

[CR94] M. Crochemore and W. Rytter. *Text Algorithms.* Oxford University Press, 1994.

[CR95] M. Crochemore and W. Rytter. Squares, cubes and time-space efficient string-searching. *Algorithmica*, 13(5):405–425, 1995.

[Cro92] M. Crochemore. String-matching on ordered alphabets. *Theoretical Computer Science*, 92(1):33–47, 1992.

[CV97a] M. Crochemore and R. Vérin. Direct construction of compact directed acyclic word graphs. In *Proceedings of the 8th Annual Symposium on Combinatorial Pattern Matching*, number 1264 in Lecture Notes in Computer Science, pages 116–129. Springer-Verlag, 1997.

[CV97b] M. Crochemore and R. Vérin. On compact directed acyclic word graphs. In *Structures in Logic and Computer Science*, number 1261 in Lecture Notes in Computer Science, pages 192–211. Springer-Verlag, 1997.

[CW79] B. Commentz-Walter. A string matching algorithm fast on the average. In *Proceedings of the 6th International Colloquium on Automata, Languages and Programming*, number 71 in Lecture Notes in Computer Science, pages 118–132. Springer-Verlag, 1979.

[DGM94] M. Dubiner, Z. Galil, and E. Magen. Faster tree pattern matching. *Journal of the ACM*, 41(2):205–213, 1994.

[FLSS92] V. A. Fischetti, G. M. Landau, J. P. Schmidt, and P. H. Sellers. Identifying periodic occurrences of a template with applications to protein struture. In *Proceedings of the 3rd Annual Symposium on Combinatorial Pattern Matching*, number 664 in Lecture Notes in Computer Science, pages 111–120. Springer-Verlag, 1992.

[FM00] P. Ferragina and G. Manzini. Opportunistic data structures with applications. In *Proceedings of the 41st IEEE Annual Symposium on Foundations of Computer Science*, pages 390–398. IEEE Computer Society Press, 2000.

[FM01] P. Ferragina and G. Manzini. An experimental study of an opportunistic index. In *Proceedings of the 12th ACM-SIAM Annual Symposium on Discrete Algorithms*, pages 269–278. ACM Press, 2001.

[FNU01] K. Fredriksson, G. Navarro, and E. Ukkonen. *Faster than FFT: Rotation Invariant Combinatorial Template Matching*, volume II. Transworld Research Network, 2001. To appear.

[FP74] M. J. Fischer and M. Paterson. String matching and other products. In R. M. Karp, editor, *Proceedings SIAM-AMS Complexity of Computation*, pages 113–125. AMS, 1974.

[FT98] M. Farach and M. Thorup. String matching in Lempel-Ziv compressed strings. *Algorithmica*, 20(4):388–404, 1998.

[FU98] K. Fredriksson and E. Ukkonen. Rotation invariant filters for multidimensional string matching and orientation search. In *Proceedings of the 9th Annual Symposium on Combinatorial Pattern Matching*, number 1448 in Lecture Notes in Computer Science, pages 118–125. Springer-Verlag, 1998.

[Gag94] P. Gage. A new algorithm for data compression. *The C Users Journal*, 12(2), 1994.

[Gal79] Z. Galil. On improving the worst case running time of the Boyer-Moore string searching algorithm. *Communications of the ACM*, 22(9):505–508, 1979.

[GBY90] G. H. Gonnet and R. A. Baeza-Yates. An analysis of the Karp-Rabin string matching algorithm. *Information Processing Letters*, 34(5):271–274, 1990.

[GBYS92] G. Gonnet, R. Baeza-Yates, and T. Snider. *Information Retrieval: Data Structures and Algorithms*, chapter 3: New indices for text: Pat trees and Pat arrays, pages 66–82. Prentice-Hall, 1992.

[GG97] R. Giancarlo and R. Grossi. Multi-dimensional pattern matching with dimensional wildcards: Data structures and optimal on-line search algorithms. *Journal of Algorithms*, 24(2):223–265, 1997.

[Glu61] V-M. Gluskov. The abstract theory of automata. *Russian Mathematical Surveys*, 16:1–53, 1961.

[GP90] Z. Galil and K. Park. An improved algorithm for approximate string matching. *SIAM Journal on Computing*, 19(6):989–999, 1990.

[GS81] Z. Galil and J. Seiferas. Linear-time string matching using only a fixed number of local storage locations. *Theoretical Computer Science*, 13(3):331–336, 1981.

[Gus97] D. Gusfield. *Algorithms on Strings, Trees and Sequences: Computer Science and Computational Biology*. Cambridge University Press, 1997.

[GV00] R. Grossi and J.S. Vitter. Compressed suffix arrays and suffix trees with applications to text indexing and string matching. In *Proceedings of the 32th ACM Symposium on the Theory of Computing*, pages 397–406. ACM Press, 2000.

[Han93] C. Hancart. On Simon's string searching algorithm. *Information Processing Letters*, 47(2):95–99, 1993.

[HBFB99] K. Hofmann, P. Bucher, L. Falquet, and A. Bairoch. The PROSITE database, its status in 1999. *Nucleic Acids Research*, 27:215–219, 1999.

[HM98] C. Hagenah and A. Muscholl. Computing epsilon-free NFA from regular expressions in $O(n\log^2(n))$ time. In *Proceedings of the 23th Symposium on Mathematical Foundations of Computer Science*, number 1450 in Lecture Notes in Computer Science. Springer-Verlag, 1998.

[HN01] H. Hyyrö and G. Navarro. Faster bit-parallel approximate string matching. Technical Report TR/DCC-2002-1, University of Chile, Santiago, Chile, 2002.

[Hor80] R. N. Horspool. Practical fast searching in strings. *Software Practice and Experience*, 10(6):501–506, 1980.

[HSW97] J. Hromkovic, S. Seibert, and T. Wilke. Translating regular expressions into small epsilon-free nondeterministic finite automata. In *Proceedings of the 14th Annual Symposium on Theoretical Aspects of Computer Science*, number 1200 in Lecture Notes in Computer Science, pages 55–66. Springer-Verlag, 1997.

[HU79] J. E. Hopcroft and J. D. Ullman. *Introduction to Automata, Languages and Computations*. Addison-Wesley, 1979.

[Huf51] D. A. Huffman. A method for the construction of minimum redundancy

codes. *Proceedings of the Institute of Electrical and Radio Engineers*, 40:1098–1101, 1951.

[Hyy01] H. Hyyrö. Explaining and extending the bit-parallel algorithm of Myers. Technical Report A-2001-10, University of Tampere, Finland, 2001.

[IHS+01] S. Inenaga, H. Hoshino, A. Shinohara, M. Takeda, S. Arikawa, G. Mauri, and G. Pavesi. On-line construction of compact directed acyclic word graphs. In *Proceedings of the 12th Annual Symposium on Combinatorial Pattern Matching*, number 2089 in Lecture Notes in Computer Science, pages 169–180. Springer-Verlag, 2001.

[JTU96] P. Jokinen, J. Tarhio, and E. Ukkonen. A comparison of approximate string matching algorithms. *Software Practice and Experience*, 26(12):1439–1458, 1996.

[Kär99] J. Kärkkäinen. *Repetition-Based Text Indexing.* PhD thesis, Department of Computer Science, University of Helsinki, December 1999.

[Kil92] P. Kilpeläinen. *Tree Matching Problems with Applications to Structured Text Databases.* PhD thesis, University of Helsinki, Finland, 1992.

[KK99] R. Kolpakov and G. Kucherov. Finding maximal repetitions in a word in linear time. In *Proceedings of the 40th IEEE Annual Symposium on Foundations of Computer Science.* IEEE Computer Society Press, 1999.

[KLPC99] D. K. Kim, J. S. Lee, K. Park, and Y. Cho. Efficient algorithms for approximate string matching with swaps. *Journal of Complexity*, 15:128–147, 1999.

[KM92] P. Kilpeläinen and H. Mannila. Grammatical tree matching. In *Proceedings of the 3rd Annual Symposium on Combinatorial Pattern Matching*, number 644 in Lecture Notes in Computer Science, pages 162–174. Springer-Verlag, 1992.

[KM93] S. K. Kannan and E. W. Myers. An algorithm for locating non-overlapping regions of maximum alignment score. In *Proceedings of the 4th Annual Symposium on Combinatorial Pattern Matching*, number 684 in Lecture Notes in Computer Science, pages 74–86. Springer-Verlag, 1993.

[KM95] J. R. Knight and E. W. Myers. Approximate regular expression pattern matching with concave gap penalties. *Algorithmica*, 14(1):85–121, 1995.

[KMP77] D. E. Knuth, J. H. Morris, Jr, and V. R. Pratt. Fast pattern matching in strings. *SIAM Journal on Computing*, 6(1):323–350, 1977.

[Knu73] D. E. Knuth. *The Art of Computer Programming: Sorting and Searching*, volume 3. Addison-Wesley, 1973.

[KNU00] J. Kärkkäinen, G. Navarro, and E. Ukkonen. Approximate string matching over Ziv-Lempel compressed text. In *Proceedings of the 11th Annual Symposium on Combinatorial Pattern Matching*, number 1848 in Lecture Notes in Computer Science, pages 195–209. Springer-Verlag, 2000.

[Kos89] S. R. Kosaraju. Efficient tree pattern matching. In *Proceedings of the 30th IEEE Annual Symposium on Foundations of Computer Science*, pages 178–183. IEEE Computer Society Press, 1989.

[KR87] R. M. Karp and M. O. Rabin. Efficient randomized pattern-matching algorithms. *IBM Journal of Research and Development*, 31(2):249–260, 1987.

[KR95] G. Kucherov and M. Rusinowitch. Matching a set of strings with variable length don't cares. In *Proceedings of the 6th Annual Symposium on Combinatorial Pattern Matching*, number 937 in Lecture Notes in Computer Science, pages 230–247. Springer-Verlag, 1995.

[Kri87] K. Krithivasan. Efficient two-dimensional parallel and serial approximate pattern matching. Technical Report CAR-TR-259, University of Maryland, 1987.

[KS87] K. Krithivasan and R. Sitalakshmi. Efficient two-dimensional pattern matching in the presence of errors. *Information Science*, 43:169–184, 1987.

[KS95] J. Kececioglu and D. Sankoff. Exact and approximation algorithms for sorting by reversals, with application to genome rearrangement. *Algorithmica*, 13(1/2):180–210, 1995.

[KS98] J. Kärkkäinen and E. Sutinen. Lempel-Ziv index for q-grams. *Algorithmica*, 21(1):137–154, 1998.

[KTS+98] T. Kida, M. Takeda, A. Shinohara, M. Miyazaki, and S. Arikawa. Multiple pattern matching in LZW compressed text. In *Proceedings of the 8th Data Compression Conference*, pages 103–112. IEEE Computer Society Press, 1998.

[KTSA99] T. Kida, M. Takeda, A. Shinohara, and S. Arikawa. Shift-And approach to pattern matching in LZW compressed text. In *Proceedings of the 10th Annual Symposium on Combinatorial Pattern Matching*, number 1645 in Lecture Notes in Computer Science, pages 1–13. Springer-Verlag, 1999.

[KU94] J. Kärkkäinen and E. Ukkonen. Two and higher dimensional pattern matching in optimal expected time. In *Proceedings of the 5th ACM-SIAM Annual Symposium on Discrete Algorithms*, pages 715–723. ACM Press, 1994.

[KU96] J. Kärkkäinen and E. Ukkonen. Lempel-Ziv parsing and sublinear-size index structures for string matching. In N. Ziviani, R. Baeza-Yates, and K. Guimarães, editors, *Proceedings of the 3rd South American Workshop on String Processing*, pages 141–155, Recife, Brazil. Carleton University Press, 1996.

[Lev65] V. I. Levenshtein. Binary codes capable of correcting spurious insertions and deletions of ones. *Problems of Information Transmission*, 1:8–17, 1965.

[LT97] D. P. Lopresti and A. Tomkins. Block edit models for approximate string matching. *Theoretical Computer Science*, 181(1):159–179, 1997.

[LV89] G. M. Landau and U. Vishkin. Fast parallel and serial approximate string matching. *Journal of Algorithms*, 10(2):157–169, 1989.

[LW75] R. Lowrance and R. A. Wagner. An extension of the string-to-string correction problem. *Journal of the ACM*, 22(2):177–183, 1975.

[Man89] U. Manber. *Introduction to Algorithms.* Addison-Wesley, 1989.

[Man97] U. Manber. A text compression scheme that allows fast searching directly in the compressed file. *ACM Transactions on Information Systems*, 15(2):124–136, 1997.

[MBY91] U. Manber and R. A. Baeza-Yates. An algorithm for string matching with a sequence of don't cares. *Information Processing Letters*, 37(3):133–136, 1991.

[McC76] E. M. McCreight. A space-economical suffix tree construction algorithm. *Journal of Algorithms*, 23(2):262–272, 1976.

[Meh84] K. Mehlhorn. *Data Structures and Algorithms 1: Sorting and Searching.* Springer-Verlag, 1984.

[Mel96] B. Melichar. String matching with k differences by finite automata. In *Proceedings of the 13th International Conference on Pattern Recognition*, volume II, pages 256–260, Vienna, Austria. IEEE Computer Society Press, 1996.

[MKT+00] T. Matsumoto, T. Kida, M. Takeda, A. Shinohara, and S. Arikawa. Bit-parallel approach to approximate string matching in compressed texts. In *Proceedings of the 8th String Processing and Information Retrieval*, pages 221–228. IEEE Computer Society Press, 2000.

[MM89] E. W. Myers and W. Miller. Approximate matching of regular expressions. *Bulletin of Mathematical Biology*, 51:7–37, 1989.

[MM91] G. Mehldau and E. W. Myers. A system for pattern matching applications on biosequences. *Computer Applications in Biosciences*, 9(3):299–314, 1991.

[MM93] U. Manber and E. W. Myers. Suffix arrays: a new method for on-line string searches. *SIAM Journal on Computing*, 22(5):935–948, 1993.

[MM96] R. Muth and U. Manber. Approximate multiple string search. In *Proceedings of the 7th Annual Symposium on Combinatorial Pattern Matching*, number 1075 in Lecture Notes in Computer Science, pages 75–86. Springer-Verlag, 1996.

[MNZBY00] E. Moura, G. Navarro, N. Ziviani, and R. Baeza-Yates. Fast and flexible word searching on compressed text. *ACM Transactions on Information Systems*, 18(2):113–139, 2000.

[MP70] J. H. Morris, Jr and V. R. Pratt. A linear pattern-matching algorithm. Report 40, University of California, Berkeley, 1970.

[MP94] S. Muthukrishnan and K. Palem. Non-standard stringology: Algorithms and complexity. In *Proceedings of the 26th ACM Symposium on the Theory of Computing*, pages 770–779. ACM Press, 1994.

[MRS96] H. M. Mahmoud, M. Régnier, and R. T. Smythe. Analysis of Boyer-Moore-Horspool string-matching heuristic. Report 9634, The George Washington University, 1996.

[MW94] U. Manber and S. Wu. GLIMPSE: A tool to search through entire file systems. In *Proceedings of the USENIX Winter 1994 Technical Conference*, pages 23–32, San Francisco, CA. USENIX Association, 1994.

[Mye92] E. W. Myers. A four russians algorithm for regular expression pattern matching. *Journal of the ACM*, 39(2):430–448, 1992.

[Mye94] E. W. Myers. A sublinear algorithm for approximate keyword searching. *Algorithmica*, 12(4/5):345–374, 1994.

[Mye95] E. W. Myers. Approximately matching context-free languages. *Information Processing Letters*, 54(2):85–92, 1995.

[Mye96] E. W. Myers. Approximate matching of network expression with spacers. *Journal of Computational Biology*, 3(1):33–51, 1996.

[Mye99] E. W. Myers. A fast bit-vector algorithm for approximate string matching based on dynamic programming. *Journal of the ACM*, 46(3):395–415, 1999.

[Nav98] G. Navarro. *Approximate Text Searching.* PhD thesis, Department of Com-

puter Science, University of Chile, 1998.

[Nav00] G. Navarro. Improved approximate pattern matching on hypertext. *Theoretical Computer Science*, 237:455–463, 2000.

[Nav01a] G. Navarro. A guided tour to approximate string matching. *ACM Computing Surveys*, 33(1):31–88, 2001.

[Nav01b] G. Navarro. Nr-grep: a fast and flexible pattern matching tool. *Software Practice and Experience (SPE)*, 2001. To appear.

[Nav01c] G. Navarro. Regular expression searching over Ziv-Lempel compressed text. In *Proceedings of the 12th Annual Symposium on Combinatorial Pattern Matching*, number 2089 in Lecture Notes in Computer Science, pages 1–17. Springer-Verlag, 2001.

[NBY99] G. Navarro and R. Baeza-Yates. Very fast and simple approximate string matching. *Information Processing Letters*, 72:65–70, 1999.

[NBY00] G. Navarro and R. Baeza-Yates. A hybrid indexing method for approximate string matching. *Journal of Discrete Algorithms*, 1(1):205–239, 2000. Special issue on Matching Patterns.

[NKT+01] G. Navarro, T. Kida, M. Takeda, A. Shinohara, and S. Arikawa. Faster approximate string matching over compressed text. In *Proceedings of the 11th Data Compression Conference*, pages 459–468. IEEE Computer Society Press, 2001.

[NMN+00] G. Navarro, E. Moura, M. Neubert, N. Ziviani, and R. Baeza-Yates. Adding compression to block addressing inverted indexes. *Information Retrieval*, 3(1):49–77, 2000.

[NR99a] G. Navarro and M. Raffinot. Fast regular expression search. In *Proceedings of the 3rd Workshop on Algorithm Engineering*, number 1668 in Lecture Notes in Computer Science, pages 199–213. Springer-Verlag, 1999.

[NR99b] G. Navarro and M. Raffinot. A general practical approach to pattern matching over Ziv-Lempel compressed text. In *Proceedings of the 10th Annual Symposium on Combinatorial Pattern Matching*, number 1645 in Lecture Notes in Computer Science, pages 14–36. Springer-Verlag, 1999.

[NR00] G. Navarro and M. Raffinot. Fast and flexible string matching by combining bit-parallelism and suffix automata. *ACM Journal of Experimental Algorithmics (JEA)*, 5(4), 2000. `http://www.jea.acm.org`.

[NR01a] G. Navarro and M. Raffinot. Compact DFA representation for fast regular expression search. In *Proceedings of the 5th Workshop on Algorithm Engineering*, number 2141 in Lecture Notes in Computer Science, pages 1–12, 2001.

[NR01b] G. Navarro and M. Raffinot. Fast and simple character classes and bounded gaps pattern matching, with application to protein searching. In *Proceedings of the 5th Annual International Conference on Computational Molecular Biology*, pages 231–240. ACM Press, 2001.

[NSF01] P. Nicodème, B. Salvy, and P. Flajolet. Motif statistics. *Theoretical Computer Science*, 2001. To appear.

[NT00] G. Navarro and J. Tarhio. Boyer-Moore string matching over Ziv-Lempel

compressed text. In *Proceedings of the 11th Annual Symposium on Combinatorial Pattern Matching*, number 1848 in Lecture Notes in Computer Science, pages 166–180. Springer-Verlag, 2000.

[Par96] K. Park. Analysis of two-dimensional approximate pattern matching algorithms. In *Proceedings of the 7th Annual Symposium on Combinatorial Pattern Matching*, number 1075 in Lecture Notes in Computer Science, pages 335–347. Springer-Verlag, 1996.

[Pea91] W. R. Pearson. Searching protein sequence libraries: comparison of the sensitivity and selectivity of the Smith-Waterman and FASTA algorithms. *Genomics*, 11:635–650, 1991.

[Pev00] P. A. Pevzner. *Computational Molecular Biology: An Algorithmic Approach.* MIT Press, 2000.

[Pin85] R. Y. Pinter. Efficient string matching with don't care pattern. In A. Apostolico and Z. Galil, editors, *Combinatorial Algorithms on Words*, pages 11–29. Springer-Verlag, 1985.

[PL88] W. R. Pearson and D. J. Lipman. Improved tools for biological sequence comparison. *Proceedings of the National Academy of Sciences of the U.S.A.*, 85:2444–2448, 1988.

[Raf97] M. Raffinot. On the multi backward dawg matching algorithm (MultiBDM). In R. Baeza-Yates, editor, *Proceedings of the 4th South American Workshop on String Processing*, pages 149–165, Valparaíso, Chile. Carleton University Press, 1997.

[Rég89] M. Régnier. Knuth-Morris-Pratt algorithm: an analysis. In *Proceedings of the 14th Symposium on Mathematical Foundations of Computer Science*, number 379 in Lecture Notes in Computer Science, pages 431–444. Springer-Verlag, 1989.

[RH91] S. Ranka and T. Heywood. Two-dimensional pattern matching with k mismatches. *Pattern Recognition*, 24(1):31–40, 1991.

[RS97] M. Régnier and W. Szpankowski. On the approximate pattern occurrence in a text. In *Proceedings Compression and Complexity of SEQUENCES'97.* IEEE Press, 1997.

[Ryt80] W. Rytter. A correct preprocessing algorithm for Boyer-Moore string searching. *SIAM Journal on Computing*, 9(3):509–512, 1980.

[Sad99] K. Sadakane. A modified Burrows-Wheeler transformation for case-insensitive search with application to suffix array compression. In *Proceedings of the 9th Data Compression Conference*, page 548, 1999. Poster. `http://www-imai.is.s.u-tokyo.ac.jp/~sada/papers/Sada99b.ps.gz`.

[Sad00] K. Sadakane. Compressed text databases with efficient query algorithms based on the compressed suffix array. In *Proceedings of the 11st International Symposium on Algorithms and Computation*, number 1969 in Lecture Notes in Computer Science, pages 410–421. Springer-Verlag, 2000.

[Sch98] J. P. Schmidt. All highest scoring paths in weighted grid graphs and their application to finding all approximate repeats in strings. *SIAM Journal on Computing*, 27(4):972–992, 1998.

[Sed88] R. Sedgewick. *Algorithms*. Addison-Wesley, 1988.

[Sel80] P. H. Sellers. The theory and computation of evolutionary distances: Pattern recognition. *Journal of Algorithms*, 1(4):359–373, 1980.

[Sim93] I. Simon. String matching algorithms and automata. In R. Baeza-Yates and N. Ziviani, editors, *Proceedings of the 1st South American Workshop on String Processing*, pages 151–157, Brazil. Universidade Federal de Minas Gerais, 1993.

[SK83] D. Sankoff and J. B. Kruskal. *Time Warps, String Edits, and Macromolecules: the Theory and Practice of Sequence Comparison*. Addison-Wesley, 1983.

[SM98] M.-F. Sagot and E. W. Myers. Identifying satellites and periodic repetitions in biological sequences. *Journal of Computational Biology*, 5:539–554, 1998.

[SMT+00] Y. Shibata, T. Matsumoto, M. Takeda, A. Shinohara, and S. Arikawa. A Boyer-Moore type algorithm for compressed pattern matching. In *Proceedings of the 11th Annual Symposium on Combinatorial Pattern Matching*, number 1848 in Lecture Notes in Computer Science, pages 181–194. Springer-Verlag, 2000.

[Sri86] M. A. Sridhar. Efficient algorithms for multiple pattern matching. Technical Report 661, University of Wisconsin-Madison, 1986.

[ST95] E. Sutinen and J. Tarhio. On using q-gram locations in approximate string matching. In *Proceedings 3rd Annual European Symposium*, number 979 in Lecture Notes in Computer Science, pages 327–340. Springer-Verlag, 1995.

[Sun90] D. M. Sunday. A very fast substring search algorithm. *Communications of the ACM*, 33(8):132–142, 1990.

[SV96] S.C. Sahinalp and U. Vishkin. Efficient approximate and dynamic matching of patterns using a labeling paradigm. In *Proceedings of the 37th IEEE Annual Symposium on Foundations of Computer Science*. IEEE Computer Society Press, 1996.

[SW81] T. F. Smith and M. S. Waterman. Identification of common molecular sequences. *Journal of Molecular Biology*, 147:195–197, 1981.

[SZ97] D. Shasha and K. Zhang. Approximate tree pattern matching. In *Pattern Matching Algorithms*, pages 341–371. Oxford University Press, 1997.

[Tho68] K. Thompson. Regular expression search algorithm. *Communications of the ACM*, 11:419–422, 1968.

[TSM+01] M. Takeda, Y. Shibata, T. Matsumoto, T. Kida, A. Shinohara, S. Fukamachi, T. Shinohara, and S. Arikawa. Speeding up pattern matching by text compression: The dawn of a new era. *Journal of the Information Processing Society of Japan (IPSJ)*, 42(3), 2001. To appear.

[TU88] J. Tarhio and E. Ukkonen. A greedy approximation algorithm for constructing shortest common superstrings. *Theoretical Computer Science*, 57(1):131–145, 1988.

[TU93] J. Tarhio and E. Ukkonen. Approximate Boyer-Moore string matching. *SIAM Journal on Computing*, 22(2):243–260, 1993.

[Ukk85] E. Ukkonen. Finding approximate patterns in strings. *Journal of Algorithms*, 6(1–3):132–137, 1985.

[Ukk92] E. Ukkonen. Approximate string matching with q-grams and maximal matches. *Theoretical Computer Science*, 92(1):191–212, 1992.

[Wat96] B. W. Watson. A new regular grammar pattern matching algorithm. In *Proceedings of the 4th Annual European Symposium*, number 1136 in Lecture Notes in Computer Science, pages 364–377. Springer-Verlag, 1996.

[WM92a] S. Wu and U. Manber. Agrep – a fast approximate pattern-matching tool. In *Proceedings USENIX Winter 1992 Technical Conference*, pages 153–162. USENIX Association, 1992.

[WM92b] S. Wu and U. Manber. Fast text searching allowing errors. *Communications of the ACM*, 35(10):83–91, 1992.

[WM94] S. Wu and U. Manber. A fast algorithm for multi-pattern searching. Report TR-94-17, Department of Computer Science, University of Arizona, Tucson, AZ, 1994.

[WMB99] I. Witten, A. Moffat, and T. Bell. *Managing Gigabytes*. Van Nostrand Reinhold, 2nd edition, 1999.

[WMM95] S. Wu, U. Manber, and E. W. Myers. A subquadratic algorithm for approximate regular expression matching. *Journal of Algorithms*, 19(3):346–360, 1995.

[WMM96] S. Wu, U. Manber, and E. W. Myers. A subquadratic algorithm for approximate limited expression matching. *Algorithmica*, 15(1):50–67, 1996.

[Yao79] A. C. Yao. The complexity of pattern matching for a random string. *SIAM Journal on Computing*, 8(3):368–387, 1979.

[ZL77] J. Ziv and A. Lempel. A universal algorithm for sequential data compression. *IEEE Transactions on Information Theory*, 23:337–343, 1977.

[ZL78] J. Ziv and A. Lempel. Compression of individual sequences via variable length coding. *IEEE Transactions on Information Theory*, 24:530–536, 1978.

[ZS97] K. Zhang and D. Shasha. Tree pattern matching. In A. Apostolico and Z. Galil, editors, *Pattern Matching Algorithms*, chapter 11, pages 341–371. Oxford University Press, 1997.

[ZSW94] K. Zhang, D. Shasha, and J. T. L. Wang. Approximate tree matching in the presence of variable length don't cares. *Journal of Algorithms*, 16(1):33–66, 1994.

[ZT89] R. F. Zhu and T. Takaoka. A technique for two-dimensional pattern matching. *Communications of the ACM*, 32(9):1110–1120, 1989.

[ZWM99] K. Zhang, L. Wang, and B. Ma. Computing similarity between RNA structures. In *Proceedings of the 10th Annual Symposium on Combinatorial Pattern Matching*, number 1645 in Lecture Notes in Computer Science, pages 281–293. Springer-Verlag, 1999.

索　引

符号

closure　ε-闭集　5.3.2
example　示例　5.3.2

BNDM algorithm　ABNDM 算法　6.5, 6.5.2, 6.5.3, 6.6.3, 6.8, 6.9
example　示例　6.5.2
cyclic　无环的　6.7.1
dvanced Aho-Corasick algorithm　高级 Aho-Corasick 算法　3.2.3, 3.5
ho-Corasick algorithm　Aho-Corasick算法　3.2.2, 3.3.1, 3.5
advanced　高级的　3.2.3
complexity　复杂度　3.2.2
example　示例　3.2.2
preprocessing　预处理　3.2.2
pproximate extended searching　近似扩展搜索　6.7
pproximate matching　近似匹配　6.1
utomaton　自动机　2.2.2, 6.3, 6.6.3

ackward Dawg Matching algorithm　BDM算法　2.4.1
ackword Nondeterministic Dawg Matching algorithm　BNDM 算法　2.4.2
Backward Oracle Matching algorithm　BOM 算法　2.4.3
BDM algorithm　BDM 算法　2.4.1, 4.2.1, 6.5.2
related reference　相关参考　2.6
BestFactor procedure　BestFactor 过程　5.5.2
Bison software　Bison 软件　5.8
Bit-parallelism　位并行　2.2.2, 3.2, 3.4, 4.1, 5.4, 5.5.3, 6.3, 6.4, 6.7.3
block distance　块距离　6.9
BNDM algorithm　BNDM 算法　6.9
example　示例　2.4.2
multiple　多模式　3.4.1
BNDM_preproc procedure　BNDM_preproc 过程　5.5.3
BOM algorithm　BOM 算法　2.4.3, 4.2.1
example　示例　2.4.3
multiple　多串的 BOM 算法　3.4.3
bounded length gaps　限长空位　4.1, 4.3
Boyer-Moore algorithm　Boyer-Moore 算法　2.3.1, 3.3
complexity　复杂度　2.3.1
preprocessing　预处理　2.3.1
related reference　相关参考　2.6
BPD algorithm　BPD算法　6.4.1, 6.5.2, 6.6.3, 6.7.3, 6.9

example 示例 6.4.1.2
BPGlushkov algorithm BPGlushkov 算法 5.4.2, 5.6
bit-parallel 位并行 5.4.2
BPM algorithm BPM 算法 6.4.2, 6.5.1, 6.5.2, 6.6.3, 6.9
example 示例 6.4.2
BPR algorithm BPR 算法 6.4.1, 6.5.2, 6.6.3, 6.9
example 示例 6.4.1
BPThompson algorithm BPThompson 算法 5.4.1, 5.4.2, 5.6
example 示例 5.4.1
reducing space 减少空间 5.4.1
Build_AC procedure Build_AC 过程 3.2.2
Build_Oracle_Multiple procedure Build_Oracle_Multiple 过程 3.4.3
BuildDFA algorithm BuildDFA 算法 5.3.2
BuildEps procedure BuildEps 过程 5.4.1
BuildState procedure BuildState 过程 5.3.2
BuildTran procedure BuildTran 过程 5.4.2

C

classes of characters 字符组 4.1, 4.2
in the pattern 模式串的字符组 4.2.1
in the text 文本中的字符组 4.2.2
Commentz-Walter algorithm Commentz-Walter 算法 3.3.1, 3.6
complexity 复杂度 3.3.1
Compute_pref procedure Compute_pref 过程 5.5.1
Compute_Reach procedure Compute_Reach 过程 5.5.3
[illegible] 常数空间算法 2.6
context-free grammar 上下文无关语法 6.9
CreateSuperp procedure CreateSuperp 过程 6.7
CreateTree algorithm CreateTree 算法 6.5.1

D

deletion 删除 6.1, 6.4.1.2
DFA 5.3.1, 5.3.2, 5.3.3, 5.4, 5.5.1, 6.3, 6.7.2, 6.7.3
searching 搜索 5.3.2
DFAClassical algorithm DFAClassical 算法 5.3.2, 5.6
DFA Modules algorithm DFA Modules 算法 5.3.2
diagonal 对角线的 6.3, 6.7.3
diagonal-wise bit-parallelism 对角线式位并行 6.4.1
difference 差异 6.1
DP algorithm DP 算法 6.2.3, 6.4, 6.4.1
dynamic multiple string matching 动态多串匹配 3.6
dynamic programming algorithm 动态规划算法 6.2, 6.7.1

E

edit distance 编辑距离 6.1, 6.7.1
EpsClosure algorithm EpsClosure 算法 5.3.2
example 示例 5.3.2
experimental map 实验图
approximate searching 近似搜索 6.8
multiple string 多字符串 3.5
regular expre ssion 正则表达式 5.6
single string 单字符串 2.5
experiments 实验 2.5, 3.5, 5.6, 6.8
extended string matching 扩展串匹配 4.1
Extended-BNDM algorithm Extended-BNDM 算法 4.5.2, 4.6
[illegible] 示例 4.5.2

ended-Shift_And algorithm Extended-Shift_And 算法 4.5.1
xample 示例 4.5.1

tor oracle Oracle 因子
ultiple string 多字符串 3.4.3
earch 搜索 2.4.3, 3.4.3
ingle string 单字符串 2.4.3.1
or searching 子串搜索
ultiple string matching 多字符串匹配 3.1, 3.4
ingle string matching 单字符串匹配 2.1, 2.4, 2.4.3
ring 过滤 6.5, 6.5.3
egular expression 正则表达式 5.5
x software Flex 软件 5.8
ur-Russians approach Four-Russians 逼近法 6.3, 6.7.2

ps-BNDM algorithm Gaps-BNDM 算法 4.3.2
ps-Shift-And algorithm Gaps-Shift-And 算法 4.3.1
xample 示例 4.3.1
ushkov algorithm Glushkov 算法 5.2.2, 5.4.2
bit-parallel 位并行 5.4.2
ushkov automaton Glushkov 自动机 5.2.2, 5.3, 5.5
xample 示例 5.2.2, 5.4.2
ushkov construction Glushkov 构建 5.2, 5.4.2
omplexity 复杂度 5.2.2
roperties 性质 5.2.2
ushkov_variables procedure Glushkov_variables 过程 5.2.2
graph 图 6.7.1

H

Hamming sidtance Hamming 距离 6.9
hash table 哈希表 3.3.3, 6.6.1
hashing 哈希
  approximate searching 近似搜索 6.6.1
  multiple string matching 多字符串匹配
  single string matching 单字符串匹配 2.6
hierarchical verification 分级校验 6.5.1, 6.6.2
Horspool algorithm Hospool 算法 2.3.2, 3.2.3, 4.2.1
  example 示例 2.3.2
  multiple 多串的 2.3.2

I

insertion 插入 6.1, 6.4.1

K

Knuth-Morris-Pratt algorithm Knuth-Morris-Pratt 算法 2.2.1
  complexity 复杂度 2.2.1
  preprocessing 预处理 2.2.1
  related references 相关参考 2.6

L

Levenshtein distance Levenshtein 距离 6.1
Lex software Lex 软件 5.8
lmin procedure lmin 过程 5.5

M

module 模块 5.3.3
MultiBP algorithm MultiBP 算法 6.6.3, 6.8
multidimensional text 多维文本 6.9

MultiFactRE algorithm MultiFactRE 算法 5.5.2
MultiHash algorithm MultiHash 算法 6.6, 6.8
MultiPEX algorithm MultiPEX 算法 6.6, 6.8
multiple approximate searching 多串近似搜索 6.6
multiple BNDM 多串 BNDM 3.4.1, 5.6
  example 示例 3.4.1
multiple Horspool algorithm 多串 Horspool 算法 3.3.2
multiple pattern searching 多模式串搜索 4.6
multiple Shift-And 多串 Shift-And 3.2.1, 3.4
  example 示例 3.2.1
multiple string matching 多字符串匹配 3
  regular expression 正则表达式 5.5.1
MultiStringRE algorithm MultiStringRE 算法 5.5.1, 5.5.3, 5.6
  example 示例 5.5.1

N

network expressions 网络表达式 6.7.2
NFA 5.2, 5.3, 5.5, 6.5.2, 6.3, 6.6
NFAModules algorithm NFAModules 算法 5.6, 5.7, 5.8

O

optimal path 最优路径 6.2.1, 6.7.1
optional characters 可选字符 4.1, 4.4
Oracle-on-line algorithm Oracle-on-line 算法 2.4.3
Oracle_add_letter procedure Oracle_add_letter 过程 2.4.3

P

Parse algorithm Parse 算法 5.8
parsing 解析 5.1, 5.9
  example 示例 5.8
partitioning 划分 6.5.1, 6.6.2
permutation 重排 6.9
PEX algorithm PEX 算法 6.5.1, 6.5.3, 6.6.2, 6.9
  example 示例 6.5.3
positive-length gaps 长度大于 0 的空位 6.7.3
Pref procedure Pref 过程 5.5.1
  mMultiple string matching 多字符串匹配 3.2
  single string matching 单字符串匹配 2.2
PROSITE 4.1, 4.3, 4.7, 6.7.2

R

regular expression 正则表达式 5.1, 6.7, 6.7.2
  filtering 过滤 5.5
regular expression matching 正则表达式匹配 5.1
regular expression parsing 正则表达式解析 5.1, 5.7, 5.8
regular expression search 正则表达式搜索 5.3
RegularBNDM algorithm RegularBNDM 算法 5.5.3, 5.6
  example 示例 5.5.3
repeatable characters 可重复字符 4.1, 4.5
reversals 反转的 6.9
Reverse_Arrows procedure Reverse_Arrows 过程 5.5.3
row-wise bit-parallelism 行式位并行 6.4.1

S

SBDM algorithm SBDM 算法 3.4
SBOM algorithm SBOM 算法 3.4, 3.5
  complexity 复杂度 3.4.3
  example 示例 3.4.3

Backward Dawg Matching algorithm Set Backward Dawg 匹配算法 3.4
Backward Oracle Matching algorithm Set Backward Oracle 匹配算法 3.4
Horspool algorithm Set Horspool 算法 3.3.2, 6.5.1
complexity 复杂度 3.3.2
example 示例 3.3.2
of regular expression 正则表达式集合 5.7
Shift-And algorithm Shift-And 算法 2.2, 2.2.2, 4.1, 5.4.1
Shift-Or algorithm Shift-Or 算法 2.2, 2.2.2, 4.1
complexity 复杂度 2.2.2
example 示例 2.2.2
shortest path 最短路径 6.7.1
signatures 签名 6.5.3
string matching 字符串匹配 2.1
substitution 替换 6.1, 6.4.1
suffix automaton 后缀自动机
multiple string 多字符串 3.4.2
single string 单字符串 2..4.1
suffix searching 后缀搜索
multiple string matching 多字符串匹配 3.1, 3.3
single string matching 单字符串匹配 2.1, 2.3
Sunday algorithm Sunday 算法 2.3.2
superimposed automata 重叠自动机 6.6.3
supply function supply 函数
Aho-Corasick algorithm Aho-Corasick 算法 3.2.2
factor oracle Oracle 因子 2.4.3, 3.4.3

T

Thompson algorithm Thompson 算法 5.2.1
bit-parallel 位并行 5.4
complexity 复杂度 5.2.1
example 示例 5.2.1
search 搜索 5.3
Thompson automaton Thompson 自动机 5.2.1, 5.4
example 示例 5.2.1
properties 性质 5.2.1
Thompson construction Thompson 构建 5.2
Thompson_recur procedure Thompson_recur 过程 5.2.1
tree representation 树表示 5.1, 5.2.2, 5.8
trie 3.1, 3.2, 3.3, 3.4.3
Trie algorithm Trie 算法 3.1

U

unbounded alphabets 无限字母表 3.6

V

Verify procedure 校验过程 6.6.2

W

wild cards 通配符 4.5
Wu-Manber algorithm Wu-Manber 算法 3.3.3, 3.5
example 示例 3.3.3

Y

Yacc software Yacc 软件 5.8

中文书名	英文书名	作者	译审者
学（第六版）	Discrete Mathematics, 6E	Richard Johnsonbaugh	石纯一
构与算法分析（C++版）（第二版）	A Practice Introduction to Data Structures and Algorithm Analysis, 2E	Clifford A. Shaffer	张　铭
构与问题求解（Java版）（第二版）	Data Structures and Problem Solving Using Java, 2E	Mark Allen Weiss	陈清夷
计技巧与分析	Algorithms Design Techniques and Analysis	M. H. Alsuwaiyel	朱　洪
算法导论（第二版）	Introduction to Distributed Algorithms, 2E	Gerard Tel	
算法导论	Introduction to Computing and Algorithms	Russell Shackelford	章小莉
算全书	The Sourcebook of Parallel Computing	Ian Foster	李晓梅
系统结构：量化研究方法（第三版）	Computer Architecture:A Quantitive Approach, 3E	John L. Hennessy	郑纬民
理器设计：超标量处理器基础	Modern Processor Design: Fundamentals of Superscalar Processors	John Paul. Shen	戴　葵
编语言程序设计（第四版）	Assembly Language for Intel-Based Computers, 4E	Kip R. Irvine	罗云彬
数处理器基础：理论与应用 (第四版)	Digital and Microprocessor Fundamentals: Theory and Application, 4E	William Kleita	张太镒
系统	Computer System: A Programmer's Perspective	Randal E. Bryant	
处理器结构、编程与接口（第六版）	The Intel Microprocessors Architecture, Programming, and Interfacing, 6E	Barry B. Brey	金惠华
系统设计与体系结构（第二版）	Computer Systems Design and Architecture, 2E	Vincent P. Heuring	邹恒明
统：并发与分布式软件设计	Operating Systems: Concurrent and Distributed Software Design	Jean Bacon	陈向群
统：内核与设计原理（第四版）	Operating Systems：Internals and Design Principles, 4E	William Stallings	邹　鹏
门级教程（第四版）	UNIX Unbounded: A Beginning Approach, 4E	Amir Afzal	张焕国
算机互连网络技术：一种工程方法	Interconnection Networks: An Engineering Approach	Jose Duato	谢伦国
通信系统：技术、标准与网络	Multimedia Communication Systems: Techniques, Standards, and Networks	K. R. Rao	冯　刚
P 进行网际互联（第一、二、三卷）	Internetworking with TCP/IP Vol 1,Vol 2,Vol 3	Douglas E. Comer	谢希仁
计算机通信（第七版）	Data and Computer Communications, 7E	William Stallings	谢希仁
码学与网络安全: 原理与实践(第三版)	Cryptography and Network Security: Principles and Practice, 3E	Williams Stallings	张焕国
缩原理与应用（第二版）	Data Compression: The Complete Reference, 2E	David Salomon	吴乐南
理器与网络系统设计	Network System Design	Douglas E. Comer	
全：公众世界中的私人通信（第二版	Network Security: Private Communication in a Public World，2E	Charlie Kaufman	许剑卓
安全学：安全的艺术与科学	Computer Security: Art and Science	Matt Bishop	陈克非
码学理论与实践	Modern Cryptography: Theory & Practice	Wenbo Mao	王育民
析、体系结构与设计（第二版）	Network Analysis, Architecture, and Design, 2E	James D. McCabe	李　刚
全原理与应用（第三版）	Security in Computing, 3E	Charles P. Pfleeger	秦志光
学教程	C++ How to Program	Harvey M. Deitel	邱仲潘
程：从问题分析到程序设计	C++ Programming: From Problem Analysis to Program Design	D. S. Malik	钟书毅
程导论（第二版）	An Introduction to Programming With C++, 2E	Diane Zak	马良荔
心思想（第三版）	Computing Concepts with C++ Essentials, 3E	Cay Horstmann	晏海华
程问题求解：面向对象方法	Engineering Problem Solving with C++: An Object-Oriented Approach	Delores M. Etter	周　斌
a 2 大学教程	Advanced Java 2 Platform How to Program	Harvey M. Deitel	钱　方
明教程	Simply Java Programming	Harvey M. Deitel	张琛恩
向对象程序设计（第二版）	Java, Java, Java: Object-Oriented Problem Solving, 2E	Ralph Morelli	董永乐
UML 面向对象设计与编程	Java the UML Way: Integrating Object-Oriented Design Programming	Else Lervik	王　柏
译器的Java 实现（第二版）	Modern Compiler Implementation in Java, 2E	Andrew W. Appel	程　虎

中文书名	英文书名	作者	译审者	版
言：原理与实践（第二版）	Programming Languages: Principles and Practice, 2E	Kenneth C. Louden	黄林鹏	中、
象编程（第二版）	Object-Oriented Programming Using C++, 2E	Ira Pohl		英
第三版）	Human-Computer Interaction, 3E	Alan Dix Janet Finlay	蔡利栋	中、
超越人机交互	Interaction Design: Beyond Human-Computer Interaction	Jennifer Preece	麦中凡	中
计（第三版）	Designing the User Interface: Strategies for Effective Human-Computer Interaction	Ben Shneiderma	张国印	中
理	Digital Image Processing	K. R. Castleman	朱志刚	中
学（第二版）	Computer Graphics, 3E	Donald Hearn	蔡士杰	中、
一种现代方法	Computer Vision: A Modern Approach	David A. Forsyth	林学言	中
统：接口、应用与设计	Understanding Virtual Reality: Interface, Application, and Design	William Sherman	吴玲达	中
形学：造型、绘制与动画技术	Non-Photorealistic Computer Graphics: Modeling, Rendering, and Animation	Thomas Strothotte	叶修梓	中
实（第二版）	Virtual Reality Technology, 2E	Grigore C. Burdea	吴玲达	中
理论与实践	Artificial Intelligence: Theory and Practice	Thomas Dean	顾国昌	中、
	Artificial Intelligence	Rob Callan	黄厚宽	中
导引论	An Introduction to Multiagent Systems	Michael Wooldridge	石纯一	中
结构、设计与控制	Intelligent Systems: Architecture, Design and Control	Alexander M. Meystel	冯祖仁	中、
则（第二版）	Statistical Pattern Recognition, 2E	Andrew Webb	林孔元	中
第二版）	Pattern Recognition, 2E	Sergios Theodoridis	李晶皎	中
言学基础	Foundation of Statistical Natural Language Processing	Christopher D. Manning	苑春法	中
解（第二版）	Natural Language Understanding, 2E	James F. Allen	刘　群	中
理综论	Speech and Language Processing: An Introduction to Natural Language Processing, Computational Linguistics and Speech Recognition	Daniel Jurafsky	冯志伟	中
习	Statistical Learning Theory	Vladimir Naumovich Vapnik	张学工	中
据：数据挖掘、推理与预测	The Elements of Statistical Learning: Data Mining, Inference, and Predication	Trevor Hastie	范明柴	中
导论	An Introduction to Support Vector Machines and Other Kernel-based Learning Methods	Nello Cristianini	李国正	中
反馈控制（第四版）	Feedback Control of Dynamic Systems, 4E	Gene F. Franklin	朱齐丹	中
人：分析、系统与应用	Introduction to Robotics: Analysis, Systems, Applications	Saeed B. Niku	孙增圻	中
导论	An introduction to AI Robotics	Robin R. Murphy	孙增圻	中
工程实践技术手册	Robotic Explorations: A Hands-on Introduction to Engineering	Fred G. Martin	刘　荣	中
应用开发与管理（第二版）	Database Design, Application Development and Administration, 2E	Michael V. Mannino	唐常杰	中
管理（第六版）	Modern Database Management, 6E	Jeffrey A. Hoffer		英
型、语言与设计	Database: Models, Languages, Design	Jams L. Johnson	李天柱	中
调优：原理与技术	Database Tuning: Principles, Experiments, and Troubleshooting Techniques	Dennis Shasha	孟小峰	中
设计与模式	Object-Oriented Design and Patterns	Cay Horstmann	张琛恩	中
统工程：模拟、证明与应用指南	Petri Nets for Systems Engineering: A Guide to Modeling, Verification, and Applications	Claude Girault	袁崇义	中
量：度量与模型（第二版）	Metrics and Models in Software Quality Engineering, 2E	Stephen H. Kan	何志均	中
	The B-Book: Assigning Programs to Meanings	J-R Abrial	裘宗燕	中
言（第二版）	Numerical Recipes in C++: The Art of Scientific Computing, 2E	William H. Press	胡健伟	中、
二版）	The Grid 2: Blueprint for a New Computing Infrastructure	Ian Foster		中